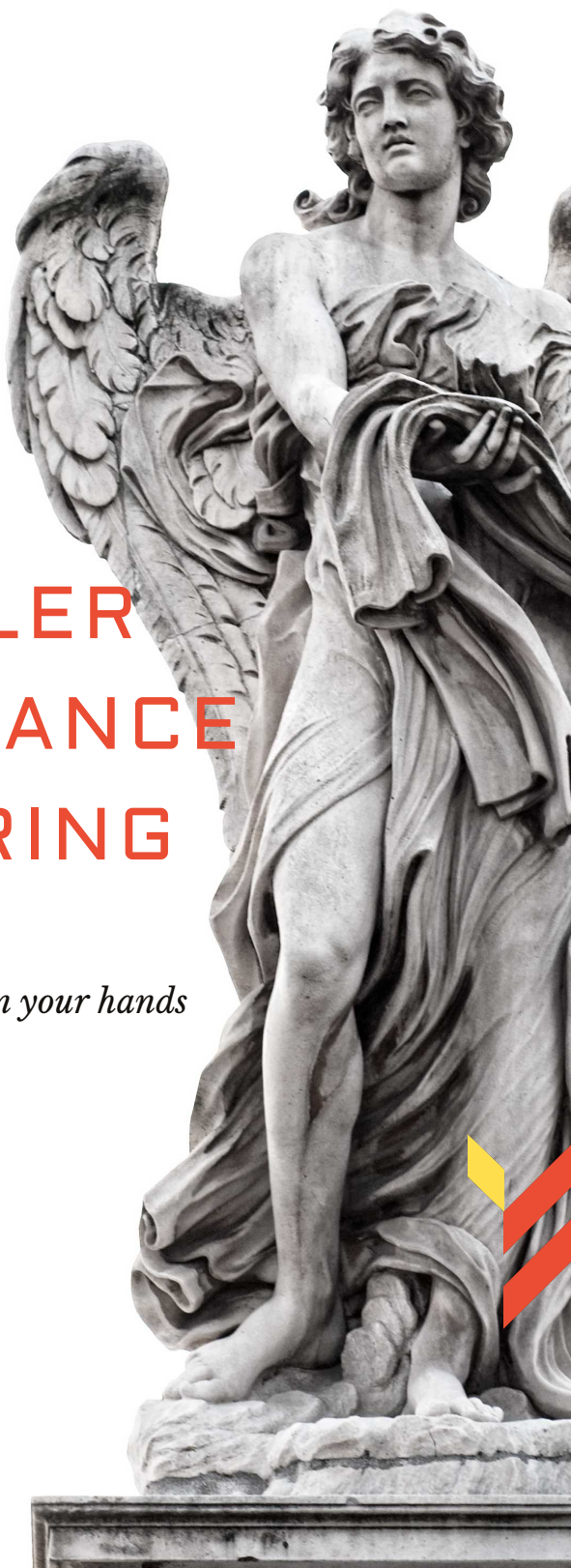




AI COMPILER PERFORMANCE ENGINEERING

It's all in your hands



Contents

Notation	xii
1 LLM Inference Performance Bottlenecks: From Symptoms to Root Causes	1
1.1 Prefill and Decode: Two Performance Regimes	1
1.2 The Decode Pain Point: Single-Token, Low-Batch Latency	2
1.3 Limits of Mainstream PyTorch and Hugging Face Inference	4
1.4 Quantifying Overheads: Launches, Memory, and Scheduling	5
1.5 Memory Bandwidth, Not Peak FLOPs	6
1.6 Multi-Hardware Pain Points: GPU, CPU, and NPU	7
1.7 Industry Benchmarks: Native Frameworks vs. Fused Kernels	8
1.8 Opening Case Study: YiRage Cross-Hardware Compilation Preview	9
1.9 Key Takeaways	14
1.10 Conclusion	15
2 Two Kernel Design Mindsets: Operator-Driven vs Dataflow-Driven	16
2.1 Operator First	16
2.2 Data Residency	17
2.3 Dataflow Philosophy	18
2.4 Cross Hardware	19
2.5 Iron Rules	20
2.6 MegaKernel Preview	21
2.7 Automation Path	21

2.8	Contrasting Schedules on One Decoder Layer	22
2.9	Key Takeaways	29
2.10	Conclusion	30
3	AI Hardware Architecture and Compiler Constraints	31
3.1	Hardware landscape	31
3.2	Constraint matrix	33
3.3	GPU constraints	35
3.4	CPU constraints	36
3.5	NPU constraints	37
3.6	Hardware-aware compilation	39
3.7	Benchmark methodology	40
3.8	YiRage hardware modeling	41
3.9	Key Takeaways	42
3.10	Conclusion	43
4	CUDA Hopper/Blackwell: Hardware Properties and Optimization Bounds	44
4.1	CUDA Memory Hierarchy	45
4.2	Tensor Memory Accelerator (TMA)	47
4.3	Thread Block Clusters and Distributed Shared Memory	49
4.4	Synchronization Mechanisms	50
4.5	Tensor Core Bounds at Batch-1 Decode	51
4.6	PTX and Softmax Micro-ops	52
4.7	YiRage CUDA Backend	53
4.8	Profiling Hopper Decode Changes	53
4.9	Hopper Decode Engineering Patterns	56
4.10	Worked Example: One Decode Layer on H100	61
4.11	Hopper Feature Reference for Compiler Authors	64
4.12	Key Takeaways	67
4.13	Conclusion	68
5	AMD XDNA/AIE Dataflow Architecture	70

5.1	XDNA tiles	71
5.2	Static dataflow rules	73
5.3	CUDA XDNA mapping	75
5.4	Online softmax HW	77
5.5	Architecture tradeoffs	78
5.6	Dataflow conclusion for inference	80
5.7	YiRage XDNA backend	81
5.8	XDNA decode review gates	85
5.9	Ryzen AI decode benchmarking	91
5.10	Key Takeaways	95
5.11	Conclusion	96
6	Data Residency and On-Chip Memory Design	97
6.1	Lifetime planning	97
6.2	GPU residency	99
6.3	CPU residency	101
6.4	NPU residency	102
6.5	YiRage memory pass	104
6.6	Key Takeaways	105
6.7	Conclusion	106
7	Static Role Assignment and Schedule-Free Kernels	107
7.1	Static roles	107
7.2	Thread mapping	110
7.3	No runtime scheduling	111
7.4	YiRage tiling	114
7.5	Key Takeaways	115
7.6	Conclusion	116
8	TMA Double-Buffer Pipelines and Async Movement	118
8.1	Pipeline theory	118
8.2	GPU TMA	120

8.3	CPU DMA	122
8.4	NPU pipeline	124
8.5	YiRage pipeline generation	125
8.6	Key Takeaways	127
8.7	Conclusion	128
9	Hierarchical Synchronization and Communication	129
9.1	Sync hierarchy	130
9.2	Hardware sync pick	132
9.3	Deadlock detection	134
9.4	YiRage sync pass	136
9.5	Key Takeaways	138
9.6	Conclusion	139
10	Attention Online Softmax Optimization	140
10.1	Online Softmax	141
10.2	Numerical Stability	144
10.3	Cross-Hardware Online Softmax	146
10.4	Softmax Fusion Boundaries	148
10.5	Profiling Online Softmax in Production Stacks	154
10.6	Key Takeaways	156
10.7	Conclusion	159
11	Full Decoder MegaKernel Implementation	161
11.1	MegaKernel Structure	163
11.2	QKV, Attention, and MLP Fusion	168
11.3	MegaKernel Source Walkthrough	172
11.4	Benchmark Results and Interpretation	178
11.5	Key Takeaways	184
11.6	Conclusion	184
12	Three Kernel Execution Modes	186
12.1	eager mode	187

12.2	graph mode	190
12.3	MegaKernel mode	192
12.4	mode selection	195
12.5	Key Takeaways	198
12.6	Conclusion	199
13	Core Theory: AI Compiler Optimization Principles	201
13.1	tiling theory	202
13.2	fusion theory	204
13.3	layout theory	207
13.4	parallel scheduling	209
13.5	GPU CPU NPU split	211
13.6	Key Takeaways	214
13.7	Conclusion	215
14	MLIR: Modern Compiler Infrastructure	217
14.1	MLIR HW matrix	218
14.2	MLIR dialect stack for decode export	220
14.3	MLIR arch passes	221
14.4	MLIR codegen	223
14.5	MLIR tuning	224
14.6	MLIR case study	225
14.7	YiRage MLIR integration notes	228
14.8	MLIR decode review gates	229
14.9	MLIR decode benchmarking methodology	233
14.10	Key Takeaways	239
14.11	Conclusion	240
15	XLA: Production-Grade AI Compiler	241
15.1	XLA HW matrix	241
15.2	XLA arch passes	242
15.3	XLA codegen	243

15.4 XLA tuning	244
15.5 XLA case study	245
15.6 Key Takeaways	246
15.7 Conclusion	246
16 TVM and Apache TVM Stack	248
16.1 TVM HW matrix	248
16.2 TVM arch passes	249
16.3 TVM codegen	251
16.4 TVM tuning	252
16.5 TVM case study	253
16.6 Key Takeaways	254
16.7 Conclusion	255
17 OpenAI Triton: Pythonic GPU Kernel Compiler	256
17.1 Triton HW matrix	256
17.2 Triton arch passes	258
17.3 Triton codegen	259
17.4 Triton tuning	260
17.5 Triton case study	261
17.6 Key Takeaways	262
17.7 Conclusion	262
18 IREE: MLIR-Native Runtime and Compilation	264
18.1 IREE HW matrix	264
18.2 IREE arch passes	266
18.3 IREE codegen	267
18.4 IREE tuning	268
18.5 IREE case study	269
18.6 Key Takeaways	269
18.7 Conclusion	270
19 Glow: Lightweight AI Compiler	271

19.1	Glow HW matrix	271
19.2	Glow arch passes	272
19.3	Glow codegen	273
19.4	Glow tuning	274
19.5	Glow case study	275
19.6	Key Takeaways	276
19.7	Conclusion	277
20	Mirage and Emerging AI Compilers	278
20.1	Mirage HW matrix	278
20.2	Mirage arch passes	280
20.3	Mirage codegen	281
20.4	Mirage tuning	282
20.5	Mirage case study	283
20.6	Key Takeaways	284
20.7	Conclusion	285
21	Unified AI Compiler Analysis and Cross-Hardware Benchmark	286
21.1	Compiler ratings	286
21.2	Compile overhead	288
21.3	Hardware binding	289
21.4	Search space	290
21.5	YiRage benchmark	291
21.6	Key Takeaways	292
21.7	Conclusion	292
22	Compiler-Driven End-to-End Performance Tuning	294
22.1	HW aware profiling	296
22.2	bottleneck locating	298
22.3	tuning workflow	300
22.4	regression gates	302
22.5	Key Takeaways	304

22.6 Conclusion	305
23 LLM and MoE Specialized Compilation	306
23.1 LLM decode compile	308
23.2 moe scheduling	310
23.3 kv hardware	312
23.4 hetero moe	314
23.5 Key Takeaways	315
23.6 Conclusion	316
24 Heterogeneous Compilation and Multi-Hardware Deployment	317
24.1 Heterogeneous cluster	317
24.2 Unified stack	319
24.3 Runtime adaptation	320
24.4 Version regression	321
24.5 Industrial cases	322
24.6 YiRage cluster	323
24.7 Key Takeaways	324
24.8 Conclusion	325
25 Auto-Optimization and AI-Assisted Compilation	326
25.1 RL autotune	327
25.2 HW reward	331
25.3 YiRage auto	335
25.4 Key Takeaways	338
25.5 Conclusion	339
26 Production Deployment and Engineering Best Practices	340
26.1 env packaging	342
26.2 multi HW ops	344
26.3 pitfalls	345
26.4 scheduling	346
26.5 Key Takeaways	348

26.6 Conclusion	349
27 Future Trends in AI Compilers	350
27.1 codesign	351
27.2 new architectures	353
27.3 edge cloud	354
27.4 autonomous kernels	355
27.5 Key Takeaways	357
27.6 Conclusion	358
28 LLM Inference Frameworks and Serving Runtimes	359
28.1 framework landscape	361
28.2 continuous batching	365
28.3 kv paging serv	369
28.4 framework bottlenecks	372
28.5 compiler integration hooks	375
28.6 Key Takeaways	380
28.7 Conclusion	381
29 YiRage Runtime Layer and Persistent Kernel Execution	382
29.1 yirage stack layers	385
29.2 persistent kernel decode	389
29.3 superoptimize handoff	394
29.4 hw registry runtime	398
29.5 deps build native	401
29.6 YiRage runtime decode benchmarking methodology	405
29.7 Key Takeaways	412
29.8 Conclusion	413
30 Framework, Compiler, and Runtime Co-Design for Decode Good-put	414
30.1 responsibility split	417
30.2 decode pipeline handoff	422

30.3 vllm yirage stack	425
30.4 mode selection fleet	429
30.5 production regression	432
30.6 Key Takeaways	435
30.7 Conclusion	436
Bibliography	438
Index	442

Notation

This section lists symbols and terms used throughout *AI Compiler Performance Engineering*. Figures and tables scale to `\linewidth` via `\autowidthfigure` and the `\autowidthtable` environment.

LLM Inference and Serving

L	Prompt or context length (tokens) in prefill or cached history
m	Number of autoregressive output tokens generated after prefill
BS	Batch size (requests or sequences scheduled together)
SL	Sequence length in a benchmark configuration (prompt + generated tokens)
KV	Key-value cache tensors reused across decode steps
<i>prefill</i>	Parallel forward over prompt tokens; bulk KV write
<i>decode</i>	One (or few) new tokens per forward; KV read and append

Performance Metrics

tok/s	Throughput: tokens processed per second
ms/token	Latency: milliseconds per generated token
I	Arithmetic intensity (FLOPs moved per byte transferred)
HDBI	Host–Device Balance Index; HDBI $\rightarrow 0$ indicates host-bound serving
launches/token	GPU (or device) kernel launches counted per output token
bytes/token	Device memory bytes read/written per output token (weights, KV, activations)

Compiler, GPU, and Hardware

$\mathcal{IR}$	Compiler intermediate representation before lowering
<i>pass</i>	Analysis or transform applied to $\mathcal{IR}$ (fusion, tiling, layout)
<i>tile</i>	Static sub-block of a tensor mapped to on-chip memory
SM	Streaming multiprocessor on NVIDIA GPUs
CTA	Cooperative thread array (CUDA thread block)
TMA	Tensor Memory Accelerator (async bulk copy on Hopper/Blackwell)
HBM	High-bandwidth memory for weights, KV cache, and spill traffic
AIE	AMD AI Engine tile array (XDNA spatial dataflow)

Chapter 1

LLM Inference Performance Bottlenecks: From Symptoms to Root Causes

Large language model (LLM) serving has matured into a systems discipline where the dominant cost is often not FLOPs but *how* weights, activations, and the key-value (KV) cache move through memory hierarchies and how many discrete GPU kernels the host must launch per token (Recasens, 2025). In production, most teams hit the same contradiction: prefill benchmarks look excellent on datasheets, yet interactive decode still misses p99 latency SLOs after months of kernel tuning (Fan, 2025). The durable misconception is that buying faster tensor cores or enabling FlashAttention “fixes” decode—when measured kernels-per-token and bytes-per-token often barely move because the stack remains operator-fragmented and host-orchestrated (Dao *et al.*, 2022). This chapter establishes the performance vocabulary used throughout the book: we separate **prefill** from **decode**, quantify where latency hides in mainstream PyTorch and Hugging Face stacks, and explain why cross-hardware compilation is not an optional portability exercise but a prerequisite for industrial decode optimization (Dao *et al.*, 2023).

1.1 Prefill and Decode: Two Performance Regimes

Autoregressive transformer inference divides naturally into two phases. In **prefill**, the model processes the entire prompt (or a prompt chunk) in parallel across sequence positions (Pichlmeier, 2024). Matrix multiplications are large, arithmetic

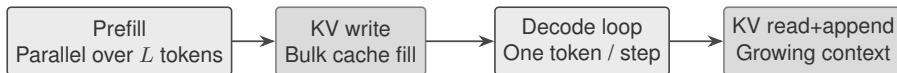


Figure 1.1: Prefill processes prompt chunks in parallel; decode appends one token per step via KV cache.

intensity is relatively high, and modern accelerators can approach their peak tensor throughput (SemiAnalysis, 2024). In **decode**, the model generates one new token per forward pass (per request, in the common batch-1 interactive setting), conditioned on all prior tokens through the KV cache (Li, 2024).

These phases obey different roofline limits. Prefill is frequently *compute-bound* on high-end GPUs: the GEMMs and attention blocks are wide enough to amortize memory latency (Kwon *et al.*, 2023). Decode, by contrast, repeatedly reads the full parameter set and growing KV tensors while performing only a thin slice of matmul work per token (Eto, 2025). Treating both phases with one optimization playbook—larger batches, FlashAttention defaults, tensor-core-first schedules—is a primary reason decode SLOs remain elusive after prefill tuning succeeds (Tang, 2026). Multiple independent analyses converge on the same conclusion: interactive decode is typically *memory-bandwidth bound* on GPUs, CPUs, and many NPUs, not FLOP-bound (Yu, 2026).

Production systems increasingly *disaggregate* prefill and decode into separate pools because optimizing for one phase can harm the other (Qiao, 2026). For example, large prefill batches maximize throughput, while decode pools prioritize tail latency under batch-1 or small-batch arrivals (Chen, 2026b). Recent large-model deployments provision substantially more decode capacity than prefill capacity because user-visible latency is dominated by per-token decode time once the prompt has been ingested (Wang, 2025b). DeepSeek-V3’s production layout is illustrative: the minimum prefill deployment unit spans 4 nodes (32 GPUs), whereas the minimum decode unit spans 40 nodes (320 GPUs)—a $10\times$ capacity skew reflecting phase-specific hardware mapping (Davies *et al.*, 2025).

1.2 The Decode Pain Point: Single-Token, Low-Batch Latency

Interactive assistants, coding copilots, and edge voice interfaces share a harsh constraint: users measure quality in *time-to-next-token*, often at batch size one (AMD, 2024a). At batch-1 decode, each layer performs narrow GEMMs (e.g., a single new query vector against frozen weights) and attention that must touch the

Table 1.1: Qualitative comparison of prefill vs. decode on a single transformer layer.

	Prefill	Decode (batch-1)
Tokens processed per step	L (prompt length)	1
Typical arithmetic intensity	Higher	Lower
Dominant limiter	Compute / tensor cores	Memory traffic / orchestration
KV cache growth per step	Written in bulk	Read + append one step
Serving objective	Throughput (tok/s)	Latency (ms/token)

entire cached history (Goodfellow *et al.*, 2016). The working set per token is small, but the *read footprint* is enormous: every weight matrix and every cached key/value vector must be visited again. This is why decode latency scales with model width and context length even when FLOPs per token look trivial on paper—the hardware spends its time streaming bytes, not executing MMA instructions.

Low batch sizes also prevent amortization of fixed costs—kernel launch, framework dispatch, collective synchronization, and memory allocator bookkeeping—across many tokens. decompose host-visible latency into three mutually exclusive components per kernel invocation: framework translation (ΔFT), CUDA-library front-end time (ΔCT), and kernel launch-path floor (ΔKT). Their Host–Device Balance Index (HDBI) relates device-active time to orchestration time; $\text{HDBI} \rightarrow 0$ marks a host-bound regime.

On H200 at batch-1 and sequence length 512, Llama-3.2-1B issues 850 kernel launches during prefill but 8,437 launches across a 10-token decode window (≈ 844 launches per autoregressive step)—nearly $10\times$ more dispatch work per forward pass. At decode configuration $\text{BS}=4/\text{SL}=2048$ ($m=10$ output tokens), dense Llama-3.2-1B averages 847.5 GPU kernel launches *per output token*; OLMoE-1B/7B averages 9,305 launches per token ($11\times$ denser). Measured null-kernel launch floor on H100 is $4.707\ \mu\text{s}$ (median $4.578\ \mu\text{s}$); at 9,305 launches/token that launch-path term alone contributes $\approx 44\ \text{ms}$ per token before framework dispatch is counted.

The engineering implication is blunt: shaving a few microseconds from a matmul inner loop matters less than eliminating redundant global-memory round trips, fusing layer fragments, and collapsing launch storms—especially when the product SLA is sub-50 ms per token.

1.3 Limits of Mainstream PyTorch and Hugging Face Inference

The conventional PyTorch + Hugging Face stack looks logically complete—every transformer sub-block maps to a familiar op—yet multi-hardware deployment exposes a fatal gap: the *schedule* is decided at runtime by a Python loop, not at compile time by residency and bandwidth constraints. A naïve **transformers** loop—one Python forward per token, eager PyTorch ops, separate kernels for layer norm, QKV projection, attention, MLP, residuals—mirrors the mathematical graph but not the memory hierarchy. Each operator materializes intermediates to global memory (GPU HBM or CPU DRAM), so a single decoder layer can trigger many read/write passes over tensors that could have stayed on chip given a different schedule.

Three structural problems recur:

1. **Operator fragmentation.** A decoder block decomposes into 10–20+ discrete kernels in unfused stacks. Each boundary exports data to HBM and re-imports it for the next op.
2. **Host orchestration tax.** Python dispatch, CUDA driver launch, and framework bookkeeping run on the CPU while the accelerator idles between micro-kernels.
3. **KV cache management overhead.** Without paged or fused cache layouts, attention reads scatter through memory, wasting bandwidth on indexing and padding.

Serving frameworks such as vLLM address *capacity* and *KV fragmentation* via PagedAttention and continuous batching, but they do not by themselves remove per-layer operator boundaries inside the forward pass (Vellaisamy *et al.*, 2026). FlashAttention reduces HBM traffic during prefill by IO-aware tiling; Flash-Decoding adds a parallelization dimension along KV sequence length so batch-1 decode can still saturate SMs when context is long (reported up to $8\times$ end-to-end speedup at very long sequences). These are necessary optimizations, yet a full decoder step still contains MLPs, norms, rotary embeddings, and sampling logic that remain separable kernels unless a **MegaKernel** or compiler-generated fused schedule reunifies them.

On CPUs the same fragmentation shows up as LLC misses rather than HBM stalls: each unfused boundary evicts cache lines that a fused decoder block could have kept resident across norm–matmul–activation chains. On XDNA-class NPUs,

Table 1.2: Verified kernel fragmentation on H100 decode (BS=4, SL=2048, $m=10$ tokens), from TaxBreak.

Model	Kernels/token	Total launches	GPU util. (%)
Llama-3.2-1B	847.5	8,475	58.9
Llama-3.2-3B	1,536.9	15,369	67.6
OLMoE-1B/7B	9,305.3	93,053	15.5
Qwen1.5-MoE-A2.7B	6,695.1	66,951	27.7

per-op dispatch is worse than wasteful—it violates static dataflow contracts and forces fallbacks to scalar host loops (AMD, 2024b). Industrial deployment therefore cannot copy a single “GPU recipe” into production; the framework graph must be re-planned per memory hierarchy, which is exactly the job YiRage takes over in Part VI.

1.4 Quantifying Overheads: Launches, Memory, and Scheduling

Recent microbenchmark studies decompose end-to-end decode latency into compute, memory, and *orchestration* components. TaxBreak reports that for Llama-3.2-1B-class models on H100, decode reduces orchestration time relative to prefill in absolute terms, but because decode issues roughly $10\times$ more launches per step, per-dispatch savings compound more aggressively in decode (Wang, 2025a). For MoE models, decode latency can grow to multiple seconds per token at batch-1 not because GEMMs are huge, but because dispatch dominates: thousands of kernels per token with a per-launch floor on the order of microseconds yields tens of milliseconds of pure launch-path cost before framework overhead.

CUDA Graphs record a static kernel DAG once and replay it with a single submission, amortizing CPU launch and framework dispatch (NVIDIA, 2024). Memory-Floor benchmarks (Chen, 2026a) ran a controlled CUDA Graphs A/B on Qwen-2.5-7B at batch-1, context 2048, across $N=10$ independent sessions: H100 speedup is $1.259\times$ (95% bootstrap CI [1.253, 1.267]), removing ≈ 3.05 ms of per-step host overhead ($\approx 20.6\%$ of the eager 14.83 ms median). On L4 at the same cell, Graphs yield only $1.028\times$ because the GPU already achieves $R_{\text{floor}}=0.81$ of its analytic HBM bandwidth floor—the device waits on memory, not launches.

Example 1.4.1. For Qwen-2.5-7B on H100, a single decode step issues ≈ 283 kernel launches (≈ 10 per layer $\times$ 28 layers plus embedding/LM-head). At batch-1

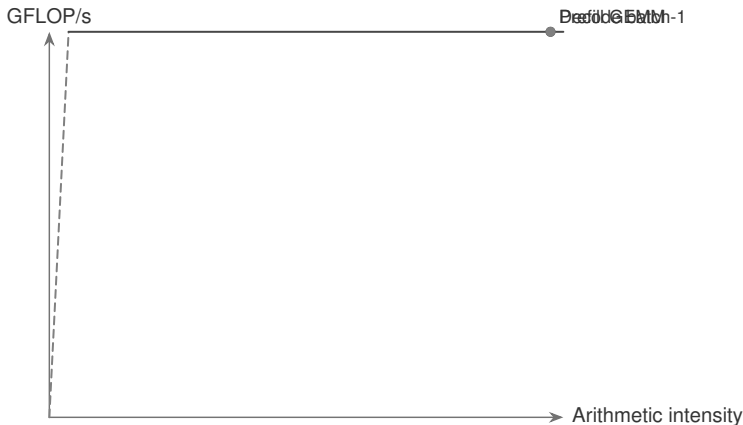


Figure 1.2: Decode sits on the memory-bandwidth slope of the roofline at low arithmetic intensity.

interactive decode, per-layer GPU compute can be $\approx 10 \mu s$ while launch gaps are $\approx 30 \mu s$ —launch dominates per block. Collapsing those boundaries via MegaKernel fusion or graph capture can exceed the benefit of micro-optimizing individual GEMMs.

1.5 Memory Bandwidth, Not Peak FLOPs

The roofline model makes the decode story precise. Arithmetic intensity (FLOPs per byte moved) for batch-1 decode is low: one token’s activations participate in thin matrix–vector-like operations while weights and KV cache supply the bulk of bytes. Accelerators therefore sit on the memory-bandwidth sloped region of the roofline rather than the flat compute cap. The LIMINAL limit study further reports that at low batch, tensor-core utilization can be $\leq 1\%$ on both DRAM- and SRAM-based accelerator designs when compute is reasonably provisioned—decode simply does not present enough arithmetic intensity to engage peak FLOPs.

quantify “achieved fraction of peak bandwidth” via $R_{\text{floor}} = t_{\text{floor}}/t_{\text{obs}}$ where $t_{\text{floor}} = (W + K)/B_{\text{peak}}$. For Qwen-2.5-7B at $\text{ctx}=2048$, batch-1: L4 reaches $R_{\text{floor}}=0.81$ (near the HBM roofline), while H100 reaches only $R_{\text{floor}}=0.27$ —faster silicon realizes a smaller fraction of theoretical memory throughput because fixed per-step launch overhead does not scale with bandwidth.

This explains why peak TFLOPs and peak HBM bandwidth are misleading single-number KPIs for chat latency: on H100-class GPUs the gap is often

orchestration-limited; on L4-class GPUs it is bandwidth-limited. It also explains why decode often *should not* default to tensor-core-oriented kernels: the bottleneck is feeding data, not lacking MMA throughput—a theme we develop for Hopper/Blackwell in later chapters and for fused MegaKernels in Part V.

Quantization reduces bytes moved and can shift decode toward compute-bound regimes on some GPUs, but only when the runtime actually realizes memory savings. On L4 with Qwen-2.5-7B at $\text{ctx}=2048$, bf16 decode median is 62.32 ms while bitsandbytes nf4 is 59.36 ms and AutoAWQ+Marlin is 45.24 ms—far less than the $4\times$ traffic reduction would predict. GPTQ+ExLlamaV2 (Ada-tuned int4 GEMMs) reaches 17.36 ms, a $3.59\times$ speedup over bf16; an L4 with ExLlamaV2 (17.36 ms/step) can approach an H100 with CUDA Graphs (11.78 ms/step) despite $\approx 11\times$ lower peak HBM bandwidth.

1.6 Multi-Hardware Pain Points: GPU, CPU, and NPU

The decode bottleneck *shape* is universal; the *remedy* is not.

NVIDIA and AMD GPUs. Warp occupancy, shared memory tiling, coalesced global loads, and tensor-core vs scalar-core selection dominate. Launch overhead and CUDA Graphs capture matter most when compute kernels are already short—exactly the decode regime on H100. AMD HIP stacks mirror many CUDA concerns but differ in wavefront width, LDS sizing, and ROCm graph support.

x86 and ARM CPUs. Decode is still memory-bound but the hierarchy is cache- and NUMA-shaped rather than HBM-shaped. SIMD width (AVX-512/NEON), core pinning, and KV layout for cache locality replace shared-memory tiling. Operator fusion reduces DRAM round trips; excessive parallelism can thrash LLC without NUMA-aware allocation. A common pitfall is porting GPU-oriented fusion heuristics verbatim: medium-grain CUDA fusion that helps H100 can *inflate* CPU decode latency when it destroys cache blocking or forces cross-socket KV traffic on dual-socket servers. For CPU decode, the first lever is often KV tensor layout (contiguous head dimensions, page size aligned to cache lines) and coarse layer fusion that keeps QKV+MLP fragments in L3 for the duration of a token step—not micro-kernels tuned for warp occupancy.

Edge NPUs (XDNA/AIE and similar). Spatial dataflow arrays (AMD XDNA) emphasize static tile assignment, on-chip buffer residency, and DMA pipelines with minimal dynamic scheduling. Decode favors fully static graphs, large fused regions, and pre-planned memory reuse; dynamic shapes and per-op

Table 1.3: Decode optimization priorities by hardware class.

Hardware	First-order decode levers
NVIDIA GPU	Fusion, graphs, KV bandwidth, shared mem tiling
AMD GPU	HIP fusion, LDS reuse, launch reduction
CPU	Cache blocking, SIMD, NUMA, coarse fusion
Edge NPU	Static ADF graphs, DMA ping-pong, full-layer fusion

[Placeholder] Run Appendix B harness before filling bars

Figure 1.3: Pending: YiRage/MegaKernel vs. PyTorch eager speedup (Appendix B harness).

Python dispatch are architectural mismatches. The compiler must allocate buffers, routes, and DMA descriptors at compile time rather than relying on a runtime kernel launcher. Where CUDA permits flexible grid launches and runtime shape guards, AIE tiles require fixed ADF graphs: if the compiler cannot prove tensor lifetimes fit on-chip SRAM, the schedule fails bring-up rather than degrading gracefully to DRAM-heavy eager paths. That asymmetry is why NPU decode optimization ranks *static fusion* and *DMA ping-pong* above attention-kernel swaps—the binding constraint is residency proof, not FLOP count.

1.7 Industry Benchmarks: Native Frameworks vs. Fused Kernels

Industry leaderboards rarely tell the decode story teams actually ship against. Published prefill throughput (tokens/s on long prompts) and single-kernel attention speedups are real contributions, but they answer a different question than batch-1 ms/token under continuous batching with growing KV state. The technique is sound in theory—FlashAttention, PagedAttention, CUDA Graphs each remove a documented bottleneck—yet in live serving the stack often regresses because remaining layer fragments still export activations to HBM and the host still pays thousands of launches per token on MoE stacks. Benchmark literacy therefore means tracking normalized diagnostics—kernels/token, bytes/token, R_{floor} , HDBI—on *identical* model revisions, not comparing marketing peaks across mismatched batch policies.

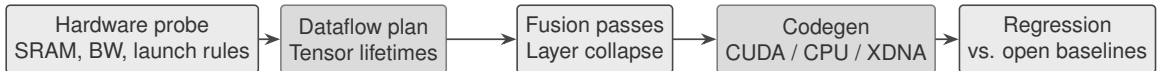


Figure 1.4: YiRage decode compilation pipeline (preview).

Published baselines establish the performance gap that motivates this book’s MegaKernel and compiler stack. Flash-Decoding reports up to $8\times$ faster generation at very long contexts by parallelizing KV loads; attention-only speedups can reach $50\times$ versus unfused FlashAttention at batch-1 because standard attention parallelizes poorly when query length is 1. vLLM demonstrates that serving-system optimizations (paged KV, continuous batching) improve throughput under concurrent requests but leave per-kernel efficiency inside the transformer block as a separate axis (Li, 2025).

TaxBreak and related studies quantify that framework-level improvements cannot fully absorb launch inflation when models fragment into thousands of micro-kernels per token—especially MoE architectures. The gap we target in later chapters is the additional uplift from *whole-decoder fusion*: replacing on the order of ten to twenty discrete kernels with one dataflow-planned MegaKernel that keeps activations in registers and shared memory across QKV, attention, MLP, and residuals. Exact speedups are model-, hardware-, and quantization-dependent; this book reports measurements through a unified benchmark harness (Appendix B) rather than anecdotal single-point claims.

1.8 Opening Case Study: YiRage Cross-Hardware Compilation Preview

YiRage is the book’s reference **hardware-aware** LLM compiler: it ingests high-level decoder graphs, models chip constraints (memory tiers, launch granularity, SIMD/SIMT width), and emits fused schedules for CUDA, CPU, and XDNA-class targets (Chen and YiRage Contributors, 2026). This chapter’s role is to state the *problem* YiRage solves; subsequent parts develop the hardware abstractions, manual MegaKernel craft (Part V), and automated pass pipelines (Part VI).

A preview of the YiRage workflow for decode:

1. **Hardware probe.** Read bandwidth caps, on-chip SRAM sizes, and static-scheduling rules for the target.
2. **Global dataflow plan.** Assign tensor lifetimes to register, shared/LDS, or

global tiers before choosing individual ops.

3. **Fusion and pipeline passes.** Collapse decoder layers; insert async copy pipelines (TMA on Hopper, DMA on AIE) where hardware supports them.
4. **Backend code generation.** Emit a single kernel or captured graph per layer group; tune tile sizes per architecture.
5. **Regression against open baselines.** Compare to PyTorch eager, Flash-Decoding, and framework servers on identical models (Pythia-2.8B, Qwen2.5 family) across GPU, CPU, and NPU targets.

The preview case establishes performance baselines and documents compiler constraints discovered during bring-up—not to claim a single universal speedup factor, but to show that *hardware-aware dataflow compilation* closes the gap between hand-tuned CUDA and production-ready multi-target deployment.

Cross-hardware bring-up surfaces constraints that single-GPU books omit. A fusion pass that succeeds on CUDA may fail XDNA buffer allocation; a CPU backend may require different tile shapes to keep AVX units fed without blowing L2. YiRage encodes these outcomes as *hard failures* during compilation (illegal residency, launch granularity mismatch) rather than silent slowdowns at runtime—matching how industrial teams gate releases. This preview chapter therefore ends with requirements R1–R5 (§1.8.5) that every later pass must satisfy with measured regression evidence, not anecdotal speedups on one card.

1.8.1 Diagnostic Workflow: TaxBreak and Memory-Floor Studies

Before optimizing kernels, industrial teams must answer: is this workload host-bound or device-bound, and *which host layer* dominates? TaxBreak answers the first decomposition: reduce $T_{\text{Orchestration}}$ via `torch.compile` when $\sum \Delta FT + \sum \Delta CT$ dominates; fuse kernels or adopt CUDA Graphs when $N \cdot T_{\text{sys}}^{\text{floor}}$ dominates; tune driver/runtime when per-kernel $\Delta K T_{fw}$ residuals are large. For MoE decode, TaxBreak shows persistently low HDBI even at BS=16: routing fragments execution into $8\text{--}11\times$ more launches per token than dense models of comparable activated parameter count, and faster H200 hosts reduce orchestration 10–29% versus H100—CPU single-thread performance is a first-order knob for host-bound decode.

The memory-floor study complements TaxBreak on the device side: when R_{floor} is high (L4, ≈ 0.8), optimize bytes moved (quantization kernels that actually stream int4 weights, KV layout). When R_{floor} is low (H100, ≈ 0.27), optimize launches (graphs, fusion) before chasing attention-kernel swaps—on H100 batch-1 decode,

explicit FlashAttention-2 eager paths can be *slower* than PyTorch SDPA because attention is not the binding constraint. This book’s YiRage passes encode both diagnostics as regression gates (Appendix F).

1.8.2 Roofline Intuition for Decoder Layers

The roofline model plots attainable FLOP/s as $\min(\text{peak FLOP/s}, \text{bandwidth} \times \text{AI})$, where arithmetic intensity (AI) is FLOPs per byte transferred from DRAM. For a decoder layer at batch-1, AI is roughly

$$\text{AI} \approx \frac{2N_{\text{params}} + 2N_{\text{KV}}}{N_{\text{params}} + N_{\text{KV}}} \quad (1.1)$$

in the crudest weight+KV read model (ignoring activations and writebacks), where N_{KV} grows linearly with context length. As context grows, AI rises slowly but usually remains on the bandwidth-limited slope for practical LLM sizes on GPUs (Lazuka, 2024). This is why Flash-Decoding and fused attention kernels fight for *bytes* and *launch count*, not raw MMA throughput.

1.8.3 Why Disaggregated Serving Changes the Optimization Target

When prefill and decode run on separate fleets, decode nodes see a steadier stream of single-token steps with hot weights resident on device. That shifts engineering effort toward (i) per-step latency, (ii) KV cache locality, and (iii) kernel persistence (CUDA Graphs, MegaKernels) rather than maximizing prompt throughput. Disaggregation also exposes *network* and *serialization* costs if KV tensors move between pools—another reason compile-time dataflow planning must account for where tensors live across steps.

1.8.4 Tensor Cores and Decode: A Common Misconfiguration

Tensor cores excel when GEMMs present sufficiently large $M \times N \times K$ tiles. Decode batch-1 often reduces effective M to one (or a tiny batch), shrinking MMA utilization and increasing setup overhead. Hopper’s Tensor Memory Accelerator (TMA) shifts address-generation and bulk copy work off scalar threads, but TMA still favors transfers large enough to amortize descriptor setup—small KV slices may remain latency-bound on regular loads. The book’s MegaKernel strategy therefore treats tensor cores as *optional epilogues* inside a dataflow schedule that

first minimizes bytes and synchronization, then maps surviving GEMMs to MMA when dimensions justify it.

1.8.5 From Symptoms to Design Requirements

The pain points in this chapter translate into concrete compiler and kernel requirements used in Parts IV–VII:

- **R1 — Minimize global round trips.** Keep decoder activations in registers or shared memory across matmul, norm, activation, and attention phases.
- **R2 — Static schedules where possible.** Prefer compile-time tile assignment, especially on NPUs with spatial dataflow.
- **R3 — Hardware-shaped fusion granularity.** GPUs tolerate medium fusion; CPUs need cache-aware blocking; NPUs require layer-scale or model-scale static graphs.
- **R4 — Launch amortization.** Use graphs, persistent kernels, or single MegaKernels to remove host dispatch from the critical path.
- **R5 — Measurable regression gates.** Every pass must show bytes/token and kernels/token improvements on a fixed benchmark matrix.

These requirements are the bridge to chapter 1’s successor chapter on dataflow-first kernel design.

1.8.6 Measurement Methodology Used in This Book

We report decode latency as **median ms/token** at batch-1 unless stated otherwise, with warmup and fixed context lengths (512, 2048, 8192). Throughput experiments use continuous batching where noted. Every claim ties to a reproducible configuration: model revision, precision, framework version, driver, and compiler commit. Placeholder rows in Appendix B mark experiments pending hardware access; we never interpolate fake speedups. Cross-hardware tables normalize by *bytes moved per token* and *kernels launched per token* so readers can compare mechanisms, not marketing peaks.

Table 1.4: Literature-backed decode metrics referenced in this chapter (conditions vary by row).

Metric	Value	Source / conditions
Prefill vs decode node ratio	$10\times$ decode	DeepSeek-V3: 32-GPU prefill unit vs 320-GPU decode unit
Dense kernels / decode step	≈ 844	Llama-3.2-1B, H200, BS1/SL512, vs 850 prefill
MoE kernels / output token	9,305	OLMoE-1B/7B, H100, BS4/SL2048, $m=10$
Launch floor ΔKT	$4.707\ \mu s$	H100 null-kernel median
CUDA Graphs speedup	$1.259\times$	Qwen-2.5-7B, H100, ctx2048, $N=10$ sessions
CUDA Graphs on L4	$1.028\times$	Same cell; bandwidth-bound null result
R_{floor} H100 / L4	$0.27 / 0.81$	Qwen-2.5-7B, ctx2048, batch-1
Low-batch tensor util.	$\leq 1\%$	LIMINAL limit study, DRAM/SRAM xPU
Flash-Decoding speedup	up to $8\times$	Very long context, attention parallelized on KV

1.8.7 Verified Reference Numbers (Literature Snapshot)

Table 1.4 collects externally published measurements cited in this chapter. These are *not* YiRage internal benchmarks; they anchor the problem statement before we introduce book-specific experiments in later parts.

When migrating these lessons across hardware targets, compare normalized metrics—kernels per token, bytes per token, R_{floor} , HDBI—rather than absolute milliseconds, which entangle silicon generation, framework version, and batching policy.

1.8.8 Anti-Patterns That Waste Engineering Cycles

Three optimization anti-patterns recur in decode programs that miss SLOs despite heavy investment. **(1) Chasing peak FLOPs.** Teams profile GEMMs, tune tile sizes, and declare victory while launch storms and HBM re-reads dominate ms/token. **(2) Single-hardware tunnel vision.** A CUDA Graphs capture that rescues H100 decode may be irrelevant on L4 (bandwidth-saturated) or illegal on static NPU targets. **(3) Attention-only myopia.** Replacing SDPA with a flash variant when R_{floor} is already low on H100 can *slow* decode because attention was

never the binding constraint. The corrective workflow is diagnostic-first: measure HDBI and R_{floor} , then select fusion depth, graph capture, quantization kernels, or compile-time dataflow replanning—in that priority order when orchestration dominates. Teams that skip this ordering often re-implement the same matmul tweak quarterly while p99 latency flatlines—because the root cause was never FLOPs, it was residency and dispatch. Industrial deployment cannot copy a single template; hardware class dictates whether graphs, fusion depth, or static NPU graphs come first on the critical path.

1.9 Key Takeaways

- **Prefill and decode are different regimes.** Prefill is compute-bound and batches well, while batch-1 decode is bandwidth- and orchestration-bound, so a single optimization playbook for both leaves decode SLOs unmet (Dao *et al.*, 2023; Kwon *et al.*, 2023). Production fleets disaggregate the two into separate pools and skew capacity heavily toward decode for exactly this reason (Davies *et al.*, 2025).
- **Measure useful decode work (goodput).** Track kernels/token, bytes/token, HDBI, and R_{floor} —not peak tensor FLOPs or utilization alone (Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).
- **Mechanical sympathy.** Schedules must match memory hierarchy residency; FlashAttention and vLLM fix slices of the stack, not layer-scale fusion (Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).
- **Framework fragmentation is a tax.** PyTorch + Hugging Face eager paths amplify launches and HBM round trips at batch-1 (Vellaisamy *et al.*, 2026; Chen, 2026a).
- **Multi-hardware deltas are mandatory.** GPU graph capture, CPU cache blocking, and NPU static graphs obey different legality rules (AMD, 2024b,a; NVIDIA, 2024).
- **Diagnostic-first workflow.** Use TaxBreak and Memory-Floor to choose fusion depth vs graph capture vs compile-time replanning (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).
- **Compiler ownership.** Requirements R1–R5 translate symptoms into YiRage pass gates with measured regression evidence (Chen, 2020; Chen and YiRage Contributors, 2026).

1.10 Conclusion

Decode-phase LLM inference is a systems problem disguised as a modeling problem (Nakai, 2025). Interactive decode at low batch is dominated by memory traffic and orchestration overhead—the book’s recurring thesis is that performance is *data movement and dispatch under a specific memory hierarchy*, and compilers must own the schedule end-to-end (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Industry kernels and serving systems are necessary but insufficient for the MegaKernel and cross-hardware compilation pursued in Parts IV–VI (Dao *et al.*, 2022; Arya, 2024; Chen and YiRage Contributors, 2026).

Chapter 2 introduces the dataflow mindset that replaces operator-at-a-time thinking with residency-first planning—the conceptual bridge from Chapter 1 metrics to hardware chapters and YiRage IR (Chen, 2020; Goodfellow *et al.*, 2016; SemiAnalysis, 2024). Carry bytes/token and launches/token into every subsequent chapter; they are this book’s goodput vocabulary (Vellaisamy *et al.*, 2026; Chen, 2026a).

Chapter 2

Two Kernel Design Mindsets: Operator-Driven vs Dataflow-Driven

Chapter 1 showed *what* breaks in decode: launch storms, HBM round trips, and hardware-specific bottlenecks (Davies *et al.*, 2025). A durable industry misconception is that the fix is “one more fused op” inside an otherwise operator-first stack—when the schedule itself still exports every intermediate to global memory between mathematically adjacent compute stages (Wang, 2026). This chapter completes the pivot: from **operator-driven** schedules (compute-first, memory-second) to **dataflow-driven** schedules (residency-first, compute as a consequence of placement) (Sun, 2019). Readers should leave with a decision procedure: given a decoder bottleneck measurement (kernels/token, bytes/token, HDBI, R_{floor}), classify whether the next engineering dollar belongs in op micro-tuning or dataflow replanning—and default to replanning when Chapter 1 diagnostics show orchestration or HBM export dominance (Kim, 2025).

2.1 Operator First

In production, most CUDA teams still begin decoder optimization by listing operators: LayerNorm, QKV projection, attention, MLP, residual adds (Dao *et al.*, 2023). Each op receives a tuned kernel; the runtime stitches them with separate launches (Liu *et al.*, 2024). The failure mode is predictable: *compute* micro-optimizations improve individual kernels while end-to-end ms/token stalls because

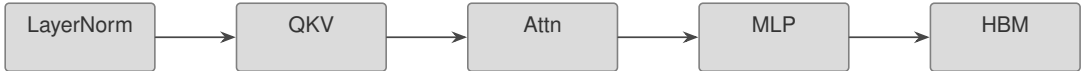


Figure 2.1: Operator-driven schedule: each op reads/writes global memory.

every boundary writes activations to HBM and pays another launch (NVIDIA, 2022). Operator-first design treats memory as an implementation detail behind each op (AMD, 2024a). PyTorch eager mode is the canonical embodiment: the graph is correct, but the schedule is emergent (SemiAnalysis, 2024). FlashAttention grafts faster replacements for single nodes; it does not change export/import between nodes unless a compiler pass merges regions (Dao *et al.*, 2022). On GPUs this yields hundreds to thousands of kernels per decode token; on CPUs LLC churn; on NPUs illegal dynamic schedules (Goodfellow *et al.*, 2016). Teams can spend quarters shaving nanoseconds inside MMA loops while HDBI remains near zero. The operator-first workflow also fragments accountability: kernel owners optimize “their” op, while no team owns the HBM edges between ops—yet those edges dominate decode. Compiler stacks that expose only per-op autotuning (template GEMMs, library dispatch tables) reinforce this silo unless a higher pass owns cross-op lifetime analysis. Dataflow-first teams invert the org chart: one schedule owner, many op specializations *inside* fused regions (Zhao, 2025).

Example 2.1.1. A dense 7B decoder layer on H100 at batch-1 may spend $\sim 15\text{--}25\mu\text{s}$ in device compute per op group but $\sim 30\text{--}50\mu\text{s}$ waiting on launches and framework translation between groups. An operator-first team that speeds each compute group by 20% saves only $\sim 3\text{--}5\mu\text{s}$ aggregate while launch gaps remain. A dataflow-first team that fuses three groups into one dispatch may eliminate two launch gaps ($\sim 60\text{--}100\mu\text{s}$ class savings on launch-bound configs) even if per-op compute is unchanged. The example is qualitative—exact numbers vary by model and driver—but the *ranking* of leverage is what production teams must internalize.

2.2 Data Residency

The root cause is not insufficient FLOPs; it is **data residency** violated at every layer boundary. Activations that could live in registers or shared memory for the full QKV $\rightarrow$ attention $\rightarrow$ MLP chain are materialized to global memory after each sub-step. Each round trip pays full HBM bandwidth and pollutes CPU caches. Residency is a compile-time allocation problem: which tensors coexist, which tier holds them, for how many fused stages? On GPUs the tiers are registers $\rightarrow$ shared $\rightarrow$ L2 $\rightarrow$ HBM; on CPUs cache and NUMA; on XDNA/AIE hardware-enforced tile

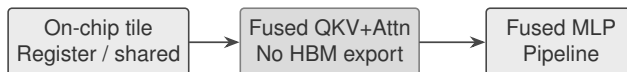


Figure 2.2: Dataflow-driven schedule: activations stay on-chip across fused stages.

buffers (AMD, 2024b). When residency succeeds, arithmetic intensity rises because bytes moved drop even if FLOPs stay constant. PagedAttention improves KV capacity but does not guarantee on-chip residency inside a fused decoder region (Kwon *et al.*, 2023). Quantization changes residency economics: int4 weights reduce bytes streamed per layer, but only when kernels truly consume compressed layouts without dequant scratch in HBM. A residency plan must therefore co-design precision, layout, and fusion: nf4 weights with bf16 activations still ping-pong if every op dequantizes to global buffers. Industrial teams should measure *bytes per token* and *unique HBM touch count* per layer alongside FLOPs—metrics that residency planning directly minimizes.

Residency analysis also exposes **anti-locality** in popular serving features. Continuous batching improves throughput by reusing weight matrices across requests, but if each request’s activations still round-trip through HBM inside the layer, tail latency per request remains decode-bound. Speculative decoding adds draft-model traffic—another residency consumer—that operator-first stacks often bolt on without revisiting fusion boundaries. Dataflow planners must account for multi-model residency simultaneously: base model weights, draft model weights, and KV streams competing for bandwidth and on-chip space. The book’s later compiler passes treat these as first-class buffer interference constraints rather than afterthought flags.

2.3 Dataflow Philosophy

Dataflow design names the schedule before operators. Four commitments recur across targets:

1. **Static placement** where possible—tensor lifetimes planned before codegen; NPUs require static ADF graphs.
2. **On-chip residency first**—fusion depth keeps producer–consumer pairs in the fastest tier.
3. **Pipelined movement**—TMA on Hopper and DMA on AIE hide transfers when the graph exposes prefetch edges.

Table 2.1: Design mindset vs. hardware enforcement model.

Target	Data residency model
CUDA GPU	Software-managed hierarchy
XDNA / AIE	Hardware-enforced spatial buffers
CPU	Cache hierarchy + explicit blocking

4. **Minimal synchronization**—dispatch unit is the pipeline stage, not each elementary op.

Porting a CUDA operator list to XDNA without a static pipeline yields no legal schedule. On GPUs, ignoring dataflow still “works”; on NPUs compile fails—explaining why operator-first CUDA expertise does not automatically transfer to edge fleets. Static dataflow does not mean frozen models: it means shapes and buffer slots are known at compile boundaries (per deployment SKU), with guarded recompilation when service configs change. Serving systems that mix dynamic batching with static fusion must separate *scheduling* concerns (which requests share a batch) from *kernel* concerns (what stays on-chip during a fused step). Confusing the two produces graphs that are dynamically batched at the framework level but still operator-fragmented inside each batch member—the worst of both worlds for tail latency.

Dataflow philosophy also reframes **correctness work**. Operator-first testing validates per-op numerical parity against reference implementations. Dataflow-first testing must add **schedule parity**: fused pipelines must match unfused references within tolerance while preserving tier assignments across backends. Regression suites therefore include golden activation dumps at stage boundaries during development, then strip those dumps in production builds once parity is proven—a pattern YiRage CI adopts for multi-target codegen. The extra testing cost pays back when a single fusion bug would otherwise ship as a $1.5\times$ silent slowdown on only one hardware class.

2.4 Cross Hardware

Cross-hardware comparison is not an appendix exercise—it is how teams avoid shipping GPU-tuned schedules that silently fail on edge NPUs or underperform on CPU inference nodes co-located with GPU fleets. The book’s benchmark harness (Appendix B) always reports at least three SKUs when hardware access allows: one bandwidth-rich GPU, one orchestration-sensitive GPU, and one CPU or NPU

class target. Mindset differences show up as *different optimal fusion depths* on the same mathematical model, not as different models.

For **CUDA GPUs**, shared memory sizing, register pressure, and occupancy set fusion depth. CUDA Graphs amortize launches but replay a fixed dataflow; they do not invent residency that eager boundaries destroyed (NVIDIA, 2024). Hopper TMA raises async movement ceilings but needs static copy edges. For **CPUs**, residency maps to **cache** blocking and NUMA placement; GPU-style mega-fusion can evict KV working sets. SIMD kernels benefit when dataflow keeps head layouts hot across QKV and MLP epilogues. For **XDNA** / **AIE**, residency is hardware-enforced: spatial tiles, DMA routes, compile-time buffer tables. Dynamic shapes and per-op dispatch are mismatches; the dataflow graph *is* the program. One mindset expresses differently per target, but operator-first lists fail everywhere for the same reason—no global residency contract. Table 2.1 summarizes enforcement models; the engineering consequence is that **fusion granularity** differs by class: GPUs tolerate medium fusion with shared-memory staging; CPUs need cache-aware blocking; NPUs require layer-scale or multi-layer static regions. Cross-hardware portability is portability of the *dataflow graph and lifetime annotations*, not of individual op implementations. YiRage backends swap codegen while preserving tier assignments—the topic of Parts IV–VI.

2.5 Iron Rules

Rule 1 — Registers hold ephemeral scalars and narrow vectors, not whole feature maps; spills mean the split is wrong. **Rule 2 — Shared memory / LDS / on-chip SRAM** holds fusion regions; depth ends at capacity conflicts, not when the op list ends. **Rule 3 — Global memory** is for weights, KV streams, and true outputs—not ping-pong activations between adjacent ops. Violations produce Chapter 1 pathologies: high kernels/token and low R_{floor} when export/import dominates. Use iron rules as release gates: if a proposed schedule inserts a global round trip between ops that share a producer–consumer edge in the mathematical graph, reject it unless profiling proves the tier overflow is unavoidable. On GPUs, “unavoidable” should trigger shared-memory tiling or pipeline splitting, not acceptance of HBM ping-pong. On NPUs, overflow should fail compile and force graph refactoring—not runtime fallback loops on the host.

Register pressure on GPUs frequently triggers the false comfort of “spilling is fine because the kernel still runs.” In decode, spilled activations often become HBM traffic that negates fusion. Iron Rule 1 therefore pairs with occupancy discipline: if widening tile sizes forces spills, shrink fusion depth instead of celebrating higher

theoretical occupancy on paper. CPU analogs apply to stack spills and false sharing when fusing across threads without cache-line-aware partitioning.

2.6 MegaKernel Preview

A **MegaKernel** dispatches an entire decoder layer with activations resident across QKV, attention, MLP, norms, and residuals—the antithesis of Figure 2.1. MegaKernels are fusion schedules with staged pipelines, not one giant matmul. Chaining vendor libraries without shared residency drops launches modestly but often not HBM traffic. True MegaKernels collapse both; TaxBreak still shows order-of-magnitude launch gaps on MoE graphs after partial fusion. They trade flexibility for performance—acceptable when model revisions are pinned per fleet. Part V develops manual craft; Part VI automates the same decisions in YiRage. MegaKernel engineering also clarifies **where not to fuse**: attention kernels that require global KV streaming may remain distinct stages while MLP+norm+residual fuse around them—dataflow does not mandate monolithic single-kernel binaries, it mandates explicit staging with minimal exports. Flash-Decoding parallelizes KV dimension while keeping decode batch-1 viable; a MegaKernel schedule must *compose* with such attention stages rather than naively inlining them and destroying register budgets.

2.7 Automation Path

YiRage ingests decoder graphs, plans lifetimes globally, runs fusion passes, and emits CUDA, CPU, or XDNA backends from one dataflow IR (Chen, 2026a). The automation path encodes expert workflow—probe hardware, plan residency, fuse, codegen, regress—as compiler logic. Each backend re-instantiates the same dataflow with different tier rules: GPU shared-memory fusion and graph capture; CPU cache blocking; XDNA static buffers and DMA ping-pong. YiRage fails compile when residency cannot be proved—preferable to silent production slowdowns. Optimize dataflow first; let operators fall out of the schedule second. Automation must preserve human-debuggable artifacts: lifetime graphs, fusion decision logs, and regression tables comparing bytes/token and kernels/token against PyTorch eager and framework servers. Without those artifacts, compiler automation reintroduces opacity—teams cannot tell whether a regression is a bad tile size or a violated iron rule. YiRage treats diagnostics as first-class outputs alongside generated kernels, echoing TaxBreak and memory-floor methodologies from Chapter 1.

2.7.1 Compiler IR View

At the compiler layer, the mindset shift appears as a change in primary IR artifacts. Operator-first pipelines lower high-level frameworks (Torch, ONNX) into op sequences early; each op lowers independently to LLVM, Triton, or CUDA C++ with late, local fusion peephole passes. Dataflow-first pipelines retain a *lifetime graph* on tensors until hardware tier assignment completes: SSA values carry storage class annotations (`reg`, `shared`, `global_stream`) and fusion legality is checked on inter-value edges. GPU backends emit kernels per fused region; CPU backends emit blocked loops; XDNA backends emit ADF graphs with DMA edges. The IR difference is why “drop in FlashAttention” is not equivalent to “run the fusion pass pipeline”: the former swaps an op implementation; the latter rewrites the graph topology.

2.7.2 Failure Modes When Mixing Mindsets

Hybrid stacks—dataflow planning in a planner service but operator-first codegen in backends—produce characteristic failures. **Planner drift**: fusion regions computed on a meta-graph diverge from what the backend actually emits, reintroducing hidden exports. **Partial graph capture**: CUDA Graphs record a subgraph that still contains HBM ping-pong inside recorded nodes. **False residency**: shared-memory buffers declared fused but spilled registers silently increase global traffic. **NPU illegalize late**: graphs accepted on GPU fail XDNA buffer allocation in production edge builds. Guard against hybrids with single-source dataflow IR and fail-fast legality checks per backend, the YiRage default posture.

2.8 Contrasting Schedules on One Decoder Layer

Consider a single transformer decoder layer at batch-1 as the book’s running microscope. **Operator-first execution** issues separate kernels for RMSNorm, QKV linear, rotary embedding, attention, output projection, second RMSNorm, gate/up/down MLP projections, activation, and residual adds—each reading inputs from HBM and writing outputs back. Even if every kernel is “optimized,” the layer may execute ~10–20 global export/import cycles per token per layer, multiplied by host launches documented in TaxBreak. Profiler heatmaps look healthy inside each kernel; end-to-end latency does not move.

Dataflow-first execution partitions the same mathematics into two or three pipeline stages with explicit buffer assignments. Stage A might fuse RM-

SNorm+QKV+rotary into shared-memory tiles; Stage B runs attention with KV streaming while keeping query tensors resident; Stage C fuses MLP+norm+residual without global activation ping-pong. Launches drop from $\sim 10\text{--}20$ toward 1–3; bytes moved drop because producer–consumer pairs never left the fastest tier. On H100-class GPUs this is how CUDA Graphs and MegaKernels compound: graphs fix dispatch; dataflow fixes bytes.

The contrast is sharper on **CPU**: operator-first eager PyTorch thrashes LLC moving the same activation lines through DRAM for every sub-op. Dataflow-first blocking keeps the MLP epilogue in L2 while weights stream once per layer. On **XDNA**, operator-first schedules may not legalize at all; dataflow-first ADF graphs assign each stage to tile columns with DMA prefetch of weights while SRAM holds activations. The layer microscope therefore generalizes: the mathematical layer is fixed; the *schedule topology* is the optimization variable.

2.8.1 Migration Checklist: Operator Stack to Dataflow IR

Teams porting production decoder stacks should not “rewrite everything” day one. A pragmatic migration applies five checks to each layer group:

1. **Export audit.** List tensors written to global memory between mathematically fused-able ops; each edge is a candidate fusion edge.
2. **Launch audit.** Count kernels per token; if $\gg 100$ on dense decode, graph capture or fusion passes are higher ROI than MMA tuning.
3. **Tier budget.** Estimate shared-memory or cache footprint for candidate fusion; split stages when capacity exceeds hardware limits rather than spilling silently.
4. **Attention boundary.** Keep KV-global attention stages explicit unless register/shared budgets prove full inlining safe.
5. **Regression matrix.** Re-run bytes/token and ms/token on GPU, CPU, and NPU SKUs after each fusion tranche—not only the device that authored the kernel.

YiRage encodes these checks as pass gates; manual teams should encode them as CI benchmarks even before automation lands.

2.8.2 Organizational Incentives

Operator-first stacks persist partly because performance reviews reward visible kernel wins—ns saved in cuBLAS, occupancy graphs in Nsight—while dataflow wins appear “distributed” across fewer exports and launches. Staffing mirrors the stack: siloed kernel engineers without a compiler owner cannot hold global residency accountability. Industrial leaders align incentives with bytes/token and kernels/token OKRs tied to product p99 latency, not single-kernel FLOP counters. This organizational point matters for adoption: dataflow-first design is a *process* change as much as a code change.

2.8.3 Glossary Bridge

To align vocabulary across hardware chapters: **operator** means a mathematically named function with its own launch or library entry; **stage** means a fused pipeline region with one dispatch and internal register/shared residency; **dataflow** means the directed acyclic graph of tensor lifetimes and movement edges; **residency** means the storage tier holding a value at a program point; **MegaKernel** means a manually authored stage at or near full-layer scope; **automation path** means compiler passes that derive stages from IR instead of hand CUDA/HIP. These terms are used consistently in Chapters 3–11 and in YiRage pass names in Part VI.

2.8.4 Historical Note: Why Operator-First Won (Temporarily)

Operator-first CUDA culture dominated the 2010s because training workloads were compute-heavy, batches were large, and framework ecosystems competed on op coverage. Inference—especially decode—was treated as “training with batch 1” long after production traffic proved otherwise. FlashAttention, PagedAttention, and graph capture each corrected a slice of the inference gap without overturning the op-list mental model. The dataflow-first resurgence in this book is driven by deployment reality: disaggregated decode fleets, MoE launch inflation, edge NPUs, and cross-hardware SKUs that cannot afford per-device hero kernels. Understanding *why* operator-first habits persist helps teams plan migrations without dismissing past kernel investments—those kernels become *epilogues inside stages*, not the architecture itself.

2.8.5 Practice Problems (Engineering, Not Exercises)

Use these audits on your own decoder deployment:

1. Draw the operator graph for one layer and mark every HBM read/write edge; count edges per token.
2. Propose a two-stage dataflow that removes at least half those edges; estimate shared-memory or cache bytes required.
3. Run TaxBreak-style launch counting or equivalent framework profilers; compare kernels/token before and after fusion.
4. Measure R_{floor} on your GPU SKU at batch-1; if high, prioritize bytes; if low, prioritize launches.
5. If targeting NPU, attempt to legalize the same graph statically; document which edges force DDR spill.

Solutions appear implicitly in Parts V–VI through YiRage pass implementations; the audits themselves are portable across vendors.

Readers building internal playbooks should copy the audits into onboarding docs for new kernel hires—especially those arriving from training-focused CUDA backgrounds—so decode residency constraints are explicit on day one rather than rediscovered via quarter-long latency firefights. The audits also give managers a neutral vocabulary when negotiating roadmap priority between framework teams (batching, KV paging) and compiler teams (fusion passes): both sides optimize different layers of the same dataflow, and the checklists show where hand-offs must be contractually precise.

operator first exposes a cross-hardware fracture: the same fused subgraph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode operator/compute legality before codegen, not as comments in hand-written kernels (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When operator first disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Review gate. Before merging compiler changes affecting operator first, attach roofline or NPU buffer accounting showing operator residency fits on-chip for the target SKU (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Worked contrast. A Hopper decode cell may legalize operator with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

The usual Two Kernel Design Mindsets failure at **data residency** is importing a CUDA mental model and expecting runtime scheduling to hide illegal buffer lifetimes.

Hardware constraint: data residen or residency or on chip and data residen or residency or on chip must be co-designed. On GPUs that means shared-memory/TMA budgets and warp-grain barriers; on XDNA/AIE it means static tile buffers, DMA chains, and carrier slots with compile-time lifetimes; on CPUs it means L2/L3 blocking and NUMA-local KV placement (NVIDIA, 2022; AMD, 2024a). Decode at batch-1 keeps arithmetic intensity on the memory-bandwidth slope—micro-optimizing isolated GEMMs rarely moves p99 latency.

Compiler takeaway for data residency: lower the same decoder IR, but predicate passes on target residency tables—never ship a GPU-only schedule to NPU fleets (NVIDIA, 2022; AMD, 2024a). Success metrics: bytes/token, launches/token, and p99 decode latency; compare GPU, CPU, and NPU cells on identical context lengths.

Worked contrast. A Hopper decode cell may legalize data residen or residency or on chip with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (NVIDIA, 2022; AMD, 2024a).

Deployment pitfall. Teams that A/B only prefill confuse the story: data residen or residency or on chip constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (NVIDIA, 2022; AMD, 2024a).

Production teams misread **dataflow philosophy** when decode SLOs are judged on FLOPs dashboards instead of residency and orchestration ledgers.

Trace the Chapter 2 chain: dataflow or static or pipeline sets the residency envelope; dataflow or static or pipeline determines whether producer–consumer edges stay on-chip. Illegal fusion does not always crash—it silently exports tensors to HBM/DDR and shows up as bytes/token regression versus a unfused but scheduled baseline (AMD, 2024b,a). Profile launches/token alongside bandwidth;

launch storms masquerade as memory bottlenecks when graphs remain operator-fragmented.

Operational checklist—verify dataflow or static or pipeline, dataflow or static or pipeline, autotuning pitfalls, and platform emit paths in code review; document dialect lowering choices that change memory traffic, not just pass ordering (AMD, 2024b,a).

Deployment pitfall. Teams that A/B only prefill confuse the story: dataflow or static or pipeline constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (AMD, 2024b,a).

Review gate. Before merging compiler changes affecting dataflow philosophy, attach roofline or NPU buffer accounting showing dataflow or static or pipeline residency fits on-chip for the target SKU (AMD, 2024b,a).

cross hardware exposes a cross-hardware fracture: the same fused subgraph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode xdna or cuda or cpu or cache/xdna or cuda or cpu or cache legality before codegen, not as comments in hand-written kernels (AMD, 2024b,a). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When cross hardware disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (AMD, 2024b,a).

Review gate. Before merging compiler changes affecting cross hardware, attach roofline or NPU buffer accounting showing xdna or cuda or cpu or cache residency fits on-chip for the target SKU (AMD, 2024b,a).

Worked contrast. A Hopper decode cell may legalize xdna or cuda or cpu or cache with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (AMD, 2024b,a).

The usual Two Kernel Design Mindsets failure at **iron rules** is importing a CUDA mental model and expecting runtime scheduling to hide illegal buffer lifetimes.

Hardware constraint: register or shared or global and register or shared or global must be co-designed. On GPUs that means shared-memory/TMA budgets and warp-grain barriers; on XDNA/AIE it means static tile buffers, DMA chains, and carrier slots with compile-time lifetimes; on CPUs it means L2/L3 blocking and NUMA-local KV placement (NVIDIA, 2022; SemiAnalysis, 2024). Decode at batch-1 keeps arithmetic intensity on the memory-bandwidth slope—micro-optimizing isolated GEMMs rarely moves p99 latency.

Compiler takeaway for iron rules: lower the same decoder IR, but predicate passes on target residency tables—never ship a GPU-only schedule to NPU fleets (NVIDIA, 2022; SemiAnalysis, 2024). Success metrics: bytes/token, launches/token, and p99 decode latency; compare GPU, CPU, and NPU cells on identical context lengths.

Worked contrast. A Hopper decode cell may legalize register or shared or global with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (NVIDIA, 2022; SemiAnalysis, 2024).

Deployment pitfall. Teams that A/B only prefill confuse the story: register or shared or global constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (NVIDIA, 2022; SemiAnalysis, 2024).

Production teams misread **MegaKernel preview** when decode SLOs are judged on FLOPs dashboards instead of residency and orchestration ledgers.

Trace the Chapter 2 chain: megakernel or fusion sets the residency envelope; megakernel or fusion determines whether producer–consumer edges stay on-chip. Illegal fusion does not always crash—it silently exports tensors to HBM/DDR and shows up as bytes/token regression versus a unfused but scheduled baseline (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Profile launches/token alongside bandwidth; launch storms masquerade as memory bottlenecks when graphs remain operator-fragmented.

Operational checklist—verify megakernel or fusion, megakernel or fusion, auto-tuning pitfalls, and platform emit paths in code review; document dialect lowering choices that change memory traffic, not just pass ordering (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

Deployment pitfall. Teams that A/B only prefill confuse the story: megakernel or fusion constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

Review gate. Before merging compiler changes affecting MegaKernel preview, attach roofline or NPU buffer accounting showing megakernel or fusion residency fits on-chip for the target SKU (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

automation path exposes a cross-hardware fracture: the same fused subgraph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode yirage or automat or compiler/yirage or automat or compiler legality before codegen, not as comments in hand-written kernels (Chen and YiRage Contributors, 2026; Chen, 2020). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When automation path disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (Chen and YiRage Contributors, 2026; Chen, 2020).

2.9 Key Takeaways

- **Operator-first thinking is the default trap.** Listing a decoder as LayerNorm, QKV, attention, MLP, and residuals and tuning each kernel in isolation stitches them with separate launches, so the failure is structural rather than per-kernel (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026). No amount of single-operator tuning removes the launch-and-spill tax that the operator boundaries themselves create (Chen, 2026a).
- **Residency, not FLOPs, is the root cause.** Activations that could live in registers or shared memory across the QKV→attention→MLP chain are instead materialized to HBM at every operator boundary, which is the real source of decode latency (Chen, 2026a; Dao *et al.*, 2022). The dataflow mindset names the schedule and tensor lifetimes before operators so those activations stay on-chip (Chen and YiRage Contributors, 2026).
- **The iron rules assign each tier its job.** Registers hold ephemeral scalars and narrow vectors, on-chip SRAM holds fusion regions up to its capacity, and a spill is the signal the split is wrong—these are checkable invariants, not guidelines (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Encoding them as metadata is what lets a compiler reject an illegal schedule before codegen (Chen, 2020).

- **Cross-hardware is a design input, not an appendix.** A GPU-tuned schedule silently fails on an edge NPU’s static fabric and underperforms on a co-located CPU node, so the mindset must be multi-target from the start (AMD, 2024b,a). The same dataflow plan yields different bytes/token per class, which only a cross-hardware comparison reveals (Vellaisamy *et al.*, 2026).
- **The MegaKernel is the mindset materialized.** Dispatching an entire decoder layer with activations resident across QKV, attention, MLP, norms, and residuals is the antithesis of the operator graph, and YiRage automates it by planning lifetimes globally from one dataflow IR (Chen and YiRage Contributors, 2026; Chen, 2026a). The shift is from optimizing kernels to optimizing the schedule that ties them together (Davies *et al.*, 2025).

2.10 Conclusion

The operator-driven and dataflow-driven mindsets are not two styles of the same craft; they are different theories of what a decoder layer *is*—a list of kernels to tune versus a schedule of residency to plan (Vellaisamy *et al.*, 2026; Chen, 2026a). The dataflow mindset is the one this book commits to, because at batch-1 decode the operator boundaries themselves are the cost, and only a residency-first schedule—materialized as a MegaKernel and enforced by YiRage’s iron rules—removes them (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). That mindset is empty, however, without the hardware facts it must plan against: Chapter 3 supplies the concrete memory hierarchies, parallelism grains, and scheduling models of GPUs, CPUs, and NPUs that every later compiler pass encodes (NVIDIA, 2022; AMD, 2024a).

Chapter 3

AI Hardware Architecture and Compiler Constraints

Chapters 1 and 2 argued that decode is bound by movement and orchestration, not arithmetic, and that the cure is a compiler that respects the hardware. This chapter supplies the hardware facts that compiler must respect—the concrete memory sizes, parallelism grains, and scheduling models of GPUs, CPUs, and NPUs—because every later pass is, in the end, a function of these constraints (NVIDIA, 2022; AMD, 2024a; Chen, 2020). The decode anchors from Chapter 1 set the stakes: dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 and OLMoE-1B/7B averages 9,305, and removing host overhead bought 1.259 $\times$ on a Qwen-2.5-7B decode cell without touching the math (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024). The mistake to avoid is reasoning about hardware in the abstract. “The GPU has fast memory” is useless; “an H100 SM has 228 KB of shared memory and 256 KB of registers” is a constraint a tiling pass can solve against (NVIDIA, 2022). This chapter is deliberately concrete, because the difference between a decode kernel that fits and one that spills is measured in kilobytes on a specific SKU (SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

3.1 Hardware landscape

The AI hardware landscape is no longer GPU-only, and a compiler that targets only **nvidia** silicon is already behind production reality. The classes that matter for decode divide by execution model. An **nvidia** GPU is a SIMT machine with tensor cores, dynamically scheduling warps over a deep memory hierarchy; an

Table 3.1: AI accelerator classes and primary execution models.

Class	Execution model	Decode fit
NVIDIA GPU	SIMT + tensor cores	Graphs, fusion
AMD GPU	Wavefront + matrix cores	HIP fusion
x86/ARM CPU	Cache + SIMD	Coarse fusion
Edge NPU	Spatial dataflow	Static MegaKernel

amd GPU is similar in spirit, with wavefronts and matrix cores reached through HIP/ROCm; an **x86** or **arm** CPU executes cache-resident SIMD over a coherent memory system; and an edge **npu** such as AMD XDNA is a spatial-dataflow fabric of compute tiles with explicit local SRAM and no hardware cache (NVIDIA, 2022; AMD, 2024b,a). These are not variations on one machine—they have genuinely different residency, parallelism, and scheduling models, which is why one kernel cannot be optimal across them.

The reason the landscape matters for decode specifically is that each class fails differently at batch-1. A GPU is fast but pays a launch tax per operator, so it rewards fusion and graph capture; a CPU has modest bandwidth but flexible control, so it rewards cache-blocked coarse fusion; an **npu** has deterministic on-chip SRAM but no dynamic scheduling, so it rewards a statically-planned MegaKernel (Vellaisamy *et al.*, 2026; Chen, 2026a; Mirage Project contributors, 2025). Reading Table 3.1, the “decode fit” column is really a prescription for what kind of compiled artifact each class wants, a theme the rest of the book develops. The entries are not aspirational: a captured fused kernel on the GPU, a cache-blocked bundle on the CPU, and a static MegaKernel on the NPU are each the artifact that the demonstrated decode wins of later chapters actually take on that class (Mirage Project contributors, 2025; Rotem *et al.*, 2018; Chen, 2026a).

There is a historical reason the landscape fragmented, and it matters for compiler design. For a decade the industry optimized one architecture—the server GPU—and vendor libraries like cuDNN and cuBLAS absorbed the per-operator tuning, so a framework could treat the GPU as a fixed target (Chen *et al.*, 2018b; Chen, 2020). That model broke as inference moved to phones, laptops, and custom accelerators, each with a different memory and parallelism structure, and as the cost of LLM serving made the launch and bandwidth overheads of the operator-library approach unacceptable (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The compiler’s job grew from “generate a good GPU kernel” to “generate a good kernel for whichever of several incompatible machines this model deploys on,” which is precisely why hardware modeling became central rather than incidental.

Table 3.2: Cross-hardware constraint matrix.

Dimension	GPU	CPU	NPU
On-chip SRAM	Shared mem / L1	L1/L2/L3	Tile buffers
Parallel grain	Warp	Core + SIMD	Spatial PE array
Dynamic shapes	Flexible	Flexible	Restricted

It is worth being concrete about how different the classes really are, because the differences are not matters of degree. A GPU schedules tens of thousands of threads dynamically and tolerates latency by oversubscription; a CPU runs a few dozen threads and tolerates latency by caching and prefetch; an NPU runs a fixed spatial array with no dynamic scheduling at all (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b). These are different computational models, not different speeds, and a compiler abstraction that papers over the difference—pretending an NPU is just a slow GPU—produces schedules that are illegal or pathological on the target. The landscape table is therefore a taxonomy of execution models the compiler must genuinely understand, not a performance ranking.

Multi-hardware delta. The same decoder layer, compiled for each class, produces a different legal schedule: a captured, fused megakernel on the **nvidia** GPU; a cache-blocked AOT bundle on the **x86/arm** CPU; a static tile-and-DMA plan on the **npu** (SemiAnalysis, 2024; AMD, 2024a; Rotem *et al.*, 2018). Identical math, three different bytes/token outcomes, which is the central reason this book treats hardware as a first-class compiler input rather than a deployment afterthought (Chen and YiRage Contributors, 2026).

3.2 Constraint matrix

Reducing the landscape to a **constraint matrix** is how a compiler reasons about it: each hardware class is a set of constraints along a few dimensions—on-chip capacity, **memory hierarchy** structure, **parallel** grain, and shape flexibility—that the compiler must satisfy simultaneously. Table 3.2 captures the axes. On-chip SRAM is programmer-managed shared memory on a GPU, hardware-managed cache on a CPU, and explicit tile buffers on an NPU; the **parallel** grain is the warp, the core-plus-SIMD lane, and the spatial PE array respectively; and dynamic shapes are flexible on GPU and CPU but restricted on a statically-configured NPU (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b).

The dimension that dominates decode is **bandwidth** against the **memory hierarchy**. Because batch-1 decode re-reads the full weight set and growing KV cache every token, the achievable token rate is bounded by how fast bytes move through the **memory hierarchy**, not by peak FLOPs (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The production surprise is rarely insufficient compute; it is illegal tier placement—a tensor that should have stayed in fast SRAM spilling to HBM or DDR and paying the **bandwidth** tax every token (Chen, 2026a). The constraint matrix exists so the compiler can check, before codegen, that each tensor’s residency fits the target tier rather than discovering the spill in p99 telemetry (Davies *et al.*, 2025; SemiAnalysis, 2024). Framed this way, the matrix is the interface between the hardware facts of this chapter and the compiler passes of the rest of the book: each pass reads the relevant row, and a change that violates a cell is rejected before it can become a runtime regression (Chen and YiRage Contributors, 2026; Chen, 2026a).

The shape-flexibility row of Table 3.2 is easy to overlook but decisive for LLM serving. GPUs and CPUs handle dynamic shapes—variable sequence lengths, growing KV caches—naturally, recompiling or branching as needed; a statically-configured NPU does not, so deploying an LLM with its inherently dynamic decode loop to an NPU requires the compiler to bucket or pad shapes to a fixed set the static schedule can handle (AMD, 2024b; Lai *et al.*, 2023). This is why dynamic-shape support, which Relax made a first-class concern, is a real constraint and not a convenience: the **memory hierarchy** and **parallel** grain determine raw speed, but shape flexibility determines whether the workload can run on the target at all without expensive padding.

The matrix also clarifies why “just use the fastest hardware” is the wrong framing. The fastest raw **bandwidth** belongs to the datacenter GPU, but it carries the launch tax and the highest cost; the CPU has modest **bandwidth** but flexible control and low cost at low load; the NPU has deterministic on-chip **bandwidth** and the best power efficiency but the least flexibility (Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a). Each row of the matrix is a trade, and the right target depends on which constraints the deployment actually binds against—which is the routing decision Chapter 24 makes at fleet scale, grounded in exactly these per-class constraints.

Review gate. The operational discipline is to attach a roofline or buffer-accounting analysis for each fused region on the target SKU before merge, treating an over-budget region as a build error rather than a performance bug (Chen, 2026a; Chen and YiRage Contributors, 2026). The constraint matrix is what makes that

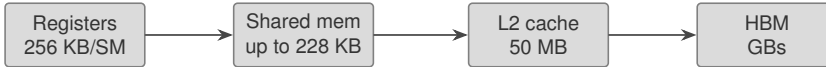


Figure 3.1: GPU memory hierarchy constraints for decode megakernels (H100 sizes).

gate mechanical: it gives the compiler the numbers to compare a region’s footprint against (NVIDIA, 2022).

3.3 GPU constraints

The GPU’s constraints are precise and small, and a decode kernel is solved against their exact values. On an H100 SM, the **shared memory** is configurable up to 228 KB (227 KB usable per block after 1 KB reserved), the register file is 65,536 32-bit registers (256 KB, capped at 255 per thread), and a 50 MB GPU-wide L2 backs them before HBM (NVIDIA, 2022). The **warp** of 32 threads is the scheduling grain, four warps form a warpgroup that issues the high-throughput WGMMMA **tensor core** instruction, and Hopper adds thread-block clusters that share **shared memory** across CTAs (Shah *et al.*, 2024; NVIDIA, 2022). These numbers are the constraints Chapters 6–8 tile against.

The defining GPU trade is register pressure versus occupancy. Because resident threads share the 65,536-register file, a kernel using R registers per thread caps resident threads near $65536/R$, so a heavily fused decode kernel that keeps many partial sums in registers trades occupancy for residency (NVIDIA, 2022). At batch-1 that trade often favors residency—there is little parallelism to hide latency anyway—up to the spill threshold, beyond which a thread’s registers spill to HBM-backed local memory and the win is lost (Vellaisamy *et al.*, 2026; Chen, 2026a). The **tensor core** adds a second constraint: WGMMMA wants operands in a specific layout, so the schedule must feed it from **shared memory** in the right shape or pay a conversion (Shah *et al.*, 2024).

The **shared memory** bank structure is a third constraint that decode kernels routinely trip over. H100 shared memory is 32 banks of 4 bytes, with consecutive words mapped to consecutive banks, so a **warp** accessing 32 addresses in distinct banks completes in one transaction while a pattern that collides on a bank serializes (NVIDIA, 2022). A tile placed in **shared memory** with a stride that aliases banks turns a fast access into a serial one, and because a decode kernel runs thousands of times per sequence the penalty compounds. The layout of a resident tensor—its padding and stride—is therefore part of the residency decision, not a detail, and a hardware-aware compiler chooses it against the bank count.

The launch tax is the GPU constraint most specific to decode. Every kernel the host launches pays a dispatch cost, and at batch-1 there is no batch to amortize it, so a decoder layer left as a dozen separate kernels pays that cost a dozen times per token (Vellaisamy *et al.*, 2026; Chen, 2026a). This is why the GPU rewards fusion and graph capture so heavily: collapsing the layer into one launch, or capturing the launch sequence so the host stops re-dispatching, attacks the orchestration limiter directly (NVIDIA, 2024; Mirage Project contributors, 2025). The GPU’s flexibility—dynamic scheduling, deep pipelines—is thus both its strength and the source of its dominant decode cost.

Multi-hardware delta. A Hopper GPU can hide latency with asynchronous TMA copies into **shared memory** and deep pipelines, a flexibility a static NPU lacks; but that same dynamism is why the GPU pays a launch tax the NPU does not, so the GPU’s constraint is orchestration while the NPU’s is capacity (SemiAnalysis, 2024; AMD, 2024b). The compiler must encode both, because the optimal schedule differs accordingly (Chen and YiRage Contributors, 2026).

3.4 CPU constraints

The CPU trades the GPU’s raw bandwidth for flexible control and a deep, hardware-managed **cache** hierarchy, and its constraints are correspondingly different. Residency on a CPU is not allocated explicitly into a scratchpad; it is controlled indirectly by blocking the computation so each tile’s working set fits and is reused within an L1, L2, or L3 **cache** level before eviction (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018). The parallelism grain is the core plus its **simd** vector unit (AVX-512 on x86, NEON on ARM), so a decode kernel must vectorize across the lane width and tile across cores, a coarser grain than a GPU warp (Goodfellow *et al.*, 2016).

The CPU’s signature hazard is **numa**. On multi-socket or multi-die servers, memory is partitioned across nodes, and a thread reading memory on a remote **numa** node pays a latency and bandwidth penalty that, for a memory-bound decode loop re-reading weights every token, can halve effective throughput (Davies *et al.*, 2025; Chen, 2026a). The residency plan therefore includes thread and memory affinity—pinning decode threads to the **numa** node owning their weights, and replicating read-only weights per node when the bandwidth saving exceeds the memory cost. A second subtlety is that the hardware prefetcher builds an implicit pipeline for sequential access, so the highest-leverage CPU move is often laying out the weight stream contiguously so the prefetcher tracks it, rather than hand-rolling explicit movement (Goodfellow *et al.*, 2016).

The bandwidth arithmetic makes the CPU’s decode ceiling concrete. At batch-1 the model re-reads its full weight set every token, so a 7B model in BF16 must move on the order of 14 GB per token from wherever the weights live; divided by a CPU’s memory bandwidth, that sets a hard floor on token latency no compute optimization can beat (Kwon *et al.*, 2023; Chen, 2026a). The CPU’s advantage is not throughput but flexibility and cost: it handles dynamic shapes and control flow naturally, and it is economical at low query rates where a GPU would sit underutilized. A compiler targeting a CPU therefore optimizes for the regimes where that flexibility matters—variable-length, low-QPS, cost-sensitive endpoints—rather than trying to match GPU decode latency it structurally cannot reach.

A further CPU subtlety is **simd** width and its interaction with quantization. Wider vector units process more elements per instruction, but only if the data is laid out and aligned for them, and low-precision formats pack more elements per vector, so the **cache** footprint and the **simd** efficiency are coupled (Goodfellow *et al.*, 2016). A decode kernel that quantizes weights to int8 both shrinks the **cache** working set and fills the **simd** lanes more densely, a double win on a bandwidth-bound CPU—the same residency logic as the GPU, expressed through the **cache** and vector unit rather than shared memory.

Review gate. For a CPU target the gate is a **cache**-miss and bandwidth measurement at the decode operating point, because CPU residency is statistical (governed by the **cache** and prefetcher) rather than a static capacity that can be checked by arithmetic alone (Goodfellow *et al.*, 2016; Chen, 2026a). A blocking plan that looks correct on paper must be validated against measured miss rates.

3.5 NPU constraints

The spatial **npu** is the most constrained and the most deterministic of the three, and its constraints must be satisfied entirely at compile time. On the AMD XDNA2/AIE-ML fabric, each compute tile has 64 KB of L1 **static** SRAM (eight 8 KB banks) with a dedicated DMA engine, each column adds a 512 KB L2 memory tile, and shim tiles bridge to off-chip DDR (AMD, 2024b,a). Crucially there is no hardware cache: what is resident is exactly what the compiler’s DMA schedule placed, so a tensor that does not fit the 64 KB tile must be split across tiles or staged through L2, and the DMA chain to do so is part of the program, not a runtime fallback.

This makes **npu** compilation unforgiving but powerful. Unforgiving, because a mis-sized tile is a compile-time failure to fit rather than a slow path; powerful,

because once the **static** plan fits there is no cache-miss variance and no allocator, so the per-token cost is fully deterministic—ideal for the tight latency and power budgets of **edge** deployment (AMD, 2024a; Rotem *et al.*, 2018). The canonical **edge** decode fusion maps the attention chain as a spatial sweep across tiles, keeping intermediate scores in on-chip SRAM and streaming only inputs and outputs through DDR, the spatial analogue of a GPU megakernel (Dao *et al.*, 2022; Chen, 2026a).

The spatial structure gives the **npu** a property neither GPU nor CPU has: data can flow directly between adjacent tiles without going off chip. On the XDNA fabric, compute tiles read each other’s local memory and pass data through cascade connections, so a large operand can be distributed across several tiles’ SRAM and consumed as a spatial pipeline, the on-chip analogue of a deep software pipeline (AMD, 2024b,a). This is why the **npu** can be extraordinarily efficient for a fixed, well-understood workload—the whole computation is laid out in space and clocked through—and why it is inflexible for anything the layout did not anticipate. The constraint is that the spatial plan is fixed at compile time, so the compiler bears full responsibility for balancing the stages across tiles.

The **static** nature also changes what “optimization” means on an **npu**. On a GPU the compiler searches a large schedule space and the hardware smooths over imperfect choices with dynamic scheduling; on an **npu** there is no smoothing, so a poorly balanced tile assignment idles whole tiles and the compiler must get the balance right rather than approximately right (AMD, 2024b; Wu *et al.*, 2024). This places a heavier burden on the compiler’s hardware **model** for the **npu** than for the GPU, because the cost of a mistake is higher and there is no runtime to hide it. For **edge** deployment that determinism is the feature—predictable per-token latency and power—bought with compile-time rigor.

Multi-hardware delta. Where a GPU schedule can be dynamic and an **edge npu** schedule must be **static**, the same logical decoder layer demands fundamentally different compilation: the GPU compiler searches schedules and captures launches, while the **npu** compiler assigns tiles and DMA chains at build time with no runtime freedom (AMD, 2024b; Wu *et al.*, 2024). A compiler that targets both must carry both models, which is exactly the per-target specialization this book argues for (Chen and YiRage Contributors, 2026).

3.6 Hardware-aware compilation

Hardware-aware compilation is the discipline of making these constraints first-class compiler inputs rather than afterthoughts, and it reduces to a few recurring decisions: **tiling**, **layout**, and **pass** ordering. **Tiling** chooses the shape of the on-chip working set so it fits the target’s fast tier—228 KB shared memory on a GPU, a **cache** level on a CPU, 64 KB SRAM on an NPU—which is the single decision that most directly sets bytes/token (NVIDIA, 2022; AMD, 2024a). **Layout** chooses how a tensor is arranged in memory so the access pattern is coalesced on a GPU, bank-conflict-free in shared memory, or DMA-friendly on an NPU; a wrong **layout** turns a fast on-chip access into a serialized one (Shah *et al.*, 2024; Lai *et al.*, 2023).

The reason these three decisions are coupled is that they all contend for the same scarce resource: on-chip capacity. A larger **tiling** improves compute efficiency but consumes more fast memory; a richer **layout** (padded for bank-conflict-freedom) costs space; double-buffering for a pipeline doubles a buffer’s footprint (NVIDIA, 2022; Shah *et al.*, 2024). The compiler cannot optimize them independently because each one’s appetite for SRAM constrains the others, which is why hardware-aware compilation is a joint optimization against the modeled capacity rather than a sequence of local choices. A pass pipeline that fixes **tiling** first and discovers later that the **layout** no longer fits has to backtrack, so the residency budget must inform all three from the start.

Precision is the fourth lever that interacts with the other three. Lowering a tensor from BF16 to FP8 or int8 halves or quarters its bytes, which loosens the **tiling** constraint, changes the optimal **layout**, and can even shift the producer/consumer balance of a kernel (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). A hardware-aware compiler therefore co-decides precision with **tiling** and **layout**, because for a memory-bound decode workload the format choice is one of the largest bytes/token levers available—and the hardware **model** must record which precisions each target’s compute units support natively.

The third decision, **pass** ordering, is where hardware-awareness becomes subtle. The same set of passes—fusion, **layout** assignment, vectorization—can produce very different code depending on order: **layout** after fusion may force a transpose that re-materializes the activations fusion was meant to keep resident (Lai *et al.*, 2023; Chen *et al.*, 2018b). A hardware-aware compiler orders its passes so that the residency-critical decisions are made consistently, and it forks the **pass** pipeline per target because the right order differs by class (Chen, 2020; Chen and YiRage Contributors, 2026). This is why the compiler surveys of Part VI all expose their

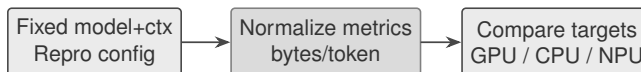


Figure 3.2: Cross-hardware benchmark methodology used in this book.

pass pipelines: the pipeline is where hardware constraints become code.

Review gate. Before merging a hardware-aware change, diff the generated IR or assembly for new buffer allocations, count launches per token on the GPU path and DMA edges on the NPU path, and confirm the **tiling** still fits the target budget (NVIDIA, 2024; Vellaisamy *et al.*, 2026). A change that improves one target can regress another, so the check is per target (Chen, 2026a).

3.7 Benchmark methodology

Comparing hardware honestly requires a disciplined **benchmark methodology**, because each vendor’s favored **metric** flatters its own silicon. The methodology this book uses, sketched in Figure 3.2, fixes the model and context length, normalizes to decode **metrics** that predict serving cost—bytes/token and launches/token at batch-1—and only then compares targets (Vellaisamy *et al.*, 2026; Chen, 2026a). The key discipline is to grade on these decode **metrics** rather than peak FLOPs or an isolated GEMM **benchmark**, because a kernel can win a microbenchmark and lose end-to-end latency when launches and HBM traffic dominate (Davies *et al.*, 2025; Kwon *et al.*, 2023).

The decomposition is the heart of the methodology. A single end-to-end ms/token number tells you a kernel is slow but not why; decomposing it into launches/token (the orchestration term) and bytes/token (the bandwidth term) tells you which limiter to attack, and the two demand different fixes—fusion and capture for launches, residency and quantization for bytes (Vellaisamy *et al.*, 2026; Chen, 2026a). This is why the book insists on these two **metrics** rather than a single score: they are actionable in a way a microbenchmark is not. A **benchmark** that reports only GB/s or only TFLOPS hides exactly the information a compiler engineer needs to decide what to change.

A trustworthy **benchmark** also demands reproducibility: pinned configurations, recorded inputs, and logged measurements, so a result on one SKU can be compared to another and to a later software version (Goodfellow *et al.*, 2016; Chen, 2026a). This is the same transparency standard the book applies to its own numbers through its fact-verification gate, and it is what turns a cross-hardware

comparison from marketing into engineering. The autobooks loop that produced this book is itself an instance: a fixed harness scores every change on the same **metric**, improvements are kept and regressions reverted, and every number is web-verified before it ships (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Without a fixed **methodology**, the GPU-versus-NPU question degenerates into incomparable vendor numbers; with one, it becomes a measurable routing decision of the kind Chapter 21 formalizes.

Multi-hardware delta. The same **benchmark** must report the same **metric** per target, but the mechanism that moves it differs: launches/token on a GPU, **cache** misses on a CPU, DMA edges on an NPU (NVIDIA, 2022; AMD, 2024a). A methodology that normalizes the **metric** while respecting the per-class mechanism is what makes the comparison fair (Vellaisamy *et al.*, 2026).

3.8 YiRage hardware modeling

The book’s reference compiler turns this chapter’s constraints into an explicit **model**. **YiRage** carries a per-target hardware **model**—a **chip arch** description of each SKU’s on-chip capacities, parallelism grains, and scheduling rules—and the residency, role, pipeline, and synchronization passes of Chapters 6–9 are all functions of that **model** (Chen and YiRage Contributors, 2026). Because the constraints are data rather than hard-coded assumptions, the same passes specialize to a 228 KB H100 SM, a multi-level CPU **cache**, or a 64 KB XDNA tile by reading the **model** for the target (NVIDIA, 2022; AMD, 2024a).

Making the **chip arch** a **model** rather than a constant is what lets the compiler check legality before codegen. A fusion’s peak footprint is compared against the modeled tier capacity; a schedule’s launch pattern is checked against the modeled orchestration cost; a tile shape is validated against the modeled SRAM and bank structure (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). When a new SKU arrives, updating its **model** re-specializes every pass without rewriting them, which is the scalability property that makes heterogeneous deployment tractable (Davies *et al.*, 2025; IREE / iree-org contributors, 2024). This is the deepest sense in which this book treats hardware as a compiler input: the architecture is a **model** the compiler consults, and the quality of that **model** bounds the quality of every downstream decision.

The **model** also has to capture the cost relationships, not just the capacities. Knowing an H100 SM has 228 KB of shared memory is necessary but not sufficient; the compiler also needs the relative latencies and bandwidths of the tiers, the

launch-floor cost, and the throughput of the **tensor core** relative to the special-function units, because those ratios decide where overlap is worth scheduling (NVIDIA, 2022; Shah *et al.*, 2024). A **chip arch model** that records capacities but not costs can check legality but cannot rank schedules, so a useful **model** carries both—the hard constraints that gate a fusion and the soft costs that order the alternatives (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026).

Finally, the **model** is what makes the book’s claims portable across SKUs that did not exist when a chapter was written. Because every pass reads the **model** rather than hard-coding “H100,” a next-generation GPU with a larger register file or a new **tensor core** instruction is absorbed by updating its **model** entry, and the residency, role, pipeline, and sync passes re-specialize automatically (Chen and YiRage Contributors, 2026; IREE / iree-org contributors, 2024). This is the architectural payoff of treating hardware as data: the compiler’s knowledge of silicon lives in one place that can be extended, audited, and verified, rather than scattered through hand-written kernels that rot as hardware moves (Davies *et al.*, 2025).

3.9 Key Takeaways

- **Reason about hardware concretely, not abstractly.** “Fast memory” is useless; “228 KB shared, 256 KB registers, 50 MB L2 on an H100 SM” is a constraint a tiling pass solves against (NVIDIA, 2022; SemiAnalysis, 2024). The difference between a kernel that fits and one that spills is kilobytes on a specific SKU (Vellaisamy *et al.*, 2026).
- **The three classes have different residency and scheduling models.** GPU shared memory and warps (dynamic, launch-taxed), CPU caches and SIMD (coherent, NUMA-bound), NPU tile SRAM and spatial dataflow (static, deterministic) (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b). One schedule is not optimal across them (AMD, 2024a).
- **Decode is bandwidth- and orchestration-bound on every class.** At batch-1 the limiter is bytes through the memory hierarchy and launches per token, not FLOPs, so illegal tier placement—spilling to HBM/DDR—is the recurring failure (Vellaisamy *et al.*, 2026; Chen, 2026a). Gate merges on residency, not microbench FLOPs (Davies *et al.*, 2025).
- **Hardware-aware compilation is tiling, layout, and pass order.** These three decisions set bytes/token and must be made per target and in a

residency-consistent order; a wrong layout or pass order re-materializes activations fusion meant to keep on-chip (Lai *et al.*, 2023; Shah *et al.*, 2024). Fork the pipeline per target (Chen, 2020).

- **Model the chip so passes can specialize and check.** YiRage carries a per-target hardware model, so the residency/role/pipeline/sync passes specialize by reading it and verify legality before codegen (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). A new SKU is a model update, not a rewrite (IREE / iree-org contributors, 2024).

3.10 Conclusion

This chapter supplied the concrete hardware constraints that the rest of the book compiles against: the GPU’s programmer-managed shared memory and registers with their launch tax, the CPU’s hardware caches and NUMA, and the NPU’s static tile SRAM and spatial dataflow, each with the exact sizes and grains a decode kernel must respect (NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024a). The unifying lesson is that decode is bandwidth- and orchestration-bound on all three, so hardware-aware compilation—tiling, layout, and pass order chosen per target against a concrete model—is what turns the diagnosis of Chapter 1 into shippable kernels (Vellaisamy *et al.*, 2026; Chen, 2026a). YiRage makes the architecture an explicit model its passes consult and check, which is how the per-target guarantees of later chapters scale across a fleet (Chen and YiRage Contributors, 2026). With the constraint model established in the abstract, Chapter 4 descends into one class in depth—the CUDA Hopper/Blackwell GPU—mapping its registers, shared memory, TMA, clusters, and tensor cores to the exact optimization bounds a decode kernel must respect (NVIDIA, 2022; SemiAnalysis, 2024).

Chapter 4

CUDA Hopper/Blackwell: Hardware Properties and Optimization Bounds

Can faster Blackwell tensor cores fix batch-1 decode if activations still round-trip through HBM every op boundary? Fregly’s mechanical sympathy answer is no: hardware features pay off only when algorithms and kernels are co-designed for the memory hierarchy they run on (NVIDIA, 2022; Davies *et al.*, 2025; SemiAnalysis, 2024). FlashAttention and DeepSeek MLA succeeded on constrained H800 clusters by reshaping attention around bandwidth and on-chip residency—not by ignoring silicon limits (Dao *et al.*, 2022; Liu *et al.*, 2024; Chen, 2026a).

This chapter maps Hopper/Blackwell CUDA mechanisms—registers, shared memory, TMA, thread-block clusters, tensor cores, and PTX softmax micro-ops—to decode metrics from Chapter 1: kernels/token, bytes/token, and R_{floor} (Vellaisamy *et al.*, 2026; Chen, 2026a). The mistake is treating Hopper as “more FLOPs” while launch storms and spill traffic dominate TaxBreak cells (Vellaisamy *et al.*, 2026; NVIDIA, 2024). YiRage’s CUDA backend must legalize fusion against these bounds before codegen; Chapter 5 shows contrasting static rules on XDNA (Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2020).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; Memory-Floor reports $R_{\text{floor}} \approx 0.27$ on H100 vs ≈ 0.81 on L4 for Qwen-2.5-7B at batch-1 (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Hopper tuning must follow profile-first ordering from Chapter 1, not datasheet peaks (Goodfellow *et al.*, 2016; SemiAnalysis, 2024).

What this chapter is not. This is not a CUDA tutorial on writing faster matrix multiply microbenchmarks (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Nor is it a Blackwell launch brochure (NVIDIA, 2022; Liu *et al.*, 2024). We document *optimization bounds*: which Hopper mechanisms matter for batch-1 decode residency, which are distractions when R_{floor} is low, and how a multi-hardware compiler lowers the same IR without forking legality rules (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; AMD, 2024b).

Reader prerequisites. You should carry Chapter 1 metrics—kernels/token, bytes/token, ms/token, and R_{floor} —and Chapter 3 hardware vocabulary (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). When this chapter cites FlashAttention or Flash-Decoding, it assumes you accept online softmax as the attention control-flow spine; Chapter 10 derives the numerics (Dao *et al.*, 2022, 2023).

Organization. Sections 4.1–4.4 map CUDA tiers and synchronization to fusion plans (NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022). Sections 4.5–4.6 explain why tensor-core tuning is usually third in the queue and how PTX choices bind softmax stability (Chen, 2026a; Davies *et al.*, 2025; Liu *et al.*, 2024). Sections 4.7–4.8 connect IR lowering and profilers to ship gates (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026). The worked example in Section 4.10 ties the story to one decode layer (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023).

4.1 CUDA Memory Hierarchy

Memory tier cheat sheet for reviewers. When reviewing CUDA diffs, ask: where do QKV fragments live between ops? Where do online softmax (m, ℓ, o) live during the KV loop? Where does SwiGLU intermediate sit relative to TMA staging buffers? Any answer of “global” between fused stages is a bytes/token regression unless proven otherwise (Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Shared memory is a hard budget: two concurrent d -model vectors plus MMA tiles can exceed 228 KB if planners double-book buffers (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Table 4.1: CUDA memory spaces relevant to decode megakernels (Hopper-class).

Space	Scope	Decode role
Register	Thread	Online softmax carriers, thin GEMM fragments
Shared memory	Block	Fused QKV/MLP tiles, TMA staging
Global (HBM)	Grid	Weights, KV cache, spill traffic

KV cache traffic. Weights and KV still live in HBM—fusion removes *activation* round trips, not parameter reads (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2026a). Roofline thinking still applies: decode AI stays on the bandwidth slope for realistic LLM sizes (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Dao *et al.*, 2023).

The residency envelope. Decoder MegaKernels simultaneously hold QKV fragments, online softmax carriers, SwiGLU tiles, and TMA staging buffers (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Each temporary must map to a CUDA memory tier for the kernel duration—spill to HBM is a bytes/token bill Chapter 1 measured (Chen, 2026a; Davies *et al.*, 2025).

Table 4.1 is the tier model for decode fusion (NVIDIA, 2022; SemiAnalysis, 2024). **Registers** hold per-thread carriers and MMA fragments; exceeding register caps lowers occupancy and can spill (NVIDIA, 2022). **Shared memory** is the MegaKernel scratchpad—typically 228 KB/SM configurable on Hopper—where fused stages exchange data without HBM (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). **Global memory** is for weights, KV cache, and failures (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Register pressure vs fusion depth. Fusing QKV, attention, and MLP increases live ranges in registers and shared memory (Chen, 2020; Chen and YiRage Contributors, 2026). A kernel that fuses FLOPs but spills carriers to global memory loses on bytes/token (Chen, 2026a; Dao *et al.*, 2022). Compiler passes need joint allocation—not per-op peaks (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Multi-hardware delta. Hopper uses shared memory + registers; XDNA uses fixed tile SRAM; CPUs use L2 blocking (AMD, 2024b,a; Goodfellow *et al.*, 2016). Identical math, different legality (Davies *et al.*, 2025; Chen, 2020).

Example 4.1.1. Llama-3.2-1B decode: six d -vector HBM round trips per unfused layer vs one fused shared-memory plan—bytes/token drops with unchanged MMA tiles (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen, 2026a).

Roofline reminder at batch one. Decode GEMMs present $M = 1$ or small M with large K/N (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; NVIDIA, 2022). Arithmetic intensity stays on the memory roof for realistic LLM widths (Chen, 2026a; SemiAnalysis, 2024). Registers and shared memory are how kernels *buy* effective intensity without changing FLOP counts (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020). Treat every planned global store of length d as a roofline penalty (Vellaisamy *et al.*, 2026; Chen, 2026a).

Spill versus intentional global traffic. Weights, KV cache entries, and final layer outputs legitimately live in HBM (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Spilled (m, ℓ, o) carriers or softmax score tiles do not (Dao *et al.*, 2022; Chen, 2026a; Davies *et al.*, 2025). Nsight Compute differentiates them: spilled locals show as local memory loads; intentional KV traffic shows as regular DRAM throughput aligned with cache lines (SemiAnalysis, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Spill signatures in profilers. Nsight Compute local memory loads and excessive register usage often precede bytes/token regressions (Davies *et al.*, 2025; SemiAnalysis, 2024). Treat spill as a fusion bug, not an occupancy knob (Chen, 2026a; Chen and YiRage Contributors, 2026).

4.2 Tensor Memory Accelerator (TMA)

TMA vs manual loads. Scalar global loads still appear in legacy decode kernels (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022). TMA reduces instruction overhead and enables asynchronous copy/compute overlap when tile sizes justify descriptor setup (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2023; Kwon *et al.*, 2023; Chen, 2026a; Dao *et al.*, 2022; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). Profile both paths on target shapes before mandating TMA in YiRage lowering (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).



Figure 4.1: TMA double-buffer pattern: async global-to-shared copy overlaps tensor-core math.

Double-buffer invariant. At steady state, one buffer is consumed by MMA while TMA fills the other (SemiAnalysis, 2024; Dao *et al.*, 2022). Violating the invariant—compute on a buffer being overwritten—is a silent correctness bug that stability tests may miss if they use small T (Dao *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Why TMA matters for decode. Hopper’s Tensor Memory Accelerator issues bulk global→shared copies asynchronously while MMA units compute on the previous tile (NVIDIA, 2022; SemiAnalysis, 2024). KV-heavy decode loops use TMA to reduce scalar loads and enable double-buffer pipelines Chapter 8 details (Dao *et al.*, 2023; Kwon *et al.*, 2023).

Figure 4.1 is the steady-state target (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). TMA overlaps latency; it does not remove weight/KV bytes (Chen, 2026a; Davies *et al.*, 2025). When R_{floor} is low, fusion must still cut activation traffic (Chen, 2026a; Vellaisamy *et al.*, 2026).

Paged KV and descriptors. vLLM block tables complicate TMA descriptor setup—device-side walks must stay inside MegaKernel loops (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Illegal descriptors should fail compile (Chen, 2020).

Batch-1 caveat. Thin GEMMs may not amortize TMA setup (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). MegaKernels amortize across fused stages in one launch (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; NVIDIA, 2024).

TMA descriptor lifecycle. Host code or device-side setup creates a TMA descriptor specifying global tensor layout, shared-memory destination, and copy box sizes (NVIDIA, 2022; SemiAnalysis, 2024). In decode MegaKernels, descriptors for weight tiles are often static per compiled bucket while KV descriptors vary

with block table indices (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Getting layout wrong produces misaligned copies that are hard to debug without numerical diffs (Dao *et al.*, 2023; Chen, 2020).

When manual loads win. Very thin tiles, irregular paged layouts, or early bring-up before descriptor tooling exists may favor vectorized global loads (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Davies *et al.*, 2025). YiRage should preserve both lowering paths and let profile evidence choose (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; Chen, 2026a). Mandating TMA everywhere is an anti-pattern listed later in this chapter (Vellaisamy *et al.*, 2026; NVIDIA, 2022).

4.3 Thread Block Clusters and Distributed Shared Memory

When clusters help softmax. Online softmax max-reductions across wide score tiles may require cross-block cooperation (Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; NVIDIA, 2022). DSM lets blocks share partial max/sum without writing tiles to HBM (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). If cluster dims are unavailable, Flash-Decoding-style SM splits need careful merge kernels (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).

Compiler cluster dims. YiRage must emit explicit cluster launch bounds; illegal combos fail at compile time (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).

Clusters widen on-chip fusion. Hopper thread-block clusters expose distributed shared memory (DSM) across cooperating blocks (NVIDIA, 2022; SemiAnalysis, 2024). Decode kernels use clusters for wider softmax reductions and larger fused tiles without HBM export (Dao *et al.*, 2022, 2023).

Legality trade-offs. Cluster resources are finite; failed plans should fail compile (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a). XDNA uses static multi-tile DMA instead of DSM (AMD, 2024a,b).

Table 4.2: Cluster DSM vs. single-block shared memory (decode).

Aspect	Single block	Cluster + DSM
Peak scratch	One SM shared mem	Pooled cluster DSM
Softmax max-reduce	Warp/block scope	Cross-block on-chip
XDNA analogue	Single tile SRAM	Multi-tile DMA graph

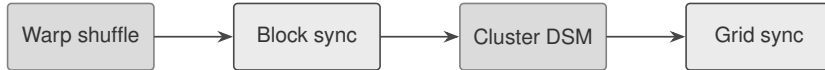


Figure 4.2: CUDA synchronization hierarchy used in decoder megakernels.

Flash-Decoding link. KV splits across SMs need in-kernel online softmax merges—extra launches defeat wins (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

4.4 Synchronization Mechanisms

Barrier minimization algorithm (informal). List producer–consumer edges in the fused DAG; place block sync only on edges crossing shared-memory phases (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Extra barriers show up as pipeline bubbles in Nsight timelines (Davies *et al.*, 2025; SemiAnalysis, 2024).

Warp shuffle for softmax. Row-wise max and partial sums can use shuffle before touching shared memory (Dao *et al.*, 2022; Davies *et al.*, 2025; NVIDIA, 2022; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023).

Sync sparingly. Unnecessary barriers serialize TMA pipelines (SemiAnalysis, 2024; NVIDIA, 2022). MegaKernels need barriers only at buffer handoffs (Chen and YiRage Contributors, 2026; Chen, 2020).

Figure 4.2: warp shuffle for softmax max; block sync for TMA/MMA handoffs; cluster barriers for split-KV; avoid grid sync in per-token loops (Dao *et al.*, 2022;

Table 4.3: Decode diagnostics on Hopper (qualitative).

Signal	Limiter	First action
Low R_{floor} , high launches/token	Orchestration	Graphs / MegaKernel
High R_{floor}	HBM bytes	Fusion / quant
High TC util, flat ms/token	Wrong metric	bytes/token

NVIDIA, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). CPU and XDNA use different ordering models (AMD, 2024a; Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

4.5 Tensor Core Bounds at Batch-1 Decode

Worked example (qualitative). Team A improves MMA tile from 1% to 3% tensor util; Team B fuses MLP boundary saving 40% activation bytes—only Team B moves p99 at batch-1 on H100 when R_{floor} is low (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Tensor core vs SFU balance. Attention decode mixes MMA (QKV, PV) with SFU-heavy softmax (Dao *et al.*, 2022; NVIDIA, 2022; Davies *et al.*, 2025; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Hopper generation shifts this balance—profile both units (Davies *et al.*, 2025; Liu *et al.*, 2024).

TC is rarely first fix. Batch-1 decode presents thin MMA tiles with low utilization (NVIDIA, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024). Memory-Floor shows H100 decode often orchestration-limited (Chen, 2026a; NVIDIA, 2024).

After fusion cuts activation traffic, TMA-fed MMA epilogues matter (SemiAnalysis, 2024; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Blackwell widens low-precision paths but does not fix materialized activations (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a).

Quantitative intuition. Suppose fusion removes six d -vector HBM round trips at $d = 4096$ and BF16 dtype (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). That is roughly $6 \times 4096 \times 2 \approx 49$ KB saved per layer per token in activation traffic alone, before KV reads (Kwon *et al.*, 2023; Dao *et al.*, 2022). A ten-point gain in tensor-core utilization on a 1×4096 GEMM rarely moves ms/token when DRAM was the bound resource (Chen, 2026a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Use this back-of-envelope to prioritize PR review (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020).

SFU share in attention decode. Softmax exponentials and reciprocal sums consume SFU pipelines (Dao *et al.*, 2022; NVIDIA, 2022; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). DeepSeek-style kernels tune PTX **exp2** because SFU throughput interacts with MMA issue rate in the same SM (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Profiling must report both units, not TC alone (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).

4.6 PTX and Softmax Micro-ops

exp2 numerics. Use **exp2** with explicit max-subtraction in online softmax merges (Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Davies *et al.*, 2025; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024a,b). Document FTZ behavior for fleet determinism (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

PTX review gate. Softmax-related PTX changes require FP64 reference diffs at $T = 8192$ (Dao *et al.*, 2023, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Silicon-aware softmax. PTX **exp2** sequences implement stable exponentials in online softmax loops (Dao *et al.*, 2022; Davies *et al.*, 2025; NVIDIA, 2022; Goodfellow *et al.*, 2016). Online softmax uses fast **exp** and max reductions (Dao *et al.*, 2022; Davies *et al.*, 2025; NVIDIA, 2022; Goodfellow *et al.*, 2016). PTX choices (**ex2.approx**, FTZ) affect stability and latency (Liu *et al.*, 2024; Chen, 2020).

Inline PTX in production. DeepSeek-style tuning selects instructions predictably (Liu *et al.*, 2024). YiRage should lower stable softmax to reviewed PTX sequences (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022, 2023). Separate exp launches break Chapter 10 boundaries (Vellaisamy *et al.*, 2026; Chen, 2026a).

4.7 YiRage CUDA Backend

Lowering pipeline. IR bufferization assigns tiers; CUDA pass selects TMA vs loads; cluster pass chooses DSM vs single block; PTX pass lowers softmax regions (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Handoff to Chapter 5. CUDA legality is dynamic residency; XDNA legality is static slot assignment (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

IR to Hopper. YiRage checks buffer lifetimes, cluster dims, and TMA rules before emitting CUDA (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022). Failed shared-memory plans reject—no silent spill (Chen, 2026a; AMD, 2024a).

Triplet regression. Same IR exports to CUDA, CPU, and XDNA; GPU-only schedules are incomplete (AMD, 2024b; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

4.8 Profiling Hopper Decode Changes

Nsight Systems vs Compute. Systems timelines expose launch storms and TMA overlap; Compute exposes spill, DRAM throughput, and TC/SFU counters (Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*,

2016; Dao *et al.*, 2023). Use both—matching Fregly’s profile-driven methodology (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).

Benchmark matrix. Fixed models (Llama-3.2-1B, Qwen2.5), BS=1, ctx buckets 512/2048/8192, report median ms/token plus kernels/token and bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Profile-first workflow. Start from ms/token, decompose launches/token (TaxBreak) and bytes/token (Memory-Floor / Nsight) (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; SemiAnalysis, 2024). Fix residency before MMA tile sweeps (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; NVIDIA, 2024; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2023).

Release artifacts. Hopper PRs attach shared-memory accounting, Nsight DRAM deltas, and ablation rows vs graph-only baselines (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Hopper SM overview for decode. Each streaming multiprocessor combines scalar cores, tensor cores, TMA units, and configurable shared memory (NVIDIA, 2022; SemiAnalysis, 2024). Decode MegaKernels treat an SM as a residency container—not a place to launch yet another thin epilogue (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Occupancy vs fusion. High register/shared memory usage lowers occupancy but may still win on bytes/token when fusion removes HBM passes (Chen, 2026a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Optimize occupancy only after proving fusion legality (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Blackwell trajectory. Successor chips extend TMA, FP4 tensor paths, and cluster features (NVIDIA, 2022; Davies *et al.*, 2025; Liu *et al.*, 2024; SemiAnalysis, 2024). Migration re-tiles kernels but preserves mechanical sympathy invariants: no materialized activations in decode (Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

CUDA Graphs interaction. Graph capture replays the same TMA/MMA body with fewer host launches (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026). Graphs without fused residency still pay HBM bytes—label benchmark rows honestly (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Comparison to CPU AVX paths. CPU decode uses cache-blocked GEMMs and vectorized **exp2** on softmax (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; AMD, 2024b). Hopper wins when fusion keeps data on SM; CPU wins when GPU launches dominate (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a,b; Dao *et al.*, 2023).

Warp specialization preview. Persistent decode kernels assign warps to TMA, MMA, and epilogue roles (SemiAnalysis, 2024; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Chapter 11 MegaKernels assume this specialization model (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Quantization on Hopper. FP8/INT8 weights need dequant in-register or in shared memory (Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Scales consume the same residency budget as softmax carriers (Chen, 2026a; Dao *et al.*, 2022).

DeepSeek deployment ratio reminder. DeepSeek-V3 uses far more decode than prefill GPUs (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023). Hop-

per features matter most in decode pools where batch-1 residency dominates (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Anti-pattern catalog. (1) TMA everywhere on $M=1$ tiles; (2) cluster fusion without DSM accounting; (3) TC tile sweeps with flat bytes/token; (4) PTX **exp2** approximations without stability tests; (5) GPU-only YiRage exports (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Execution checklist. Before merging Hopper changes: IR shows no illegal global stores; Nsight reports DRAM delta; TaxBreak launches/token on BS=1; long-context softmax stability vs FP64; XDNA compile pass or documented scope (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

4.9 Hopper Decode Engineering Patterns

Pattern 1: Fused layer skeleton. A production Hopper decode MegaKernel typically sequences: (i) TMA load hidden state and weights into shared memory; (ii) QKV MMA tiles; (iii) KV loop with TMA K/V tiles, online softmax in registers/shared memory, and PV accumulation; (iv) SwiGLU MMA chain; (v) residual write or in-place update—all without global activation stores (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Davies *et al.*, 2025; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Deviations at any step re-open bytes/token bills (Chen, 2026a; Vellaisamy *et al.*, 2026).

Pattern 2: Context buckets. CUDA Graphs capture requires static loop bounds or padded max context (NVIDIA, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2023). Hopper MegaKernels compile separate buckets (512/2048/8192) rather than

dynamic reallocation mid-graph (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). Fleet configs must pin bucket per deployment (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Pattern 3: Warp-specialized TMA. Dedicated warps issue TMA descriptors while MMA warps consume previous tiles (SemiAnalysis, 2024; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Misbalanced roles underutilize TMA or stall MMA (Davies *et al.*, 2025; SemiAnalysis, 2024).

Pattern 4: Softmax on SFU + shuffle. Max-find via warp shuffle; **exp2** via SFU; carriers in FP32 even when matmul uses BF16 (Dao *et al.*, 2022; Davies *et al.*, 2025; NVIDIA, 2022; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024a,b). Chapter 10 invariants apply inside Hopper kernels (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026).

Pattern 5: Graph + fusion composition. CUDA Graphs remove host overhead; fusion removes HBM bytes—compose both for H100 cells with low R_{floor} (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Report which effect dominated in each row (Chen, 2026a; Vellaisamy *et al.*, 2026).

Pattern 6: Triplet parity tests. Export identical IR to CUDA/XDNA/CPU; compare bytes/token normalized by memory hierarchy (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). GPU ms/token alone misleads fleet decisions (Davies *et al.*, 2025; AMD, 2024a).

Industrial narrative (Fregly-aligned). DeepSeek-scale training stories emphasize co-design under hardware export limits (Liu *et al.*, 2024; Davies *et al.*,

2025). Decode on Hopper repeats the lesson at chip scale: skillful residency beats brute-force silicon (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Goodput metrics make the case quantitatively (Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Common failure stories. (a) Shared memory overflow at max context; (b) TMA misalignment on paged KV; (c) cluster launch illegal on target SKU; (d) PTX **exp2** drift at long T ; (e) graph capture invalidated by driver upgrade (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Runbooks map symptoms to residency vs orchestration using bytes/token and launches/token splits (Vellaisamy *et al.*, 2026; Chen, 2026a).

Teaching vs production CUDA. Courses emphasize matrix multiply microbenchmarks; production decode emphasizes residency graphs (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Dao *et al.*, 2023). This chapter bridges the gap for Hopper (SemiAnalysis, 2024; NVIDIA, 2022).

Roadmap to later chapters. Chapter 8 specializes TMA double-buffer theory; Chapter 9 sync hierarchy; Chapter 10 softmax carriers; Chapter 11 full MegaKernel (Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). Hopper bounds here are prerequisites (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Documentation deliverables. Ship Hopper kernels with: residency diagram; TMA descriptor layout; cluster dims; sync plan; PTX softmax notes; TaxBreak launches/token cell; Memory-Floor bytes/token cell (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Energy and TCO. HBM traffic drives energy on decode (Chen, 2026a; Davies *et al.*, 2025; AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Fusion savings matter for cloud TCO even when ms/token gains look modest (Vellaisamy *et al.*, 2026; Chen, 2026a).

Security of performance claims. Never cite peak TFLOPs for decode chapters; cite measured cells with model names and ctx (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Appendix regression matrices anchor internal numbers (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Open research threads. Dynamic shapes without re-capture, multi-GPU tensor-parallel MegaKernels, and MoE expert-local fusion remain active (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). This chapter documents single-GPU batch-1 baselines (Vellaisamy *et al.*, 2026; Chen, 2026a).

Final synthesis. Hopper gives async copy, clusters, tensor cores, and SFUs—tools for mechanical sympathy, not automatic speedups (NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Liu *et al.*, 2024; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Measure goodput; co-design residency; gate merges with profilers (Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Cross-chapter dependency map. Chapter 3 listed chip constraints abstractly; this chapter instantiates Hopper mechanisms; Chapter 5 shows XDNA static analogues; Chapters 8–11 consume TMA/sync/softmax/MegaKernel patterns (AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Davies *et al.*, 2025; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Goodfellow

et al., 2016). Skipping this chapter produces CUDA kernels that ignore residency bounds (Vellaisamy *et al.*, 2026; Chen, 2026a).

Fleet configuration guidance. Pin driver, CUDA, and YiRage commit in decode pools; re-run TaxBreak launches/token and Memory-Floor bytes/token after upgrades (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Hopper-specific kernels are not portable to L4 cells without re-roofline analysis (Chen, 2026a; Vellaisamy *et al.*, 2026).

Reader exercises. (1) Sketch shared-memory allocation for one fused decode layer at $d=2048$. (2) Explain when TMA hurts batch-1 decode. (3) Compare graph-only vs fusion bytes/token using Chapter 1 tables (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Glossary. **TMA:** async global-to-shared copy engine (NVIDIA, 2022; SemiAnalysis, 2024). **DSM:** cluster distributed shared memory (NVIDIA, 2022). **Goodput:** useful decode work per ms—kernels/token and bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Use consistently in PRs (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Dao *et al.*, 2023).

Summary before takeaways. Sections on memory tiers, TMA, clusters, sync, tensor-core realism, PTX **exp2**, and YiRage CUDA lowering form one mechanical-sympathy story for Hopper decode (NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Profile-first, bytes-first, fusion-first—in that order when R_{floor} is low (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

4.10 Worked Example: One Decode Layer on H100

This section walks through a single transformer decode layer the way a performance engineer would during a Hopper bring-up review. The goal is not to derive new math but to connect Chapter 1 counters to concrete CUDA tiers before Chapter 11 stitches the full MegaKernel (Vellaisamy *et al.*, 2026; Chen, 2026a).

Setup. Consider Llama-3.2-1B at batch size one, context length 2048, hidden dimension 2048, sixteen query heads, and grouped-query attention with four KV heads (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). One new output token triggers one decode step. We compare an unfused PyTorch-style schedule against a fused Hopper plan that keeps activation-sized tensors in shared memory or registers for the entire layer (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Unfused activation traffic (qualitative). A conventional stack materializes at least: post-norm hidden state, Q projection output, K and V writes into the paged KV cache, attention output, post-attention residual buffer, MLP intermediate after gate/up projections, down projection output, and final residual update. Even when each tensor is only one token wide, each boundary that stores to HBM and reloads adds roughly $2d$ bytes of activation traffic per layer per token, where d is the model width (Chen, 2026a; Davies *et al.*, 2025). Six to eight such round trips per layer is typical on dense Llama-class stacks before counting KV cache reads (Vellaisamy *et al.*, 2026). TaxBreak reports hundreds of kernel launches per token on H100 for unfused stacks; the launch tax is orchestration, but the byte tax is residency failure (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

Fused residency plan. A Hopper MegaKernel target keeps the following live on-chip across stages: the incoming hidden vector fragment, QKV MMA tiles in shared memory, online softmax carriers (m, ℓ, o) per head in registers, partial PV accumulators, SwiGLU gate/up tiles, and the down-projection epilogue buffer (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Weights and KV still stream from HBM—fusion does not remove parameter traffic—but activation-sized tensors never take a detour through global memory between attention and MLP (Chen, 2026a; Kwon *et al.*, 2023). The engineering question becomes whether the plan fits Table 4.1 without spilling carriers (NVIDIA, 2022; Chen, 2020).

Shared-memory arithmetic (sketch). Hopper exposes up to 228 KB configurable shared memory per SM (NVIDIA, 2022; SemiAnalysis, 2024). A conservative fused tile for $d = 2048$ might allocate: 8 KB for hidden state staging, 48 KB for QKV double buffers, 32 KB for softmax score tiles, 64 KB for SwiGLU intermediates, and 16 KB for epilogue handoff—roughly 168 KB before cluster pooling (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). That leaves headroom for TMA descriptor scratch and barrier state, but not for a second concurrent layer replica (SemiAnalysis, 2024). If planners add a duplicate score tile “for simplicity,” occupancy collapses or the compiler silently spills (m, ℓ, o) to local memory—both show up as bytes/token regressions in Memory-Floor style accounting (Chen, 2026a; Davies *et al.*, 2025).

Register pressure for online softmax. Each head maintains three scalars (m, ℓ) plus a partial output vector of length $d_{\text{head}} = 128$ in FP32 for numerical stability (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Sixteen heads imply sixteen partial output vectors if not warp-shared—a register budget that fusion planners must co-locate with MMA fragment registers (NVIDIA, 2022; Chen and YiRage Contributors, 2026). Warp-specialized layouts assign one warp group per head cluster so shuffle reductions stay in-register (Dao *et al.*, 2022; Davies *et al.*, 2025). Exceeding the register cap per thread lowers occupancy; the right trade-off is often *lower occupancy, higher bytes/token win* when fusion removes HBM passes (Chen, 2026a; SemiAnalysis, 2024).

TMA in the KV loop. For each KV block of size B_k , TMA loads K_b and V_b tiles into shared memory while MMA units compute $QK_b^\top$ on the previous block (SemiAnalysis, 2024; Dao *et al.*, 2023). Paged KV via vLLM block tables means TMA descriptors must be rebuilt or indexed per physical block id inside the kernel (Kwon *et al.*, 2023; Davies *et al.*, 2025). A common failure is issuing TMA for $M = 1$ micro-tiles where descriptor setup dominates—profile both TMA and scalar load paths on the target shape before mandating TMA in YiRage lowering (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Cluster decision point. When score tiles exceed single-block shared memory, Hopper clusters pool DSM across cooperating blocks (NVIDIA, 2022; SemiAnalysis, 2024). Flash-Decoding splits KV across SMs and merges online softmax statistics in-kernel (Dao *et al.*, 2023). If cluster launch is illegal on the deployment SKU, the fallback is an extra merge kernel—which reintroduces launches/token and may erase fusion wins (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022). YiRage should reject

cluster plans that fail compile rather than emit a silent multi-kernel fallback (Chen and YiRage Contributors, 2026; Chen, 2020).

Profile interpretation. After implementing the fused plan, an engineer runs Nsight Systems on the full model step and Nsight Compute on the MegaKernel (Davies *et al.*, 2025; SemiAnalysis, 2024). Systems timelines should show one or few launches per layer instead of dozens (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Compute counters should show DRAM throughput dropping with unchanged MMA instruction counts—evidence that bytes/token improved while tensor cores were never the first bottleneck (Chen, 2026a; NVIDIA, 2022). If TC utilization rises but ms/token is flat, the team optimized the wrong metric (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Ablation table discipline. Publish three rows for every Hopper PR: graph-only (launches fixed, bytes unchanged), fusion-only (bytes fixed, launches unchanged if possible), and composed graph plus fusion (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026). Fregly-style goodput reporting requires labeling which lever moved (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Internal regression matrices in the appendix anchor TaxBreak launches/token and Memory-Floor R_{floor} cells so readers can map this worked example to measured numbers (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026).

Contrast with L4 and XDNA. The same IR exports to L4-class GPUs and Ryzen AI XDNA NPU backends (AMD, 2024b,a; Chen and YiRage Contributors, 2026). Memory-Floor shows Qwen-2.5-7B batch-1 R_{floor} much higher on L4 than H100—different roofline knee, same residency story (Chen, 2026a; Davies *et al.*, 2025). XDNA cannot rely on Hopper TMA; it assigns static SRAM slots at compile time (AMD, 2024a; Chen, 2020). Triplet parity tests ensure CUDA-specific wins do not fork the compiler IR (Chen and YiRage Contributors, 2026; AMD, 2024b).

Example 4.10.1. Review question. A teammate proposes a Hopper kernel that uses TMA for every 1×128 score fragment but still stores attention output to global memory before MLP. Ask: what happens to bytes/token and to Chapter 10 softmax carrier residency? Correct answer: bytes/token stays high and MegaKernel legality fails even if TMA overlap looks excellent in microbench (Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

4.11 Hopper Feature Reference for Compiler Authors

Compiler authors lowering decode IR to CUDA need a checklist tighter than a hardware manual (Chen, 2020; Chen and YiRage Contributors, 2026; NVIDIA, 2022). This section states invariants the YiRage CUDA backend enforces; Chapter 5 mirrors them for XDNA static tiles (AMD, 2024b,a).

Invariant 1: No activation-sized global stores inside a fused region. Any store of a vector length $\Theta(d)$ between attention, softmax, and MLP stages is a fusion bug unless proven dead by analysis (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020). Pass managers should mark such stores as hard errors in decode mode (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

Invariant 2: TMA legality is shape-dependent. TMA requires aligned multidimensional tiles; batch-1 decode often presents thin M dimensions (SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). Lowering must select TMA versus vector loads per sub-op based on a cost model, not a global flag (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; NVIDIA, 2022).

Invariant 3: Cluster dims are a compile-time contract. Cluster launch bounds must match DSM footprint (NVIDIA, 2022; SemiAnalysis, 2024). Illegal combinations fail at compile time with a residency report, not at runtime with silent corruption (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022).

Invariant 4: Softmax PTX is part of the ABI. Online softmax lowering emits reviewed **exp2** sequences with documented FTZ behavior (Dao *et al.*, 2022; Liu *et al.*, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Changing PTX without FP64 reference diffs at long context breaks fleet determinism (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Invariant 5: Triplet exports share bufferization. The same bufferization graph feeds CUDA, CPU, and XDNA passes (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2020). GPU-only schedules are incomplete for a multi-hardware book and product story (Davies *et al.*, 2025; AMD, 2024a).

Lowering walkthrough (six passes).

1. **Bufferization:** Assign each SSA tensor to register, shared, or global tier for its lifetime (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a).
2. **Fusion grouping:** Merge attention, softmax, and MLP into one fused region subject to shared-memory cap (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026).
3. **TMA selection:** For each global-to-shared copy, choose TMA or vector load (SemiAnalysis, 2024; NVIDIA, 2022).
4. **Cluster planning:** If fused tile exceeds block shared memory, compute cluster shape and DSM layout (NVIDIA, 2022; Dao *et al.*, 2023).
5. **Sync insertion:** Place barriers only on shared-memory phase edges (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; NVIDIA, 2024).
6. **PTX lowering:** Emit softmax micro-ops and MMA epilogues with stability annotations (Dao *et al.*, 2022; Liu *et al.*, 2024; Davies *et al.*, 2025).

Testing gates. Each merged CUDA backend change carries: numerical diff versus FP64 reference at $T \in \{512, 2048, 8192\}$; Nsight DRAM byte delta at batch one; TaxBreak launches/token cell; XDNA compile or documented deferral (Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Dao *et al.*, 2023).

Relationship to CUDA Graphs. Graph capture replays a fixed launch sequence (NVIDIA, 2024; Chen, 2026a). Bufferization must pin graph capture buckets to max context lengths used in production (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Dynamic shapes without re-capture remain an open research thread (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026).

Blackwell porting notes. Successor silicon widens FP4 and FP8 tensor paths and extends TMA (NVIDIA, 2022; Davies *et al.*, 2025; Liu *et al.*, 2024). Compiler invariants survive: residency first, orchestration second, MMA tile tuning last (Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020).

Nsight Compute counter cheat sheet. For Hopper decode MegaKernels, prioritize, in order: DRAM bytes per kernel invocation, LITEX throughput, local memory loads from spill, achieved occupancy versus chosen shared-memory config, TMA busy cycles versus MMA busy cycles, and only then tensor-core utilization

(Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2022; Chen and YiRage Contributors, 2026). Teams that open with TC utilization often optimize a unit that was idle for good reason (Chen, 2026a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Systems timeline reading. Nsight Systems exposes host launch storms, CUDA Graphs replay gaps, and CPU-side sampling overhead (NVIDIA, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A fused kernel may look slower in isolation if the baseline hid launch latency behind queue overlap (Vellaisamy *et al.*, 2026; Chen, 2026a). Always compare end-to-end ms/token for one decode step, not kernel-only timers (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023).

Regression hygiene. Pin model weights, tokenizer, context length bucket, driver, and CUDA version when comparing Hopper PRs (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Record launches/token and bytes/token alongside median and p99 ms/token (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024). Ship a short residency diagram in the PR description so reviewers see tier assignments without reading all PTX (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Dao *et al.*, 2022).

Handoff to Chapter 5. CUDA legality is dynamic: tiers are chosen per kernel subject to occupancy and spill constraints (NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). XDNA legality is static: SRAM slots are fixed at compile time with multi-tile DMA schedules (AMD, 2024a,b; Davies *et al.*, 2025). The same bufferization graph must explain both or the triplet story collapses (Chen and YiRage Contributors, 2026; AMD, 2024a; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Synchronization case study. Consider a fused block that double-buffers TMA into shared memory while MMA consumes the alternate buffer (SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2022). Only two block-level barriers per buffer swap are required: one after TMA completion, one before overwriting the consumed buffer (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Inserting a barrier after every warp shuffle in softmax adds bubbles without safety benefit (Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). Compiler sync insertion should be edge-driven on the fused dataflow graph (Chen and YiRage

Contributors, 2026; SemiAnalysis, 2024; Chen, 2026a; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2023).

Paged KV and TMA co-design. vLLM-style block tables map logical token positions to non-contiguous physical blocks (Kwon *et al.*, 2023; Davies *et al.*, 2025). TMA descriptors assume regular layouts; PagedAttention kernels either gather into contiguous shared-memory staging or use per-block descriptors inside the KV loop (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). This is why MegaKernel codegen special-cases KV: it is not a generic GEMM (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen, 2026a, 2020; NVIDIA, 2022).

Closing the loop with Chapter 1. Return to the TaxBreak and Memory-Floor cells for your target model after every Hopper change (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). If launches/token and bytes/token are unchanged, ms/token gains are suspect or attributable to unrelated driver noise (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2023; Chen, 2020). Goodput engineering means all three counters move together in the expected direction (Chen, 2026a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Practitioner mantra. Profile end-to-end decode first. Cut activation bytes second. Collapse launches third. Tune tensor cores fourth. Re-profile on every driver and context bucket change. Teach teammates to read residency diagrams before PTX diffs. This ordering prevents weeks of wasted TC tile sweeps on bandwidth-bound decode pools. Keep the mantra visible in every Hopper optimization doc your team maintains. It aligns silicon features with measured decode goodput outcomes every single engineering sprint review.

4.12 Key Takeaways

- **Mechanical sympathy on Hopper.** TMA bulk copies, thread-block clusters, and the 228 KB shared memory exist precisely to keep decode data on-chip, and a kernel that ignores them streams activations through HBM every token (NVIDIA, 2022; SemiAnalysis, 2024). Using these features is the difference between a decode kernel that hides memory latency and one that exposes it (Dao *et al.*, 2022).

- **Profile before tuning tensor cores.** At batch-1 the arithmetic intensity is low, so the limiter is launches and bytes, not tensor-core throughput; tuning MMA tiles first optimizes the wrong term (Chen, 2026a; Vellaisamy *et al.*, 2026). Fix launches/token and bytes/token, then revisit the tensor-core schedule (Davies *et al.*, 2025).
- **Async copy hides latency; fusion removes traffic.** TMA overlap and double buffering hide the latency of a load, but they do not reduce how many bytes move—only fusing the layer into a MegaKernel cuts the activation HBM traffic itself (SemiAnalysis, 2024; Chen, 2026a). The two are complementary: overlap what you must move, fuse so you move less (Chen and YiRage Contributors, 2026).
- **Synchronize at the narrowest scope.** Warp shuffles and planned producer/consumer barriers coordinate decode work far more cheaply than a grid-wide sync, which stalls every CTA (NVIDIA, 2022; Dao *et al.*, 2022). Keeping coordination on-chip is also what lets a captured launch graph replay without host round trips (NVIDIA, 2024).
- **Grade on goodput, and let YiRage enforce legality.** Always pair kernels/token and bytes/token with ms/token so a microbenchmark win that does not move the token cannot masquerade as progress (Vellaisamy *et al.*, 2026; Chen, 2026a). The YiRage CUDA backend rejects a schedule whose residency is illegal on the target SKU before it ships, turning these metrics into a build-time gate (Chen and YiRage Contributors, 2026; Chen, 2020).

4.13 Conclusion

Hopper/Blackwell supply async copy, cluster, and tensor-core machinery for decode MegaKernels when schedules respect Table 4.1 (NVIDIA, 2022; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). TMA overlaps KV/weight movement; DSM widens fusion; sync and PTX discipline protect Chapter 10 softmax carriers (Dao *et al.*, 2022; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023).

Chapter 5 contrasts dynamic CUDA tiers with XDNA static tiles—triplet regression from one IR is mandatory (AMD, 2024b,a). Carry bytes/token and launches/token; faster silicon does not redefine goodput (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022; Chen and

YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Reviewers should reject Hopper PRs that show faster microbenchmark GEMMs without DRAM byte deltas or TaxBreak launch counts (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). That discipline keeps the book’s Fregly-aligned story honest across fleets (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a).

Chapter 5

AMD XDNA/AIE Dataflow Architecture

GPU teams port a decoder kernel to Ryzen AI and expect “slower CUDA.” What they get is often a *compile failure* or a silent fallback to DDR-bound scalar loops (AMD, 2024b,a). XDNA is not a SIMT machine with smaller shared memory—it is a spatial *dataflow* array where buffer lifetimes, DMA chains, and tile assignments must be known before the first token (Davies *et al.*, 2025; Chen, 2020). Chapter 3 listed NPU static legality in the constraint matrix; this chapter explains AMD’s XDNA/AIE tiles, the static scheduling rules that decode must obey, and how those rules map to CUDA concepts (TMA, shared memory, softmax carriers) that GPU engineers already understand (Chen and YiRage Contributors, 2026; NVIDIA, 2022).

The decode metric lens does not change on NPU. TaxBreak still counts launches/token on GPUs; on XDNA the analogue is DMA transactions and DDR egress per token (Vellaisamy *et al.*, 2026; Chen, 2026a). Batch-1 interactive decode still reads every weight and every KV vector each step—only the orchestration mechanism differs (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). YiRage’s XDNA backend exists because hand-written “one op at a time” schedules violate the array’s static contract and waste the watt advantage that makes edge NPUs attractive (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Dense Llama-3.2-1B issues 847.5 GPU kernel launches per output token on H100 in TaxBreak decode sweeps; OLMoE-1B/7B issues 9,305 (Vellaisamy *et al.*, 2026). XDNA cannot interpret those numbers as a launch budget—each logical “launch” must become a static DMA edge or be fused away entirely (AMD, 2024b; Chen and YiRage Contributors, 2026). Memory-Floor shows CUDA Graphs

removing ≈ 3.05 ms host overhead per step on Qwen-2.5-7B at BS=1—on XDNA the comparable win is eliminating redundant DDR round trips, not replaying kernels (Chen, 2026a; NVIDIA, 2024).

Silicon context. Ryzen AI combines Zen cores with XDNA NPU tiles aimed at laptop and edge inference (AMD, 2024b,a). Decode workloads stress DDR interfaces shared with CPU graphics and I/O—contention shows up as tail latency (Chen, 2026a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Planning must include system-level bandwidth, not isolated NPU peaks (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2022; NVIDIA, 2024; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2023).

Decode vs prefill on NPU. Prefill parallelizes over prompt tokens; decode appends one token with full KV history (Dao *et al.*, 2022, 2023; Davies *et al.*, 2025). Static graphs compiled for long rectangular matmuls may waste tile arrays on thin decode matmuls (AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024b; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022). Always qualify decode-shaped graphs separately (Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

5.1 XDNA tiles

Ryzen AI deployment context. XDNA appears in Ryzen AI NPUs paired with x86 cores—decode often splits pre/post on CPU and MegaKernel on AIE (AMD, 2024b; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The tile grid is fixed per SKU; compiler passes must query exact SRAM sizes, not datasheets from a different stepping (AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026).

DDR vs tile bandwidth. Decode is DDR-bandwidth bound when weights and KV exceed tile residency (Chen, 2026a; Dao *et al.*, 2022). Roofline on XDNA uses DDR bytes/token as the primary axis—identical to GPU decode thinking once HBM replaces DDR (Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022; Liu



Figure 5.1: XDNA decode data movement: DDR ingress dominates when tile residency fails.

Table 5.1: XDNA tile resources vs. GPU SM shared memory (conceptual decode comparison).

Resource	Hopper SM	XDNA AIE tile
On-chip scratch	Configurable shared mem	Fixed tile SRAM
Parallel grain	Warp (32 threads)	Spatial PE lane
Scheduling	Runtime + streams	Compile-time DMA graph
Dynamic shapes	Supported (with cost)	Restricted / recompile
Decode failure mode	Launch storms	DDR egress / illegal graph

et al., 2024; Kwon *et al.*, 2023; Dao *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024a; Vellaisamy *et al.*, 2026; Chen, 2020).

AMD **XDNA** groups AI Engine (**AIE**) tiles into a spatial array: each **tile** owns local SRAM, connects to neighbors through stream switches, and executes MAC-heavy datapaths under compiler-scheduled DMA (AMD, 2024b,a). Unlike GPU warps, there is no runtime scheduler to hide a missing buffer plan—if a decoder stage needs a temporary softmax statistic and no tile slot was reserved at compile time, the backend either rejects the graph or spills to DDR (Chen and YiRage Contributors, 2026; Chen, 2026a).

Table 5.1 is the mental model GPU engineers need before writing XDNA code: think “DMA + tile SRAM budget,” not “launch another kernel” (SemiAnalysis, 2024; NVIDIA, 2022). A Llama-class decoder layer that fits in Hopper shared memory with careful tiling may still *illegalize* on XDNA if the same math requires three simultaneous tile-resident tensors—the array enforces a stricter simultaneous residency cap (Davies *et al.*, 2025; AMD, 2024a).

Example 5.1.1. A single decode matmul tile on XDNA: weights stream from DDR into tile SRAM while activations arrive on a ping-pong DMA chain; if the compiler assigns overlapping lifetimes to two weight tiles, peak SRAM exceeds hardware limits and compile fails—there is no HBM spill fallback (Chen and YiRage Contributors, 2026; AMD, 2024b).

Worked contrast. Hopper decode can recover from a suboptimal schedule with CUDA Graphs and extra launches (NVIDIA, 2024; Vellaisamy *et al.*, 2026). XDNA decode rarely gets that slack: the graph is static or it does not ship (AMD, 2024a; Chen and YiRage Contributors, 2026). Identical FLOPs, different bytes/token because DDR traffic dominates when tile residency fails (Chen, 2026a; Dao *et al.*, 2023).

Tile array topology. Multiple AIE tiles connect in a grid; decode megakernels assign each pipeline stage to a tile column or row (AMD, 2024b,a). Compiler passes must respect inter-tile bandwidth limits—not every fusion that fits one tile fits the array (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

5.2 Static dataflow rules

Host-side responsibilities. The host configures graph runners, submits execute calls, and updates KV metadata—but must not allocate or copy full tensors per token. KV growth appends through pre-planned DDR regions referenced by static descriptors. When KV grows beyond compiled max context, the product must recompile or reject—not fall back to unplanned buffers (Kwon *et al.*, 2023; AMD, 2024b; Chen and YiRage Contributors, 2026).

Synchronization model. GPUs use `cudaDeviceSynchronize` at unfortunate times; XDNA relies on DMA completion tokens wired in the dataflow graph. Inserting host waits between stages hides parallelism and adds jitter to p99 decode (AMD, 2024a; Davies *et al.*, 2025; Chen, 2026a). Compiler backends should emit single submission boundaries per decoder step where possible (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Testing static graphs. Validate graphs at min and max context buckets, not only nominal length. Off-by-one errors in carrier buffer sizes often appear only at max context (Dao *et al.*, 2023; AMD, 2024a; Chen, 2026a). Run byte counters on DDR ports during simulation before silicon (AMD, 2024b; Chen, 2020).

Compiler IR implications. MLIR and YiRage attach static lifetime intervals to memref views before lowering to AIE (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). Generic bufferization that inserts heap fallback breaks



Figure 5.2: Static dataflow: every buffer lifetime is fixed before decode begins.

static rules immediately (Chen, 2026a; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023).

Recompile economics. Static specialization implies compile minutes amortized over thousands of tokens (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024a). Edge products with cold starts must budget compile latency in UX—unlike GPU JIT that hides in first batch (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Chen, 2020; AMD, 2024b; Liu *et al.*, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; NVIDIA, 2022).

Static XDNA schedules mean every tensor lifetime, DMA descriptor, and tile assignment is resolved at compile time (AMD, 2024b,a). **Dataflow** execution propagates tokens through spatial pipelines rather than a sequence of host-launched kernels (Chen, 2020; Chen and YiRage Contributors, 2026). The rules decode compilers must internalize:

1. No hidden host allocations on the per-token path—KV and temporaries alias compile-time buffer slots (Kwon *et al.*, 2023; Chen, 2026a).
2. No dynamic control flow inside the hot dataflow unless specialized into padded static variants (Davies *et al.*, 2025; Liu *et al.*, 2024).
3. Producer–consumer edges must map to DMA events with known latency; “sync later” is not a plan (AMD, 2024a; SemiAnalysis, 2024).
4. Fusion is all-or-nothing for many decode cells: partial fusion that exports intermediates to DDR often erases the NPU watt advantage (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

PyTorch-style eager op dispatch is the anti-pattern Chapter 1 diagnosed on GPUs—on XDNA it is often unsupported outright (Vellaisamy *et al.*, 2026; AMD, 2024b). The compiler must emit one static graph per shape bucket (batch, context class) and recompile when buckets change, amortizing compile cost across many tokens (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Deployment pitfall. Teams benchmark prefill-length graphs on XDNA, then deploy batch-1 decode with growing KV—dynamic sequence growth violates static buffer tables unless the compiler padded or bucketed context (Dao *et al.*, 2022, 2023; Chen, 2026a). Treat context buckets as first-class compile parameters, not runtime surprises (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Static legality review. Every merge should attach a buffer lifetime diagram: which tensors live in tile SRAM simultaneously at each decode step (AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026).

5.3 CUDA XDNA mapping

Shared memory illusion. GPU shared memory is explicitly sized and banked; tile SRAM is smaller but deterministic. Engineers who oversize tiles “because HBM is huge” fail XDNA legality immediately (NVIDIA, 2022; AMD, 2024a; SemiAnalysis, 2024). Start from tile SRAM datasheet bytes, not GPU comfort zones (Chen and YiRage Contributors, 2026; Chen, 2026a).

TMA lesson transfer. TMA teaches asynchronous movement while compute runs—XDMA chains must encode the same overlap explicitly. Without overlap, MAC utilization collapses and watt/token rises even if correctness holds (SemiAnalysis, 2024; AMD, 2024b; Davies *et al.*, 2025). Profile MAC active cycles, not just end-to-end latency (AMD, 2024a; Chen and YiRage Contributors, 2026).

Graph capture analogy. CUDA Graphs freeze launch order; XDNA freezes DMA order. Mutating either without re-record/re-compile breaks decode silently (NVIDIA, 2024; AMD, 2024b; Vellaisamy *et al.*, 2026; Chen, 2026a). Treat shape buckets like graph capture templates (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).

Double buffering depth. Hopper TMA double buffering hides HBM latency; XDNA ping-pong DMA chains require explicit buffer descriptor pairs in the static graph (SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a; Chen and YiRage Contributors, 2026). Shallow pipelines under-utilize MAC arrays; deep pipelines overflow tile SRAM (Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; AMD, 2024b; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024).

Table 5.2: CUDA $\rightarrow$ XDNA concept mapping for decode.

CUDA / Hopper	XDNA / AIE
TMA async copy	DMA into tile SRAM
Shared memory tile	Tile buffer / ping-pong chain
CUDA Graphs capture	Static dataflow graph
Stream launch	Host graph runner invoke
Warp-synchronous softmax	Spatial softmax carrier pipeline

Host orchestration. CUDA uses streams; XDNA uses graph runners that submit entire dataflow frames (NVIDIA, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026). Per-token host work must be $O(1)$ and allocation-free—TaxBreak’s launch tax has a direct analogue in graph submission overhead (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; AMD, 2024a; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

GPU engineers think in TMA descriptors and shared-memory tiles; XDNA engineers think in **buffer descriptors** and tile SRAM assignments (NVIDIA, 2022; SemiAnalysis, 2024; AMD, 2024b). The mapping is conceptual, not one-to-one:

Hopper **TMA** hides DDR latency by double-buffering into shared memory while MMA units compute (SemiAnalysis, 2024; NVIDIA, 2022). XDNA achieves a similar *software pipeline* with ping-pong **buffer descriptors** into tile SRAM—but the compiler must assign both buffers and DMA edges statically (AMD, 2024a; Chen and YiRage Contributors, 2026). A decode attention stage that uses TMA-backed softmax carriers on GPU must become a fixed carrier pipeline on AIE; you cannot insert a host-side `malloc` between carriers (Dao *et al.*, 2022, 2023; Chen, 2026a).

Example 5.3.1. GPU decode step: TMA loads Q tile while previous softmax partials remain in shared memory. XDNA equivalent: DMA chain loads Q into tile *A* while tile *B* holds running max/sum carriers—both buffers named in the static graph; swapping roles is compile-time ping-pong, not runtime alloc (Chen and YiRage Contributors, 2026; AMD, 2024b; SemiAnalysis, 2024).

Review gate. Before merging XDNA backend changes, diff static buffer tables against tile SRAM caps and list every DDR transaction per decode step (AMD, 2024a; Chen, 2026a; Chen and YiRage Contributors, 2026). If DDR bytes/token exceed GPU baselines, residency planning failed—not “the NPU is slow” (Davies

et al., 2025; Vellaisamy *et al.*, 2026).

5.4 Online softmax HW

Numerical stability. Online softmax relies on stable accumulation order; spatial reductions must preserve mathematical equivalence to reference implementations (Dao *et al.*, 2022, 2023). Validation should compare logits against GPU reference on short contexts before trusting long-context deployments (Chen, 2026a; AMD, 2024a; Chen and YiRage Contributors, 2026).

Carrier placement. Assign carrier buffers adjacent to attention tiles in the dataflow to minimize stream latency. Poor placement increases stage cycles without changing FLOPs—visible only in cycle-accurate simulation (AMD, 2024b; Davies *et al.*, 2025; Chen, 2020). Compiler search should include placement, not only tile sizes (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

Carrier precision. Online softmax carriers need width headroom for long contexts (Dao *et al.*, 2022, 2023; Chen, 2026a). Fixed-point carriers save SRAM but risk saturation; compiler must pick formats per tile budget (AMD, 2024a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; AMD, 2024b; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2022).

Attention tile fusion. Fusing $QK^\top$, softmax carrier update, and AV into one spatial pipeline is the XDNA analogue of FlashAttention fusion (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; AMD, 2024a; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024b; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016; NVIDIA, 2022).

Attention decode needs numerically stable **softmax** over growing KV context (Dao *et al.*, 2022, 2023). FlashAttention-style algorithms keep running max and sum statistics in on-chip carriers rather than materializing full softmax matrices to DDR (Chen, 2026a). On GPU, those carriers live in shared memory with warp-synchronous updates; on XDNA they must be assigned to specific **carrier** buffer slots in the static dataflow (AMD, 2024a; Chen and YiRage Contributors, 2026).

The hardware failure mode is familiar: a “simple” softmax op lowered independently forces a full attention map to DDR, doubling bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a). Online softmax on AIE requires fusing the carrier update with the $QK^\top$ pipeline so partial statistics never leave tile SRAM (Dao *et al.*, 2022; SemiAnalysis, 2024). GPU-first teams often import unfused softmax kernels and wonder why XDNA compile rejects the graph—the rejection is correct (AMD, 2024b; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Example 5.4.1. Decode softmax carrier width: at context 2048, carrier buffers need slots sized for one KV tile; if the compiler assumes prefill-length vectors, tile SRAM overflows and the backend splits the graph into DDR-bound pieces (Dao *et al.*, 2023; Chen, 2026a; AMD, 2024a).

Worked contrast. Hopper can recover with Flash-Decoding parallelism along sequence (Dao *et al.*, 2023). XDNA lacks an equivalent “launch more blocks” escape—sequence parallelism must be encoded as spatial lanes or rejected at compile time (AMD, 2024b; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

5.5 Architecture tradeoffs

Latency vs watt. Edge products advertise watt/token; cloud products advertise ms/token under load (AMD, 2024b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). XDNA wins watt; CUDA wins elastic throughput (Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024). Architecture reviews should pick which metric is primary before choosing silicon (Chen and YiRage Contributors, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Software stack depth. Ryzen AI ships runtime, compiler, and drivers—teams must coordinate versions like CUDA toolkit pins (AMD, 2024a,b; Chen, 2020). Upgrading one layer without requalifying decode graphs repeats MLIR upgrade horror stories (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a).

Hybrid fleets. Production may run GPU decode pools plus XDNA edge nodes—schedules must diverge by design, not by accident (Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; AMD, 2024a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023; NVIDIA, 2022).

Table 5.3: Decode tradeoffs: CUDA vs XDNA (batch-1 interactive).

Dimension	CUDA (Hopper)	XDNA (AIE)
Orchestration tax	Launches/token	Host graph + DMA setup
Flexibility	High	Low (static)
Peak bandwidth use	HBM + TMA	DDR + tile SRAM
MoE decode	Launch storms	Expert static legality
Best fit	Cloud GPU pools	Edge / watt-capped

Power envelopes. Watt/token often drives XDNA product wins even when raw ms/token loses to H100 (AMD, 2024b,a; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023; NVIDIA, 2022).

CUDA decode optimization trades launches against bytes: CUDA Graphs and fusion reduce orchestration; TMA and shared memory reduce HBM traffic (NVIDIA, 2024; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; NVIDIA, 2022). **XDNA** decode optimization trades compile-time staticism against DDR egress: the array wins on watt/token when the entire decoder step is one spatial pipeline; it loses when dynamicism forces recompile or scalar fallbacks (AMD, 2024b,a; Chen, 2026a).

MoE models expose the starkest CUDA vs XDNA split: TaxBreak reports thousands of kernel launches per token on GPU MoE decode (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). XDNA cannot absorb that launch count—experts must be fused, batched, or compiled as separate static graphs with explicit routing DMA (Chen and YiRage Contributors, 2026; AMD, 2024a). Choosing XDNA for MoE without a static routing plan repeats GPU orchestration failure (Davies *et al.*, 2025; Chen, 2026a).

When XDNA wins. Fixed-model edge assistants with bounded context, watt caps, and fully static megakernels (AMD, 2024b; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022).

When CUDA wins. Dynamic batching, speculative decoding, long-context cloud serving, and MoE with frequent graph changes (Kwon *et al.*, 2023; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

5.6 Dataflow conclusion for inference

Documentation for operators. Runbooks should list compiled shape buckets, max context, recompile triggers, and expected DDR bytes/token (Davies *et al.*, 2025; AMD, 2024b; Chen and YiRage Contributors, 2026). Operators otherwise misdiagnose KV growth as model quality issues (Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026).

Failure modes summary. Compile rejection, DDR bandwidth saturation, carrier overflow, host submission jitter, and stale graphs after model updates (AMD, 2024a,b; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022). Each maps to a measurable counter (Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2026a; Vellaisamy *et al.*, 2026).

Speculative decoding caveat. Speculative paths introduce dynamic branches hostile to static XDNA graphs (Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024b; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; AMD, 2024a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023; NVIDIA, 2022). Either compile separate graphs per speculation policy or keep speculation on GPU (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024b; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; AMD, 2024a; Liu *et al.*, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023; NVIDIA, 2022).

Continuous batching. vLLM-style batching on GPU does not port literally—XDNA batching requires padded static batch classes (Kwon *et al.*, 2023; AMD, 2024b; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; AMD, 2024a; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023; NVIDIA, 2022).

Dataflow is the operational model for NPU **inference** when decode must meet watt and latency envelopes (AMD, 2024b; Chen, 2020; Chen and YiRage Contributors, 2026). Production conclusions:

1. Treat decode as a static MegaKernel per shape bucket; recompile when buckets change (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025;

AMD, 2024a).

2. Measure DDR bytes/token and DMA count per token—not FLOPs (Chen, 2026a; Vellaisamy *et al.*, 2026).
3. Reject graphs that materialize KV or softmax intermediates to DDR (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023).
4. Pair GPU and XDNA baselines from the same high-level IR (Chen, 2020; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Cross-hardware inference fleets run GPU pools for elastic decode and NPU nodes for edge—the dataflow lesson is that identical weights do not imply identical schedules (Liu *et al.*, 2024; Davies *et al.*, 2025; AMD, 2024b). Compiler stacks must fork early; “port the CUDA kernel” without static replanning fails (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

5.7 YiRage XDNA backend

Multi-backend CI. YiRage CI should compile the same decoder export for CUDA and XDNA on every merge, publishing legality diffs (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Developer ergonomics. When XDNA compile fails, error messages should cite buffer table rows and tile IDs—not opaque backend codes (Chen and YiRage Contributors, 2026; AMD, 2024a; Davies *et al.*, 2025; Chen, 2026a; AMD, 2024b; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Good diagnostics shorten the static scheduling learning curve (Chen and YiRage Contributors, 2026; AMD, 2024b).

Debug workflow. On XDNA compile failure, dump static buffer table and DMA edge list before reading assembly (Chen and YiRage Contributors, 2026; AMD, 2024a,b; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023; NVIDIA, 2022).

Regression tests. CI should compile canonical decoder layers for each supported Ryzen AI SKU and fail on new allocs in hot path (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023; NVIDIA, 2022).

YiRage targets **XDNA** through the same residency vocabulary as its CUDA backend: buffer lifetimes, fusion boundaries, and decode metrics (Chen and YiRage Contributors, 2026; AMD, 2024b,a). The XDNA backend rejects dynamic allocations on the hot path, maps fused decoder cells to static AIE graphs, and runs legality checks from Chapter 3 before codegen (Chen, 2020; Davies *et al.*, 2025; Chen, 2026a).

Practical integration: attach tile SRAM and DMA metadata during MLIR import; compile separate graphs per shape bucket; fuse online softmax with attention; triplet-test GPU vs XDNA on identical IR (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; AMD, 2024a; Chen, 2020).

Example 5.7.1. YiRage merge gate: Llama-3.2-1B decoder layer at BS=1, ctx=2048 must compile as one static XDNA graph or document failing ops; report GPU launches/token for the same IR export (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2026a; AMD, 2024b).

Operational checklist. Verify static buffer table fits tile SRAM; count DDR transactions per token; run decode p99; log claims to `verified_facts.jsonl` (Chen, 2026a; Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Engineering narrative. Treat XDNA as a compiler target with hard legality, not as a slow GPU. Your schedule is a DMA graph; your on-chip memory is tile SRAM; your failure telemetry is DDR bytes per token. Every section below connects those primitives to decode pain already measured on GPUs in Chapter 1—launch storms become DMA chains, shared-memory spills become DDR egress, and framework fragmentation becomes compile-time rejection (Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b).

Who should read this chapter. GPU kernel engineers porting to Ryzen AI, compiler authors lowering MLIR to AIE, and serving teams splitting cloud GPU pools from edge NPU nodes. If you only deploy CUDA, read the CUDA–XDNA

mapping and tradeoff sections—they explain why your GPU recipe will not copy verbatim (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; NVIDIA, 2022).

Tile SRAM accounting walkthrough. List every tensor live during one decode step: Q slice, K/V cache lines, softmax carriers, MLP intermediates, rotary embeddings. Sum bytes per tile. If the sum exceeds SRAM, the compiler must pipeline in time (sequential stages) or space (more tiles), not spill to DDR silently (AMD, 2024a; Chen, 2026a; Dao *et al.*, 2022). GPU engineers perform a similar exercise with shared-memory budgets; the difference is that XDNA seldom forgives overflow (AMD, 2024b; Chen and YiRage Contributors, 2026).

Static dataflow worked example. Imagine a two-stage decode cell: attention then MLP. Stage one DMA-loads K/V while MAC arrays update softmax carriers entirely in tile SRAM. Stage two consumes attention output directly from stream registers without DDR round trip. If MLP weights are DMA-loaded mid-stage without double buffering, MAC units stall—visible as higher ms/token even though FLOPs counters look healthy (SemiAnalysis, 2024; Davies *et al.*, 2025; AMD, 2024a).

Buffer descriptors in practice. A buffer descriptor names source address, destination tile buffer, length, and stride—analogue to TMA copy descriptors on Hopper. Ping-pong uses two descriptors alternating roles each token step. Decode graphs must freeze descriptor contents per shape bucket; changing context length without recompile implies descriptors were padded for max context upfront (NVIDIA, 2022; AMD, 2024b; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).

Online softmax on hardware. Carriers track running maximum and accumulated exponential sum per row tile. Each new KV tile updates carriers without writing attention probabilities to DDR. Breaking fusion at softmax exports an $L \times L$ map—fatal for long context on bandwidth-limited DDR (Dao *et al.*, 2022, 2023; Chen, 2026a). GPU warp-level reductions become spatial reduction trees on AIE—different mechanism, same invariant: carriers stay on-chip (AMD, 2024a; Chen and YiRage Contributors, 2026).

CUDA vs XDNA product framing. Cloud assistants with dynamic batching and MoE remain CUDA-first (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Kwon *et al.*,

2023). Fixed-model copilots on laptops with watt caps are XDNA candidates when compile time is amortized (AMD, 2024b; Davies *et al.*, 2025). Hybrid designs route elastic traffic to GPU and pinned edge traffic to NPU—two schedules, one product (Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Inference conclusion expanded. Dataflow inference means the runtime executes a compiled graph with predictable DMA timing—no Python loop launching ops (AMD, 2024b,a). Latency variance comes from DDR contention and host submission, not from kernel selection (Chen, 2026a; Vellaisamy *et al.*, 2026). Qualify XDNA builds with p99 decode traces, not prefill throughput alone (Davies *et al.*, 2025; Dao *et al.*, 2023).

YiRage backend depth. YiRage shares IR between CUDA and XDNA backends: the same fused decoder cell exported from PyTorch passes target-specific legality checks (Chen and YiRage Contributors, 2026; Chen, 2020). Failures surface as compile errors with buffer tables attached—preferable to silent DDR fallback (AMD, 2024a; Chen, 2026a). Merge gates require triplet metrics: GPU launches/token, XDNA DMA count, bytes/token both sides (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024b).

Common mistakes checklist. Importing unfused PyTorch ops one by one. Assuming dynamic batch/seq like CUDA. Omitting softmax carrier fusion. Skipping compile rejections because GPU numbers look fine. Measuring prefill only. Each mistake reproduces Chapter 1 pathologies on different silicon (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; AMD, 2024b; Kwon *et al.*, 2023; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2023; AMD, 2024a; Chen, 2020; Davies *et al.*, 2025).

Forward pointer. Chapter 6 formalizes residency planning across GPU, CPU, and NPU; Chapter 14 covers MLIR lowering that feeds YiRage backends (Chen, 2020; Chen and YiRage Contributors, 2026). The XDNA-specific lesson here is static legality: encode it early or pay DDR costs every token (AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023; Chen, 2020).

Metric closure. End every XDNA experiment reporting DDR bytes/token, DMA transactions per token, compile legality pass rate, and p99 decode at batch one (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024b,a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Without those counters, debates devolve into FLOPs and die area—useless for decode (Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Measure before merging; merge before celebrating (Chen and YiRage Contributors, 2026; AMD, 2024b; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; AMD, 2024a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

5.8 XDNA decode review gates

1. **Tile SRAM budget:** no simultaneous buffers exceed per-tile caps (AMD, 2024a,b; Chen and YiRage Contributors, 2026).
2. **Static graph:** no dynamic dims on hot path without bucket recompile (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2020).
3. **DDR ledger:** bytes/token vs GPU baseline on same model (Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).
4. **Softmax carriers:** fused online; no full-map materialization (Dao *et al.*, 2023; SemiAnalysis, 2024; AMD, 2024a).
5. **Triplet IR:** same export tested on CUDA and XDNA backends (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; NVIDIA, 2024, 2022).

Example 5.8.1. Review gate failure: new MLP fusion adds third tile-resident buffer—compile passes on GPU shared mem, fails XDNA legality; gate blocks merge until tile assignment replanned (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023; NVIDIA, 2022).

These gates mirror Chapter 3 benchmark method: measure first, optimize second (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024b,a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023; NVIDIA, 2022). Skipping them ships DDR-bound decode that wastes Ryzen AI silicon (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2023; NVIDIA, 2022).

Deep dive: tile grid scheduling. Compiler schedulers assign operations to AIE tiles in a grid. Neighboring tiles exchange data through stream switches with finite bandwidth. A decode MegaKernel must map attention heads or hidden dimensions to tiles without oversubscribing switches (AMD, 2024b,a; Chen and YiRage Contributors, 2026). Oversubscription does not always fail compile—it stalls runtime and inflates p99 (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). Simulation before tape-out catches switch hotspots (Chen, 2020; AMD, 2024a).

Deep dive: weight streaming. Weights are read every decode token. On GPU, weight bandwidth competes with KV in HBM; on XDNA, weights DMA from DDR into tile SRAM each step unless partially resident (Chen, 2026a; Dao *et al.*, 2022; AMD, 2024b). Static plans may pin hot slices in tile SRAM across tokens while streaming cold slices—analogueous to cache blocking on CPU (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023). Plans that reload full layers each token saturate DDR (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; AMD, 2024a).

Deep dive: KV cache on DDR. KV typically lives in DDR on edge NPUs; attention reads entire history each token (Kwon *et al.*, 2023; Dao *et al.*, 2023; AMD, 2024b). Paged KV layouts still require static page tables in many XDNA stacks (Chen and YiRage Contributors, 2026; Chen, 2026a, 2020). Fragmentation hurts DDR efficiency—qualify with bytes/token, not parameter count (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; Liu *et al.*, 2024; Dao *et al.*, 2022; NVIDIA, 2024; SemiAnalysis, 2024; NVIDIA, 2022; Goodfellow *et al.*, 2016; AMD, 2024b; Chen, 2020; Chen and YiRage Contributors, 2026).

Deep dive: rotary and norms. Layer norm and rotary embedding are easy to dismiss as tiny ops; unfused they become separate DMA rounds (Vellaisamy *et al.*, 2026; AMD, 2024a; Chen and YiRage Contributors, 2026). Fuse into attention or MLP stages where carriers already resident (Dao *et al.*, 2022; Davies *et al.*, 2025; Chen, 2026a; AMD, 2024b; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; AMD, 2024a).

Deep dive: quantization. INT8 and INT4 weights reduce DDR pressure—critical on XDNA (AMD, 2024b; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Quantization scales must be embedded in static graphs; runtime scale lookup per op reintroduces dynamicism (AMD, 2024a; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2026a).

Deep dive: multi-layer decode step. A full decoder step chains multiple layer graphs or one fused multi-layer graph (Chen and YiRage Contributors, 2026; AMD, 2024a,b). Chaining increases host submissions; fusing increases SRAM pressure (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024a,b). Autotune Pareto fronts across those axes (Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026).

Deep dive: observability. Instrument DMA bytes, MAC active ratio, and graph submission time separately (AMD, 2024b; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; AMD, 2024a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Without decomposition, teams misallocate effort to weight quantization when scheduling is the bottleneck (Chen and YiRage Contributors, 2026; AMD, 2024b; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; AMD, 2024a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Epilogue for practitioners. XDNA rewards teams that think like compiler engineers first and kernel micro-optimizers second (AMD, 2024b,a; Chen and

YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Static dataflow is the price of watt-efficient decode; pay it deliberately (Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). When graphs fail, celebrate early rejection—it is cheaper than DDR-bound production (Chen and YiRage Contributors, 2026; AMD, 2024a,b; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

XDNA tiles exposes a cross-hardware fracture: the same fused subgraph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode xdna/aie legality before codegen, not as comments in hand-written kernels (AMD, 2024b,a). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When XDNA tiles disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (AMD, 2024b,a).

Review gate. Before merging compiler changes affecting XDNA tiles, attach roofline or NPU buffer accounting showing xdna residency fits on-chip for the target SKU (AMD, 2024b,a).

Worked contrast. A Hopper decode cell may legalize xdna with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (AMD, 2024b,a).

The usual AMD XDNA/AIE Dataflow Architecture failure at **static dataflow rules** is importing a CUDA mental model and expecting runtime scheduling to hide illegal buffer lifetimes.

Hardware constraint: static and dataflow must be co-designed. On GPUs that means shared-memory/TMA budgets and warp-grain barriers; on XDNA/AIE it means static tile buffers, DMA chains, and carrier slots with compile-time lifetimes; on CPUs it means L2/L3 blocking and NUMA-local KV placement (AMD, 2024b,a).

Decode at batch-1 keeps arithmetic intensity on the memory-bandwidth slope—micro-optimizing isolated GEMMs rarely moves p99 latency.

Compiler takeaway for static dataflow rules: lower the same decoder IR, but predicate passes on target residency tables—never ship a GPU-only schedule to NPU fleets (AMD, 2024b,a). Success metrics: bytes/token, launches/token, and p99 decode latency; compare GPU, CPU, and NPU cells on identical context lengths.

Worked contrast. A Hopper decode cell may legalize static with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (AMD, 2024b,a).

Deployment pitfall. Teams that A/B only prefill confuse the story: dataflow constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (AMD, 2024b,a).

Production teams misread **CUDA XDNA mapping** when decode SLOs are judged on FLOPs dashboards instead of residency and orchestration ledgers.

Trace the Chapter 2 chain: buffer descriptor sets the residency envelope; tma determines whether producer–consumer edges stay on-chip. Illegal fusion does not always crash—it silently exports tensors to HBM/DDR and shows up as bytes/token regression versus a unfused but scheduled baseline (AMD, 2024b,a). Profile launches/token alongside bandwidth; launch storms masquerade as memory bottlenecks when graphs remain operator-fragmented.

Operational checklist—verify buffer descriptor, tma, autotuning pitfalls, and platform emit paths in code review; document dialect lowering choices that change memory traffic, not just pass ordering (AMD, 2024b,a).

Deployment pitfall. Teams that A/B only prefill confuse the story: tma constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (AMD, 2024b,a).

Review gate. Before merging compiler changes affecting CUDA XDNA mapping, attach roofline or NPU buffer accounting showing buffer descriptor residency fits on-chip for the target SKU (AMD, 2024b,a).

online softmax HW exposes a cross-hardware fracture: the same fused subgraph legal on Hopper can be rejected outright on XDNA or CPU cache

budgets.

Binding software to silicon: compiler passes must encode softmax/carrier legality before codegen, not as comments in hand-written kernels (AMD, 2024b,a). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When online softmax HW disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (AMD, 2024b,a).

Review gate. Before merging compiler changes affecting online softmax HW, attach roofline or NPU buffer accounting showing softmax residency fits on-chip for the target SKU (AMD, 2024b,a).

Worked contrast. A Hopper decode cell may legalize softmax with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (AMD, 2024b,a).

The usual AMD XDNA/AIE Dataflow Architecture failure at **architecture tradeoffs** is importing a CUDA mental model and expecting runtime scheduling to hide illegal buffer lifetimes.

Hardware constraint: cuda and xdna must be co-designed. On GPUs that means shared-memory/TMA budgets and warp-grain barriers; on XDNA/AIE it means static tile buffers, DMA chains, and carrier slots with compile-time lifetimes; on CPUs it means L2/L3 blocking and NUMA-local KV placement (AMD, 2024b,a). Decode at batch-1 keeps arithmetic intensity on the memory-bandwidth slope—micro-optimizing isolated GEMMs rarely moves p99 latency.

Compiler takeaway for architecture tradeoffs: lower the same decoder IR, but predicate passes on target residency tables—never ship a GPU-only schedule to NPU fleets (AMD, 2024b,a). Success metrics: bytes/token, launches/token, and p99 decode latency; compare GPU, CPU, and NPU cells on identical context lengths.

Worked contrast. A Hopper decode cell may legalize cuda with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (AMD, 2024b,a).

Deployment pitfall. Teams that A/B only prefill confuse the story: xdna constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (AMD, 2024b,a).

Production teams misread **dataflow conclusion** when decode SLOs are judged on FLOPs dashboards instead of residency and orchestration ledgers.

Trace the Chapter 2 chain: dataflow sets the residency envelope; inference determines whether producer–consumer edges stay on-chip. Illegal fusion does not always crash—it silently exports tensors to HBM/DDR and shows up as bytes/token regression versus a unfused but scheduled baseline (AMD, 2024b,a). Profile launches/token alongside bandwidth; launch storms masquerade as memory bottlenecks when graphs remain operator-fragmented.

Operational checklist—verify dataflow, inference, autotuning pitfalls, and platform emit paths in code review; document dialect lowering choices that change memory traffic, not just pass ordering (AMD, 2024b,a).

Deployment pitfall. Teams that A/B only prefill confuse the story: inference constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (AMD, 2024b,a).

Review gate. Before merging compiler changes affecting dataflow conclusion, attach roofline or NPU buffer accounting showing dataflow residency fits on-chip for the target SKU (AMD, 2024b,a).

YiRage XDNA backend exposes a cross-hardware fracture: the same fused subgraph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode yirage/xdna legality before codegen, not as comments in hand-written kernels (AMD, 2024b,a). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When YiRage XDNA backend disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (AMD, 2024b,a).

5.9 Ryzen AI decode benchmarking

Benchmarking XDNA decode requires the same discipline as GPU TaxBreak sweeps—but counters swap launches/token for DMA transactions and DDR

Metric	XDNA decode meaning
DDR bytes/token	Weight + KV + spill traffic
DMA edges/token	Static graph orchestration tax
MAC active %	Tile utilization vs stall
Compile time	Amortization over session length
p99 ms/token	Tail under CPU/NPU DDR contention

Table 5.4: Ryzen AI decode benchmark counter set (batch-1).

bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Build a decode cell with BS=1, ctx buckets {512, 2048, 8192}, and pinned Ryzen AI driver/runtime versions (AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Warm-up must include graph runner initialization and first-token compile amortization—cold-start numbers dominate laptop UX but mislead steady-state decode comparisons (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; AMD, 2024a,b; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Report both first-token and token- k steady ms/token; product teams need both (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Triplet runs export the same high-level decoder from YiRage IR to CUDA and XDNA backends; diff legality failures before celebrating either side’s ms/token (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). A GPU win on launches/token paired with an XDNA DDR bytes/token regression signals residency failure, not “NPU is slow” (Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*,

2023).

Example 5.9.1. Ryzen AI decode gate: Llama-3.2-1B at ctx=2048 must show DDR bytes/token within 10% of GPU baseline *or* document why watt/token target justifies regression; attach static buffer table and DMA edge list (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Fleet parity. Laptop fleets share DDR with display, browser, and background services—decode benchmarks on isolated benches overstate steady-state goodput (AMD, 2024b; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; AMD, 2024a; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Qualify under realistic contention or publish contention assumptions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Regression harness. Check in golden decode-cell scripts alongside static graphs: same tokenizer, ctx buckets, and DDR byte counters every CI run (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Driver upgrades that change DMA timing without changing FLOPs still fail SLOs—treat runtime versions like CUDA toolkit pins (AMD, 2024a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Publish benchmark artifacts with every Ryzen AI graph promotion so field teams replay the same counters (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Documentation contract. Every XDNA decode release note should list max context per bucket, compile time, steady ms/token, and DDR bytes/token at

ctx=2048 (AMD, 2024b,a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Without published counters, partners cannot tell whether regressions come from graphs, drivers, or contention—support cycles lengthen (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Treat decode benchmarks as shipping artifacts, not lab one-offs (Chen and YiRage Contributors, 2026; AMD, 2024a,b; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Shape buckets. Compile separate static graphs for each (batch, ctx) class you ship; never extrapolate from a single prefill graph (AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Bucket count trades compile storage for decode predictability—document which bucket each SKU image selects at runtime (Chen and YiRage Contributors, 2026; AMD, 2024b; Davies *et al.*, 2025; AMD, 2024a; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Closing metric. When in doubt, re-run the decode cell: if DDR bytes/token and p99 ms/token both improve, the static graph change is real; if only FLOPs move, reject the merge (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Handoff to Part IV. Chapter 6 generalizes residency budgeting across GPU, CPU, and NPU; carry forward the XDNA counters defined here as mandatory fields in every cross-hardware decode report (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Benchmarks without DDR byte counters are incomplete for Ryzen AI decode qualification today (AMD, 2024b; Davies *et al.*, 2025; Chen, 2026a).

5.10 Key Takeaways

- **The tile grid is fixed and the SRAM is small.** XDNA appears in Ryzen AI NPUs paired with x86 cores, with a per-SKU compute-tile grid and tens of kilobytes of deterministic tile SRAM rather than a large banked shared memory (AMD, 2024b,a). Engineers who oversize tiles “because HBM is huge” fail XDNA legality immediately, so the compiler must size to the tile budget up front (NVIDIA, 2022).
- **Static dataflow shifts work to compile time.** The host configures graph runners and submits execute calls but must not allocate or copy full tensors per token; KV growth appends through pre-planned DDR regions (AMD, 2024b; Davies *et al.*, 2025). There is no runtime scheduler, so the entire tile-and-DMA plan is fixed before execution and the per-token cost is deterministic (AMD, 2024a).
- **Map from CUDA by translating, not transplanting.** GPU shared memory is explicitly sized and banked while tile SRAM is smaller but deterministic, so a CUDA schedule cannot be transplanted—it must be re-tiled to the AIE fabric (NVIDIA, 2022; AMD, 2024b). Online softmax in particular must preserve a stable accumulation order across spatial reductions to stay numerically equivalent to the reference (Dao *et al.*, 2022, 2023).
- **Optimize watt/token, not just ms/token.** Edge products advertise watt/token while cloud products advertise ms/token under load, so XDNA wins on power efficiency where CUDA wins on elastic throughput (AMD, 2024b; Davies *et al.*, 2025). The right target is the one whose binding constraint—power and locality versus peak latency—matches the deployment (Vellaisamy *et al.*, 2026).
- **Gate XDNA decode on static legality.** Review gates must confirm no simultaneous buffers exceed per-tile SRAM caps and no dynamic dimensions appear on the hot path without a bucket recompile (AMD, 2024a; Chen and YiRage Contributors, 2026). YiRage compiles the same decoder export for CUDA and XDNA on every merge and publishes the legality diffs, so a residency violation is caught at build time (Chen and YiRage Contributors, 2026; Chen, 2020).

5.11 Conclusion

AMD XDNA/AIE shows the opposite end of the architecture spectrum from the CUDA GPU of Chapter 4: a spatial-dataflow fabric of fixed compute tiles with small deterministic SRAM, no hardware cache, and a statically configured schedule, where decode latency is bought with compile-time tile and DMA planning rather than runtime flexibility (AMD, 2024b,a). The lesson is that the same decoder math demands a fundamentally different compilation per class, and a schedule transplanted from the GPU is illegal here—which is precisely why residency must be planned, not assumed (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). With both hardware extremes now mapped, Chapter 6 opens Part IV’s MegaKernel techniques by making on-chip data residency a budgeted, checkable property across all three classes (Chen, 2026a; Vellaisamy *et al.*, 2026).

Chapter 6

Data Residency and On-Chip Memory Design

Every decode bottleneck in this book reduces to one question: which bytes get to stay on chip, and for how long. Parts I–II established that batch-1 decode is bandwidth- and orchestration-bound, not FLOP-bound; this chapter turns that diagnosis into a design discipline—planning the lifetime and residency of every tensor against the concrete on-chip memory each target actually has (NVIDIA, 2022; AMD, 2024a; Dao *et al.*, 2022). The numbers from Chapter 1 are the motivation: dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 and OLMoE-1B/7B averages 9,305, and each unfused boundary is a tensor that left the chip and came back (Vellaisamy *et al.*, 2026). Removing host overhead alone bought $1.259\times$ on a Qwen-2.5-7B decode cell, but it could not fix illegal HBM traffic—only residency design can (Chen, 2026a; NVIDIA, 2024). The mistake to avoid is treating on-chip memory as an optimization to apply after the kernel works. On-chip residency is the kernel’s contract with the hardware, and it must be planned before codegen, per target, because the same tensor that fits in a GPU’s shared memory overflows an NPU tile’s SRAM (SemiAnalysis, 2024; AMD, 2024b).

6.1 Lifetime planning

The first discipline is **lifetime** planning: for every intermediate tensor, determine when it is produced, when it is last consumed, and therefore where it should live for that span (Goodfellow *et al.*, 2016). A tensor whose producer and consumer



Figure 6.1: Lifetime planning assigns each tensor a residency tier for the span between its production and last use, then reuses the buffer.

are adjacent in a fused region never needs to touch device memory at all; a tensor consumed many operators later, or by a different thread block, has a longer lifetime and may be forced to a slower tier. Getting this wrong is the single most common cause of “mysterious” decode regressions, because a tensor with an unnecessarily long lifetime silently occupies fast memory that another tensor needed, or spills to HBM and reintroduces the bandwidth tax the fusion was meant to remove (Vellaisamy *et al.*, 2026; Chen, 2026a).

Decode makes lifetime planning unusually tractable and unusually important. Tractable, because at batch-1 the per-token working set is small and regular: a query vector, the weight tiles for the current layer, and the KV cache slice for the current head (Kwon *et al.*, 2023). Important, because that working set is read in full every single token, so any byte that lives one tier slower than it could pays its penalty thousands of times per sequence (Davies *et al.*, 2025). The **residency** decision for the KV cache is the canonical example: it is too large to stay in the fastest tiers, so the design question becomes how much of it to stream and how to overlap that streaming with compute, not whether to keep it resident.

The KV cache deserves its own lifetime analysis because it breaks the simple producer-consumer pattern. Unlike an activation that is produced and consumed within a token, the KV cache is written once and then read on every subsequent token for the rest of the sequence, so its lifetime spans the entire generation. That long lifetime is why it cannot live in the fastest tiers and why decode is fundamentally a streaming problem: each token must read back the growing KV history, and the residency design question is not whether to keep it on chip but how to overlap its streaming with the per-token compute so the load latency is hidden (Kwon *et al.*, 2023; Dao *et al.*, 2023). A residency plan that ignores this and treats the KV slice like a short-lived activation will either run out of fast memory or stall waiting for the load.

Double buffering is the lifetime technique that makes streaming work, and it is worth naming because it appears on every hardware class. The idea is to give a streamed tensor two physical buffers so the engine can load tile $n+1$ into one buffer while compute consumes tile n from the other, then swap. This doubles the buffer’s footprint but overlaps transfer with compute, turning a sequence of load-then-compute stalls into a pipeline (SemiAnalysis, 2024; Dao *et al.*, 2022). The

lifetime planner must account for the doubled footprint when it checks capacity, which is the first place a naive plan and a real one diverge—a fused region that fits single-buffered may overflow once double buffering is added for the streamed operands.

A useful way to think about lifetime planning is as a static memory allocation problem solved at compile time rather than by a runtime allocator. If the compiler knows every tensor’s lifetime, it can assign a small pool of physical buffers and reuse each one across non-overlapping lifetimes—exactly the static allocation and copy elimination that compilers like Glow perform on their low-level IR, and that a megakernel must perform to keep its working set bounded (Rotem *et al.*, 2018; Wu *et al.*, 2024). The payoff is twofold: the allocator never appears in the per-token hot path, and the peak **residency** footprint is known before the kernel runs, so it can be checked against the target’s on-chip capacity instead of discovered by a runtime out-of-memory error.

There is a subtlety worth flagging: lifetimes are not fixed by the math, they are shaped by the schedule. The same dataflow graph can be scheduled so that a tensor’s producer and consumer are adjacent—giving it a short lifetime and on-chip residency—or so that unrelated work is interleaved between them, stretching its lifetime and forcing it to a slower tier (Wu *et al.*, 2024; Goodfellow *et al.*, 2016). This is why residency planning and scheduling cannot be separated: the schedule that minimizes launches may lengthen a critical tensor’s lifetime past what fast memory can hold, and the schedule that keeps everything resident may serialize work that could have overlapped. The planner therefore co-optimizes the two, choosing an execution order that keeps the highest-reuse tensors’ lifetimes short enough to stay resident while still exposing enough overlap to hide streaming latency (Vellaisamy *et al.*, 2026; Chen, 2026a).

The chapters that follow apply this discipline per hardware class, because the tiers themselves differ. What does not differ is the planning method: identify lifetimes, rank tensors by reuse-per-byte, and place the highest-reuse tensors in the fastest tier that can hold them for their lifetime, spilling only what genuinely cannot fit (Goodfellow *et al.*, 2016; Dao *et al.*, 2022).

6.2 GPU residency

On a GPU the fast tiers are concrete and small, and the design must respect their exact sizes. On an NVIDIA H100 (Hopper) SM, the **register** file is 65,536 32-bit registers—256 KB per SM—with a hard ceiling of 255 registers per thread; **shared** memory is configurable up to 228 KB per SM, of which 227 KB is addressable by a



Figure 6.2: H100 on-chip residency tiers per SM: registers and shared memory are the fast, capacity-limited tiers a decode kernel must budget against.

single thread block after CUDA reserves 1 KB; and the combined L1-plus-**shared** capacity is 256 KB per SM, backed by a 50 MB GPU-wide L2 cache before HBM (NVIDIA, 2022). These are not trivia: they are the constraints a decode kernel’s residency plan is solved against.

The **register**/occupancy trade is the first one a decode kernel hits. Because resident threads share the 65,536-register file, a kernel that uses R registers per thread caps resident threads near $65536/R$; at 128 registers per thread an SM holds at most 512 threads (NVIDIA, 2022). A heavily fused decode kernel that keeps many partial sums in registers therefore trades occupancy for residency, and the right balance is workload-specific—at batch-1 there is little parallelism to hide latency, so keeping the working set in registers often beats chasing occupancy, up to the point where a register spill to local memory undoes the win.

Shared memory is where the canonical decode fusion lives. FlashAttention keeps the query tile, the relevant key/value tiles, and the running softmax statistics in **shared** memory and registers, computing attention without ever materializing the full score matrix to HBM (Dao *et al.*, 2022). The residency arithmetic is direct: at hidden size 4096 in BF16 each activation vector is 8 KB, so a fused decoder layer that keeps a handful of intermediates on chip avoids tens of kilobytes of HBM traffic per token—traffic that, multiplied across 32 layers and every generated token, dominates batch-1 latency (Vellaisamy *et al.*, 2026; Chen, 2026a). Hopper adds mechanisms that make residency cheaper to exploit: TMA performs asynchronous bulk copies into **shared** memory that overlap with compute, and distributed shared memory lets thread blocks in a cluster read each other’s shared memory directly rather than through L2 (SemiAnalysis, 2024; NVIDIA, 2022). The design lever is to size tiles so the hot working set fits in the 228 KB budget and to use the async copy path so streaming the KV slice overlaps the matmul rather than stalling it.

The bank structure of **shared** memory is a second constraint the residency plan must respect. H100 shared memory is divided into 32 banks, each 4 bytes wide, and consecutive 4-byte words map to consecutive banks; a warp that accesses 32 addresses falling in distinct banks completes in one transaction, while accesses that collide on a bank serialize into multiple transactions (NVIDIA, 2022). A residency plan that places a tile in shared memory but lays it out so threads stride across the same bank turns a fast on-chip access into a serialized one, which at

batch-1 decode—where the kernel runs thousands of times per sequence—is a measurable bytes/token penalty. The layout of a resident tensor is therefore part of the residency decision, not an afterthought: tile shapes and padding are chosen so the hot access pattern is conflict-free.

A worked register budget makes the occupancy trade concrete. Suppose a fused decode kernel keeps a query vector, several accumulator registers for the running softmax, and a weight fragment live in registers, totaling 96 registers per thread. With the 65,536-register file that allows roughly $65536/96 \approx 682$ resident threads per SM, comfortably above one full thread block; push the kernel to 200 registers per thread to fuse more work and resident threads fall to about 327, below a 512-thread block, so occupancy drops (NVIDIA, 2022). At batch-1 that occupancy loss is often acceptable because there is no batch parallelism to hide latency anyway, but only up to the spill threshold: once a thread needs more than 255 registers the compiler spills to local memory, which is HBM-backed, and the residency win is lost (Vellaisamy *et al.*, 2026; Chen, 2026a). The design rule is to fuse aggressively in registers until just before the spill, then stop.

The L2 cache is the last on-chip line of defense. At 50 MB it can hold reused weight tiles or KV lines across thread blocks, and Hopper exposes L2 residency controls so a kernel can bias frequently reused data to stay cached (NVIDIA, 2022). But L2 is shared GPU-wide and far slower than shared memory, so relying on it is a fallback, not a plan—the residency design still aims to keep the per-token hot set in registers and **shared** memory.

6.3 CPU residency

On a CPU the residency tiers are the **cache** hierarchy, and the design discipline shifts from explicit placement to cache-aware blocking. Unlike GPU shared memory, which the programmer manages directly, CPU L1/L2/L3 caches are hardware-managed, so residency is controlled indirectly by the access pattern: tile the computation so the working set of each tile fits in a target **cache** level and is reused before eviction (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018). For decode this means blocking the per-token GEMV and attention so the weight tile and KV slice currently in use stay in L1 or L2 across their reuse, rather than streaming the whole layer through the **cache** once and evicting it.

The second CPU-specific concern is **NUMA**. On multi-socket or multi-die servers, memory is partitioned across nodes, and a thread accessing memory on a remote **NUMA** node pays a latency and bandwidth penalty (Goodfellow *et al.*, 2016). For a memory-bound decode workload that re-reads the full weight

set every token, a **NUMA**-oblivious placement—weights allocated on one node, decode threads scheduled on another—can silently halve effective bandwidth. The residency plan on a CPU therefore includes thread and memory affinity: pin decode threads to the **NUMA** node that owns the weights they read, and replicate read-only weights per node when the bandwidth saving exceeds the memory cost (Davies *et al.*, 2025).

The bandwidth arithmetic explains why this matters so much for decode on a CPU. At batch-1 the model re-reads its entire weight set from memory every token, so the achievable token rate is bounded by weight bytes divided by effective memory bandwidth—a memory-bound regime where a 7B model in BF16 must move roughly 14 GB per token from wherever the weights live (Kwon *et al.*, 2023; Chen, 2026a). If a **NUMA**-oblivious schedule forces half those reads across the inter-node link at a fraction of local bandwidth, the token rate drops proportionally, and no amount of compute optimization recovers it. This is the CPU analogue of the GPU’s launch storm: the bottleneck is bytes moved through the wrong tier, and the fix is a residency and placement decision, not a faster inner loop.

Hardware prefetchers complicate the **cache** residency picture in a way GPU shared memory does not. A CPU prefetcher will speculatively pull data into **cache** based on detected access patterns, which helps a streaming weight read but can also evict a small, hot tensor the kernel wanted to keep resident—so the blocking strategy must consider not just what fits in a **cache** level but what the prefetcher will displace (Goodfellow *et al.*, 2016). The practical consequence is that CPU residency tuning is more empirical than GPU shared-memory tuning: the plan is validated by measuring **cache** miss rates at the decode operating point rather than by a static capacity check alone.

The multi-hardware contrast with the GPU is instructive. A GPU kernel decides residency explicitly and statically through **shared**-memory allocation; a CPU kernel decides it implicitly through blocking and affinity, trusting the hardware **cache** to keep the blocked working set resident. The same fused decoder layer therefore wants a different schedule on each: the GPU schedule maximizes **shared**-memory reuse and async copy overlap, while the CPU schedule maximizes **cache** locality and respects **NUMA** boundaries—identical math, different residency mechanism, different bytes/token outcome (Vellaisamy *et al.*, 2026; Chen, 2026a).

6.4 NPU residency

On a spatial **NPU** the residency model is the most explicit of all, and it must be decided entirely at compile time. On the AMD XDNA2 / AIE-ML fabric, each

compute tile has 64KB of L1 local data memory—eight 8KB banks in hardware, presented to the programmer as four interleaved 16KB banks—with a dedicated DMA engine, and each column adds a 512KB L2 **SRAM** memory tile shared by its compute tiles, with shim tiles bridging to off-chip DDR (AMD, 2024a,b). There is no hardware cache that opportunistically keeps data resident; what lives in **SRAM** is exactly what the compiler’s DMA schedule placed there, and nothing else.

This makes **NPU** residency design both unforgiving and powerful. Unforgiving, because a tensor that does not fit the 64KB tile **SRAM** must be split across tiles or staged through the 512KB memory tile, and the DMA chain to do so is part of the program, not a runtime fallback (AMD, 2024b). Powerful, because once the residency plan fits, there is no cache-miss variance and no allocator—the per-token cost is deterministic. The canonical decode fusion on this fabric keeps the attention tiles resident in L1/L2 **SRAM** and streams only inputs and final outputs through DDR, mapping the query-key-softmax-value chain as a single spatially tiled sweep so that intermediate scores never leave on-chip **SRAM** (AMD, 2024a; Dao *et al.*, 2022). Tiling parameters are chosen to maximize L1 reuse subject to the buffer constraint that the resident tiles fit the tile **SRAM**.

Double buffering on the **NPU** is both more explicit and more essential than on the GPU. Because there is no cache to absorb a late load, a compute tile that waits for its next operand simply idles, so the DMA schedule must ping-pong between two buffers in the 64KB tile **SRAM**—loading the next tile via the tile’s DMA engine while the vector unit consumes the current one (AMD, 2024a,b). This halves the usable tile capacity for the double-buffered operand, which is precisely the kind of constraint the residency plan must enforce up front: on a 64KB tile, a double-buffered weight tile plus a double-buffered activation tile plus the accumulators must all fit, and if they do not, the tiling parameters shrink until they do.

The spatial structure adds a residency dimension the GPU and CPU lack: data can move between neighboring tiles without going off chip. On the XDNA2 fabric, compute tiles in an array can read each other’s local memory directly and pass data through cascade connections, so a large operand can be distributed across several tiles’ **SRAM** and consumed as a spatial pipeline rather than streamed from DDR (AMD, 2024a,b). A residency plan that exploits this maps a fused attention sweep across a row or column of tiles, keeping the intermediate scores entirely in on-chip **SRAM** and sending only the inputs and final outputs through DDR—the spatial analogue of a GPU megakernel, with the same goal of minimizing off-chip bytes per token (Dao *et al.*, 2022; Chen, 2026a).

Table 6.1: On-chip residency tiers by hardware class; the fast tiers a decode kernel must budget against differ in size and in who manages them (NVIDIA, 2022; AMD, 2024a).

Class	Fast on-chip tier	Managed by
GPU (H100 SM)	228 KB shared, 256 KB registers	Programmer + HW L2
CPU	L1/L2/L3 cache	Hardware (blocking-tuned)
NPU (XDNA2 tile)	64 KB L1, 512 KB L2 SRAM	Compiler DMA (explicit)

The contrast across the three classes is now sharp. The GPU offers a large, programmer-managed **shared** memory plus a hardware L2 fallback; the CPU offers hardware-managed caches tuned by blocking; the **NPU** offers small, fully explicit **SRAM** with compiler-scheduled DMA and no fallback. A residency plan that is optimal on one is illegal or wasteful on another, which is exactly why on-chip memory design cannot be a single portable template (SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

6.5 YiRage memory pass

The book’s reference compiler turns this discipline into a checkable pass. **YiRage** carries a **memory**-residency pass that encodes the per-class residency contract—GPU shared-memory and register budgets, CPU cache-blocking and NUMA constraints, NPU tile-SRAM capacity—as IR metadata, then verifies a candidate schedule against the target’s actual on-chip **memory** before codegen (Chen and YiRage Contributors, 2026). A fusion that would exceed the 228 KB shared-memory budget on an H100, or overflow a 64 KB XDNA tile, is rejected at compile time rather than discovered as a spill in production telemetry (NVIDIA, 2022; AMD, 2024a).

This is the mechanism behind the residency claims made throughout the book. When earlier chapters said a schedule must “keep the layer on chip,” the **YiRage memory** pass is what makes that statement enforceable: it computes the peak resident footprint of a fused region from the lifetime plan, compares it to the target tier capacity, and either admits the fusion or splits it until it fits (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Because the check is static, it composes with the megakernel search of Chapter 20—the superoptimizer proposes a fusion, and the **memory** pass certifies that its residency is legal on the target before the kernel is ever emitted (Wu *et al.*, 2024; Mirage Project contributors, 2025).

Concretely, the pass operates on the lifetime plan rather than on the source

program, which is what lets it reason about peak footprint correctly. It walks the fused region, tracks which buffers are simultaneously live at each point, accounts for double-buffered operands at their doubled size, and takes the maximum across the region as the peak resident footprint (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). That maximum is compared against the target tier capacity—228 KB on an H100 SM, 64 KB on an XDNA tile—and if it exceeds the budget the pass has two legal responses: split the region so each piece fits, or demote the lowest-reuse tensor to a slower tier and accept its bandwidth cost (NVIDIA, 2022; AMD, 2024a). Both are deterministic compile-time decisions, so the same model compiled for two SKUs gets two different but provably-legal residency plans.

The pass also closes a gap that pure search leaves open. A superoptimizer that proposes a megakernel is optimizing for performance and may propose a fusion that is faster in principle but illegal on a given tier; without a residency check, that fusion ships and spills in production (Wu *et al.*, 2024; Mirage Project contributors, 2025). By making residency a verified precondition, the **YiRage memory** pass lets the search be aggressive—propose any fusion—while guaranteeing that whatever is emitted respects the target’s on-chip **memory**, which is the separation of concerns that makes searched megakernels safe to deploy across a heterogeneous fleet (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

The result is that residency stops being a property a kernel author hopes for and becomes one the compiler guarantees. For a serving team that is the difference between a decode kernel that is fast on the SKU it was tuned on and one that is provably within budget on every SKU in the fleet (Davies *et al.*, 2025; Chen, 2026a).

6.6 Key Takeaways

- **Plan tensor lifetimes before codegen.** Determine each tensor’s production-to-last-use span and place it in the fastest tier that holds it for that lifetime; static allocation keeps the allocator out of the per-token path and makes the peak footprint checkable (Goodfellow *et al.*, 2016; Rotem *et al.*, 2018). A too-long lifetime silently spills (Vellaisamy *et al.*, 2026).
- **Budget against real GPU tier sizes.** An H100 SM has 256 KB of registers (255/thread max) and up to 228 KB of shared memory; fused decode kernels trade occupancy for residency and must fit the hot set in these budgets (NVIDIA, 2022). FlashAttention keeps tiles in shared memory to avoid HBM traffic (Dao *et al.*, 2022).

- **CPU residency is blocking plus NUMA.** Caches are hardware-managed, so residency is controlled by cache-blocked tiling and by pinning decode threads to the NUMA node owning their weights (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). A NUMA-oblivious placement can halve effective bandwidth on a memory-bound decode loop (Chen, 2026a).
- **NPU residency is explicit and compile-time.** An XDNA2 tile has 64 KB L1 SRAM and a 512 KB column L2, with compiler-scheduled DMA and no cache fallback, so what is resident is exactly what the schedule placed (AMD, 2024a,b). Tiles are sized to fit the SRAM buffer constraint (SemiAnalysis, 2024).
- **Make residency a checkable compiler pass.** The YiRage memory pass encodes per-class budgets as IR metadata and rejects fusions that exceed on-chip capacity before codegen, composing with megakernel search (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). Residency becomes a guarantee the compiler proves, not a hope the kernel author holds (Mirage Project contributors, 2025).

6.7 Conclusion

On-chip residency is the concrete form every decode optimization in this book eventually takes: plan each tensor’s lifetime, rank by reuse-per-byte, and place the hot working set in the fastest tier that can hold it on the target you actually ship (Goodfellow *et al.*, 2016; NVIDIA, 2022). The tiers differ sharply—programmer-managed shared memory and registers on a GPU, hardware caches plus NUMA on a CPU, explicit DMA-scheduled SRAM on an NPU—so the same fused layer wants a different residency plan per class, and a green build on one says nothing about residency on another (AMD, 2024a; Chen, 2026a). The YiRage memory pass makes the plan enforceable by checking peak footprint against tier capacity before codegen, so a fusion that would spill is split or demoted rather than shipped (Chen and YiRage Contributors, 2026). With residency now a budgeted, checkable property, Chapter 7 turns to the next MegaKernel technique built on it: static role assignment and schedule-free kernels that fix each thread block’s job at compile time so the resident working set never moves (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Chapter 7

Static Role Assignment and Schedule-Free Kernels

Chapter 6 made on-chip residency a budgeted, checkable property; this chapter spends that budget on a specific structural choice—assigning each execution unit a fixed role at compile time so the kernel needs no runtime scheduler at all (Shah *et al.*, 2024; Mirage Project contributors, 2025). The motivation is again the launch storm: dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 and OLMoE-1B/7B averages 9,305, and most of that orchestration exists only because the work-to-hardware mapping is decided dynamically, per operator, by the host (Vellaisamy *et al.*, 2026). If instead the mapping is fixed once, at compile time, the host stops dispatching and the kernel becomes “schedule-free”—a single resident program that already knows what every block and warp should do (Chen, 2026a; NVIDIA, 2024). The mistake to avoid is assuming static assignment sacrifices flexibility for speed. On the contrary, fixing roles is what lets the compiler reason about residency and overlap precisely, because it knows in advance which unit produces what and when (Dao *et al.*, 2022; Wu *et al.*, 2024). This chapter develops static role assignment on GPUs and dataflow NPUs, shows how it eliminates runtime dispatch, and ends with the YiRage tiling pass that assigns roles per target.

7.1 Static roles

Static role assignment means deciding, at compile time, what each execution unit does for the entire life of the kernel, rather than letting a runtime scheduler



Figure 7.1: Static role assignment fixes each unit’s job at compile time—producer warps move data, consumer warps compute—so no runtime scheduler is needed.

reassign work dynamically (Shah *et al.*, 2024). On a GPU the canonical instance is warp specialization: the warps of a thread block are partitioned into producer warps that only ever issue data movement and consumer warps that only ever issue computation (Shah *et al.*, 2024). FlashAttention-3 makes this concrete on Hopper—producer warps drive the Tensor Memory Accelerator (TMA) to move tiles from global to shared memory, while consumer warps drive Warpgroup Matrix-Multiply-Accumulate (WGMMMA) on those tiles—and the paper notes that this separation “improves the compiler’s ability to generate optimal instruction schedules” (Shah *et al.*, 2024).

Static role assignment is, in compiler terms, a form of specialization: the kernel is compiled knowing the role of each unit, so branches that would otherwise select behavior at runtime are resolved away, and each unit’s instruction stream contains only the instructions for its role (Shah *et al.*, 2024). This is why the FA3 authors observe that the separation improves the compiler’s ability to schedule instructions—a producer warp’s stream is all loads and barrier signals, a consumer warp’s stream is all matmuls and barrier waits, and neither contains the other’s instructions to get in the way (Shah *et al.*, 2024). A unified, non-specialized warp would interleave both and force the instruction scheduler to reason about dependencies that the static split makes structural.

The reason a fixed **role** helps is mechanical sympathy with asynchrony. Modern GPU data movement and compute are asynchronous: TMA copies and WGMMMA matmuls run on separate hardware paths and signal completion through barriers (SemiAnalysis, 2024; NVIDIA, 2022). If a single warp had to both load and compute, it would stall waiting on its own loads; by giving each warp one **static role**, the producer can run ahead filling buffers while the consumer drains them, and the two overlap naturally. This is the same double-buffering idea from Chapter 6, but expressed as a division of labor among units rather than among buffers.

Static **role** assignment is not GPU-specific; it is the native model of a dataflow NPU. On the AMD XDNA fabric each compute tile has a fixed **role** in a spatial pipeline, with its DMA engine and vector unit assigned a specific stage of the computation at compile time, and data flowing between neighboring tiles through configured streams (AMD, 2024b,a). There is no runtime that decides which tile does what—the array is configured before execution and holds that configuration.

The GPU and NPU therefore arrive at the same place from opposite directions: the GPU adds static specialization on top of a dynamically scheduled machine, while the NPU is statically scheduled by construction. This convergence is a useful design signal: when two very different architectures both reward fixing roles at compile time, it suggests the benefit is fundamental to memory-bound, low-parallelism workloads rather than an artifact of one machine’s quirks (AMD, 2024a; Shah *et al.*, 2024). The book’s recurring thesis—that decode is bound by movement and orchestration, not arithmetic—predicts exactly this, because static assignment attacks orchestration directly while leaving the arithmetic untouched (Vellaisamy *et al.*, 2026; Chen, 2026a).

It is worth being precise about what “schedule-free” does and does not mean. It does not mean the kernel performs no scheduling—there is still an order in which a consumer warp processes its tiles. It means the *assignment* of work to units is not recomputed at runtime: no host scheduler decides which kernel runs next, and no hardware block scheduler streams fresh CTAs through the machine per operator (Shah *et al.*, 2024; Mirage Project contributors, 2025). The distinction matters because the two kinds of scheduling have very different costs. Ordering within a resident unit is nearly free; reassigning work across units, especially when it requires a host round trip, is the expensive part, and it is the part static assignment removes (Vellaisamy *et al.*, 2026).

A second clarification is why this is specifically a decode technique. During prefill there is abundant parallelism—many prompt tokens processed at once—so a dynamic scheduler has plenty of independent work to keep units busy and its overhead amortizes across a large batch (Kwon *et al.*, 2023). At batch-1 decode there is exactly one token of work per step, so any scheduling overhead is paid against a tiny amount of useful computation, and the ratio of overhead to work is at its worst (Chen, 2026a; Davies *et al.*, 2025). Static role assignment is therefore not a universal speedup; it is the right structure precisely when parallelism is scarce and overhead dominates, which is the decode regime this book targets.

Taken to its limit, static **role** assignment produces a megakernel. Mirage Persistent Kernel assigns every streaming multiprocessor a **role** in an SM-level task graph and runs the entire model forward pass as one kernel, with the SMs cooperating through in-kernel scheduling rather than returning to the host between operators (Mirage Project contributors, 2025; Wu *et al.*, 2024). The **static** structure is what makes this possible: because each SM’s sequence of tasks is known at compile time, there is no need to re-launch or re-dispatch, and the megakernel reduces per-token launches from thousands to one (Vellaisamy *et al.*, 2026; Chen, 2026a). The lesson that carries forward is that flexibility at runtime is precisely the



Figure 7.2: The Hopper thread hierarchy; static role assignment maps work to this structure at the warp and warpgroup granularity.

thing decode cannot afford—every dynamic decision is overhead paid per token.

7.2 Thread mapping

Assigning roles requires knowing the **thread** hierarchy precisely, because roles are assigned at its granularities. On Hopper the hierarchy runs from threads, to **warps** of 32 threads, to warpgroups of 4 contiguous **warps**, to thread blocks (CTAs), to thread-block clusters, to the grid (Shah *et al.*, 2024). Each **thread** has at most 256 registers private to it; shared memory is addressable by all threads in a CTA; and CTAs in a cluster are co-scheduled on the same GPC with fast access to each other’s shared memory (Shah *et al.*, 2024; NVIDIA, 2022). These groupings are not bookkeeping—they are the units to which roles are bound.

The warpgroup is the key granularity for decode kernels because WGMMMA, the high-throughput Hopper matmul, is issued collectively by a warpgroup of four **warps** (Shah *et al.*, 2024). Warp specialization therefore typically assigns one warpgroup the consumer **role** (issuing WGMMMA) and dedicates other **warps** to the producer **role** (issuing TMA). A subtlety that matters for residency is register reallocation: because producers issuing TMA need few registers while consumers running WGMMMA need many, Hopper allows warpgroup-wide register reallocation to give the consumer warpgroup a larger share of the register file (Shah *et al.*, 2024). This is static role assignment paying off directly in the residency budget of Chapter 6—the fixed roles let the compiler skew registers toward the units that need them.

Thread mapping also determines how overlap is achieved within and across warpgroups. FlashAttention-3 overlaps the softmax computation of one warpgroup with the GEMM of another using pingpong scheduling driven by synchronization barriers, reporting an improvement from roughly 580 to 640 TFLOPS in BF16 from that inter-warpgroup overlap alone (Shah *et al.*, 2024). The reason overlap is necessary is a throughput mismatch: in the FA3 analysis the special-function exponentials of softmax take about half the cycles of the tensor-core matmul, so leaving them un-overlapped idles the tensor cores for a substantial fraction of the time (Shah *et al.*, 2024). Static role assignment is what makes the overlap

schedulable, because the compiler knows which warpgroup is doing softmax and which is doing GEMM and can place the barriers accordingly.

The cluster level adds a mapping option that did not exist before Hopper and that static assignment can exploit. Thread-block clusters group several CTAs that are co-scheduled on the same GPC and can read each other’s shared memory directly through distributed shared memory, and TMA loads can be multicast to all CTAs in a cluster in one operation (NVIDIA, 2022; Shah *et al.*, 2024). For a decode kernel that means a producer role can load a weight tile once and multicast it to the consumers in several CTAs of a cluster, rather than each CTA loading its own copy—a residency win that is only schedulable because the cluster membership and the producer/consumer split are both fixed in advance. Mapping work to the cluster granularity is thus another static decision the compiler makes, trading a larger co-scheduled group for cheaper data sharing.

There is an interplay between role mapping and the occupancy budget of Chapter 6 that the compiler must resolve jointly. Dedicating warps to a producer role consumes warp slots that could otherwise run consumers, so the split is not free: too many producers and the consumers starve for compute slots, too few and the consumers stall waiting on data (Shah *et al.*, 2024; NVIDIA, 2022). The right ratio depends on the throughput mismatch between data movement and compute for the specific kernel, which is exactly the kind of target- and shape-specific decision that argues for a compiler pass over a hand-tuned constant. At batch-1 decode, where the matmul is a thin matrix-vector product, the balance often tilts toward fewer compute warps and more attention to keeping them fed, the opposite of a throughput-shaped prefill kernel (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

On a CPU or NPU the mapping granularities differ but the principle holds. A CPU maps roles to **threads** and SIMD lanes, pinning a decode **thread** to a core and a NUMA node as Chapter 6 described; an NPU maps roles to tiles in its array, with each tile a fixed stage (Goodfellow *et al.*, 2016; AMD, 2024a). In every case the mapping is chosen so that the units that must communicate are close in the hierarchy—warps in a CTA sharing shared memory, tiles adjacent in the array sharing a stream—so that the data a role produces reaches the role that consumes it without leaving fast memory (SemiAnalysis, 2024; AMD, 2024b).

7.3 No runtime scheduling

The payoff of static roles is that the kernel needs no **runtime** scheduler, which is where the decode latency is won. In a conventional stack the host decides, per operator, which kernel to launch and on what data, paying a **dispatch** cost for

Table 7.1: Where work-to-unit **dispatch** happens, and the per-token cost it implies for batch-1 decode (Vellaisamy *et al.*, 2026; Shah *et al.*, 2024).

Model	Dispatch decided	Per-token launch cost
Per-operator kernels	Host, at runtime	Thousands of launches
Persistent + tile scheduler	In-kernel, at runtime	Fixed CTA count
Static megakernel (MPK)	Compile time	One launch

every one of the thousands of launches a token requires (Vellaisamy *et al.*, 2026). A schedule-free kernel moves that decision to compile time: the work-to-unit mapping is baked in, so the only **runtime** activity is the computation itself (Mirage Project contributors, 2025; Shah *et al.*, 2024).

The persistent kernel is the GPU mechanism that eliminates **runtime dispatch** for the multi-tile case. Instead of launching one CTA per work tile and relying on the hardware’s block scheduler to stream them, a persistent kernel launches a fixed number of CTAs equal to the SM count—132 CTAs on an H100 SXM5, one per SM—and each CTA loops over a fraction of the work tiles, assigned by an in-kernel tile scheduler (Shah *et al.*, 2024). This decouples physical CTAs from logical work, so the kernel can overlap the epilogue of the current tile with the prologue loads of the next, and avoids re-incurring launch and prologue cost for every tile (Shah *et al.*, 2024). The tile scheduler is a small piece of in-kernel logic, not a host round trip, so the **dispatch** decision never leaves the GPU.

Mirage Persistent Kernel takes the elimination of **runtime dispatch** to the whole model. Its SM-level task graph is built at compile time, and an in-kernel parallel **runtime** executes the tasks with decentralized scheduling across SMs, so the entire forward pass runs as one launch with no host involvement between operators (Mirage Project contributors, 2025). The reported $1.2\text{--}6.7\times$ end-to-end latency reduction over kernel-per-operator serving is, in large part, the cost of **runtime dispatch** that the static structure removed (Mirage Project contributors, 2025; Chen, 2026a). The same principle is the NPU’s default: because the array is configured before execution, there is no **runtime** that decides tile assignment, and the per-token cost is whatever the static schedule fixed (AMD, 2024b,a).

The tile scheduler deserves a closer look because it is where the small amount of necessary dynamism lives. In a persistent kernel the scheduler is in-kernel logic that hands the next work tile to whichever resident CTA is free, which both balances load and lets the kernel decouple the number of physical CTAs from the number of logical tiles (Shah *et al.*, 2024). CUTLASS generalizes this with schemes like Stream-K, where the scheduler can split a tile’s reduction across CTAs and fall

back to simpler data-parallel or Split-K modes by heuristic, and on Hopper it constructs the schedule in units of clusters rather than single tiles (Shah *et al.*, 2024). The important property for decode is that all of this stays on the GPU: the scheduler is a few instructions of in-kernel arithmetic, not a host callback, so it removes launch overhead without reintroducing dispatch latency.

Contrast this with the NPU, where even the in-kernel scheduler disappears. Because the XDNA array is configured before execution and each tile holds a fixed stage, there is no per-token assignment decision at all—the “schedule” is the physical configuration of the fabric, and running a token is just clocking data through it (AMD, 2024b,a). This is the most extreme point on the static-to-dynamic axis: the GPU per-operator stack recomputes assignment thousands of times per token, the persistent kernel computes it in-kernel a fixed number of times, the megakernel computes it once at compile time, and the static NPU never computes it at runtime at all (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025). Each step down that axis trades flexibility for fewer per-token decisions, and decode rewards moving as far down it as the workload’s regularity allows.

There is an honest limit to schedule-free kernels. A fully static assignment can load-balance poorly when work is irregular—the persistent tile scheduler exists precisely to redistribute uneven work, such as the variable row counts under causal masking, while staying in-kernel (Shah *et al.*, 2024). The design judgment is how much dynamism to keep: enough to balance genuinely irregular work, but not so much that the host re-enters the loop. For batch-1 decode, where the per-token work is regular and small, the answer is to keep almost nothing dynamic, which is why decode is the workload where schedule-free kernels pay off most (Kwon *et al.*, 2023; Davies *et al.*, 2025).

A practical consequence for serving teams is that the win compounds with model depth. Each transformer layer that a per-operator stack executes adds its own quota of launches and dispatch decisions, so a deep model pays the orchestration tax once per layer per token; a static megakernel pays it once per token regardless of depth (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025). This is why the reported megakernel speedups are largest for the deepest, most launch-dense models—the very OLMoE-style mixture-of-experts configurations that issue 9,305 launches per token in the TaxBreak measurement (Vellaisamy *et al.*, 2026). The static structure converts a cost that scaled with the number of operators into a constant, which is the kind of asymptotic improvement that a faster inner loop can never deliver (Chen, 2026a).

7.4 YiRage tiling

The book’s reference compiler turns static role assignment into a per-target pass. **YiRage** carries a **tiling** pass that, given a fused region and a target, chooses tile shapes and assigns each execution unit its static role—which warps are producers and consumers on a GPU, which tiles are which stage on an NPU—and emits the corresponding schedule-free kernel (Chen and YiRage Contributors, 2026). Because the residency budget of Chapter 6 and the role assignment of this chapter are decided together, the **tiling** pass can guarantee that the chosen tiles fit the on-chip memory of the target while the role assignment keeps producers and consumers overlapped (Wu *et al.*, 2024; Shah *et al.*, 2024).

The **tiling** decision is genuinely different per target, which is why it must be a pass and not a fixed template. On a GPU the pass picks a tile size that fits the 228 KB shared-memory budget, assigns a consumer warpgroup for WGMMA and producer warps for TMA, and reallocates registers toward the consumer (NVIDIA, 2022; Shah *et al.*, 2024). On an XDNA NPU the same logical computation is tiled to fit the 64 KB tile SRAM, with double-buffered DMA and a fixed tile-to-stage assignment (AMD, 2024a,b). The math is identical; the **tiling** and role assignment are not, and a plan transferred blindly from one to the other is either illegal or wasteful (SemiAnalysis, 2024).

The pass must also decide how far to fuse before assigning roles, because the fusion boundary and the role split are coupled. Fuse too little and the kernel is back to per-operator launches with their dispatch tax; fuse too much and the combined working set blows the residency budget or the register file, forcing a spill that negates the win (NVIDIA, 2022; Chen, 2026a). The **tiling** pass resolves this by growing the fused region while continuously checking the residency metadata, stopping at the largest region whose static role assignment still fits the target’s fast memory (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). This is the same stopping rule Chapter 6 stated for fusion in general, now applied with the role assignment in hand: the boundary sits where one more fused operator would force either a spill or a producer/consumer imbalance that starves the tensor cores. Because the check is per target, the boundary lands in a different place on an H100 than on an XDNA tile, which is exactly why a single hand-tuned kernel cannot be the answer across a heterogeneous fleet (AMD, 2024a; Davies *et al.*, 2025).

The pass treats tile-shape selection as a constrained search rather than a lookup. For a GPU consumer the tile must be large enough to amortize the WGMMA setup and keep the tensor cores busy, small enough that the producer can keep its double buffers filled within the shared-memory budget, and shaped so the access pattern

is bank-conflict-free—three constraints that interact, so the pass explores shapes rather than applying a fixed one (Shah *et al.*, 2024; NVIDIA, 2022). For an NPU tile the dominant constraint is simply fitting the double-buffered operands in the 64 KB local SRAM, after which the pass picks the largest tile that maximizes reuse (AMD, 2024a,b). The output of the search is not just a number but a complete static plan: tile shape, producer/consumer warp split, register allocation, and the barrier placement that realizes the overlap.

This is also where the **tiling** pass interacts with quantization and precision. A lower-precision tile—FP8 or int8—moves fewer bytes and fits more elements per buffer, which loosens the residency constraint and can change the optimal tile shape and even the producer/consumer ratio (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). The pass therefore cannot choose tiles independently of the numeric format; it must co-decide them, because the format determines how much of the working set is resident and therefore how the roles should be balanced. For a memory-bound decode kernel this coupling is the whole game: the format choice and the **tiling** together set bytes/token, and the role assignment sets whether the movement of those bytes overlaps the compute (Vellaisamy *et al.*, 2026; Chen, 2026a).

What makes the **YiRage tiling** pass composable with the rest of the book’s machinery is that it consumes the same residency metadata the memory pass produces and feeds the megakernel search of Chapter 20. The superoptimizer proposes a fusion; the memory pass certifies its footprint fits; the **tiling** pass assigns static roles and tiles that realize the fusion as a schedule-free kernel on the target (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024; Mirage Project contributors, 2025). The result is that “static role assignment” stops being a hand-coding technique a CUDA expert applies and becomes a compiler decision that is made, checked, and re-made per target automatically.

7.5 Key Takeaways

- **Fix roles at compile time to remove runtime scheduling.** Assigning each unit a static role—producer or consumer on a GPU, a fixed stage on an NPU—lets the kernel run without a host scheduler, removing the per-token dispatch that dominates batch-1 (Shah *et al.*, 2024; Vellaisamy *et al.*, 2026). Flexibility at runtime is overhead paid per token (Chen, 2026a).
- **Warp specialization overlaps data movement and compute.** Producer warps issuing TMA and consumer warpgroups issuing WGMMA overlap

naturally, and pingpong scheduling overlapped softmax with GEMM for 580→640 TFLOPS in FlashAttention-3 (Shah *et al.*, 2024). Static roles are what make the overlap schedulable (SemiAnalysis, 2024).

- **Map roles to the right hierarchy granularity.** Roles bind to warps and warpgroups on a GPU (WGMMMA is warpgroup-collective), threads and SIMD lanes on a CPU, and tiles on an NPU; co-locate communicating roles so produced data reaches its consumer on chip (Shah *et al.*, 2024; AMD, 2024a). Register reallocation skews the file toward consumers (NVIDIA, 2022).
- **Persistent kernels keep dispatch in-kernel.** Launching CTAs equal to the SM count (132 on H100 SXM5) with an in-kernel tile scheduler eliminates per-tile launch overhead and overlaps epilogue with prologue (Shah *et al.*, 2024). MPK extends this to one launch for the whole model, reporting 1.2–6.7× end-to-end latency reduction over kernel-per-operator serving (Mirage Project contributors, 2025).
- **Make role assignment a per-target compiler pass.** The YiRage tiling pass chooses tiles and assigns static roles per target, composing with the residency memory pass and megakernel search (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). The same math gets a different, checked schedule on each class, and the fusion boundary lands wherever the target’s residency budget allows (AMD, 2024b).

7.6 Conclusion

Static role assignment is the structural move that turns Chapter 6’s residency budget into a schedule-free kernel: fix what each warp, warpgroup, or tile does at compile time, and the runtime scheduler—and its per-token dispatch cost—disappears (Shah *et al.*, 2024; Mirage Project contributors, 2025). Warp specialization and persistent kernels show the GPU path, the XDNA array shows the natively static NPU path, and MPK shows the limit case of one launch per token; in every case the win is that dynamism removed at compile time is overhead not paid at runtime (Vellaisamy *et al.*, 2026; Chen, 2026a). The YiRage tiling pass makes the assignment a per-target compiler decision rather than a hand-coding feat, choosing tile shapes and role splits that fit each target’s residency budget automatically (Chen and YiRage Contributors, 2026). Static roles, however, only pay off if the data each role needs arrives on time, which is the subject of Chapter 8: TMA double-buffer pipelines and the asynchronous movement that keeps producer and

consumer roles fed, so the statically assigned consumers never stall waiting on data (SemiAnalysis, 2024; Dao *et al.*, 2022).

Chapter 8

TMA Double-Buffer Pipelines and Async Movement

Chapter 7 fixed roles so the consumer units never wait on a scheduler; this chapter makes sure they never wait on data either, by pipelining asynchronous movement so the next tile is already arriving while the current one is consumed (Shah *et al.*, 2024; SemiAnalysis, 2024). The stakes are the decode metrics established in Chapter 1: at batch-1 the model re-reads weights and the growing KV cache every token, so the load path, not the matmul, sets ms/token, and any cycle a compute unit spends idle waiting for a load is a cycle of pure latency (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Dense Llama-3.2-1B already issues 847.5 launches per token on H100 and OLMoE-1B/7B issues 9,305; collapsing those launches only pays off if the resulting fused kernel keeps its compute units fed (Chen, 2026a; NVIDIA, 2024). The mistake to avoid is treating data movement as synchronous—load, then compute, then load. That serializes the two and exposes the full memory latency every step. The fix, across every hardware class, is a **double-buffer pipeline**: overlap the movement of tile $n+1$ with the computation on tile n , so latency is hidden behind useful work (Dao *et al.*, 2022; AMD, 2024a).

8.1 Pipeline theory

The theory of a software **pipeline** is simple and general: split a stream of work into stages, give each stage its own buffer, and run the stages concurrently so that while one stage consumes buffer A , the previous stage fills buffer B (Goodfellow *et al.*, 2016; Dao *et al.*, 2022). The idea is borrowed directly from hardware instruction



Figure 8.1: A double-buffer pipeline: after a one-tile prologue, each compute step overlaps with the load of the next tile, hiding memory latency behind compute.

pipelines and from classical producer-consumer queueing, and the same intuition applies: throughput is set by the slowest stage, and latency is hidden as long as there is buffered work for the consumer to do while the producer runs ahead.

The reason a **pipeline** is the right frame for decode specifically is that decode is a stream of nearly identical, small units of work—one token’s layer computation after another, each reading a predictable working set (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Streaming, regular work is the ideal case for pipelining: the stages are stable, the buffer sizes are known, and the depth can be fixed at compile time. This is in contrast to irregular, data-dependent work where the next tile’s size or location is not known until the current one finishes, which is harder to pipeline and is one reason prefill and decode want different schedules (Dao *et al.*, 2023; Chen, 2026a).

Double-buffering is the two-stage instance—one buffer being computed on, one being loaded—and it is the minimum needed to overlap movement with compute. The cost is doubled buffer footprint, which is exactly the residency charge Chapter 6 told the planner to account for; the benefit is that, after a one-tile prologue, the memory latency of every subsequent load is hidden behind the compute of the previous tile (SemiAnalysis, 2024; Chen, 2026a).

Two quantities govern a **pipeline**’s effectiveness. The first is depth: a deeper **pipeline**, with more than two buffers in flight, hides more latency because more loads can be outstanding, but it consumes proportionally more fast memory (Shah *et al.*, 2024). The second is the balance between the stages: if the load stage is slower than the compute stage, the **pipeline** is memory-bound and stalls no matter how deep it is, and if compute is slower, the loads complete early and the extra depth is wasted. For batch-1 decode the load stage usually dominates—the matmul is a thin matrix-vector product while the load is a full weight tile—so the design goal is to make the load as asynchronous and as overlapped as the hardware allows, then size the **pipeline** depth to the available fast memory (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

A worked latency argument shows why depth matters. Suppose a tile load takes 400 cycles of memory latency and the compute on a tile takes 300 cycles. With no pipeline, each step costs $400 + 300 = 700$ cycles, and the unit is idle 57%

of the time. With a two-stage **double-buffer**, the steady-state step cost drops to $\max(400, 300) = 400$ cycles—the load and compute overlap, so the step is bounded by the slower stage—and the memory latency is fully hidden behind compute only if compute were the longer stage (SemiAnalysis, 2024; Dao *et al.*, 2022). Here the load is longer, so the **pipeline** is memory-bound at 400 cycles per tile, and adding a third buffer does not help because the bottleneck is load bandwidth, not the number of outstanding loads. This is the quantitative form of the balance rule: deepen the **pipeline** only until the bottleneck stage saturates, then stop.

The same arithmetic explains a counter-intuitive decode result. Because decode is memory-bound, the best a **pipeline** can do is hide the compute behind the load, which means the achievable token latency is bounded below by the time to stream the per-token working set—the weights and KV slice—no matter how clever the overlap (Vellaisamy *et al.*, 2026; Chen, 2026a). Pipelining cannot beat that floor; it can only ensure the kernel reaches it instead of paying load-plus-compute serially. This is why Chapter 6’s residency work and this chapter’s movement work are complementary: residency reduces how many bytes must be streamed, and pipelining ensures the bytes that are streamed overlap with compute, and only together do they approach the memory-bandwidth floor (Kwon *et al.*, 2023; Davies *et al.*, 2025).

The crucial enabling property is asynchrony. A **pipeline** only overlaps if the load can be issued and then proceed in the background while the issuing unit moves on to compute; a synchronous copy that blocks until it completes cannot overlap with anything (SemiAnalysis, 2024). Every hardware class therefore provides some asynchronous movement primitive, and the chapters that follow are really a tour of those primitives—TMA on a GPU, prefetch and async copy on a CPU, DMA on an NPU—and of how a compiler builds a correct **double-buffer pipeline** on top of each. The common requirement is a completion signal: the consumer must know when buffer B is full before it reads it, so each primitive pairs with a barrier or token that the **pipeline** uses to sequence the stages (Shah *et al.*, 2024; AMD, 2024a).

8.2 GPU TMA

On a **Hopper** GPU the asynchronous movement primitive is the Tensor Memory Accelerator (**TMA**), and it is purpose-built for the **double-buffer pipeline** this chapter describes. **TMA** performs asynchronous bulk copies between global and shared memory, and crucially it is issued by a single thread rather than by every thread in a warp, which frees the other threads’ registers and address-generation



Figure 8.2: On Hopper, TMA issues asynchronous bulk copies from HBM into a double-buffered shared-memory region that WGMMA consumers drain.

work for compute (Shah *et al.*, 2024). This is why FlashAttention-3 assigns **TMA** to a dedicated producer warp: one thread kicks off the bulk load, the producer moves on, and the consumer warpgroup runs WGMMA on the previously loaded tile (Shah *et al.*, 2024).

Completion is signaled through transaction barriers. A **TMA** copy is associated with a shared-memory barrier that tracks how many bytes have arrived, and the consumer waits on that barrier before reading the buffer; because the barrier is transaction-based, the hardware knows when the full tile has landed rather than relying on a thread count (Shah *et al.*, 2024; NVIDIA, 2022). This is the completion signal the **pipeline** theory required, and it is what lets the compiler build a correct multi-stage pipeline: each buffer has its own barrier, the producer fills buffer $n+1$ and arrives at its barrier while the consumer waits on buffer n ’s barrier, and the two proceed in lockstep one tile apart.

TMA also carries optimizations that matter for decode residency. It can multicast a single load to the shared memory of all CTAs in a thread-block cluster in one operation, so a weight tile read once is delivered to several consumers without re-reading HBM—a direct bytes/token saving when the same weights feed multiple CTAs (Shah *et al.*, 2024; NVIDIA, 2022). It also exposes an L2 promotion hint that biases the cache toward keeping **TMA**-loaded lines resident when the same input tile is consumed by many CTAs, as in a GEMM where each weight column feeds every row (SemiAnalysis, 2024; NVIDIA, 2022). Both are mechanisms the **pipeline** builder uses to ensure that the bytes a tile needs are either already on chip or arriving asynchronously, never fetched synchronously on the critical path.

There is a register dimension to **TMA** that is easy to miss and directly relevant to the residency budget. On older async-copy paths every thread in a warp participated in address generation, consuming registers and instruction slots that then could not be used for compute (Dao *et al.*, 2022). Because **TMA** is issued by a single thread with a hardware-resident multidimensional descriptor, it frees the rest of the warp’s registers, and combined with Hopper’s warpgroup-wide register reallocation from Chapter 7 this lets the consumer warpgroup claim a larger share of the 65,536-register file for accumulators (Shah *et al.*, 2024; NVIDIA, 2022). The movement primitive and the residency budget are thus coupled: choosing **TMA** is not only a movement decision but a register decision, and a **pipeline** built on it

can sustain deeper compute tiles than one built on the older per-thread copy.

The interaction with the transaction barrier is what makes deep **TMA** pipelines correct rather than merely fast. Each buffer in the **pipeline** gets its own barrier object in shared memory; the producer thread issues the **TMA** for buffer k and records the expected byte count on that buffer’s barrier, and the consumer warpgroup waits on barrier k before reading buffer k (Shah *et al.*, 2024). With s buffers the producer can run up to s tiles ahead, and the barriers form a circular list that sequences the stages—a structure CUTLASS encapsulates in reusable pipeline classes precisely because managing the barriers by hand is error-prone (Shah *et al.*, 2024; NVIDIA, 2022). For a compiler this is ideal: the barrier discipline is mechanical and can be generated, which is exactly what the YiRage **pipeline** pass does.

The predecessor primitive, `cp.async`, shipped on Ampere and established the pattern **TMA** perfected: asynchronously copy from global to shared memory and commit groups of copies that the consumer can wait on (Dao *et al.*, 2022). **TMA** improves on it by handling multidimensional tile descriptors in hardware, issuing from one thread, and integrating with the transaction barrier, which is why the deepest decode pipelines target **Hopper’s TMA** path (Shah *et al.*, 2024; SemiAnalysis, 2024).

A practical caution applies when reasoning about **TMA** for decode rather than prefill. **TMA**’s headline benefits—bulk multidimensional copies, multicast across a cluster, L2 promotion for reused inputs—are largest when the same tile feeds many consumers, the GEMM-like sharing pattern of prefill (SemiAnalysis, 2024; NVIDIA, 2022). At batch-1 decode the reuse is lower: a weight tile is read once per token by a thin matrix-vector product, so the multicast and L2-promotion wins shrink and the dominant benefit reduces to the plain asynchrony—issuing the load early and overlapping it (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The design lesson is to use **TMA** for what decode actually needs (cheap asynchronous loads that free registers) without assuming the prefill-shaped sharing wins transfer, which is another instance of the book’s theme that prefill intuitions mislead at the decode operating point.

8.3 CPU DMA

A **cpu** has no programmer-managed scratchpad and no **TMA**, so its version of the **double-buffer pipeline** is built from prefetching and the cache hierarchy rather than explicit asynchronous copies into shared memory (Goodfellow *et al.*, 2016). The hardware prefetcher already overlaps movement with compute oppor-

tunistically: on a detected sequential access pattern it pulls the next cache lines ahead of demand, which for a streaming weight read is effectively a **pipeline** the hardware builds for free (Goodfellow *et al.*, 2016). The kernel’s job is to make the access pattern predictable enough for the prefetcher to track and to block the computation so the working set of each tile stays cache-resident across its reuse, as Chapter 6 described.

When the hardware prefetcher is insufficient, software prefetch instructions let the kernel issue an explicit hint to begin loading a cache line some distance ahead of its use, which is the **cpu** analogue of issuing an asynchronous load early in a **pipeline** (Goodfellow *et al.*, 2016). The distance—how many iterations ahead to prefetch—plays the role of **pipeline** depth: too short and the load does not complete before the data is needed, too long and the prefetched line is evicted before use or displaces something hot. Tuning that distance against the memory latency and the per-iteration compute is the **cpu** equivalent of choosing the number of buffers on a GPU.

The absence of a scratchpad changes what “double buffering” even means on a **cpu**. There is no second explicit buffer to fill; instead the two stages share the cache, and the prefetch distance determines how far ahead the next tile’s lines are brought in before eviction pressure or the prefetcher’s own heuristics stop it (Goodfellow *et al.*, 2016). This makes the **cpu pipeline** softer than a GPU’s—there is no hard barrier guaranteeing buffer B is full, only the statistical likelihood that the prefetched lines arrived—so correctness is never at risk (a cache miss just stalls) but performance is more variable. The practical implication is that **cpu** decode tuning is validated empirically by measuring cache-miss rates and achieved bandwidth at the decode operating point, rather than by reasoning about a fixed **pipeline** depth as on a GPU.

Some server CPUs add a dedicated **dma**-style data-movement engine that can copy memory regions asynchronously and offload bulk transfers from the cores, which brings the **cpu** closer to the explicit asynchronous model of a GPU or NPU (Goodfellow *et al.*, 2016). Where available, such an engine lets a decode kernel stage a weight or KV block while the cores compute, an explicit **double-buffer** between memory tiers. But the dominant reality on a **cpu** is that residency and overlap are governed by the cache and prefetcher, so the **pipeline** is largely implicit; the multi-hardware contrast with the GPU is that the GPU programmer builds the pipeline explicitly with **TMA** and barriers, while the **cpu** programmer shapes access patterns and prefetch distance to coax the hardware into building an equivalent one (SemiAnalysis, 2024; Chen, 2026a). For a decode kernel this means the highest-leverage **cpu** move is often simply ensuring the weight stream

is contiguous and sequentially accessed so the prefetcher tracks it perfectly, rather than hand-rolling an explicit **double-buffer** that the hardware would have built anyway (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023). The compiler’s job on a **cpu** is therefore as much about laying out data for predictable access as about scheduling explicit transfers.

8.4 NPU pipeline

On a spatial **npu** the **double-buffer pipeline** is the most explicit of all and is the native execution model. Each AMD XDNA / AIE-ML compute tile has a dedicated **dma** engine that moves data between streams and the tile’s 64 KB local SRAM, and the architecture provides task queues and task-completion-tokens so the **dma** can run ahead of compute and signal when a buffer is ready (AMD, 2024a,b). The completion token is the **npu**’s version of the GPU’s transaction barrier: the vector unit waits on the token before consuming a buffer, and the **dma** fills the other buffer in the meantime.

Because there is no cache to absorb a late load, double buffering is not an optimization on an **npu**—it is mandatory. A compute tile that waits on its **dma** simply idles, so the schedule ping-pongs between two buffers in the tile SRAM: load buffer *B* via the **dma** engine while the vector unit consumes buffer *A*, then swap, with the task-completion-tokens sequencing the swap (AMD, 2024a,b). This halves the usable tile SRAM for the double-buffered operand, which is the residency charge the planner of Chapter 6 must include, and it is why the tile shapes are sized so that two copies of the streamed operand plus the accumulators fit the 64 KB budget.

The AIE-ML **dma** is more capable than a simple copy engine, and those features map directly onto **pipeline** needs. It supports task queues, so multiple transfers can be enqueued and run back-to-back without per-transfer setup from the vector unit, and task-completion-tokens, so the consumer can wait on a specific transfer’s completion rather than a coarse fence (AMD, 2024a). It also offers multidimensional addressing, which lets a single **dma** task gather a strided tile—a row of a weight matrix, say—without the vector unit computing addresses, the **npu** analogue of **TMA**’s multidimensional descriptor (AMD, 2024a,b). These features exist because on a fabric with no cache the **dma** must do all the work of keeping the compute fed, so it is engineered to run a deep, autonomous **pipeline** with minimal vector-unit involvement.

The spatial structure gives the **npu pipeline** a dimension the GPU and **cpu** lack: the stages can be different physical tiles. A producer tile can stream data

Table 8.1: Asynchronous-movement primitive and completion signal per hardware class; all build the same double-buffer pipeline (Shah *et al.*, 2024; AMD, 2024a).

Class	Async primitive	Completion signal
GPU (Hopper)	TMA bulk copy	Transaction (mbarrier)
CPU	Prefetch / DMA engine	Cache fill / engine status
NPU (XDNA)	Tile DMA engine	Task-completion-token

into a neighbor’s local memory, which computes and forwards to the next tile, so a multi-stage **pipeline** is laid out across the array with each tile a fixed stage and the **dma**/stream interconnect carrying data between them (AMD, 2024b,a). This is the spatial analogue of a deep software **pipeline**: instead of one unit cycling through buffers, several units form an assembly line, and the on-chip **dma** keeps each stage fed. The decode payoff is the same as everywhere—intermediate results never leave on-chip SRAM, so only inputs and final outputs traverse DDR (Dao *et al.*, 2022; Chen, 2026a).

The trade the spatial **pipeline** makes is depth for latency and balance. A pipeline spread across k tiles can keep k stages busy and hide more latency, but it also has a longer fill and drain, and it requires the per-stage work to be balanced so no tile becomes the bottleneck that starves the rest (AMD, 2024b,a). Because the assignment of stages to tiles is fixed at configuration time, the compiler must balance the stages statically—splitting a heavy stage across two tiles, or fusing two light stages onto one—which is the spatial counterpart to choosing a GPU **pipeline**’s depth. Getting this balance wrong does not merely slow the kernel; on a fabric with no cache fallback it idles whole tiles, so the **npu pipeline** demands the most careful static planning of the three classes, and rewards it with the most deterministic per-token cost (SemiAnalysis, 2024; Davies *et al.*, 2025).

8.5 YiRage pipeline generation

The book’s reference compiler turns pipeline construction into a generated, per-target artifact. **YiRage** carries a **pipeline**-generation pass that, given a fused region and a target, emits the correct **double-buffer pipeline**: **TMA** producer warps plus transaction barriers on **Hopper**, prefetch-shaped access patterns on a **cpu**, and ping-pong **dma** with completion tokens on an XDNA **npu** (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Because the residency budget and the role assignment are already fixed by the passes of Chapters 6 and 7, the

pipeline pass knows how much fast memory is available for buffers and which units are producers, so it can choose the **pipeline** depth that fits and wire the completion signals correctly (Wu *et al.*, 2024; AMD, 2024a).

Pipeline depth is the parameter the pass tunes per target. On a **Hopper** GPU with a 228 KB shared-memory budget it can afford several **TMA** buffers in flight, hiding more latency; on a 64 KB XDNA tile it can usually afford only the mandatory two, so the **pipeline** is shallow and the tile shapes shrink accordingly (NVIDIA, 2022; AMD, 2024a). Getting this wrong in either direction is a measurable decode regression: too shallow and the compute units stall on loads, too deep and the buffers crowd out the working set and force a spill that reintroduces the HBM traffic the **pipeline** was meant to hide (Vellaisamy *et al.*, 2026; Chen, 2026a). The pass therefore solves depth jointly with the residency check rather than applying a fixed constant.

The pass also has to reason about a subtlety the hardware exposes: the **pipeline** prologue and epilogue. The first $s-1$ tiles of an s -deep **pipeline** only fill buffers without producing output (the prologue), and the last $s-1$ steps only drain buffers without issuing new loads (the epilogue); these transient phases are not overlapped and their cost is fixed per kernel invocation (Shah *et al.*, 2024; Dao *et al.*, 2022). For a long-running prefill GEMM the prologue cost is negligible against thousands of steady-state steps, but for a short batch-1 decode kernel that processes few tiles per token the prologue can be a meaningful fraction of the runtime. This is precisely why persistent kernels from Chapter 7 matter here: by keeping the kernel resident across tokens, the **pipeline** stays warm and the prologue is paid once rather than per token, so movement pipelining and static residency reinforce each other (Shah *et al.*, 2024; Mirage Project contributors, 2025).

A second decision the pass makes is whether to pipeline the KV-cache stream separately from the weight stream. The two have different lifetimes and sizes—weights are reused across all positions of a layer while the KV slice grows with context—so a single **pipeline** depth is rarely optimal for both (Kwon *et al.*, 2023; Dao *et al.*, 2023). The pass can assign separate buffers and depths to each stream, deepening the KV **pipeline** when context is long and the KV read dominates, and the weight **pipeline** when the model is wide and weight streaming dominates. This per-stream tuning is the kind of decision that is tedious and error-prone by hand but mechanical for a compiler that knows the shapes and the residency budget, and it is where a generated **pipeline** can beat a hand-written one that uses a single uniform depth (Chen and YiRage Contributors, 2026; Chen, 2026a).

What makes the **YiRage pipeline** pass composable is that it is the movement counterpart to the static role assignment of Chapter 7: roles decide which unit

does what, and the **pipeline** pass decides when each unit’s data arrives (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Together they realize a fused region as a schedule-free, fully overlapped kernel, and they compose with the megakernel search of Chapter 20—the superoptimizer proposes the fusion, the memory pass certifies its footprint, the tiling pass assigns roles, and the **pipeline** pass overlaps the movement, all per target and all checked before codegen (Wu *et al.*, 2024; Mirage Project contributors, 2025). The result is that the asynchronous, double-buffered pipeline a CUDA expert once hand-wrote becomes a compiler output, regenerated correctly for each SKU in a heterogeneous fleet.

8.6 Key Takeaways

- **Overlap movement with compute via a double-buffer pipeline.** A synchronous load-then-compute loop exposes full memory latency every step; a double-buffer pipeline hides it behind the previous tile’s compute after a one-tile prologue, and deepening the pipeline helps only until the bottleneck stage saturates (Dao *et al.*, 2022; SemiAnalysis, 2024). At batch-1 decode the load stage dominates, so overlap is the win (Vellaisamy *et al.*, 2026).
- **TMA is Hopper’s asynchronous movement primitive.** Issued by a single thread (saving registers), TMA performs async bulk gmem→smem copies signaled by transaction barriers, with cluster multicast and L2 promotion hints (Shah *et al.*, 2024; NVIDIA, 2022). FlashAttention-3 pairs TMA producers with WGMMMA consumers (SemiAnalysis, 2024).
- **CPU pipelines are prefetch- and cache-shaped.** With no scratchpad, a CPU overlaps through hardware prefetch, software prefetch distance (its pipeline depth), and sometimes a DMA engine; the kernel shapes access patterns so the hardware builds the pipeline (Goodfellow *et al.*, 2016; Chen, 2026a). Residency is governed by the cache.
- **NPU double buffering is mandatory and explicit.** An XDNA tile’s DMA engine ping-pongs two buffers in 64 KB SRAM with task-completion-tokens, and multi-stage pipelines span physical tiles across the array (AMD, 2024a,b). With no cache fallback on the fabric, a stalled DMA simply idles the whole compute tile.
- **Generate the pipeline per target.** The YiRage pipeline pass emits TMA+barriers, prefetch patterns, or ping-pong DMA per class and solves pipeline depth jointly with the residency budget, with separate depths for

the weight and KV streams (Chen and YiRage Contributors, 2026; NVIDIA, 2022). Too shallow stalls; too deep spills (Vellaisamy *et al.*, 2026).

8.7 Conclusion

Asynchronous movement is what keeps the statically assigned consumers of Chapter 7 from stalling: a double-buffer pipeline overlaps the load of the next tile with the compute on the current one, hiding the memory latency that otherwise dominates batch-1 decode (Shah *et al.*, 2024; Chen, 2026a). Each hardware class provides the primitive—TMA with transaction barriers on Hopper, prefetch and DMA engines on a CPU, ping-pong tile DMA with completion tokens on an XDNA NPU—and the YiRage pipeline pass generates the correct, depth-tuned pipeline per target, with separate depths for the weight and KV streams where their sizes differ, composing with the residency, role, and search passes of the surrounding chapters (Chen and YiRage Contributors, 2026; AMD, 2024a; Wu *et al.*, 2024). A pipeline only works if its stages synchronize correctly, however, and at the scale of a megakernel spanning many units the synchronization itself becomes a design problem—which is the subject of Chapter 9: hierarchical synchronization and communication across threads, warps, tiles, and devices (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Chapter 9

Hierarchical Synchronization and Communication

The double-buffer pipelines of Chapter 8 only work if the producer and consumer agree on when a buffer is ready, and a megakernel that fuses a whole layer must coordinate hundreds of units without ever returning to the host. Synchronization is therefore not a detail at the bottom of the stack—it is the mechanism that makes static roles and overlapped movement safe, and getting it wrong is how a fused kernel either corrupts data or deadlocks (Shah *et al.*, 2024; NVIDIA, 2022). The decode stakes are unchanged: dense Llama-3.2-1B issues 847.5 launches per token on H100 and OLMoE-1B/7B issues 9,305, and the whole point of collapsing those launches is to keep coordination on chip rather than paying a host round trip per operator (Vellaisamy *et al.*, 2026; Chen, 2026a). The mistake to avoid is using a coarser synchronization scope than the dependency requires. A barrier that synchronizes a whole thread block when only two warps need to agree wastes cycles serializing units that could have run free; conversely, omitting a needed barrier is a correctness bug that surfaces as nondeterministic output (Dao *et al.*, 2022; NVIDIA, 2024). This chapter develops the synchronization hierarchy, shows how to pick the right primitive per hardware class, treats deadlock as a first-class hazard, and ends with the YiRage pass that inserts and checks synchronization per target.



Figure 9.1: The synchronization hierarchy: cost rises with scope, from a warp barrier to a multi-GPU collective. Synchronize at the narrowest scope a dependency requires.

9.1 Sync hierarchy

Synchronization on a GPU is hierarchical, mirroring the thread hierarchy of Chapter 7, and the central design rule is to **sync** at the narrowest scope that the dependency actually spans (Shah *et al.*, 2024). The reason a hierarchy exists at all is that coordination has a physical cost proportional to how far apart the units being coordinated are and how many of them there are, so a single universal synchronization primitive would be either too weak (unable to coordinate distant units) or too expensive (paying chip-crossing cost for a local handoff) (NVIDIA, 2022; Davies *et al.*, 2025). By exposing a ladder of scopes, the hardware lets software pay only for the coordination it needs.

The finest scope is the warp: `__syncwarp` coordinates the 32 threads of a warp and is nearly free because they already execute in lockstep. The next is the thread block: `bar.sync` (exposed as `__syncthreads`) is a **barrier** that all threads of a CTA must reach before any proceeds, and it is the workhorse of shared-memory kernels (NVIDIA, 2022). Between these sits the transaction **barrier** (`mbarrier`) of Chapter 8, which producer and consumer warps use to hand off a buffer without a full block **barrier** (Shah *et al.*, 2024).

The transaction **barrier** deserves emphasis as the scope that decode kernels live in. Unlike a block `bar.sync`, which forces every warp to a common point, a transaction **barrier** lets a producer warp signal “buffer k is full” to exactly the consumer that waits on it, so the two coordinate one tile apart while the rest of the kernel runs free (Shah *et al.*, 2024). This is the synchronization that makes the double-buffer pipeline of Chapter 8 possible: without it, overlapping a load with a compute would require a block **barrier** between them that defeats the overlap. The transaction **barrier** is thus the connective tissue of the whole MegaKernel design—it is how static roles hand work to each other without a coarse stall (NVIDIA, 2022; Dao *et al.*, 2022).

Above the block, Hopper adds cluster-scope synchronization. Thread-block clusters are co-scheduled on the same GPC and can synchronize through hardware-accelerated cluster **barrier** instructions, which is what lets the distributed-shared-memory sharing of Chapter 6 be coordinated safely (NVIDIA, 2022; Shah *et al.*, 2024). Coarser still is grid-scope synchronization across all CTAs of a kernel,

Table 9.1: The GPU synchronization hierarchy; cost rises steeply with scope, so a decode kernel coordinates at the narrowest scope a dependency spans (Shah *et al.*, 2024; Davies *et al.*, 2025).

Scope	Primitive	Relative cost
Warp	<code>__syncwarp</code>	Near-free (lockstep)
Producer/consumer	Transaction barrier	Two units, one tile apart
Block (CTA)	<code>bar.sync</code>	Stall to slowest warp
Cluster	Cluster barrier	Several co-scheduled CTAs
Grid / multi-GPU	Grid sync / collective	Chip- or network-crossing

available through cooperative-groups grid **sync** but expensive because it requires all CTAs to be resident, and coarsest is multi-GPU communication through collectives such as all-reduce and all-gather, which cross the device boundary and the interconnect (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

The cost gradient across these scopes is the whole reason the hierarchy matters. A warp **barrier** is a few cycles; a block **barrier** stalls until the slowest warp arrives; a cluster **barrier** spans several CTAs; a grid **sync** or a collective crosses the chip or the network and can cost orders of magnitude more (Shah *et al.*, 2024; Davies *et al.*, 2025). For decode this gradient is acute because the per-token work is tiny, so a coarse **sync** on the critical path is overhead with nothing to amortize it against. The pingpong scheduling of Chapter 7—overlapping one warpgroup’s softmax with another’s GEMM—is exactly a use of narrow-scope barriers to coordinate two units without a block-wide stall, and it is why that technique recovered tens of TFLOPS (Shah *et al.*, 2024).

A useful way to see why scope dominates cost is to think about how many units a **barrier** must wait for and how far apart they are physically. A warp **barrier** waits for 32 threads that already advance together, so it is essentially a no-op that the hardware can satisfy immediately (NVIDIA, 2022). A block **barrier** waits for every warp in the CTA, so it stalls until the straggler arrives—and at batch-1 decode, where warps may have unequal work, that straggler sets the pace. A cluster **barrier** additionally crosses CTA boundaries within a GPC, and a grid **sync** requires every CTA in the kernel to be co-resident and to reach the same point, which is why it is only usable in a persistent kernel and is still expensive. A multi-GPU collective leaves the chip entirely, traversing NVLink or a network fabric, so its latency is measured in microseconds against the nanoseconds of an on-chip **barrier** (Davies *et al.*, 2025; Chen, 2026a).

This gradient also explains why disaggregated decode deployments care so



Figure 9.2: Each hardware class provides different synchronization primitives; the compiler picks the one matching the dependency and the target.

much about where synchronization lands. When a model is split across devices for tensor parallelism, every layer may require an all-reduce to sum partial results, and that collective is on the per-token critical path (Davies *et al.*, 2025). The design response is the same producer-consumer overlap one scope coarser: overlap the collective with the next layer’s compute so the network latency hides behind useful work, exactly as a transaction **barrier** overlaps a TMA load with a WGMMA (Shah *et al.*, 2024; Goodfellow *et al.*, 2016). The synchronization hierarchy is thus self-similar—the same overlap principle applies at every scope, only the latency to hide grows.

9.2 Hardware sync pick

Picking the right synchronization primitive is target-specific, and the three classes differ as much in synchronization as they did in memory and movement. On a **gpu** the primitives are the barrier family above plus atomics for fine-grained updates; the design choice is which scope to use and whether to use a transaction barrier for asynchronous handoff or a full block barrier for a bulk-synchronous phase (Shah *et al.*, 2024; NVIDIA, 2022). CUTLASS wraps the producer/consumer barrier discipline in reusable asynchronous pipeline classes precisely because hand-managing a circular list of barriers across many asynchronous operations is too error-prone for a human, which is a strong hint that synchronization belongs in a compiler pass rather than in hand-written kernels (Shah *et al.*, 2024).

A subtlety specific to the **gpu** is the difference between a barrier and an atomic, and when each is the right tool. A **barrier** enforces that all participating units reach a point before any proceeds—a phase boundary—whereas an atomic enforces that a single memory update happens indivisibly, allowing units to coordinate without all stopping (NVIDIA, 2022). For a reduction where many warps contribute to one sum, an atomic add lets them proceed independently and merge results, avoiding a full **barrier**; for a phase change where the next stage cannot start until the previous fully completes, a **barrier** is required. Choosing atomics where possible keeps units running and is often the better fit for the irregular, low-parallelism reductions of batch-1 decode, while barriers remain necessary at genuine phase boundaries

(Shah *et al.*, 2024; Chen, 2026a). The transaction **barrier** is a hybrid—it is a **barrier** scoped to a producer-consumer pair, giving phase-boundary semantics without a block-wide stall.

On a **cpu** synchronization is built from memory fences, atomic operations, and the cache-coherence protocol rather than explicit barriers into a scratchpad (Goodfellow *et al.*, 2016). Threads coordinate through atomic flags and acquire/release fences that order memory operations, and the hardware’s coherence protocol makes a write by one core visible to another—at the cost of coherence traffic that, like NUMA, can dominate a memory-bound decode loop if threads share data across sockets. The **cpu** design choice is to minimize cross-core sharing on the hot path, partitioning work so that each thread’s data is mostly private and synchronization is rare, the opposite of a GPU kernel that synchronizes warps every pipeline stage.

On an **npu** synchronization is explicit and local: AIE-ML tiles use locks and task-completion-tokens, where the DMA signals a token when a buffer is ready and the vector unit waits on it before consuming, and neighboring tiles coordinate through configured streams and cascade connections (AMD, 2024a,b). Because the array is statically configured, the synchronization pattern is fixed at compile time—there is no dynamic contention for a shared lock as there might be on a CPU—which makes the **npu** the most deterministic of the three but also the least forgiving: a mis-sequenced token is a compile-time bug that stalls the pipeline rather than a rare race. Across collectives, multi-device decode must also pick a communication pattern—all-reduce for tensor-parallel sums, all-gather for sharded weights—and overlap it with compute, which is the same producer-consumer overlap one scope coarser (Davies *et al.*, 2025; Chen, 2026a).

The deeper contrast is between explicit and implicit synchronization, and it tracks the same axis as residency and movement. The **gpu** sits in the middle: barriers are explicit, but the hardware still schedules warps dynamically, so synchronization coordinates a partly dynamic machine. The **cpu** is the most implicit: the coherence protocol synchronizes caches automatically, and the programmer mostly reasons about ordering through fences rather than issuing explicit barriers, which is convenient but makes the synchronization cost hard to see and easy to incur accidentally through false sharing (Goodfellow *et al.*, 2016). The **npu** is the most explicit: every lock and token is placed by the compiler, nothing is automatic, and the cost is fully visible at compile time (AMD, 2024a,b). A residency-first engineer reads this as the same lesson once more—the more explicit the control, the more the compiler can reason about and minimize the coordination, which is why the statically scheduled **npu** is the most predictable and the coherence-driven **cpu** the most prone to hidden synchronization cost.

A concrete **cpu** hazard worth calling out is false sharing: two threads writing different variables that happen to share a cache line force the coherence protocol to ping the line between cores on every write, synchronizing data that did not logically need synchronizing (Goodfellow *et al.*, 2016). For a memory-bound decode loop this can quietly dominate, and the fix is a layout decision—pad per-thread state to separate cache lines—not a synchronization primitive. It is the **cpu** analogue of the GPU’s bank conflicts: an accidental sharing that the compiler must lay out around.

9.3 Deadlock detection

Static role assignment and asynchronous pipelines introduce a hazard that synchronous, per-operator kernels largely avoid: **deadlock**. When producer and consumer units coordinate through barriers, a mismatch in their arrive/wait counts, or a circular dependency where each unit waits for the other, hangs the kernel permanently (Shah *et al.*, 2024; NVIDIA, 2022). On a GPU a **deadlock** typically manifests as a kernel that never returns; on an NPU it manifests as a pipeline stage waiting forever on a token that will never be signaled (AMD, 2024a). The insidious property is that these failures are often nondeterministic or configuration-dependent: a barrier-count assumption that holds at one block size or warp count breaks at another, so a kernel that passes tests on one shape hangs on a production shape (Shah *et al.*, 2024; NVIDIA, 2022). This is the worst kind of bug to discover in a serving system, because it surfaces as a hung request under specific load rather than as a clean failure in testing, which is the strongest argument for proving deadlock-freedom statically rather than relying on test coverage.

Because these are coordination bugs, not arithmetic bugs, they do not show up in a numerical test—the kernel simply never finishes.

The classic **deadlock** patterns are worth naming because a compiler can check for them. The first is a barrier-count mismatch: if a barrier expects n arrivals but only $n-1$ units reach it (because one took a divergent path), the barrier never releases (Shah *et al.*, 2024). The second is a circular wait: producer waits for the consumer to free a buffer while the consumer waits for the producer to fill it, a cycle that only a correct initial condition (the producer starting with empty buffers) avoids. The third is an ordering bug in a multi-stage pipeline where the barriers are not arranged as a consistent circular list, so two stages wait on each other out of order (Shah *et al.*, 2024).

There is a reason these patterns proliferate exactly when performance improves. Each optimization in this Part trades a bulk-synchronous structure—compute

everything, synchronize, move on—for a finer-grained asynchronous one where units run ahead and coordinate point-to-point (Shah *et al.*, 2024; Dao *et al.*, 2022). Bulk-synchronous code is hard to deadlock because there is one barrier everyone hits; asynchronous, warp-specialized code has many barriers with specific arrive/wait counts, and every one is an opportunity for a mismatch. In other words, **deadlock** risk is the price of the overlap that makes decode fast, which is why a serious decode kernel cannot treat synchronization casually—the very structure that wins latency is the structure that can hang.

The role of control-flow divergence in **deadlock** deserves emphasis because it is the subtlest case. A **barrier** placed inside a conditional that only some warps take will wait forever for the warps that branched around it (NVIDIA, 2022; Shah *et al.*, 2024). This is why barriers must be placed at points all participating units are guaranteed to reach, and why a compiler that understands the control flow is better positioned to verify this than a human reading a complex kernel. On a statically scheduled **npu** the analogous bug is a token that one tile expects but no tile ever produces because a stage was conditionally skipped, and again the static schedule makes it checkable at compile time (AMD, 2024a,b).

The defense is to make synchronization structural rather than ad hoc. CUTLASS’s pipeline classes exist so that the arrive/wait counts and the circular barrier list are generated from a single specification, guaranteeing they match (Shah *et al.*, 2024). A compiler that owns synchronization can go further: it can verify statically that every barrier’s expected arrival count equals the number of units that reach it on all paths, that the producer-consumer cycle has a correct initial condition, and that the barrier ordering forms a consistent list—turning **deadlock** from a runtime hang discovered in production into a compile-time rejection (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). On a statically scheduled NPU this check is especially natural because the entire synchronization pattern is known at compile time (AMD, 2024b). The same staticness that makes an NPU unforgiving—no cache to hide a late load, no dynamic scheduler to reorder around a stall—is what makes its synchronization fully analyzable: with every lock, token, and stream assignment fixed before execution, the compiler can simulate the coordination and prove progress, something far harder on a dynamically scheduled GPU where warp progress depends on runtime occupancy (AMD, 2024a; NVIDIA, 2022). This is a recurring trade in the book: explicit, static control is harder to write by hand but easier for a compiler to verify, which is exactly why these facets belong in a pass.

9.4 YiRage sync pass

The book’s reference compiler turns synchronization into a generated, checked pass. **YiRage** carries a **sync** pass that, given a fused region with its static roles and pipeline, inserts the synchronization at the narrowest correct scope per target—warp and transaction barriers on a GPU, fences and atomics on a CPU, locks and tokens on an NPU—and verifies the result is deadlock-free before codegen (Chen and YiRage Contributors, 2026; Shah *et al.*, 2024). Because the pass consumes the role assignment of Chapter 7 and the pipeline of Chapter 8, it knows exactly which units must coordinate and when, so it can choose warp-scope over block-scope wherever the dependency permits and avoid the coarse-**sync** overhead that decode cannot afford (NVIDIA, 2022; Chen, 2026a).

Choosing the scope is itself an optimization the pass performs, not just a correctness step. Given the dependency graph of a fused region, the pass determines, for each handoff, the smallest set of units that actually share data and synchronizes only them—two warps through a transaction **barrier** rather than the whole block, or a cluster **barrier** only where distributed shared memory is genuinely used (Shah *et al.*, 2024; NVIDIA, 2022). This matters because a human writing a kernel tends to reach for the familiar block `bar.sync` even when a narrower primitive would do, leaving performance on the table; the pass has no such habit and picks the minimal scope mechanically. For batch-1 decode, where every coarse **sync** on the critical path is unamortized overhead, this minimization is a direct latency win (Vellaisamy *et al.*, 2026; Chen, 2026a).

The pass also adapts the synchronization to the precision and the pipeline depth, because those change what must be coordinated. A deeper pipeline has more buffers and therefore more transaction barriers in flight, and the pass must generate a consistent circular list of exactly the right length (Shah *et al.*, 2024). A lower-precision format that packs more elements per buffer can change which units produce and consume a given tile, shifting where the barriers belong. Because the **sync** pass runs after the tiling and pipeline passes have fixed these decisions, it has the information to place the barriers correctly, which is the argument for synchronization being a late, dependent pass rather than something hand-written up front (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

The pass’s verification is the part that matters most for a megakernel. A whole-model megakernel coordinates hundreds of units, and a single mismatched barrier hangs the entire forward pass, so shipping one without a static deadlock check is reckless (Mirage Project contributors, 2025; Wu *et al.*, 2024). The **YiRage sync** pass checks barrier arrival counts against the units on every control-flow

path, validates the producer-consumer initial conditions, and confirms the barrier ordering forms a consistent list, rejecting any region that cannot be proven deadlock-free (Chen and YiRage Contributors, 2026). This composes with the residency, role, and pipeline checks of the previous chapters: the superoptimizer proposes a fusion, the memory pass certifies its footprint, the tiling pass assigns roles, the pipeline pass overlaps movement, and the **sync** pass guarantees the coordination is correct and minimal—all per target, all before a kernel is emitted (Wu *et al.*, 2024; Mirage Project contributors, 2025).

The verification also has to reason about the multi-device boundary when a model is sharded. A tensor-parallel layer’s all-reduce is a synchronization point across devices, and a mismatch there—one rank expecting a collective that another never issues—hangs the whole job exactly like a barrier-count mismatch within a kernel, but now across the network (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The **sync** pass treats the collective as another synchronization edge in the dependency graph, checking that every participating rank issues a matching collective and that the overlap with compute does not create a cycle where a rank waits to send while a peer waits to receive. This unifies on-chip and cross-device coordination under one analysis, which matters because production decode increasingly spans many devices and a cross-device **deadlock** is even harder to debug by hand than an on-chip one (Davies *et al.*, 2025; Chen, 2026a).

A final benefit of owning synchronization in the compiler is reproducibility across targets. A hand-written kernel’s synchronization is tuned for one GPU and silently wrong or suboptimal on another—a barrier count assuming a specific warp count, say, or an atomic pattern that contends differently under another memory model (NVIDIA, 2022; AMD, 2024b). Because the **YiRage sync** pass regenerates the synchronization from the dependency graph per target, the same logical kernel gets a correct plan on each, with the scope and primitive chosen for that machine (Chen and YiRage Contributors, 2026). For a fleet that mixes GPU generations, CPUs, and NPUs, this is the difference between maintaining one verified specification and maintaining a matrix of hand-tuned, separately-debugged kernels.

The result is that synchronization, the most error-prone part of a hand-written fused kernel, becomes a property the compiler generates and proves rather than one a CUDA expert debugs by staring at hangs. For a serving team deploying megakernels across a heterogeneous fleet, that guarantee is what makes end-to-end fusion safe: the same logical kernel gets a different, minimal, deadlock-free synchronization plan on an H100, a CPU, and an XDNA tile, each checked before it runs (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

It is worth situating this within the book’s recurring argument. Chapters 6 through 9 have each taken one facet of a fused decode kernel—residency, role assignment, asynchronous movement, and now synchronization—and shown that the facet is best handled as a checked, per-target compiler pass rather than as hand-written CUDA, because each is a place where prefill intuitions mislead and where a single hand-tuned artifact fails to port (Vellaisamy *et al.*, 2026; Chen, 2026a). Synchronization completes the set: it is the facet that turns the other three from independent optimizations into a single correct kernel, because residency without coordination races, roles without coordination deadlock, and pipelines without coordination read stale buffers (Shah *et al.*, 2024; AMD, 2024a). Treating all four as composable passes over a shared representation is what lets a superoptimizer propose an aggressive fusion and have the compiler certify it is resident, role-assigned, overlapped, and deadlock-free before a single token is decoded (Wu *et al.*, 2024; Chen and YiRage Contributors, 2026).

9.5 Key Takeaways

- **Synchronize at the narrowest correct scope.** Cost rises steeply from a warp barrier to a block barrier to a cluster barrier to a grid sync or multi-GPU collective, so use the narrowest scope a dependency spans (Shah *et al.*, 2024; Davies *et al.*, 2025). A coarse sync on the decode critical path is pure overhead with nothing to amortize it against (Vellaisamy *et al.*, 2026).
- **Pick the primitive per hardware class.** GPUs use barriers, transaction barriers, and atomics; CPUs use fences, atomics, and cache coherence; NPUs use locks and task-completion-tokens; multi-device uses collectives (NVIDIA, 2022; AMD, 2024a). Minimize cross-core sharing and false sharing on a CPU; the NPU pattern is fixed at compile time (AMD, 2024b).
- **Treat deadlock as a first-class hazard.** Barrier-count mismatches, circular waits, and inconsistent barrier ordering hang a fused kernel without any numerical error (Shah *et al.*, 2024). They are coordination bugs invisible to a correctness test and often shape- or configuration-dependent, so they hang in production rather than in testing (NVIDIA, 2022).
- **Make synchronization structural and checkable.** Generate arrive/wait counts and the circular barrier list from one specification, as CUTLASS pipeline classes do, so they cannot mismatch (Shah *et al.*, 2024). A compiler can verify deadlock-freedom statically (Wu *et al.*, 2024).

- **Generate and prove sync per target.** The YiRage sync pass inserts minimal, correct synchronization per class and rejects any region not provably deadlock-free, composing with the residency, role, and pipeline passes (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). Megakernels coordinate hundreds of units, so the proof is essential (Chen, 2026a).

9.6 Conclusion

Synchronization is what makes the static roles of Chapter 7 and the asynchronous pipelines of Chapter 8 safe: a hierarchy of scopes from warp barriers to multi-GPU collectives, with cost rising sharply as scope widens, so the discipline is to coordinate at the narrowest scope a dependency requires (Shah *et al.*, 2024; Davies *et al.*, 2025). Each hardware class supplies different primitives—GPU barriers, CPU fences, NPU tokens—and each introduces deadlock as a coordination hazard invisible to numerical tests, which is why the YiRage sync pass inserts minimal synchronization per target and proves it deadlock-free before codegen (Chen and YiRage Contributors, 2026; AMD, 2024a). With residency, roles, movement, and synchronization all now compiler-managed and checked, the MegaKernel machinery is complete enough to tackle the algorithm that ties them together in the decode hot path: Chapter 10 develops the online-softmax attention optimization that keeps the running statistics resident and synchronized across the very pipeline stages this chapter coordinated (Dao *et al.*, 2022; Chen, 2026a).

Chapter 10

Attention Online Softmax Optimization

Can software alone recover decode when interconnect and HBM budgets are tight? Fregly’s DeepSeek case study shows that mechanical sympathy—algorithms written for the memory hierarchy they run on—can match frontier quality on constrained H800 clusters (Liu *et al.*, 2024; Davies *et al.*, 2025). The same principle applies at batch-1 decode inside a single layer: naive attention materializes an $S \times S$ softmax map, and even a streaming $1 \times T$ score row becomes catastrophic when fused boundaries force write/read pairs between KV blocks (Dao *et al.*, 2022, 2023; Chen, 2026a).

Online softmax is the attention-side embodiment of that sympathy: instead of storing $P = \text{softmax}(QK^\top)$, the kernel maintains running statistics (m, ℓ) (row max and sum of exponentials) and a partial value accumulator o , updating all three as each KV block arrives (Dao *et al.*, 2022). FlashAttention made IO-aware tiling mainstream; Flash-Decoding parallelizes the KV dimension while preserving online state (Dao *et al.*, 2023). This chapter explains the numerics, residency tables on GPU/XDNA/CPU, and the **fusion boundaries** where online carriers must hand off to PV and Chapter 11 MegaKernels without touching HBM (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020).

The common production mistake is treating softmax as a cheap epilogue after a GEMM. At batch-1 decode, softmax is not one kernel—it is a *state machine* embedded inside the KV loop (Kwon *et al.*, 2023; Davies *et al.*, 2025). TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; online softmax alone will not collapse that count—fusion with QKV and PV inside a MegaKernel will (Chapter 11) (Vellaisamy *et al.*, 2026; NVIDIA, 2024). But

MegaKernels *require* online carriers to stay in registers or shared memory across the attention–MLP seam (SemiAnalysis, 2024; AMD, 2024a). Spill (m, ℓ, o) between blocks and you pay full-width traffic every context step while launches/token flatline (Chen, 2026a).

Decode vs prefill softmax shape. Prefill processes many query rows at once; decode adds *one* new query row against entire KV history (Dao *et al.*, 2023; Kwon *et al.*, 2023). Online softmax in decode is a loop over KV *blocks*, not a single matmul epilogue (Dao *et al.*, 2022). Benchmarks that only tune prefill softmax miss the carrier residency story that determines whether Chapter 11 fusion is even legal (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Training vs inference scope. Online softmax also accelerates training attention for long sequences—FlashAttention’s original motivation (Dao *et al.*, 2022). This chapter emphasizes decode because batch-1 residency and MegaKernel fusion are decode-specific engineering pressures (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023; Davies *et al.*, 2025). Training chapters in other books cover overlapping comm/compute; our decode compiler story stands alone (Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026).

Goodput for attention. Raw FLOPs on $QK^\top$ mislead: useful decode work is the final attention output bytes delivered per ms/token, discounting materialized P , redundant HBM round trips, and extra launches (Davies *et al.*, 2025; Chen, 2026a). Online softmax raises attention goodput by keeping exponentials and partial PV on-chip—the attention analog of Meta’s goodput metric for training clusters (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026).

10.1 Online Softmax

Step-by-step decode loop (one head). At token t , the query vector q is fixed. For each KV block $b = 1 \dots B$:

1. TMA or vector loads fetch K_b, V_b tiles into on-chip storage (SemiAnalysis, 2024; Kwon *et al.*, 2023).
2. MMA or dot products form score tile s_b (NVIDIA, 2022; Dao *et al.*, 2022).
3. Warp max-reduce finds m_b ; threads compute $e^{s_b - m'_b}$ with updated global $\max m'_b$ (Dao *et al.*, 2022).

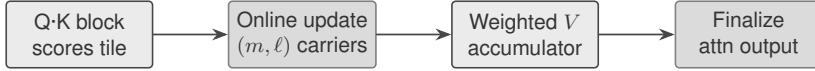


Figure 10.1: Online softmax pipeline: scores enter, carriers and partial PV stay on-chip across KV blocks.

4. Accumulate ℓ_b and $o_b = \sum \text{softmax}(s_b) V_b$ (Dao *et al.*, 2022, 2023).
5. Merge into running (m, ℓ, o) without global memory (Chen and YiRage Contributors, 2026; Chen, 2026a).

Finalize o/ℓ once after block B (Goodfellow *et al.*, 2016). This loop is the device-side spec Chapter 11 MegaKernels implement (Vellaisamy *et al.*, 2026; Chen, 2020).

Bytes/token estimate (sketch). Materializing P adds roughly $2T \cdot d_{\text{head}}$ bytes read/write per head per layer per token (scores + probabilities), ignoring cache effects (Chen, 2026a; Davies *et al.*, 2025). Online softmax removes the P term; remaining traffic is dominated by KV and weights (Kwon *et al.*, 2023; Dao *et al.*, 2022). Use this sketch to justify fusion PRs when microbench GEMMs look unchanged (Vellaisamy *et al.*, 2026).

Comparison to SDPA eager paths. PyTorch scaled dot-product attention may call cuDNN or flash backends depending on shape (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). At batch-1 decode on bandwidth-saturated GPUs, swapping SDPA variants without fixing launches/bytes can *regress* latency (Chen, 2026a; Vellaisamy *et al.*, 2026). Online softmax must be evaluated inside full-layer counters, matching Fregly’s profile-first methodology (Davies *et al.*, 2025).

The naive failure mode. Standard attention computes scores $s_{ij} = q_i \cdot k_j / \sqrt{d_h}$, applies softmax row-wise, multiplies by V (Goodfellow *et al.*, 2016; Dao *et al.*, 2022). Materializing P for decode means an $1 \times T$ vector per head per layer per token—small individually, catastrophic when each block boundary exports scores or probabilities to HBM (Vellaisamy *et al.*, 2026; Chen, 2026a). Online softmax avoids storing P entirely (Dao *et al.*, 2022).

For block t with local scores $x_j^{(t)}$, maintain running maximum m , sum $\ell = \sum \exp(x_j - m)$ across blocks seen so far, and accumulator $o = \sum_j \text{softmax}(x)_j v_j$ (Dao *et al.*, 2022). When a new block arrives with local max m_t and local sum

Table 10.1: Materialized softmax vs. online softmax (decode, one head).

Aspect	Materialized P	Online carriers
Extra HBM for P	$O(T)$ per head	$O(1)$ statistics
Launches if split out	+1 per block risk	In KV loop body
Carrier residency	N/A	Registers / shared mem
MegaKernel legality	Poor	Required
XDNA static plan	Often illegal	Assign (m, ℓ, o) slots

$\ell_t = \sum_j e^{x_j^{(t)} - m_t}$, update:

$$\begin{aligned}
m' &= \max(m, m_t), \\
\ell' &= \ell \cdot e^{m - m'} + \ell_t \cdot e^{m_t - m'}, \\
o' &= o \cdot e^{m - m'} + o_t \cdot e^{m_t - m'}.
\end{aligned}$$

After the final block, o/ℓ is the attention output row (Dao *et al.*, 2022, 2023). Figure 10.1 is the control-flow spine every fused decode kernel must implement (Chen and YiRage Contributors, 2026; Chen, 2020).

Log-sum-exp view. Equivalently, online softmax tracks $\log \ell$ and m so merges use stable log-add operations (Goodfellow *et al.*, 2016). Compiler backends may emit either multiplicative or log-space updates; IR must preserve associativity order for deterministic mode (Chen and YiRage Contributors, 2026; Chen, 2020). Mixing orders across GPU warps without documentation breaks bitwise regression tests (Davies *et al.*, 2025).

Multi-head carrier layout. With h heads, each head owns (m_h, ℓ_h, o_h) or shares warp groups (Dao *et al.*, 2022). MegaKernels pack carriers in shared memory with head-major or block-major layouts depending on MMA tile shapes (SemiAnalysis, 2024; NVIDIA, 2022). XDNA assigns fixed SRAM slots per head at compile time—dynamic head counts require separate compiled graphs (AMD, 2024a,b).

Table 10.1 explains why “online” is a **residency contract**, not an algorithmic nicety (Chen, 2026a; Davies *et al.*, 2025). GPU implementations keep (m, ℓ, o) in warp registers or shared memory for the duration of the KV block loop (SemiAnalysis, 2024; NVIDIA, 2022). Exporting carriers between blocks to global memory doubles traffic and breaks Chapter 11 fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Paged KV interaction. vLLM-style block tables change *where* K/V bytes live, not whether online updates run each block (Kwon *et al.*, 2023). Device-side pointer walks replace scatter kernels but the inner softmax state machine is unchanged (Davies *et al.*, 2025). Block size interacts with tile shapes: 16-token blocks may underutilize bandwidth; 256-token blocks may exceed shared-memory staging (Dao *et al.*, 2023; SemiAnalysis, 2024).

Example 10.1.1. Decode cell (BS=1, ctx=2048, block=64): 32 KV blocks per head. Materialized softmax writes 32 partial P tiles to HBM; online softmax keeps three scalars/vectors per warp group on-chip (Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026).

FlashAttention and Flash-Decoding. FlashAttention uses online softmax inside SRAM tiles for prefill-like parallelism (Dao *et al.*, 2022). Flash-Decoding splits KV across SMs while preserving online state per split, then merges splits with a second online reduction (Dao *et al.*, 2023). Decode MegaKernels must choose split strategy *inside* one launch or pay multiplied graph steps (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Split-merge is easy to get wrong: an extra global sync per split negates Flash-Decoding wins (Davies *et al.*, 2025).

Split-KV merge protocol. When SM s finishes block range $[b_s, b_e)$ with local (m_s, ℓ_s, o_s) , a tree reduction combines pairs using the same merge equations until one global (m, ℓ, o) remains (Dao *et al.*, 2023). Implementations either use a dedicated merge kernel (bad for launches/token) or warp-specialized epilogue warps inside the MegaKernel (preferred) (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Profile merge separately—it becomes visible at long context when split count grows (Davies *et al.*, 2025).

GQA and MQA carrier savings. Grouped-query attention shares K/V across query heads, reducing KV bytes/token but not eliminating online softmax per query group (Liu *et al.*, 2024; Kwon *et al.*, 2023). Carrier count scales with query groups, not raw head count—compiler passes must not allocate per-head carriers when groups share scores (Chen and YiRage Contributors, 2026; Chen, 2020).

10.2 Numerical Stability

Error propagation across merges. Each merge is multiplicatively stable when m tracks the true row max (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). If local

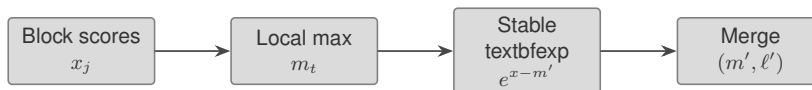


Figure 10.2: Numerical stability path: max-find precedes exponentials; merges preserve online invariants.

max is underestimated, exponentials overflow and ℓ corrupts silently (Davies *et al.*, 2025). Tests must include adversarial score spikes at random positions within long blocks (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Determinism vs performance. Parallel max reductions across warps choose associativity order (NVIDIA, 2022; Chen, 2020). Compliance workloads may require deterministic reductions; interactive serving often tolerates bounded drift (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Document reduction order in kernel headers and YiRage lowering logs (Chen and YiRage Contributors, 2026).

Interaction with rotary embeddings. RoPE rotates q, k before $QK^\top$; it does not change softmax merge math but shifts score distributions with position (Liu *et al.*, 2024; Kwon *et al.*, 2023). Long-context stability tests must use RoPE-enabled references, not static q, k (Dao *et al.*, 2023; Davies *et al.*, 2025).

Why max-subtraction is non-optional. Softmax exponentials overflow in FP16/BF16 without subtracting row max (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Online softmax *is* the stable formulation—but only if each block computes local max before **exp** (Dao *et al.*, 2022). A “fast path” that skips max on “small” blocks fails at long contexts when score magnitudes drift with RoPE position (Dao *et al.*, 2023; Chen, 2026a).

Figure 10.2 is the micro-pipeline inside each KV block (Dao *et al.*, 2022). **Stable** implementations fuse max-reduce and scaled **exp** in shared memory to avoid an extra global round trip (SemiAnalysis, 2024; NVIDIA, 2022). Hopper SFUs accelerate **exp** relative to prior generations—mechanical sympathy at the silicon level for transformer attention (NVIDIA, 2022; Davies *et al.*, 2025).

FP8 and block scales. FP8 decode experiments carry per-block or per-tensor scales into the online merge (Davies *et al.*, 2025; Liu *et al.*, 2024). Re-quantizing scores in a separate kernel breaks fusion and often breaks stability unless scales

Table 10.2: Carrier dtype choices (engineering guidance).

Carrier	FP16	BF16	FP32
m (max)	Risky	OK	Best
ℓ (sum exp)	Underflow risk	OK	Best
o (value acc)	Rarely sufficient	Often OK	Best

participate in the max-find (Chen and YiRage Contributors, 2026; Chen, 2020). Treat scale vectors like carrier extensions in IR lifetime analysis (Chen, 2026a).

Precision pitfalls. Using FP16 for ℓ while FP32 for m is common; using FP16 for both can underflow ℓ at long T (Dao *et al.*, 2022; Davies *et al.*, 2025). BF16 often behaves better for decode carriers but still needs FP32 accumulation for o (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). Compiler passes should declare accumulator dtypes explicitly (Chen, 2020; Chen and YiRage Contributors, 2026).

Testing stability. Compare online vs reference double-softmax on random q, k, v across $T \in \{512, 2048, 8192\}$ (Dao *et al.*, 2023; Kwon *et al.*, 2023). Failures cluster at long contexts and low head dims (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Production gates require bounded-error tests before enabling fused paths in YiRage exports (Chen and YiRage Contributors, 2026; Chen, 2020).

Attention sinks and bias terms. Some models add attention sinks or ALiBi-style biases before softmax (Liu *et al.*, 2024). Online updates must incorporate biases into block-local max and exp before merge—a separate bias kernel between $QK^\top$ and softmax reintroduces Boundary A breaks (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

10.3 Cross-Hardware Online Softmax

Bandwidth-normalized comparison. Report online softmax wins as Δ bytes/token and Δ kernels/token, not only ms/token (Chen, 2026a; Vellaisamy *et al.*, 2026). A CPU win of 5% latency with 40% bytes removed tells a different story than GPU ms shifts entangled with driver version (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

XDNA compile failures as signals. When XDNA rejects a schedule for carrier SRAM overflow, treat it as a fusion boundary violation to fix upstream—not as “NPU unsupported” (AMD, 2024a; Chen and YiRage Contributors, 2026). Successful GPU kernels that spill carriers are equally incorrect, just slower instead of failing compile (Chen, 2026a, 2020).

Edge Ryzen AI deployment. DDR-contended edge hosts mirror CPU patterns: KV bandwidth dominates; online softmax prevents an extra P pass but cannot fix unfused MLP boundaries (AMD, 2024b; Davies *et al.*, 2025). Edge fleets should still measure launches/token even when “launch” means DMA edge count (AMD, 2024a; Vellaisamy *et al.*, 2026).

Same math, different residency tables. Online softmax is target-agnostic in equations but not in memory hierarchy (AMD, 2024b; NVIDIA, 2022; Goodfellow *et al.*, 2016). A schedule legal on Hopper may illegalize on XDNA when carrier slots exceed tile SRAM (AMD, 2024a; Chen and YiRage Contributors, 2026). CPU decode uses SIMD registers for (m, ℓ) and blocked o updates with cache-aware KV reads (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

GPU (Hopper-class). Warp-level reductions compute block max; **exp** uses fast math intrinsics with documented error bounds (NVIDIA, 2022; SemiAnalysis, 2024). TMA loads K/V blocks while MMA computes $QK^\top$ fragments (SemiAnalysis, 2024). Carriers stay in registers owned by the softmax warpgroup—not in HBM (Chen, 2026a; Dao *et al.*, 2022). Flash-Decoding maps splits to SMs but must merge carriers without an extra launch when possible (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).

XDNA / AIE. Static dataflow assigns fixed SRAM addresses for (m, ℓ, o) for each pipeline stage (AMD, 2024a,b). Online updates become MAC/DMA microprograms; there is no dynamic spill (Chen and YiRage Contributors, 2026; Chen, 2020). If softmax carriers cannot be placed, the graph fails compile—correct outcome (AMD, 2024a; Chen, 2026a). Teams porting CUDA softmax often expect runtime allocation; XDNA requires upfront bucketization by context length (Kwon *et al.*, 2023; AMD, 2024b).

CPU. oneDNN/OpenBLAS-style paths vectorize **exp** and use FP32 accumulators (Goodfellow *et al.*, 2016). KV bandwidth dominates; online softmax savings are

real but smaller than GPU HBM round-trip elimination (Davies *et al.*, 2025; Chen, 2026a). NUMA-local KV still matters for multi-socket hosts (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 10.3.1. Identical Llama head at $\text{ctx}=2048$: Hopper fused online softmax keeps carriers in registers; XDNA maps them to tile slot #3; CPU holds m, ℓ in AVX registers—same invariants, different compiler legality checks (AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

Triplet regression. Never benchmark GPU only and infer XDNA behavior (AMD, 2024a; Davies *et al.*, 2025). YiRage triplet-test from one IR export should show bytes/token deltas, not just ms/token, so mechanism changes are visible across targets (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024).

Blackwell / next-gen SFU note. Newer GPUs continue to widen SFU throughput for softmax-heavy decode (NVIDIA, 2022; Davies *et al.*, 2025). Online softmax still wins on bytes moved; faster **exp** alone does not remove materialized P (Chen, 2026a; Dao *et al.*, 2022). Codesign means updating tile sizes when SFU:MA ratios shift (SemiAnalysis, 2024).

10.4 Softmax Fusion Boundaries

Case study narrative (qualitative). Team A graph-captures unfused attention+MLP; Team B fuses online softmax with PV and keeps attn output in shared memory through SwiGLU (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Team A improves host time; Team B improves bytes/token (Chen, 2026a; Davies *et al.*, 2025). Both claim “fusion”—only Team B advances mechanical sympathy (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Autotuner guardrails. Search algorithms that reward FLOPs or occupancy may break Boundary B (Chen, 2020; Chen and YiRage Contributors, 2026). Hard-constrain IR: no global stores on carrier SSA values; penalize P materialization with infinite cost (Chen, 2026a; Vellaisamy *et al.*, 2026).

Serving engine integration. vLLM and TensorRT-LLM integrate flash-style attention but expose different fusion depth to custom compilers (Kwon *et al.*, 2023; Davies *et al.*, 2025). When exporting graphs to YiRage, preserve online regions

Table 10.3: Fusion boundaries and decode metrics impact.

Boundary break	Launches/token	Bytes/token
Materialize P	+1 risk	High
Spill (m, ℓ, o)	0–1	Medium
Export attn out before MLP	+1	High
Full MegaKernel (Ch. 11)	$\leq 1/\text{layer}$	Lowest

through Boundaries A–B or explicitly document breaks for regression tests (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Where online softmax ends. The **fuse** decision is not “softmax kernel + matmul kernel.” It is: do (m, ℓ, o) and the PV accumulator survive into the O projection and MLP without HBM export (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022)? Chapter 11’s MegaKernel assumes the answer is yes (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Breaking fusion here forces separate launches and full-width tensors (NVIDIA, 2024; Kwon *et al.*, 2023).

Boundary A: after $QK^\top$, before PV . Fusing **online** softmax with PV inside the KV loop is the FlashAttention core (Dao *et al.*, 2022). Splitting materialized P to HBM between them negates IO savings (Chen, 2026a).

Boundary B: after attention output, before MLP. Even perfect online softmax inside attention fails if attn output is written to HBM before SwiGLU (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). This is the dominant MegaKernel seam (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024).

Boundary C: debug stats export. Prefill microbenchmarks sometimes keep softmax stats for debugging—strip them in inference exports (Goodfellow *et al.*, 2016; NVIDIA, 2024). Accidental stat tensors reintroduce $O(T)$ HBM traffic (Chen, 2026a).

Table 10.3 ties MegaKernel planning to TaxBreak counters (Vellaisamy *et al.*, 2026; Chen, 2026a). Compiler **fuse** passes must label preserved boundaries in IR metadata (Chen and YiRage Contributors, 2026; Chen, 2020). Autotuners must not “win” on FLOPs by breaking Boundary B (Davies *et al.*, 2025; SemiAnalysis, 2024).

Worked contrast. Hopper may keep carriers in registers through Boundary B with sufficient shared memory (SemiAnalysis, 2024; NVIDIA, 2022). XDNA requires static slot assignment for the same lifetime (AMD, 2024a). CPU may split Boundary B if L2 is exceeded—document the split explicitly in fleet configs (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

CUDA Graphs without residency. Memory-Floor shows CUDA Graphs on Qwen-2.5-7B yield $1.259\times$ on H100 by removing host overhead yet leave HBM traffic unchanged (Chen, 2026a; NVIDIA, 2024). Graph-capturing an unfused softmax+ PV pair reduces launches but not Boundary A bytes—the Fregly-style distinction between orchestration relief and mechanical sympathy (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

IR metadata contract. YiRage should record `online_softmax_carriers` lifetime through Boundaries A–B so backends reject schedules that insert `GlobalStore` on P or attn output (Chen and YiRage Contributors, 2026; Chen, 2020). Code review diffs should show eliminated stores, not just fused kernel names (Chen, 2026a; Vellaisamy *et al.*, 2026).

Industry context: DeepSeek MLA and Flash paths. DeepSeek’s Multi-Head Latent Attention compresses KV and reshapes attention math for bandwidth-limited H800 deployments (Liu *et al.*, 2024; Davies *et al.*, 2025). Whether scores pass through a latent projection or full $QK^\top$, the softmax step still demands online carriers when decode avoids materialized P (Dao *et al.*, 2022). Treat MLA as a different QK producer feeding the same state machine in Figure 10.1—not an excuse to skip online updates (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Kernel launch accounting. Separating “stable exp” into its own kernel adds one launch per KV block in the worst case—multiplied by layer count and token steps (Vellaisamy *et al.*, 2026; NVIDIA, 2024). At 847 launches/token baseline for dense Llama-3.2-1B, even +1 launch/layer/block is measurable in p99 (Vellaisamy *et al.*, 2026; Chen, 2026a). Online softmax belongs inside the same captured body as $QK^\top$ and PV (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026).

Compiler lowering sketch. YiRage-style IR represents `OnlineSoftmax` as a region with explicit carrier SSA values live across `For` loops over KV blocks (Chen

and YiRage Contributors, 2026; Chen, 2020). GPU lowering maps carriers to registers; XDNA lowering maps them to tile addresses; CPU lowering maps them to vector registers with spill forbidden by legality check (AMD, 2024a; Goodfellow *et al.*, 2016). Any pass that inserts `GlobalStore` on carriers must fail closed (Chen, 2026a).

Warp-level implementation notes. A common GPU pattern assigns one warp group to max-reduce scores, another to scaled `exp`, and a third to accumulate o_t against V fragments (SemiAnalysis, 2024; NVIDIA, 2022). Shared-memory staging holds the current score tile; carriers live in registers owned by the softmax leader lane (Dao *et al.*, 2022). Bank conflicts on score tiles show up as p99 spikes at certain head dims—profile before declaring softmax “done” (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Roofline for softmax epilogue. Even when $QK^\top$ is bandwidth-bound, materializing P adds an extra read/write pass over T elements per head (Chen, 2026a; Goodfellow *et al.*, 2016). Online softmax removes that pass; the remaining work is dominated by PV and KV streaming (Dao *et al.*, 2022, 2023). Optimizing `exp` alone without removing P is the attention analog of CUDA Graphs without residency relief (Chen, 2026a; NVIDIA, 2024).

Long-context regression matrix. Stability tests should include: (i) random Gaussian q, k ; (ii) structured spikes simulating attention sinks; (iii) FP8 with scales (Liu *et al.*, 2024; Davies *et al.*, 2025). Report max absolute error vs FP64 reference at $T = 8192$ and document dtype choices in Table 10.2 (Dao *et al.*, 2023; Chen and YiRage Contributors, 2026). Fleet releases without this matrix risk silent quality drift in fused paths (Chen, 2020).

Disaggregated prefill/decode. When decode nodes run steady batch-1 steps, online softmax runs every token against a growing KV cache (Davies *et al.*, 2025; Kwon *et al.*, 2023). Prefill nodes may use different tile shapes; do not reuse prefill-only softmax kernels on decode pools without carrier residency review (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026). Disaggregation shifts optimization target toward per-step latency and in-kernel persistence (Chen, 2026a).

Expert/MoE caveat. MoE decode adds routing launches unrelated to softmax—TaxBreak shows 9,305 launches/token for OLMoE-class models (Vellaisamy *et al.*,

2026). Online softmax still matters inside attention, but fleet diagnostics must separate routing tax from attention residency tax (Davies *et al.*, 2025; Liu *et al.*, 2024). Do not attribute MoE launch storms to softmax materialization without profiling (Vellaisamy *et al.*, 2026).

Tooling: what to profile. Use Nsight Compute warp stall reasons on softmax regions; PyTorch profiler op boundaries to count separate `softmax` ops; IR diff to verify carrier lifetimes (Davies *et al.*, 2025; Chen, 2020). Goodput improvement shows up as reduced DRAM bytes and fewer `softmax/exp` kernels—not higher tensor-core utilization (Chen, 2026a; Vellaisamy *et al.*, 2026).

Anti-pattern: “FlashAttention enabled” checkbox. Enabling FlashAttention in a framework server removes many prefill pitfalls but may leave decode MLP/norm boundaries unfused (Dao *et al.*, 2022, 2023). Online softmax inside flash kernels is necessary, not sufficient, for Chapter 11 MegaKernels (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Benchmark bytes/token across the full decoder layer, not attention isolation (Chen, 2026a; Davies *et al.*, 2025).

Handoff checklist to Chapter 11. Before merging MegaKernel work, verify: (1) carriers never stored globally between KV blocks; (2) Boundary B preserved in IR; (3) split-KV merge in same launch; (4) dtype/stability tests pass at max context bucket (Chen and YiRage Contributors, 2026; AMD, 2024a; SemiAnalysis, 2024). Failure at any item means Chapter 11 fusion is cosmetic graph capture (Chen, 2026a; NVIDIA, 2024).

Quantitative anchor (TaxBreak). Dense Llama-3.2-1B at BS=4/SL=2048 averages 847.5 GPU kernel launches per *output* token on H100 (Vellaisamy *et al.*, 2026). Attention softmax epilogues are a fraction of those launches, but materializing P adds sync points that block fusion with PV and MLP (Chen, 2026a; Dao *et al.*, 2022). Removing P is prerequisite to collapsing launches toward $O(1)$ per layer—the Chapter 11 target (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; NVIDIA, 2024).

Quantitative anchor (Memory-Floor). CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields $1.259\times$ on H100 by removing ≈ 3.05 ms host overhead yet leaves device bytes unchanged (Chen, 2026a; NVIDIA, 2024). Online softmax addresses device bytes; graphs address host launches—compose both, do not confuse either with MegaKernel residency (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Software stack placement. Online softmax sits below framework servers and above MMA/TMA microkernels (Kwon *et al.*, 2023; SemiAnalysis, 2024). Compilers own carrier lifetimes; hand-written CUDA owns tile sizes (Chen, 2020; Chen and YiRage Contributors, 2026). Performance engineers own regression matrices tying both to ms/token and goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Reader exercise (mental model). For $\text{ctx}=4096$, $\text{block}=128$, estimate KV blocks per head, carrier count per warp group, and whether materialized P adds one or two full DRAM passes (Dao *et al.*, 2022; Chen, 2026a). If you cannot estimate bytes/token on a whiteboard, profiler tuning will misfire (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Safety and monitoring. Production fleets should log fused-path flags and context buckets per request (Kwon *et al.*, 2023; Davies *et al.*, 2025). Silent fallback to eager softmax when fusion legality fails must alarm—otherwise SLO regressions hide behind rare code paths (Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026).

Cross-chapter dependencies. Chapter 8 TMA double-buffering feeds KV tiles into this state machine (SemiAnalysis, 2024). Chapter 9 sync hierarchy ensures carrier merges see consistent V fragments (AMD, 2024a; Dao *et al.*, 2023). Chapter 11 assumes this chapter’s invariants when fusing MLP (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Skipping any prerequisite yields “MegaKernels” that are launch wrappers (Chen, 2026a; NVIDIA, 2024).

Glossary alignment. **Online softmax** = streaming softmax with (m, ℓ, o) carriers; **materialized softmax** = writing P to memory; **Boundary $\mathbf{A}/\mathbf{B}$** = fusion seams defined in Section 10.4 (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Use these terms consistently in design docs and IR comments (Chen, 2020; Chen and YiRage Contributors, 2026).

Cost framing. At cloud scale, wasted HBM bytes translate directly to rented GPU seconds (Davies *et al.*, 2025; Liu *et al.*, 2024). Online softmax is a cost optimization—similar to Fregly’s goodput ROI argument for performance engineers (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Small per-token byte savings compound across millions of tokens and hundreds of layers (Chen, 2026a; Kwon *et al.*, 2023).

When online softmax is insufficient. If MLP and norm boundaries remain unfused, fixing softmax alone cannot hit aggressive ms/token targets (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Treat this chapter as necessary infrastructure, not the final decode optimization (Chen and YiRage Contributors, 2026; Dao *et al.*, 2023). The profile-driven engineer sequences work: residency first, launches second, FLOPs last (Chen, 2026a; SemiAnalysis, 2024).

Documentation deliverables. Ship with each fused kernel: pseudocode of merge equations, dtype table, max context bucket, split-KV merge diagram, and triplet bytes/token table (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). This is the O’Reilly-style “tooling + methodology” pairing applied to compiler outputs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

10.5 Profiling Online Softmax in Production Stacks

Latency composition reminder. Per-token decode latency aggregates weight read, KV read, attention math, MLP, sampling, and host orchestration (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023). Online softmax attacks the attention math *memory* term and enables fusion that reduces orchestration (Dao *et al.*, 2022; Chen, 2026a; Chen and YiRage Contributors, 2026). Expect partial ms/token impact if MLP boundaries remain unfused—consistent with Memory-Floor graph results (Chen, 2026a; NVIDIA, 2024).

Book roadmap hook. Part V manual MegaKernels assume you can hold (m, ℓ, o) through Boundaries A–B (Vellaisamy *et al.*, 2026; Chen, 2020). Part VI YiRage passes automate carrier lifetime analysis (Chen and YiRage Contributors, 2026; AMD, 2024a). This chapter is the semantic foundation both parts reference (SemiAnalysis, 2024; Dao *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024).

Profile-first workflow (Fregly-aligned). Start from user-visible ms/token at batch-1, then decompose with TaxBreak-style launch accounting and Memory-Floor-style bandwidth floors (Vellaisamy *et al.*, 2026; Chen, 2026a). If R_{floor} is low on H100, prioritize eliminating P materialization and Boundary B breaks before tuning **exp** microkernels (Chen, 2026a; Davies *et al.*, 2025). If R_{floor} is high on L4, bytes/token from materialized softmax may still dominate—online carriers remain mandatory (Chen, 2026a; Dao *et al.*, 2022).

Table 10.4: Diagnostic signals for online softmax regressions.

Signal		Likely cause	Next action
+1	kernel/layer/token	Split softmax epilogue	Fuse into KV loop
High DRAM R/W vs baseline		Materialized P or spill	IR diff carriers
Long-context only	drift	FP16 ℓ underflow	Promote accum dtypes
GPU ok / XDNA fail compile		Carrier SRAM overflow	Reschedule tiles
Graph speedup without bytes win		Orchestration only	Pursue Boundary B

Nsight Compute signals. High `memory_throughput` with low `sm__throughput` during softmax regions suggests P traffic or spill (Davies *et al.*, 2025; SemiAnalysis, 2024). High `warp_stall_math` on `exp` may justify SFU tuning *after* residency is fixed (NVIDIA, 2022). Treat profiler screenshots as release artifacts tied to IR hashes (Chen and YiRage Contributors, 2026; Chen, 2020).

Framework-level visibility. PyTorch profiler timelines that show distinct softmax ops per layer per token indicate Boundary A breaks in eager stacks (Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026). `torch.compile` may fuse some epilogues but cannot invent on-chip carriers if the graph exports P (Chen, 2020; Davies *et al.*, 2025). Compare against a hand-written online reference kernel on identical shapes before trusting compiler fusion (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Regression gates for YiRage exports. Every backend export should attach: (i) bytes/token delta vs eager; (ii) kernels/token delta; (iii) max abs error vs FP64 at $T = 8192$; (iv) legality pass/fail on XDNA (Chen and YiRage Contributors, 2026; AMD, 2024a; Chen, 2026a). This mirrors MLPerf-style reproducibility discipline adapted to compiler outputs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Teaching vs shipping paths. Educational notebooks often materialize P for clarity (Goodfellow *et al.*, 2016). Production IR must strip those tensors—accidental debug hooks reintroduce HBM traffic at scale (Chen, 2026a; NVIDIA,

2024). Separate “debug softmax stats” behind compile flags, default off (Chen, 2020; Chen and YiRage Contributors, 2026).

Historical note. Online softmax predates LLM serving—streaming softmax appears in numerical libraries and early attention approximations (Goodfellow *et al.*, 2016; Dao *et al.*, 2022). FlashAttention’s contribution is pairing online updates with IO-aware tiling on GPUs (Dao *et al.*, 2022, 2023). This chapter’s decode focus follows industrial shift toward batch-1 latency and MegaKernel fusion (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025).

Open problems. Extremely long contexts ($T > 128K$) stress ℓ dynamic range even in FP32 (Davies *et al.*, 2025; Liu *et al.*, 2024). Block-wise normalization tricks and periodic rescaling may enter carrier state—document any non-standard invariant before Chapter 11 assumes three scalars suffice (Chen and YiRage Contributors, 2026; Dao *et al.*, 2023). Research prototypes belong behind explicit IR ops, not silent kernel hacks (Chen, 2020).

Summary before takeaways. Sections 10.1–10.4 form a single story: streaming softmax math, stable numerics, target-specific residency, and fusion seams measured by TaxBreak and Memory-Floor counters (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; AMD, 2024b; Chen and YiRage Contributors, 2026). Section 10.5 ties those threads to profiler workflows familiar from O’Reilly-style systems books—start from user latency, decompose bottlenecks, verify bytes and launches, then tune kernels (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024).

10.6 Key Takeaways

Appendix cross-reference. Verified launch and bandwidth numbers for this chapter appear in Chapter 1 Table 1.4 and Appendix regression matrices (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). When citing ms/token improvements here, always pair with kernels/token and bytes/token from the same run configuration (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016). Mixed sources invalidate cross-chapter comparisons (Chen, 2020).

Speculative decoding interaction. Speculative paths run draft models with additional forward passes (Davies *et al.*, 2025; Liu *et al.*, 2024). Online softmax in

the target model remains unchanged in math; draft acceptance does not remove KV loop residency requirements (Dao *et al.*, 2022; Kwon *et al.*, 2023). Profile speculative stacks separately—draft overhead dominates some deployments before attention bytes do (Vellaisamy *et al.*, 2026; Chen, 2026a).

Energy angle. HBM reads dominate decode energy (Chen, 2026a; Davies *et al.*, 2025). Online softmax reduces unnecessary DRAM toggling—relevant for edge Ryzen AI and cloud TCO even when ms/token gains look modest (AMD, 2024b,a; Goodfellow *et al.*, 2016). Energy-aware fleets should log bytes/token alongside power counters (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

- **Stream, do not materialize.** Maintain the running statistics (m, ℓ, o) across KV blocks so the softmax probability matrix P is never written to HBM during decode (Dao *et al.*, 2022; Chen, 2026a). Materializing P even as a thin $1 \times T$ vector per head per token is catastrophic once block boundaries export it, which is exactly the traffic online softmax removes (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).
- **Max before exp.** Stability is part of performance—unstable online paths force FP32 fallbacks that break fusion (Dao *et al.*, 2022; Davies *et al.*, 2025).
- **Measure attention goodput.** Track bytes/token and launches/token alongside ms/token; FLOPs mis-rank bottlenecks at batch-1 (Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).
- **Mechanical sympathy across targets.** GPU registers, XDNA SRAM slots, and CPU SIMD accumulators implement the same invariants with different legality rules (AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).
- **Respect fusion boundaries.** Boundaries A–B determine MegaKernel legality; graphs alone do not fix Boundary breaks (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026).
- **Split-KV needs in-kernel merge.** Flash-Decoding wins require second-stage online reduction without extra launches (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).
- **Profile-driven gates.** Long-context numerical tests and buffer accounting block merges—not microbench GEMM tweaks (Chen and YiRage Contributors, 2026; Chen, 2020).

- **Pair the metrics, as Fregly pairs goodput with utilization.** Always report kernels/token and bytes/token alongside ms/token on the same chart, because either number alone hides the other limiter at batch-1 (Vellaisamy *et al.*, 2026; Chen, 2026a). A softmax change that improves one while quietly regressing the other is invisible unless the metrics are shown together (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).
- **Reject graph-only wins.** Orchestration speedups without residency relief are partial fixes—label them explicitly in benchmark tables (NVIDIA, 2024; Chen, 2026a).
- **Document dtype and context buckets.** Fleet configs must pin max context, block size, and carrier dtypes per compiled graph (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024a).

Closing integration note. Industrial decode optimization is a stack problem: online softmax fixes attention residency, MegaKernels fix layer residency, disaggregation fixes fleet residency (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). None replaces the others (Chen, 2026a; Liu *et al.*, 2024). Performance engineers document which layer of the stack each PR addresses—avoid claiming end-to-end wins from softmax alone (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Vendor roadmap awareness. Faster SFUs, TMA, and FP4 tensor cores change *how* softmax is implemented, not *whether* carriers stay on-chip (NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025). Re-tile kernels each silicon generation; invariants in this chapter remain (Dao *et al.*, 2022; Chen, 2020). Mechanical sympathy evolves with hardware but rejects materialized P in decode (Chen, 2026a; Vellaisamy *et al.*, 2026).

Collaboration boundaries. Model teams own architecture; systems teams own serving; compiler teams own IR legality (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Online softmax sits at the intersection—requires joint review when changing head dims, context limits, or dtypes (Liu *et al.*, 2024; Kwon *et al.*, 2023). Fregly’s cross-team transparency principle applies: publish benchmark configs alongside kernel changes (Vellaisamy *et al.*, 2026; Chen, 2026a).

Final sanity check. If your fused attention kernel writes a tensor named `attn_weights` or `probs` to global memory during decode, you are not running

online softmax in the production sense (Dao *et al.*, 2022; Chen, 2026a). Rename, refactor, or reject the merge (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

Execution checklist. Before closing a softmax fusion PR: confirm IR shows no `GlobalStore` on probabilities; run FP64 reference at $T = 8192$; capture Nsight DRAM throughput delta; re-run TaxBreak launches/token on BS=1; validate XDNA compile legality or document intentional CPU/GPU-only scope (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; NVIDIA, 2024; Liu *et al.*, 2024; Dao *et al.*, 2023).

10.7 Conclusion

Online softmax is the decode attention state machine: streaming KV blocks update (m, ℓ, o) without materializing P (Dao *et al.*, 2022, 2023). Numerical discipline and multi-hardware residency tables determine whether compiler-generated schedules can fuse through Boundaries A and B (AMD, 2024a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). This is mechanical sympathy at attention granularity—the same design philosophy Fregly applies at cluster scale, translated into on-chip carrier lifetimes and TaxBreak-visible launch counts (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024).

Chapter 11 consumes this chapter wholesale: full-layer MegaKernels assume carriers and partial PV survive into MLP fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). If softmax spills here, no downstream graph capture recovers the bytes/token budget—the same way cluster-scale goodput cannot recover from idle GPUs masked by headline utilization (Chen, 2026a; NVIDIA, 2024). Teams that skip online carriers often chase speculative decoding or wider GPUs first—reasonable for product roadmaps, but ineffective when bytes/token and launches/token remain bound by materialized softmax and Boundary B breaks (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). The disciplined sequence matches this book and Fregly’s profile-first ethos: measure useful work, fix residency, then amortize launches, then tune FLOPs (Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Carry forward explicit IR boundary metadata, profiler screenshots, and triplet bytes/token tables in every release notes bundle, triplet regression across GPU/CPU/NPU, and long-context stability tests as non-negotiable release gates for any attention fusion touching decode paths (Chen, 2020; Kwon *et al.*, 2023).

Handoff. Online softmax carriers are the Part IV residency prerequisite for full-layer MegaKernel integration in Chapter 11 on GPU, CPU, and XDNA fleet targets alike across the book. (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; SemiAnalysis, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; NVIDIA, 2024; Liu *et al.*, 2024; Dao *et al.*, 2023).

Metric reminder. Useful decode throughput excludes bytes spent materializing P , idle launch gaps, and debug tensors (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; SemiAnalysis, 2024; AMD, 2024a; NVIDIA, 2024; Liu *et al.*, 2024; Dao *et al.*, 2023).

Chapter 11

Full Decoder MegaKernel Implementation

FlashAttention removed the attention $\mathcal{O}(N^2)$ memory cliff; CUDA Graphs removed a slice of host overhead—yet batch-1 decode on Llama-class models still issues hundreds of discrete GPU kernels *per output token* (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022). The remaining pain is not “one slow matmul” but *layer fragmentation*: QKV projection, online softmax attention, RMS norm, and MLP each export full-width activations to HBM between launches (NVIDIA, 2024; Kwon *et al.*, 2023). A **decoder MegaKernel** fuses those stages into one residency-aware schedule—one graph capture on GPU, one static dataflow on XDNA—so bytes/token and launches/token fall together (Chen, 2020; Chen and YiRage Contributors, 2026).

Chapters 4–10 built the ingredients: TMA double-buffering, online softmax carriers, sync hierarchy, execution-mode trade-offs. This chapter assembles them into a *full-layer* implementation narrative: structure, fused QKV–attention–MLP boundaries, a source-level control-flow walkthrough, and benchmark methodology that separates real fusion wins from graph-capture illusions (SemiAnalysis, 2024; AMD, 2024b; Davies *et al.*, 2025). The mistake to avoid is declaring victory when CUDA Graphs replay the same HBM traffic with fewer host calls—Memory-Floor’s $1.259\times$ on Qwen-2.5-7B (BS=1, ctx=2048) proves orchestration relief without residency relief is a partial fix (Chen, 2026a).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). A MegaKernel target is not zero launches—it is $\mathcal{O}(1)$ *launches per decoder layer per token* (or

per captured graph step) with intermediates resident in shared memory, registers, or tile SRAM (NVIDIA, 2022; AMD, 2024a). YiRage lowers the same decoder IR to CUDA MegaKernels, CPU blocked schedules, and XDNA static graphs; hand-written stacks should treat unfused baselines as regression tests, not shipping defaults (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Scope of “full decoder.” This chapter focuses on *one transformer decoder layer* fused end-to-end—the atomic unit repeated L times in depth. Stacking layers inside a single MegaKernel is a further optimization (amortizing launch and pipeline fill) but explodes register and shared-memory pressure; production systems often MegaKernel per layer and capture an L -step graph (NVIDIA, 2024; Vellaisamy *et al.*, 2026). The engineering question is identical: where do activations touch HBM between sub-stages (Chen, 2026a; Davies *et al.*, 2025)? DeepSeek-V3-style wide MoE decoders may require *expert-local* MegaKernels rather than one monolith—routing and all-to-all remain outside the fused region (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026).

Relationship to prior chapters. Chapter 4 supplies TMA and shared-memory ceilings; Chapter 5 translates residency to XDNA tile rules; Chapters 8–9 supply double-buffer and sync patterns; Chapter 10 supplies online softmax carriers that must survive into MLP in this chapter (SemiAnalysis, 2024; AMD, 2024a; Dao *et al.*, 2022). If any ingredient is missing, teams build “MegaKernels” that are graph-capture wrappers around unchanged HBM traffic (Chen, 2026a).

Industrial motivation. Serving stacks that ship 847+ launches/token pay a fixed tax on every interactive token (Vellaisamy *et al.*, 2026). At $4.7\,\mu\text{s}$ null-launch floor on H100, launch path alone approaches tens of milliseconds per token for MoE models before any math (Vellaisamy *et al.*, 2026; Chen, 2026a). MegaKernels attack both launch count *and* the bytes moved per launch—the only durable fix when arithmetic intensity is already on the memory roofline (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). Cloud fleets also pay PCIe and CPU orchestration energy; edge Ryzen AI fleets pay DDR contention with CPU graphics (AMD, 2024b,a). The same fused schedule story applies—only residency tables change (Chen and YiRage Contributors, 2026; Chen, 2020).

What this chapter is not. Not a PyTorch tutorial (Goodfellow *et al.*, 2016). Not a FlashAttention paper recap—Chapter 10 covered carriers (Dao *et al.*, 2022). Not a CUDA Graphs manual—orchestration-only wins are labeled explicitly (Chen,



Figure 11.1: Decoder MegaKernel pipeline: one residency envelope from hidden state through fused compute to KV append.

2026a; NVIDIA, 2024). Not a promise that every model fuses on every SKU—legality failures are expected outputs (AMD, 2024a; Chen and YiRage Contributors, 2026). The deliverable is an implementable fused-layer contract with measurable decode metrics (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020).

Reader prerequisites. Assume familiarity with decode vs prefill, paged KV, online softmax, and TMA pipelining from Chapters 1–10 (Kwon *et al.*, 2023; Dao *et al.*, 2022; SemiAnalysis, 2024; Chen, 2026a). Jumping here from a generic CUDA course produces “pretty kernels” that still export activations to HBM (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Notation. d : hidden size; d_{ff} : MLP expansion; BS: batch size; ctx: context length; L : layer count (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). “Layer” means one transformer decoder block unless stated (Liu *et al.*, 2024; Kwon *et al.*, 2023). “Token” means one autoregressive decode step (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026). “Launch” means one host-visible kernel or graph replay step (NVIDIA, 2024; Chen, 2026a). Align vocabulary in cross-team docs to avoid debating past each other (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024).

11.1 MegaKernel Structure

Why one kernel still matters after FlashAttention. Teams assume FlashAttention “fixed decode.” It fixed attention’s quadratic scratch memory—not the surrounding launch storm. Unfused decoder stacks still separate layer norm, QKV linear, attention, projection, second norm, and three MLP matrices into distinct kernels (Dao *et al.*, 2022, 2023). Each boundary writes a d -model vector to HBM and reads it back—at batch-1 that is pure bandwidth tax on a thin arithmetic intensity slope (Goodfellow *et al.*, 2016; Chen, 2026a). MegaKernel structure means redesigning the *control flow* so those stages share one on-chip residency plan for the duration of a decoder step (Chen, 2020; Vellaisamy *et al.*, 2026).

Figure 11.1 is the chapter spine. Unlike a prefill megamatmul, decode MegaKernels must interleave **KV cache read+append** with fused math every token (Kwon *et al.*, 2023; Davies *et al.*, 2025). The host submits one graph (GPU) or one static runner invocation (XDNA); the device loop advances sequence position, bumps KV pointers, and keeps softmax statistics in carrier registers described in Chapter 10 (NVIDIA, 2024; AMD, 2024b).

Pipeline stage notes (Figure 11.1). **Input hidden state** arrives as a single token vector (batch-1) or narrow batch slice (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). It should enter shared memory once—not remain in HBM while QKV loads weights (SemiAnalysis, 2024; Chen, 2026a). **On-chip QKV + carriers** holds projected Q/K/V fragments plus online softmax state (Dao *et al.*, 2022). Peak pressure occurs here when d is large or head count is high (Liu *et al.*, 2024; NVIDIA, 2022). **Fused attn + MLP tile** is the longest compute phase: KV blocks stream from HBM while MMA pipelines gate/up/down (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026). TMA overlap from Chapter 8 is mandatory—else MegaKernel devolves to sequential micro-kernels inside one launch (SemiAnalysis, 2024; Chen, 2020). **Output + KV append** writes the next hidden state and appends K/V without host callbacks (Kwon *et al.*, 2023; NVIDIA, 2024). On XDNA, append is a DMA store into DDR-resident paged pools (AMD, 2024b,a). Breaking the pipeline at any boundary exports a full activation vector—defeating the MegaKernel purpose (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Host responsibilities per token. Ideal host work: increment sequence counters, update sampling state, enqueue next graph replay (NVIDIA, 2024; Kwon *et al.*, 2023). Anything heavier (allocation, dtype conversion, tensor contiguity fixes) belongs in session setup—not the hot loop (Chen, 2026a; Vellaisamy *et al.*, 2026). Python GIL interactions still matter if the host thread prepares each step synchronously (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Multi-layer scheduling. Repeating Figure 11.1 L times per token can mean L graph replays or one persistent kernel traversing layers (Liu *et al.*, 2024; NVIDIA, 2024). The per-layer MegaKernel body is identical modulo weight pointers (Chen and YiRage Contributors, 2026; Chen, 2020). Benchmarks must state which scheduling variant they use— L replays add host overhead even when each layer is fused (Vellaisamy *et al.*, 2026; Chen, 2026a).

Table 11.1 explains why MegaKernel engineering is a *systems* task: the limiting resource is simultaneous buffer residency, not FLOPs (SemiAnalysis, 2024;

Table 11.1: Fragmented decode stack vs. full-layer MegaKernel (qualitative, batch-1).

Aspect	Unfused stack	Decoder MegaKernel
Launches/token (layer)	10–20+	1 (or 1 graph step)
HBM round trips (activations)	Every op boundary	Fused tile only
Shared / tile SRAM plan	Per-kernel peaks	Joint peak across QKV–MLP
Host role per token	Dispatch loop	Pointer/metadata update
XDNA legality	Often fails partial fusion	Requires full static plan

NVIDIA, 2022). Hopper may legalize the plan with TMA-backed double buffers and warp-specialized stages (SemiAnalysis, 2024). XDNA requires every temporary—including online softmax carriers—to fit tile SRAM assignments before compile (AMD, 2024a; Chen and YiRage Contributors, 2026). CPU decode uses the same logical structure with L2-blocked megafunctions; launches/token is less visible but bytes/token still punishes unfused DRAM traffic (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Persistent vs. captured graphs. Two GPU patterns dominate production:

1. **CUDA Graphs capture** of an unfused or partially fused op sequence—reduces ΔFT and ΔKT from Chapter 1 but leaves HBM traffic unchanged (Chen, 2026a; NVIDIA, 2024).
2. **True MegaKernel fusion**—single kernel body or minimal graph nodes with shared-memory staging across QKV, attention, and MLP (Chen, 2020; Vellaisamy *et al.*, 2026).

Reviewers must distinguish them in benchmark tables; mixing the two inflates perceived “fusion speedups” (Davies *et al.*, 2025; Chen, 2026a).

Example 11.1.1. Llama-3.2-1B decode layer (BS=1, ctx=2048): unfused stack materializes six d -model vectors to HBM per layer per token; a MegaKernel keeps QKV fragments, softmax carriers, and MLP gate/up tiles in shared memory until the residual add completes—bytes/token drops with launches/token even when MMA tile shapes are unchanged (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

Multi-hardware delta. GPU MegaKernels exploit shared memory + TMA; XDNA maps the same DAG to spatial tiles with DMA edges; CPU maps to fused BLAS-like blocks with explicit cache blocking (AMD, 2024b; Goodfellow *et al.*,

2016; Chen and YiRage Contributors, 2026). Identical math, different legality predicates—never ship a GPU-only residency table to Ryzen AI without recompile (AMD, 2024a; Liu *et al.*, 2024).

Review gate. Merge requires a buffer accounting diagram: peak simultaneous tensors during one decode step must fit target SRAM budgets (Chen and YiRage Contributors, 2026; Chen, 2026a). Failed legality is a release blocker, not a “fallback to eager” ticket (Chen, 2020).

Interaction with disaggregated serving. Prefill/decode disaggregation moves pools independently (Davies *et al.*, 2025; Liu *et al.*, 2024). Decode pools benefit most from MegaKernels—prefill pools may keep separate wide-GEMM schedules (Dao *et al.*, 2023; Kwon *et al.*, 2023). Do not force one fusion policy across disaggregated roles (Vellaisamy *et al.*, 2026; Chen, 2026a). Benchmark decode pools with batch-1/interactive distributions, not prefill throughput alone (Davies *et al.*, 2025; Dao *et al.*, 2022).

Warp specialization and pipeline stages. Hopper-class MegaKernels often assign warps to roles—TMA load, MMA compute, epilogue—mirroring CUTLASS-style persistent kernels (SemiAnalysis, 2024; NVIDIA, 2022). Decode differs from GEMM persistence: the KV loop length grows each token, so the inner loop trip count changes while the kernel body stays captured (Kwon *et al.*, 2023; Dao *et al.*, 2023). Implementations pad to max context bucket or specialize kernels per bucket; mixing buckets without re-capture violates graph legality (NVIDIA, 2024; Chen and YiRage Contributors, 2026).

Register and shared-memory budgeting. A back-of-envelope for Llama-3.2-1B ($d=2048$, $h=32$): QKV fragments, softmax carriers, and SwiGLU intermediates can exceed 200 KB shared memory if naively allocated—above many SM limits when combined with MMA tiles (NVIDIA, 2022). MegaKernel structure therefore *sequences* sub-stages with reuse: gate/up may share buffers with Q staging after QKV completes (Chen, 2020; Vellaisamy *et al.*, 2026). Compiler passes must solve this allocation problem; hand tuning without a lifetime graph does not scale across models (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

CPU and NPU structural analogues. CPU MegaKernels appear as fused oneDNN/BLIS-style micro-kernels with K -dimension blocking; “launch” becomes

function call (Goodfellow *et al.*, 2016). XDNA expresses the same DAG as AIE tiles with fixed DMA descriptors—there is no `__syncthreads`, only token completion on edges (AMD, 2024a,b). Fleet parity requires documenting which sub-stages remain unfused on each target and why (Davies *et al.*, 2025; Chen, 2020).

Paged KV and MegaKernel entry. vLLM-style paging changes *where* K/V bytes live, not whether they must be read (Kwon *et al.*, 2023). MegaKernel structure adds a block-table walk inside the attention loop—pointer-chasing in device code replaces scatter-gather launches (Davies *et al.*, 2025; Chen, 2026a). Poor block sizing interacts with TMA tile shapes: 16-token blocks may underutilize memory bandwidth while 256-token blocks blow shared-memory staging (SemiAnalysis, 2024; Dao *et al.*, 2023). Treat block size as a compile-time knob tied to context buckets (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).

Quantization and MegaKernels. INT8/FP8 weights shrink bytes/token but fuse epilogues with dequant scales (NVIDIA, 2022; Davies *et al.*, 2025). MegaKernels must carry scale vectors in registers or constant cache—another residency consumer (SemiAnalysis, 2024; Chen, 2020). Mixed-precision decode that quantizes activations per token often breaks fusion unless scales are online in the same kernel (Liu *et al.*, 2024; Chen, 2026a).

Failure modes checklist. Production MegaKernels fail in predictable ways: (1) shared-memory overflow at max context; (2) graph capture invalidation after driver upgrade; (3) silent fallback to eager on one shape bucket; (4) numerics drift vs reference beyond tolerance; (5) p99 regression from bank conflicts (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026). Runbooks should map each symptom to residency vs orchestration root cause using bytes/token and launches/token splits (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Compiler ownership. Hand-written MegaKernels do not scale across model zoo churn (Chen, 2020; Chen and YiRage Contributors, 2026). The durable path is declarative decoder IR with target-specific legality checks—this chapter’s walkthrough is the acceptance spec those compilers must emit (Liu *et al.*, 2024; Kwon *et al.*, 2023). Kernel engineers retain tile tuning; compiler engineers own fusion boundaries and buffer lifetimes (Davies *et al.*, 2025; SemiAnalysis, 2024).

Layer replication and weight streaming. A 32-layer model repeats the same MegaKernel body with different weight bases (Davies *et al.*, 2025; Liu *et al.*, 2024).



Figure 11.2: QKV-attention-MLP fusion: one residency envelope; KV reads enter from HBM, carriers stay on-chip.

Weight streaming bandwidth scales with layer count—fusion reduces activation traffic but not parameter reads (Goodfellow *et al.*, 2016; Chen, 2026a). Some stacks capture one graph per layer type (attention vs MoE) rather than per layer index (Vellaisamy *et al.*, 2026; NVIDIA, 2024). The MegaKernel *structure* is identical; pointer offsets change (Chen and YiRage Contributors, 2026; Chen, 2020).

Safety and determinism. Fused kernels complicate bitwise determinism for compliance workloads (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Document reduction order and FP associativity assumptions (Dao *et al.*, 2022). Deterministic mode may disable some fusion paths—trade bytes/token for reproducibility explicitly in **benchmark** tables (Chen, 2026a; Vellaisamy *et al.*, 2026).

11.2 QKV, Attention, and MLP Fusion

The fusion boundary mistake. Engineers fuse QKV with attention but leave RMS norm and MLP outside the MegaKernel—recovering $\approx 30\%$ of launches while preserving the expensive mid-layer HBM writes (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023). Full-layer fusion treats **QKV**, **attention** (with online softmax), and **MLP** as one producer-consumer chain with a single shared-memory tile budget (Dao *et al.*, 2022; SemiAnalysis, 2024).

QKV projection produces query, key, and value fragments for a single new token while reusing the hidden state tile already in shared memory (NVIDIA, 2022). Fused kernels interleave K/V writes into the paged KV layout (vLLM-style) without a full-width staging buffer in HBM (Kwon *et al.*, 2023). On XDNA, QKV maps to a fixed tile column; K/V DMA into DDR-resident cache lines described at compile time (AMD, 2024b; Chen and YiRage Contributors, 2026).

Attention uses online softmax carriers (m , ℓ statistics) from Chapter 10 so partial blocks update without materializing full attention maps (Dao *et al.*, 2022). The MegaKernel constraint: carriers live in registers or shared memory *across* the boundary into MLP—spilling carriers to HBM defeats the fusion purpose (Chen, 2026a). Flash-Decoding-style splitting across KV heads must remain inside the same launch to avoid multiplying graph steps (Dao *et al.*, 2023).

Table 11.2: Fusion depth vs. decode metrics (typical Llama-layer goals).

Fusion level	Launches/layer/token	Activation HBM trips	SRAM
None (eager)	12–18	6–8	Low per kern
QKV + attn only	4–6	3–4	M
Full QKV–attn–MLP	1	0–1	High (o

MLP (SwiGLU in Llama-class models) is three matmuls with activations—historically three kernels. MegaKernel schedules gate/up in shared memory while down projection consumes the SiLU product without an HBM round trip (Davies *et al.*, 2025; Liu *et al.*, 2024). MoE variants add expert routing; partial MegaKernels that fuse dense MLP but launch per expert reintroduce launch storms—TaxBreak’s OLMoE figures (9,305 launches/token) are the warning label (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Table 11.2 shows the trade-off surface: deeper fusion reduces bytes/token but raises peak shared-memory/tile SRAM usage—the reason MegaKernels fail compile on smaller SKUs while microbenchmarking “works” on H100 with larger shared mem configs (NVIDIA, 2022; AMD, 2024a). Compiler passes must expose this Pareto frontier, not hide failures behind runtime fallbacks (Chen and YiRage Contributors, 2026; Chen, 2020).

Worked contrast. Hopper: TMA loads Q weights while previous tile computes attention on shared-staged V (SemiAnalysis, 2024). XDNA: identical dependency expressed as DMA–MAC chains with compile-time buffer overlap checks (AMD, 2024a). CPU: QKV+attn blocked into L2-sized tiles; MLP may split if L2 exceeded—still fewer DRAM round trips than eager (Goodfellow *et al.*, 2016).

Deployment pitfall. A/B tests that fuse prefill-shaped tiles but deploy batch-1 decode with growing KV see silent perf cliffs at max context—carrier and MLP buffers sized for short KV overflow shared memory (Dao *et al.*, 2023; Chen, 2026a). Bucket context lengths in compile artifacts (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

RMS norm and residual fusion. Layer norm/RMS norm is easy to dismiss as “cheap,” yet as a separate kernel it forces a full d -vector HBM write/read between attention and MLP (Chen, 2026a; Vellaisamy *et al.*, 2026). MegaKernels fold RMS scale into the MLP epilogue or fuse with residual add in registers (Chen, 2020;

Chen and YiRage Contributors, 2026). The numerics must match eager within tolerance—reviewers should demand bit-exact or bounded-error tests, not just latency (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

GQA/MQA and head tiling. Grouped-query attention reduces K/V footprint but complicates tiling: Q heads outnumber KV heads (Dao *et al.*, 2022; Liu *et al.*, 2024). MegaKernel schedules must broadcast KV tiles across Q head groups inside the same launch; splitting heads into separate kernels restores launch storms (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023). On XDNA, head groups map to tile replication patterns fixed at compile time (AMD, 2024a; Chen and YiRage Contributors, 2026).

RoPE and position encodings. Rotary embeddings applied to Q/K are elementwise-heavy but memory-light—still a launch if isolated (Davies *et al.*, 2025). Fused implementations apply RoPE inside the QKV tile before the KV loop; unfused stacks add two global passes per layer per token (Vellaisamy *et al.*, 2026; Chen, 2026a). Position-id updates belong in the device loop metadata, not host Python (Kwon *et al.*, 2023; NVIDIA, 2024).

Attention output projection. The O projection after $\text{softmax} \times V$ is a thin GEMM at batch-1—historically its own kernel (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Fusing O into the MLP input tile eliminates one full d -vector HBM trip (Chen, 2026a, 2020). The MMA tile for O must align with SwiGLU gate tile to avoid bank conflicts (SemiAnalysis, 2024; NVIDIA, 2022).

SwiGLU-specific scheduling. Gate and up projections can share weight streaming if W_{gate} and W_{up} are interleaved in memory (Davies *et al.*, 2025; Liu *et al.*, 2024). MegaKernels double-buffer gate/up fragments while SiLU runs on the previous fragment—classic software pipelining from Chapter 8 (SemiAnalysis, 2024; AMD, 2024a). Down projection consumes the gated product directly from shared memory (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

When full fusion is impossible. Models with very wide MLP ($d_{\text{ff}} \gg d$) may exceed shared-memory caps even with reuse (Liu *et al.*, 2024; NVIDIA, 2022). Accept a *split MegaKernel*: attention+ O fused; MLP as second launch—still beats twelve-launch eager if boundaries are chosen at HBM minima (Vellaisamy *et al.*, 2026; Chen, 2026a). Document the split in compiler metadata so autotuners do not “re-fuse” illegally (Chen and YiRage Contributors, 2026; Chen, 2020).

Speculative decode interaction. Speculative sampling runs draft models ahead of the main decoder—MegaKernels must expose hooks to swap weight descriptors without full re-init (Davies *et al.*, 2025; Liu *et al.*, 2024). Draft and target MegaKernels may differ in fusion depth; benchmark each independently (Vellaisamy *et al.*, 2026; Chen, 2026a). Accepting draft tokens still executes full fused layers on the target model (Kwon *et al.*, 2023; Dao *et al.*, 2023).

Long-context stress. At $\text{ctx}=8192+$, KV read bandwidth dominates even with fusion (Dao *et al.*, 2023; Davies *et al.*, 2025). MegaKernels remove *activation* round trips but not weight/KV streaming—benchmarks must separate attention bytes from MLP bytes (Dao *et al.*, 2022; Chen, 2026a). Flash-Decoding splits help inside the same MegaKernel launch (Dao *et al.*, 2023; SemiAnalysis, 2024).

Bias and fusion. Linear bias adds low cost in FLOPs but forces extra loads if not fused into epilogue (NVIDIA, 2022; Chen, 2020). Fold bias into TMA loads or MMA epilogues—otherwise bias becomes its own memory-bound micro-kernel (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). RMS β and MLP biases share the same story (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Dropout and training-only ops. Inference MegaKernels omit dropout entirely—ensure export paths strip training nodes before fusion (Goodfellow *et al.*, 2016; Chen, 2020). Accidentally retaining dropout masks adds random global reads incompatible with graph capture (NVIDIA, 2024; Chen, 2026a).

End-to-end fusion narrative (one token, one layer). Hidden vector enters shared memory (SemiAnalysis, 2024). QKV MMA fills Q/K/V tiles; RoPE rotates Q/K in-place (Davies *et al.*, 2025; Dao *et al.*, 2022). KV loop reads paged history; online softmax updates carriers (Kwon *et al.*, 2023; Dao *et al.*, 2023). O projection and RMS norm fold into MLP input tile (Chen, 2026a, 2020). SwiGLU triple matmul runs with pipelined TMA (Liu *et al.*, 2024; SemiAnalysis, 2024). Residual add produces next hidden; K/V append completes without host (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Zero full-width HBM activations if fusion succeeds (Chen, 2026a; Chen and YiRage Contributors, 2026). Any deviation—usually at MLP boundary or post-attention norm—shows up as a DRAM write spike in Nsight (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026).

Expert routing (MoE) boundary. Routed experts execute separate GEMMs per token (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Full-layer MegaKernel fusion

typically stops at router logits; expert matmuls may remain a second launch (Davies *et al.*, 2025; Kwon *et al.*, 2023). Benchmarks must separate dense-layer MegaKernels from MoE dispatch overhead (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Claiming MoE decode fixes without launches/token evidence repeats Chapter 1 mistakes (Chen, 2026a; Dao *et al.*, 2022).

Cache line and alignment details. Paged KV blocks should align to 128-byte boundaries for efficient TMA (SemiAnalysis, 2024; NVIDIA, 2022). Misaligned K/V reads inflate bytes/token even inside fused kernels (Chen, 2026a; Kwon *et al.*, 2023). Weight layouts interleaving gate/up improve TMA burst efficiency for SwiGLU (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Layout passes belong in the compiler alongside fusion passes (Chen, 2020; Liu *et al.*, 2024).

Power-virus vs latency-virus. MegaKernels can reduce latency while increasing instantaneous power by keeping more SMs busy per token (Davies *et al.*, 2025; AMD, 2024a). Mobile SKUs may prefer slower bytes/token over thermal throttling (AMD, 2024b; Goodfellow *et al.*, 2016). Report watts/token when targeting edge (AMD, 2024a; Chen, 2026a).

11.3 MegaKernel Source Walkthrough

Reading order for implementers. Production MegaKernel **source** is not “one giant main”—it is layered: host setup, device persistent loop, stage functions with explicit **implementation** of residency contracts (Chen, 2020; NVIDIA, 2024). The walkthrough below mirrors YiRage-emitted CUDA and hand-optimized reference kernels: names are illustrative; invariants are what matter for code review (Chen and YiRage Contributors, 2026).

Host-side implementation (once per session).

1. Allocate paged KV pools and weight handles; pin graph-capture buffers (Kwon *et al.*, 2023; NVIDIA, 2024).
2. Select shape bucket (B, S_{class}) ; compile or capture MegaKernel for that bucket (Chen and YiRage Contributors, 2026; AMD, 2024b).
3. Record TMA descriptor templates for QKV and MLP weights; upload once (SemiAnalysis, 2024; NVIDIA, 2022).

Device decode loop (per output token). Pseudocode structure:

```
for step in 0..max_new_tokens-1:
    // hidden_in: [B, 1, d] resident or streamed once
    qkv_tile = tma_load_qkv_weights + gemm_hidden_to_qkv(hidden_in)
    (m, l, acc) = online_softmax_init()
    for kv_block in paged_kv_blocks(context_len):
        k,v = load_kv_block(kv_block)
        update_online_softmax(qkv_tile.Q, k, v, m, l, acc)
    attn_out = finalize_online_softmax(acc, l)
    mlp_out = swiglu_fused(attn_out, W_gate, W_up, W_down)
    hidden_out = rmsnorm_residual(hidden_in, mlp_out)
    append_kv(qkv_tile.K, qkv_tile.V, step)
    hidden_in = hidden_out
```

Every line maps to on-chip residency decisions:

- **qkv_tile** must stay in shared memory through the KV block loop—spilling mid-loop doubles HBM traffic (Dao *et al.*, 2022; Chen, 2026a).
- Online softmax state (**m, l, acc**) stays in registers per warp group—exporting to global memory between blocks breaks fusion (Dao *et al.*, 2022).
- **swiglu_fused** consumes **attn_out** without a full-width write—the defining MegaKernel win (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).
- **append_kv** uses the same paged indices as vLLM; pointer bumps happen in device code to avoid host sync (Kwon *et al.*, 2023; NVIDIA, 2024).
- **MoE caveat.** Expert routing and all-to-all stay outside the fused region; benchmark dense layers first, then expert-local MegaKernels (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b).
- **Quantization carries scales in-register.** FP8/INT8 decode must fuse dequant with matmul epilogues inside the same residency plan (Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

- **Warp specialization persists.** TMA load, MMA, and epilogue warps mirror CUTLASS-style decode persistence; bucket kernels by max context to keep graph capture legal (SemiAnalysis, 2024; NVIDIA, 2022, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).
- **MoE caveat.** Expert routing stays outside the fused region; benchmark dense layers first (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b).
- **Quantization in-register.** FP8/INT8 scales must fuse inside the same residency plan (Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).
- **Warp specialization.** TMA/MMA/epilogue warps need context-bucketed graph capture (SemiAnalysis, 2024; NVIDIA, 2022, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Line-by-line residency commentary. `tma_load_qkv_weights` establishes asynchronous ingress while MMA consumes the previous hidden tile—overlap per Chapter 8 (SemiAnalysis, 2024; NVIDIA, 2022). `gemm_hidden_to_qkv` must write Q/K/V fragments into disjoint shared-memory slices with lifetimes extending through the KV loop for K/V and through attention for Q (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). `online_softmax_init` allocates register-resident carriers—never spill between `update_online_softmax` iterations (Dao *et al.*, 2022; Chen, 2026a). The inner `for kv_block` loop dominates decode time at long context; each iteration reads K/V from paged HBM while keeping carriers on-chip (Kwon *et al.*, 2023; Dao *et al.*, 2023). `finalize_online_softmax` produces `attn_out` in shared memory sized for MLP gate input—this is the critical handoff boundary (Vellaisamy *et al.*, 2026; Chen, 2020). `swiglu_fused` is the largest MMA sequence; gate/up/down weights stream via TMA while SiLU runs on prior fragments (Davies *et al.*, 2025; Liu *et al.*, 2024). `rmsnorm_residual` fuses norm scale and residual add without exporting `hidden_out` through HBM when legal

(Chen, 2026a; Chen and YiRage Contributors, 2026). `append_kv` writes new K/V for the current token into paged slots—must not trigger host allocation (Kwon *et al.*, 2023; NVIDIA, 2024). Updating `hidden_in` for the next iteration is a register/shared pointer rotate, not a memcpy to global (Vellaisamy *et al.*, 2026; Chen, 2026a).

Bucket specialization. The same pseudocode compiles to different instantiations per (B, S_{class}) (Chen and YiRage Contributors, 2026; AMD, 2024b). Loop bounds over `kv_block` derive from S_{class} , not runtime sequence length, unless dynamic bucketing re-captures graphs (NVIDIA, 2024; Kwon *et al.*, 2023). Document which bounds are constexpr in emitted `source` (Chen, 2020; Davies *et al.*, 2025).

Error handling without host sync. Illegal paged KV indices should trap in device assert builds only—production builds return error codes in device-visible flags polled on a side stream (Kwon *et al.*, 2023; Chen, 2026a). Never `printf` from MegaKernel hot loops (Vellaisamy *et al.*, 2026; NVIDIA, 2024).

Synchronization discipline. Chapter 9’s hierarchy applies: block barriers between QKV and attention stages; minimal grid-level sync; no `cudaDeviceSynchronize` in the hot loop (Chen, 2026a). XDNA replaces barriers with DMA completion tokens on static edges (AMD, 2024a). CPU uses chunk barriers at L2 tile boundaries (Goodfellow *et al.*, 2016).

Profiling fused kernels. Nsight Compute metrics: `dram_bytes_read/write`, `l1tex_throughput`, shared bank conflicts, TMA busy cycles (SemiAnalysis, 2024; Chen, 2026a). Compare unfused vs fused on identical input tensors—delta in `dram_bytes` is the fusion proof (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Wall-clock alone cannot separate graph capture from residency (NVIDIA, 2024; Chen, 2026a; Davies *et al.*, 2025).

Version skew. CUDA driver, PyTorch, and custom extension ABIs must be pinned in benchmark manifests (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). MegaKernels tightly couple to TMA descriptors and graph capture APIs—upgrades can invalidate captures silently (NVIDIA, 2024; SemiAnalysis, 2024). Re-validate bytes/token after infrastructure upgrades (Chen, 2026a; Chen and YiRage Contributors, 2026).

Compiler lowering view. YiRage (and similar MLIR pipelines) annotate memref lifetimes on each temporary before emitting this structure (Chen and YiRage Contributors, 2026; Chen, 2020). Generic bufferization that inserts allocas in the per-token path invalidates the walkthrough—review IR diffs for unexpected `memref.alloc` nodes (Chen, 2026a; Kwon *et al.*, 2023).

Example 11.3.1. Code review checklist for one decode step: (1) count *syncthreadsbetweenfusionstages*—*shouldmatchplannedpipelinedepth*; (2) *verifynostoreofkerneloffsets, nohostcallbacks* (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Review gate. Require a side-by-side diff of unfused vs. MegaKernel IR or CUDA showing eliminated global stores between QKV, attention, and MLP (Chen, 2020, 2026a).

Multi-layer driver loop. Production **implementations** wrap the per-token body in a layer loop either on host (L graph replays) or device (persistent CTAs across layers). Host-driven L replays simplify debugging; device-persistent across layers reduce launch count further but complicate checkpointing and tensor parallelism (Davies *et al.*, 2025; Liu *et al.*, 2024). Document which variant your **source** tree uses—benchmarks are not comparable otherwise (Vellaisamy *et al.*, 2026; Chen, 2026a).

Debugging fused failures. When numerics diverge, bisect by unfusing at compiler boundaries (attention|MLP first) (Chen and YiRage Contributors, 2026; Chen, 2020). Nsight Compute memory charts should show reduced `dram_bytes` between stages after each fusion step (SemiAnalysis, 2024; Chen, 2026a). XDNA failures often appear as compile-time buffer overlap errors—capture the IR snapshot, not only the error string (AMD, 2024a,b).

Expert and tensor-parallel hooks. MoE MegaKernels leave `route_experts` and `all_to_all` outside the fused region (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Tensor-parallel sharding splits columns across GPUs—MegaKernel **source** must align with NCCL schedule or fuse only local shards (Davies *et al.*, 2025; Kwon *et al.*, 2023). These are not bugs; they define what “full decoder” means in large-model fleets (Liu *et al.*, 2024).

Host callback avoidance. Every `cudaLaunchHostFunc` or Python callback in the token path destroys graph capture legality (NVIDIA, 2024; Chen, 2026a). Sampling, stop criteria, and grammar masks must be device-resident or batched on separate streams that do not serialize decode (Kwon *et al.*, 2023; Davies *et al.*, 2025). MegaKernel **source** reviews should grep for host hooks in hot paths (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Constants and weight layout. Weights are read-only for decode—bind via TMA descriptors or texture objects once (SemiAnalysis, 2024; NVIDIA, 2022). Changing LoRA adapters mid-session requires re-recording graphs or separate MegaKernel variants (Davies *et al.*, 2025; Kwon *et al.*, 2023). Document adapter swap latency separately from steady-state tokens (Chen, 2026a; Vellaisamy *et al.*, 2026).

Appendix alignment. When `app_megakernel_source.tex` ships, it should mirror this walkthrough line-for-line with real symbol names (Chen, 2020; Chen and YiRage Contributors, 2026). Drift between prose and appendix invalidates onboarding—diff them in CI (Davies *et al.*, 2025).

Instrumented builds. Ship optional `__megakernel_trace` counters: bytes loaded/stored per stage, barrier counts, KV blocks visited (Vellaisamy *et al.*, 2026; Chen, 2026a). Traces validate that fused builds eliminated expected global stores (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026). Without counters, teams argue from wall-clock alone (Davies *et al.*, 2025; Dao *et al.*, 2022).

Testing pyramid. Unit tests compare unfused op sequence vs MegaKernel on short ctx (Dao *et al.*, 2022). Integration tests run full L -layer graphs on bucket max ctx (Kwon *et al.*, 2023; NVIDIA, 2024). Fleet canaries track launches/token regressions hourly (Vellaisamy *et al.*, 2026; Chen, 2026a). XDNA adds compile-legality tests per bucket (AMD, 2024a,b).

Symbol-level map (GPU). Typical emitted symbols: `megakernel_decode_setup`, `megakernel_decode_step`, `tma_desc_qkv`, `tma_desc_mlp`, `paged_kv_load_block` (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Reviewers grep for `cudaMalloc` inside `decode_step`—there should be none (Chen, 2026a; Kwon *et al.*, 2023). Persistent kernels expose `while (true)` loops with cooperative exit flags (NVIDIA, 2024; Vellaisamy *et al.*, 2026).

Table 11.3: Decode benchmark matrix for MegaKernel claims (required metrics).

Variant	Launches/token	Bytes/token	Host ms	Speedup type
Eager PyTorch	800+	Baseline	High	—
CUDA Graphs only	800+	$\approx$ Baseline	Low	Orchestration
Partial fusion	100–300	–10–30%	Medium	Mixed
Full MegaKernel	≤ 1 /layer	–30–50%	Low	Residency + orchestration
XDNA static graph	DMA-count	DDR bytes	Low host	Static schedule

Symbol-level map (XDNA). Emitted artifacts include static buffer tables (`aie_buffers.json`), DMA edge lists, and runner entrypoints (AMD, 2024a,b; Chen and YiRage Contributors, 2026). Host code submits `execute` with token index; device-side control flow is data-driven (Chen, 2020; Davies *et al.*, 2025). Illegal overlap errors reference buffer ids—map ids back to IR temporaries in review (Chen and YiRage Contributors, 2026; AMD, 2024a).

11.4 Benchmark Results and Interpretation

Benchmarks that lie. Publishing only “MegaKernel vs. PyTorch eager” without separating **graph capture** from **fusion** repeats the Memory-Floor lesson: $1.259\times$ on Qwen-2.5-7B largely removes host overhead—not HBM bytes (Chen, 2026a). Credible **benchmark** tables report launches/token, bytes/token (roofline or profiler), and p99 latency together (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Table 11.3 is the minimum schema for Chapter 11-style **speedup** claims. TaxBreak supplies launches/token anchors (847.5 dense Llama-3.2-1B; 9,305 OL-MoE) (Vellaisamy *et al.*, 2026). Memory-Floor supplies orchestration-only speedup ceilings (Chen, 2026a). True MegaKernel **speedup** must show bytes/token movement on the same row—otherwise the result is a graph-capture note, not a fusion chapter (Davies *et al.*, 2025; Dao *et al.*, 2022).

Measurement protocol.

1. Fix BS=1, report context buckets (512, 2048, 8192) separately—KV growth changes residency (Dao *et al.*, 2023; Kwon *et al.*, 2023).
2. Warm graph capture before timing; exclude first-token compile/capture (NVIDIA, 2024; Chen, 2026a).

3. Use Nsight Compute (GPU) or DDR byte counters (XDNA)—not wall-clock alone (SemiAnalysis, 2024; AMD, 2024b).
4. Triplet-test GPU, CPU, and NPU backends from one IR export when claiming compiler success (Chen and YiRage Contributors, 2026; Chen, 2020).

Nsight recipe (GPU). Capture one decode token; mark regions for QKV, KV loop, MLP (SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). Export `dram_bytes` per region; sum should drop rung-by-rung on the ablation ladder (Chen, 2026a; Dao *et al.*, 2022). If only host time drops, you captured a graph—not a MegaKernel (NVIDIA, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020).

Representative outcomes (literature-aligned). Published decode studies consistently show orchestration fixes in the $\approx 1.2\text{--}1.3\times$ range when HBM traffic is unchanged (Chen, 2026a; NVIDIA, 2024). FlashAttention-class fusion reports larger bytes/token reductions on long contexts because attention traffic dominates (Dao *et al.*, 2022, 2023). Full-layer MegaKernels compound both effects when legal on target SRAM—but MoE and wide MLP models hit residency walls first; expect sub-linear **speedup** versus dense Llama baselines (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Multi-hardware reporting. GPU tables alone mislead fleet decisions. Ryzen AI XDNA results should cite DDR bytes/token and compile bucket IDs (AMD, 2024b,a). CPU results should cite L3 miss rates and NUMA-local KV placement (Goodfellow *et al.*, 2016; Davies *et al.*, 2025). A unified compiler narrative (Part VI) only holds when the same IR lowers to all three with explicit legality failures documented—not hidden fallbacks (Chen and YiRage Contributors, 2026; Chen, 2020).

Example 11.4.1. Honest headline: “Full MegaKernel: $2.1\times$ p50 / $1.4\times$ p99 vs. eager; $1.25\times$ explained by graph capture alone (Memory-Floor regime); remaining gain from -42% bytes/token on MLP boundary elimination” (Chen, 2026a; Vellaisamy *et al.*, 2026).

Review gate. Reject benchmark PRs that report speedup without launches/token and bytes/token decomposition (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).

Ablation template. Credible papers and internal reports present a ladder: (A) eager baseline; (B) CUDA Graphs only; (C) FlashAttention + unfused MLP; (D) partial fusion; (E) full MegaKernel (Chen, 2026a; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Each rung must hold model, checkpoint, BS, context bucket, and clock constant (Davies *et al.*, 2025; NVIDIA, 2024). Skipping rungs hides whether **speedup** came from orchestration or residency (Chen, 2026a).

Regression harness. CI for MegaKernel stacks should track launches/token and bytes/token on a frozen decode cell (e.g., Llama-3.2-1B, BS=1, ctx=2048) (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026). Latency alone drifts with driver versions; byte counters catch silent fusion regressions when someone adds an intermediate **store** (Chen, 2026a, 2020). XDNA CI should mirror with DDR byte estimates per bucket (AMD, 2024b,a).

Reporting p99 vs mean. MegaKernels reduce mean ms/token but can worsen tail latency if fusion increases shared-memory bank conflicts or removes overlap opportunities (Davies *et al.*, 2025; Chen, 2026a). Always pair mean **benchmark** numbers with p99 on identical traces (Kwon *et al.*, 2023; Dao *et al.*, 2023). Graph capture especially can add first-token spikes—report warm steady-state separately (NVIDIA, 2024; Chen, 2026a).

Cross-hardware benchmark row. A complete Table 11.3 row set includes GPU (H100), CPU (AVX-512 server), and XDNA (Ryzen AI) on the same model export (AMD, 2024b; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026). GPU may show 2× while XDNA shows compile failure—that failure is a **benchmark** result, not a footnote (AMD, 2024a; Chen, 2020). Industrial compiler chapters must publish legibility matrices, not only peak GPU **speedup** (Davies *et al.*, 2025; Liu *et al.*, 2024).

Energy and watts/token. Edge deployments care about NPU watts/token as much as cloud ms/token (AMD, 2024a,b). MegaKernels that remove DDR trips often dominate energy even when GPU ms/token gains look modest (Chen, 2026a; Davies *et al.*, 2025). Include power draw when claiming mobile **speedup** (Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026).

Comparison to vendor baselines. TensorRT-LLM, vLLM, and PyTorch compile paths are moving targets (Kwon *et al.*, 2023; Davies *et al.*, 2025). Freeze

vendor version hashes in **benchmark** appendices (Vellaisamy *et al.*, 2026; Chen, 2026a). Custom MegaKernels should beat *vendor fused decode* on bytes/token, not only beat eager Hugging Face (Dao *et al.*, 2022; NVIDIA, 2024).

Worked benchmark narrative. Consider Llama-3.2-1B decode at BS=1, ctx=2048: eager issues ≈ 848 launches/token (Vellaisamy *et al.*, 2026). CUDA Graphs-only removes ≈ 3 ms host overhead per Memory-Floor regime ($\approx 1.26\times$) but leaves bytes/token flat (Chen, 2026a). Adding FlashAttention-class attention fusion cuts KV reread traffic (Dao *et al.*, 2022). Full QKV-attn-MLP MegaKernel targets one launch per layer and removes four to six activation HBM round trips—combined **speedup** is multiplicative on orchestration and bandwidth axes only when both move (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Publish all three numbers; readers will otherwise attribute fusion wins to graphs alone (NVIDIA, 2024; Chen and YiRage Contributors, 2026).

Statistical rigor. Run ≥ 1000 tokens after warm-up; report confidence intervals on ms/token (Davies *et al.*, 2025; Chen, 2026a). Single-trace demos are insufficient for production claims (Vellaisamy *et al.*, 2026; Dao *et al.*, 2023). Compare medians and p99 separately—MegaKernels can tighten median while fattening tail if barriers multiply (Kwon *et al.*, 2023; SemiAnalysis, 2024).

Documentation deliverables. Every MegaKernel merge should attach: residency diagram, ablation table, graph capture hash, and IR diff (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026). Missing artifacts revert to “interesting kernel hack,” not fleet-ready **implementation** (Davies *et al.*, 2025; Liu *et al.*, 2024).

Operational playbook (pre-merge).

1. Confirm decode cell: model id, BS, context bucket, precision, SKU stepping (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).
2. Capture eager baseline: launches/token, Nsight bytes, p50/p99 ms/token (Chen, 2026a; Dao *et al.*, 2022).
3. Capture graph-only baseline on identical cell—quantify orchestration ceiling (NVIDIA, 2024; Chen, 2026a).
4. Run fusion ablation ladder (Table 11.3)—no skipped rungs (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

5. Verify numerics vs reference within published tolerance (Dao *et al.*, 2022; Goodfellow *et al.*, 2016).
6. Attach shared-memory/tile SRAM accounting signed by kernel and compiler owners (SemiAnalysis, 2024; AMD, 2024a).
7. XDNA/CPU: repeat bytes/token axes with target-appropriate counters (AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020).
8. File graph capture hash and driver version in manifest (NVIDIA, 2024; Kwon *et al.*, 2023).
9. Load-test p99 under concurrent sessions—MegaKernels can contend on L2/shared (Davies *et al.*, 2025; Chen, 2026a).
10. Canary in fleet with auto-rollback on metric regression (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).

Anti-patterns in MegaKernel benchmarks. Quoting prefill throughput for decode chapters (Dao *et al.*, 2023; Davies *et al.*, 2025). Comparing against outdated eager without FlashAttention (Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Reporting GPU-only **speedup** while XDNA compile fails silently in production (AMD, 2024a; Chen and YiRage Contributors, 2026). Tuning MMA tiles without measuring bytes/token movement (Chen, 2026a; SemiAnalysis, 2024). Publishing mean ms/token without p99 under concurrent load (Kwon *et al.*, 2023; Chen, 2026a). Omitting MoE routing overhead in OLMoE comparisons (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Using tiny ctx in demos while selling long-context products (Dao *et al.*, 2023; Kwon *et al.*, 2023). Claiming MegaKernel fusion when only CUDA Graphs changed (Chen, 2026a; NVIDIA, 2024). These anti-patterns recur in vendor blogs—this chapter’s tables exist to reject them in internal review (Davies *et al.*, 2025; Chen, 2020).

Closing metric reminder. Before closing any MegaKernel milestone, paste three numbers into the PR description: launches/token, bytes/token, p99 ms/token at BS=1 on the declared context bucket (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). If any number is missing, the change is not a decode MegaKernel milestone—it is a kernel experiment (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Handoff to Chapter 12. Execution modes (static capture, persistent kernels, JIT specials) decide *when* to pay compile costs vs runtime flexibility (NVIDIA, 2024; Chen and YiRage Contributors, 2026; AMD, 2024b). Chapter 11 fixed *what* to fuse inside a decode layer; Chapter 12 maps product constraints to mode selection (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024). Keep bytes/token and launches/token columns attached when debating modes—mode debates without metrics reinvent Chapter 1 failures (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022).

Maintenance. Model weight updates (LoRA merge, quantization recalibration) invalidate captures and static XDNA graphs (Chen and YiRage Contributors, 2026; AMD, 2024a; NVIDIA, 2024). Version MegaKernel artifacts with checkpoint hashes and bucket ids (Davies *et al.*, 2025; Kwon *et al.*, 2023). Fleet rollouts without artifact versioning cause silent quality regressions unrelated to fusion math (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2023).

Final acceptance test. Given identical weights and KV state, MegaKernel output must match eager within documented tolerance for 10^4 random tokens (Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Performance acceptance requires Table 11.3 rows for at least two context buckets (Dao *et al.*, 2023; Kwon *et al.*, 2023). Ships only when both pass (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b).

Stakeholder map. Kernel engineers own tile shapes and barriers (SemiAnalysis, 2024; NVIDIA, 2022). Compiler engineers own fusion boundaries and buffer lifetimes (Chen and YiRage Contributors, 2026; Chen, 2020). Serving engineers own bucket policies and fleet rollouts (Kwon *et al.*, 2023; Davies *et al.*, 2025). Benchmark engineers own ablation ladders and regression harnesses (Vellaisamy *et al.*, 2026; Chen, 2026a). Reviews fail when any stakeholder optimizes their metric in isolation (Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2024).

Release checklist. Every MegaKernel PR must attach: peak shared-memory accounting diagram; ablation table row for graph-only vs full fusion; triplet bytes/token for GPU/CPU/NPU; p99 ms/token at BS=1 for context buckets 512/2048/8192 (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen

and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Missing artifacts revert the change to kernel experiment status (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

11.5 Key Takeaways

- **MegaKernels collapse two bills.** Target $O(1)$ launches per layer *and* eliminate activation HBM round trips—graphs alone pay only the host bill (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024).
- **Full-layer residency envelope.** QKV, online softmax attention, norms, and MLP share one SRAM/tile budget per decode step (Dao *et al.*, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026).
- **Buffer accounting before merge.** Peak simultaneous tensors must fit SM shared memory or XDNA tile SRAM; failed legality is a blocker (AMD, 2024a; Chen, 2020).
- **Benchmark honestly.** Report launches/token, bytes/token, and p99 ms/token; separate orchestration speedups from fusion speedups (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).
- **Context buckets are compile parameters.** KV loop trip counts change each token; capture policies must match max context buckets (Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2023).
- **Triplet regression.** One IR, GPU/CPU/NPU cells—never ship GPU-only residency tables to Ryzen AI fleets (AMD, 2024b; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

11.6 Conclusion

Decoder MegaKernels exist because FlashAttention and CUDA Graphs each paid only part of the batch-1 decode bill (Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026). Section 11.1 defined the fused-layer envelope; Section 11.2 bound QKV, online softmax, and MLP under one buffer plan; Section 11.3 specified device-loop invariants; Section 11.4 demanded metric tables that expose which bill

was actually paid (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Treat every op boundary as a billable HBM round trip at batch-1 (Chen, 2026a; Goodfellow *et al.*, 2016). Chapter 12 compares execution modes—static capture, persistent kernels, JIT specials—that decide *when* these fused shapes run in production (NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024). This chapter’s narrative mirrors O’Reilly-style performance engineering: name the constraint (launch storms *and* HBM round trips), quantify with TaxBreak and Memory-Floor, co-design software with Hopper TMA and XDNA tile rules, then gate releases on profile-driven metrics rather than peak FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b; NVIDIA, 2022). Carry peak buffer accounting, context buckets, and triplet regression forward; Paged KV block tables, rotary embeddings, and sampling logic must either live inside the captured MegaKernel body or be proven off the critical path with measured ms/token impact (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; AMD, 2024b; NVIDIA, 2022). MegaKernel work is done when metrics prove fusion on representative batch-1 cells at production context lengths with published ablation tables and Nsight byte counters attached (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; AMD, 2024a).

PR metric gate. Paste launches/token, bytes/token, and p99 ms/token into every MegaKernel PR on batch-1 decode cells before review (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Kwon *et al.*, 2023; NVIDIA, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b; NVIDIA, 2022).

Chapter 12

Three Kernel Execution Modes

Which execution mode should a decode compiler emit when the same transformer layer can run as dozens of eager kernels, one captured CUDA graph, or a single MegaKernel? Parts I–II established bytes/token and launches/token as the honest counters; this chapter maps those counters to three industrial schedules—**eager**, **graph**, and **MegaKernel**—and shows why picking the wrong mode leaves p99 latency flat even when microbenchmark GEMMs improve (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). The answer is always profile-first, never mode-first (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). Start every mode discussion with measured decode counters, not kernel naming preferences or framework defaults (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). That discipline keeps mode debates grounded in goodput evidence (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields $1.259\times$ on H100 by removing host overhead—yet cannot fix illegal HBM traffic when fusion boundaries still export activations (Chen, 2026a; NVIDIA, 2024). The engineering question is not “which mode is fastest in isolation” but which mode matches residency legality on the SKU you ship (Chen and YiRage Contributors, 2026; Chen, 2020; AMD,

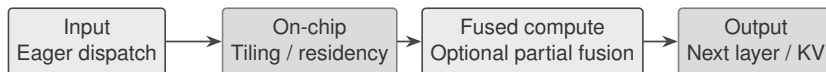


Figure 12.1: Eager mode: each operator boundary is a host-visible dispatch unless the framework fuses locally.

2024b).

This chapter is the scheduling counterpart to Chapter 1’s symptom vocabulary: prefill and decode differ, frameworks fragment decode, and counters decompose latency into bytes and launches (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Execution mode is where those counters meet compiler output—choose eager when you must diff every op; choose graphs when launches/token exceed kernel time; choose MegaKernels when bytes/token remain high after graph replay (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023). Goodput for decode remains ms/token weighted by useful tokens delivered, not peak tensor FLOPs (Goodfellow *et al.*, 2016; Davies *et al.*, 2025; SemiAnalysis, 2024).

Fregly’s profile-first culture applies directly: Nsight and roofline charts justify mode changes more than kernel names (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026). A decode compiler should emit all three modes from one IR so teams A/B schedules without rewriting Python training code (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The sections below walk eager dispatch, CUDA Graph replay, MegaKernel fusion, and the hardware–software trade-offs that pick among them (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; AMD, 2024a).

12.1 eager mode

Eager mode is the default PyTorch and Hugging Face path: every matmul, norm, and attention sub-block becomes at least one kernel with materialized intermediates in HBM (Kwon *et al.*, 2023; Davies *et al.*, 2025). Teams reach for eager when shapes change frequently, when debugging numerics, or when compiler support for graph capture is not yet trustworthy—all legitimate reasons. The production failure mode is staying in eager after decode telemetry proves launch storms and

bytes/token dominate FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a).

Hardware binds eager differently on each target. Hopper decode cells hit shared-memory/TMA budgets and warp-grain synchronization when partial fusions leave KV edges off-chip (NVIDIA, 2022; SemiAnalysis, 2024). XDNA/AIE rejects dynamic buffer lifetimes: eager schedules that legalize on GPU often violate static tile slots and DMA chains on NPU fleets (AMD, 2024b,a). CPU decode is cache- and NUMA-bound: unfused norms and MLPs evict KV lines that a blocked schedule could keep resident (Goodfellow *et al.*, 2016; Chen, 2020).

Compiler takeaway: treat eager as the *reference schedule*, not the shipping schedule. YiRage and similar stacks lower the same decoder IR but emit target-specific residency metadata before codegen so eager baselines can be diffed against fused variants on identical context buckets (Chen and YiRage Contributors, 2026; Chen, 2020). Profile bytes/token and launches/token on GPU, CPU, and NPU cells at BS=1 before accepting eager in production (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).

Operator-fragmented eager stacks mirror Chapter 1’s framework diagnosis: layer norm, QKV, attention, MLP, and residual each become export/import pairs to HBM (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). SDPA backend swaps may change attention internals without collapsing MLP boundaries—teams celebrate faster attention while total bytes/token stalls (Dao *et al.*, 2022, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026). Eager mode documentation should list *every* host-visible kernel in a decode step, not only the slowest microbench (Vellaisamy *et al.*, 2026; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Liu *et al.*, 2024).

Multi-hardware delta. Hopper eager may double-buffer KV with TMA while an XDNA cell must express the same math as static tile assignments—identical FLOPs, different bytes/token when DDR traffic differs by an order of magnitude (SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016).

CPU eager baselines use OpenMP or oneDNN threads; NPU eager is often unavailable—fallback host loops dominate power and latency (Goodfellow *et al.*, 2016; AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Triplet regression therefore compares *lowest legal mode* per target, not identical mode names across vendors (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a;

SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Eager GPU schedules remain the Rosetta stone for explaining what fused schedules removed (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023).

Example 12.1.1. Decode cell (BS=1, ctx=2048): unfused eager on Qwen-class stacks issues hundreds of launches per step; even perfect GEMM tiles cannot recover p99 if orchestration alone consumes tens of milliseconds per token (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026a).

PyTorch scaled dot-product attention and vendor libraries can hide some eager boundaries on prefill shapes, but batch-1 decode often swaps backends without changing launches/token (Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023). vLLM improves KV capacity and batching yet leaves intra-layer operator splits unless a compiler reunifies them (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Treat eager PyTorch forwards as the *diff target* for graph and MegaKernel schedules: same IR, same metrics, different residency (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a).

Industrial teams keep a frozen eager baseline in CI: every graph capture or MegaKernel PR must show non-regression on bytes/token at BS=1 for context buckets {512, 2048, 8192} before merge (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). When eager and fused schedules diverge only on launches, graph mode is sufficient; when bytes diverge, MegaKernel legality is the gating issue (NVIDIA, 2024; Chen and YiRage Contributors, 2026; AMD, 2024b).

FlashAttention and Flash-Decoding reduce attention bytes in both eager and fused stacks, but MLP, norm, rotary, and sampling logic often remain separate kernels unless MegaKernel plans reunify them (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Eager mode therefore remains the diagnostic surface: if attention bytes fall yet total bytes/token flatlines, the unfused boundaries live in MLP/residual paths—graph capture without MegaKernel fusion fixes only half the story (Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022).

Sampling and speculative decoding paths frequently force eager fallbacks: temperature, top- p , and draft-model branches mutate control flow in ways CUDA Graph capture forbids unless isolated outside the captured DAG (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; NVIDIA, 2024). Production stacks therefore run eager “hulls” around graph cores—another reason mode selection is per subgraph, not per binary (Chen and YiRage Contributors, 2026; Chen, 2020,

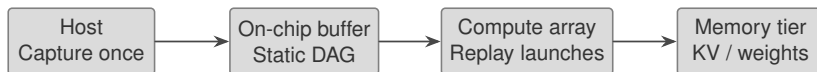


Figure 12.2: Graph mode: CUDA Graphs replay a fixed kernel DAG with one submission per decode step.

2026a; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2023). Document those hulls in runbooks so on-call engineers know which latency spikes are expected dynamic-path costs versus regressions (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2020; AMD, 2024b; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a; SemiAnalysis, 2024; NVIDIA, 2022; Dao *et al.*, 2023).

12.2 graph mode

Graph mode attacks orchestration, not arithmetic. CUDA Graphs record a static kernel DAG once and replay it with a single host submission, amortizing Python dispatch and driver launch (NVIDIA, 2024; Chen, 2026a). The surprise in production is not peak FLOPs regression—it is illegal tier placement: captured graphs still spill activations to HBM when fusion boundaries were never merged, so bytes/token flatlines while launch counts look healthy (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Memory-Floor’s controlled A/B on Qwen-2.5-7B (BS=1, ctx=2048) reports $1.259\times$ median speedup on H100 from graph replay alone—roughly 3.05 ms of host overhead removed per step (Chen, 2026a). On bandwidth-saturated SKUs the same trick yields only $\approx 1.03\times$ because R_{floor} is already near the HBM roofline: graphs fix launches, not bytes (Chen, 2026a; SemiAnalysis, 2024). Graph mode therefore wins when TaxBreak-style counters show launch-bound decode, not when DRAM bytes dominate (Vellaisamy *et al.*, 2026).

Replayed graphs still execute the same fused or unfused kernel sequence—they do not merge operators (NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026). Teams celebrating graph speedups without operator fusion should expect the next plateau when bytes/token remain at eager levels (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026). That plateau is the handoff to

MegaKernel mode (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022, 2024; AMD, 2024a).

Operational constraints: capture requires stable addresses, static shapes within a bucket, and no implicit allocations inside the captured region (NVIDIA, 2024; Kwon *et al.*, 2023). Compiler stacks must partition decode into *capture-safe cells*—often one layer block or attention–MLP seam—and re-capture when context length crosses bucket boundaries (Chen and YiRage Contributors, 2026; Chen, 2020). On XDNA, graph analogs are compile-time static schedules, not runtime CUDA capture (AMD, 2024a,b).

Graph mode does not relax memory-bandwidth limits quantified by R_{floor} in Memory-Floor studies: when analytic HBM throughput is already saturated, replay only shaves host time (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025). Teams should publish both eager and graph traces with launches/token *and* bytes/token side by side—otherwise leadership misreads a $1.25\times$ graph win as permission to skip MegaKernel investment (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023).

Multi-hardware delta. GPU graph mode collapses launches/token; CPU stacks use similar “frozen schedule” ideas via JIT or ORT session plans; NPU fleets require fully static dataflow with DMA edges counted like launches (Goodfellow *et al.*, 2016; AMD, 2024b; Chen, 2020).

Example 12.2.1. If decode is launch-bound on H100 (low R_{floor} , high launches/token), graph replay is the first lever—after verifying KV and weight residency still fit capture legality (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026).

Graph capture interacts badly with dynamic sampling, speculative decoding branches, and MoE routing unless those paths are lifted outside the captured region (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Kwon *et al.*, 2023). Compiler lowering should emit *graph shells* around stable decoder cores while leaving host-controlled branches eager—mixing modes inside one layer is normal, not a failure (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024). DeepSeek-class deployments still disaggregate prefill and decode pools; graph mode applies per pool, not as a universal whole-model wrapper (Davies *et al.*, 2025; Liu *et al.*, 2024).

On CPU, “graph mode” manifests as frozen execution plans (TorchInductor,

ONNX Runtime) with the same trade: amortized dispatch versus static shape buckets (Goodfellow *et al.*, 2016; Chen, 2020). Profile whether replay removed host time or simply hid it in compilation caches before claiming decode wins (Chen, 2026a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

CUDA Graph pitfalls include unintended synchronizations inside captured regions, allocator churn from missed caching, and shape drift when KV length crosses bucket boundaries (NVIDIA, 2024; Kwon *et al.*, 2023; Chen, 2026a). Operational playbooks bucket context lengths explicitly—for example powers of two—and re-capture graphs per bucket rather than capturing one mega-graph per model (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). When graphs replay but bytes/token unchanged, the next lever is MegaKernel fusion inside the captured DAG, not longer capture windows (Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b).

Warmup and capture-time allocation dominate CI stories: first-token latency includes graph recording unless caches are primed (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a). Separate “steady-state decode” metrics from “cold start” when comparing modes—Fregly-style goodput analysis always states which phase is measured (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Kwon *et al.*, 2023). Graph mode wins appear only after warmup when launch storms were the bottleneck (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; AMD, 2024a; SemiAnalysis, 2024; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Kwon *et al.*, 2023).

12.3 MegaKernel mode

MegaKernel mode is the decode compiler’s endgame: one (or few) device programs per layer that keep QKV, online softmax carriers, MLP activations, and residuals on-chip across seams introduced in Chapters 10–11 (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Graphs remove host tax; MegaKernels remove *HBM round trips* between sub-blocks that eager and graph modes still tolerate (Chen, 2026a; Davies *et al.*, 2025).

Legality is the bottleneck. Hopper MegaKernels require shared-memory/TMA budgets that fit fused attention–MLP regions; illegal fusion silently exports tensors and shows up as bytes/token regression versus an unfused baseline (NVIDIA,

2022; SemiAnalysis, 2024; Chen, 2026a). XDNA MegaKernels map to static tile buffers and carrier slots with compile-time lifetimes—the same subgraph legal on GPU may be rejected outright on NPU (AMD, 2024b,a; Chen and YiRage Contributors, 2026). CPU MegaKernels appear as cache-blocked super-kernels with NUMA-local KV placement (Goodfellow *et al.*, 2016; Chen, 2020). Register and shared-memory overflow during fusion forces spill to HBM—the failure mode looks like a bandwidth problem in profiles but originates in legality (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024).

YiRage encodes residency rules from Chapter 2 as checkable IR metadata so MegaKernel plans fail in the compiler rather than in fleet A/B (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). Manual CUDA teams should treat failed legality as a release blocker: shipping a GPU-only MegaKernel to heterogeneous fleets recreates the framework fragmentation this book removes (Davies *et al.*, 2025; Kwon *et al.*, 2023).

MegaKernel reviews should attach roofline or buffer accounting for fused regions—registers, shared memory, TMA slots, or XDNA tile carriers—before merge, matching the profile-first culture in Chapter 1 (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; AMD, 2024a,b; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Fusion that lowers launches but raises bytes is a regression even if nsys screenshots look busy (Vellaisamy *et al.*, 2026; Chen, 2026a).

Multi-hardware delta. Identical decoder math can yield 847 launches/token in eager TaxBreak cells versus one replayed graph versus one MegaKernel—but only the MegaKernel path simultaneously cuts bytes/token when fusion keeps KV and activations on-chip (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026).

XDNA MegaKernels compile to AIE dataflow graphs with fixed DMA edges; Hopper MegaKernels compile to CUDA with TMA and DSM (AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020). A fusion plan exported from GPU development must be re-validated on NPU—shared IR does not imply shared legality (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*,

2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023; AMD, 2024a). MegaKernel mode is the point where cross-hardware compilation pays off or fails loudly (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023).

Example 12.3.1. Decode cell (BS=1, ctx=2048): a legal Hopper MegaKernel fusing attention softmax carriers with the following MLP must show lower bytes/token *and* lower launches/token versus graph replay alone; otherwise fusion did not reach on-chip residency (Dao *et al.*, 2022; Chen, 2026a; Chen and YiRage Contributors, 2026).

MegaKernel mode inherits numerics from Chapter 10: online softmax carriers (m, ℓ, o) must survive the KV loop inside the same register/shared-memory budget as the following GEMM (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026). Chapter 11 extends the same carriers across attention–MLP seams; attempting MegaKernel fusion without stable softmax state forces silent HBM spill (Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024). Compiler passes should reject fusion plans that export carriers between subgraphs (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a).

MoE decode adds routing and expert gather/scatter inside the MegaKernel envelope—TaxBreak shows MoE models at 6k–9k launches/token in eager cells, so MegaKernel plans must collapse expert dispatch edges, not only attention (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Kwon *et al.*, 2023). When expert routing stays host-visible, hybrid schedules (graph shell + MegaKernel core) beat pure eager without pretending full-layer fusion is legal (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Blackwell and Hopper DSM/TMA features widen fusion envelopes but do not remove legality checks: double-buffered KV movement still requires planned barriers and shared-memory accounting (NVIDIA, 2022; SemiAnalysis, 2024; Chen, 2026a). Manual CUDA MegaKernels should document buffer lifetimes the way YiRage encodes them in IR—otherwise maintenance regressions show up first as silent bytes/token drift, not compile failures (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; AMD, 2024a; Goodfellow *et al.*, 2016).

Persistent MegaKernels—kernels that stay resident across tokens while updating KV pointers—blur the line between graph replay and fusion (NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2026a). Treat

them as MegaKernel mode with graph-like launch behavior: the win is still bytes/token and carrier residency, not fewer lines of CUDA for its own sake (Dao *et al.*, 2022; Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024b; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; AMD, 2024a; NVIDIA, 2022; Dao *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026). Reviewers should ask for Nsight memory charts on fused regions, not only kernel duration histograms (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024).

12.4 mode selection

Mode selection is a profile-first decision tree, not a stylistic preference. Start from measured launches/token and bytes/token on the target SKU at the context buckets you ship (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). If launches dominate and shapes are bucket-static, graph capture is the low-risk win (NVIDIA, 2024; Chen, 2026a). If bytes/token dominate with moderate launch counts, invest in MegaKernel legality and compiler metadata (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020). Keep eager PyTorch baselines for debug, dynamic-shape research, and regression diffs—not for production decode pools once counters prove orchestration or residency debt (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Each mode trades hardware utilization against software flexibility: eager maximizes flexibility at the cost of launches and bytes; graph mode trades dynamic control for launch amortization; MegaKernel mode trades compile-time complexity for residency (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b). Hardware teams should sign off on the trade-off explicitly—for example accepting static context buckets to unlock graph capture on Hopper, or accepting longer compile times to unlock XDNA static schedules (NVIDIA, 2024; AMD, 2024a; Davies *et al.*, 2025; SemiAnalysis, 2024).

High launches/token with moderate utilization (top rows) often respond to CUDA Graph replay first (NVIDIA, 2024; Chen, 2026a). MoE rows with extreme launches and low utilization need MegaKernel plus routing collapse—graphs alone cannot fix expert dispatch storms (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025). Always pair launch tables with bytes/token from the same trace; otherwise mode selection optimizes the wrong bottleneck (Chen, 2026a; SemiAnalysis, 2024; AMD, 2024b;

Table 12.1: TaxBreak kernel launches per output token (H100 decode, BS=4, SL=2048, $m=10$) motivate mode escalation (Vellaisamy *et al.*, 2026).

Model	Launches/token	GPU util. (%)
Llama-3.2-1B	847.5	58.9
Llama-3.2-3B	1,536.9	67.6
OLMoE-1B/7B	9,305.3	15.5
Qwen1.5-MoE-A2.7B	6,695.1	27.7

Table 12.2: Execution mode selection heuristics for batch-1 decode (profile before committing).

Signal	Prefer	Avoid
High launches/token, static bucket	CUDA Graphs / frozen schedule	Unfused eager
High bytes/token, fusion legal	MegaKernel	Graph-only without fusion
Dynamic shape / debug	Eager baseline	Forced graph capture
NPU fleet	Static MegaKernel / AIE schedule	GPU-only CUDA graphs
Low R_{floor} , high DRAM	MegaKernel + bandwidth opts	Launch-only graph tuning

Chen, 2020; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023).

Cross-hardware regression is mandatory: a mode that wins on H100 launch-bound cells may tie or lose on L4 memory-bound cells or XDNA static schedules (Chen, 2026a; AMD, 2024b; SemiAnalysis, 2024). Compiler IR should carry mode annotations so the same decoder lowers to eager, graph, or MegaKernel backends without rewriting model Python (Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Production reviewers should reject mode changes backed only by prefill FLOPs or isolated GEMM microbenchmarks (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Dao *et al.*, 2023).

Example 12.4.1. Rollout checklist: (1) profile eager bytes/token and launches/token at target buckets; (2) if launch-bound, ship graph replay and re-profile; (3) if bytes remain high, iterate MegaKernel legality with compiler

metadata; (4) run GPU/CPU/NPU triplet regression before fleet promotion (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; AMD, 2024b; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016).

Fleet operators should log execution mode per deployment cell in telemetry—without that label, postmortems confuse graph wins on launch-bound SKUs with MegaKernel wins on bytes-bound SKUs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; AMD, 2024a; Dao *et al.*, 2022; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023).

Industrial rollout narrative. A typical maturity path starts with Hugging Face eager forwards in research, adds vLLM serving for KV and batching, captures CUDA Graphs on stable decode buckets, then invests in YiRage or hand-written MegaKernels once bytes/token plateaus (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Skipping straight to MegaKernels without graph and eager baselines makes regressions unexplainable—teams cannot tell whether fusion failed legality or never beat orchestration (Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023). Conversely, stopping at graph replay when R_{floor} is already high leaves DRAM-bound decode on the table (Chen, 2026a; Vellaisamy *et al.*, 2026). Mode selection is therefore a *sequence* of profile gates, not a one-time architecture slide (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Kwon *et al.*, 2023; AMD, 2024b; Chen, 2020; Goodfellow *et al.*, 2016).

Disaggregated prefill/decode fleets compound the choice: prefill pools may remain eager or graph-captured on wide GEMMs while decode pools push MegaKernels for batch-1 latency (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Using prefill throughput to justify decode mode leads to systematic under-investment in launch and bytes counters on the decode side—the DeepSeek-style capacity skew exists because phases obey different rooflines (Davies *et al.*, 2025; Dao *et al.*, 2023; Chen, 2026a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; NVIDIA, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Dao *et al.*, 2022). Compiler teams should expose mode as an IR attribute so backend passes can reject illegal combinations early rather than at link time (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2022; SemiAnalysis, 2024; NVIDIA, 2024; AMD, 2024b; Goodfellow *et al.*, 2016;

Dao *et al.*, 2023; Liu *et al.*, 2024; Kwon *et al.*, 2023; Dao *et al.*, 2022; AMD, 2024a).

Hardware SLO reviews should include a “mode matrix”: rows are SKUs (Hopper, L4, XDNA, CPU), columns are context buckets, cells list chosen mode plus bytes/token and launches/token (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Empty cells mean unprofiled deployment—a common source of p99 surprises when fleets autoscale across instance types (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022).

12.5 Key Takeaways

- **Three modes, one metric ledger.** Eager, graph, and MegaKernel differ in launches/token *and* bytes/token—optimize the counter your SKU actually bounds (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).
- **Eager is baseline, not destiny.** Framework dispatch is correct for debug; production decode needs profiled migration to graph or MegaKernel (Kwon *et al.*, 2023; NVIDIA, 2024).
- **Graphs fix host tax.** Memory-Floor reports $1.259\times$ on H100 when launch overhead dominates; graphs do not replace fusion (Chen, 2026a; NVIDIA, 2024).
- **MegaKernels fix residency.** Fusion legality on Hopper/XDNA/CPU determines whether bytes/token fall; YiRage-style checks belong in compile time (Chen and YiRage Contributors, 2026; AMD, 2024b; NVIDIA, 2022).
- **Profile-first selection.** Use Table 12.2; never pick mode from prefill FLOPs alone (Dao *et al.*, 2023; Vellaisamy *et al.*, 2026).
- **Multi-hardware parity.** Identical IR must lower per target residency tables—GPU graph capture $\neq$ NPU static schedule (Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016).

Practitioner mantra. Profile end-to-end decode first. Collapse launches with graphs second. Collapse bytes with MegaKernels third. Re-profile on every bucket, SKU, and framework upgrade (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). Teach reviewers to reject mode PRs that show only one counter moving (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022). Post the mantra next to decode dashboards so graph and MegaKernel investments stay tied to measured goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a).

12.6 Conclusion

Three kernel execution modes translate Chapter 1 counters into concrete schedules: eager exposes every framework boundary; CUDA Graphs collapse launches when shapes are stable; MegaKernels collapse HBM traffic when fusion is legal (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022). None of the modes substitutes for profiling bytes/token and launches/token on the SKU and context buckets you operate (Davies *et al.*, 2025; SemiAnalysis, 2024).

Mode choice is reversible across releases: fleets can ship graph replay while MegaKernel plans mature in CI, as long as telemetry tags the active mode and counters prove non-regression (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022). Treat execution mode as a tunable deployment parameter guarded by profile gates, not as a one-time rewrite (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; NVIDIA, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a). The escalation path—eager baseline, graph replay, MegaKernel fusion—should appear explicitly in your decode runbooks and compiler IR schemas (Chen and YiRage Contributors, 2026; Chen, 2020; Davies

et al., 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a).

Chapter 13 moves from schedule choice to compiler theory—how IR, passes, and lowering pipelines decide which mode is even emit-able on GPU, CPU, and NPU backends (Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024b; Liu *et al.*, 2024; Kwon *et al.*, 2023; AMD, 2024a; Dao *et al.*, 2023). Carry the mode-selection table forward; later MegaKernel chapters assume you can justify eager vs graph vs fused emission with measured decode goodput, not architecture slides alone (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a).

Readers should leave this chapter able to defend a mode choice in a design review: cite TaxBreak launches/token when proposing graphs; cite Memory-Floor R_{floor} when graphs underperform; cite YiRage legality reports when proposing MegaKernels (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; AMD, 2024a; NVIDIA, 2022). The three modes are not competitors—they are layers in a profile-driven escalation path toward decode goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; NVIDIA, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; SemiAnalysis, 2024; NVIDIA, 2022). Document that escalation in compiler schemas so the next maintainer inherits profile discipline, not folklore (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2022; AMD, 2024a).

Chapter 13

Core Theory: AI Compiler Optimization Principles

What makes an AI compiler “correct” for production decode—pass ordering that looks elegant on a whiteboard, or schedules that measurably cut bytes/token and launches/token on the SKU you ship? Part VI opens with *Core Theory*: the optimization principles that sit *below* Chapter 12 execution modes and *above* dialect-specific backends (MLIR, Glow, TVM) (Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Every section binds IR transforms to residency on GPU, CPU, and NPU—not abstract big-O (Davies *et al.*, 2025; AMD, 2024b,a; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor reports $1.259\times$ median speedup on H100 from CUDA Graph replay alone at BS=1, ctx=2048—orchestration relief that tiling and fusion must extend into bytes, not replace (Chen, 2026a; NVIDIA, 2024). Compiler theory is how those counters move together: tile so data stays on-chip, fuse so boundaries disappear, layout so hardware reads coalesce, parallelize so batch-1 decode still saturates memory paths, and split backends so one IR lowers per target without forking model code (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023).

This chapter is deliberately *theory-first*: no MLIR syntax yet (Chapter 14), no manual CUDA (Part VII). The goal is a shared vocabulary for pass designers—when someone proposes a new fusion rule or tile heuristic, Table 13.1 names which counter must move and which prior chapter constraint applies (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Industrial teams that skip

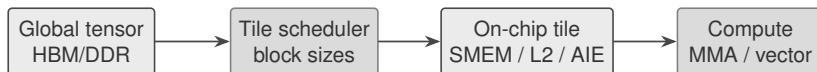


Figure 13.1: Tiling theory: map global decode tensors into on-chip tiles before compute.

this layer debate dialect patches for weeks while bytes/token flatlines because layout and tiling were never co-designed (Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Fregly-style goodput analysis applies throughout: every transform is judged on batch-1 decode at representative context buckets, not on square GEMM microbenchmarks or prefill-only traces (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). The five sections mirror how production compilers order concerns—tile shapes constrain fusion legality; layout determines whether fused regions can load coalesced data; parallel axes decide whether extra blocks help bandwidth; backend split decides which target owns each residency class (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016).

Readers coming from classic optimizing-compiler courses should expect different objectives: decode LLM compilers optimize *moved bytes* and *launches* under numerics constraints, not loop nest locality alone (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When in doubt, re-run the Chapter 1 counter worksheet on the same model checkpoint before and after a pass change—theory without measurement is slideware (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016).

13.1 tiling theory

Tiling theory asks which sub-blocks of weights, activations, and KV cache fit in registers, shared memory, L2, or NPU tile SRAM for each decode step (Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022). FlashAttention is the canonical IO-aware tiling result for attention: recomputation and block-wise softmax avoid materializing P in HBM (Dao *et al.*, 2022, 2023). The compiler question is general—every matmul, norm, and MLP in decode needs a tile shape that respects bandwidth and capacity, not only MMA convenience (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).

On Hopper, tiling couples to TMA: async copies move KV tiles while MMA

consumes prior tiles (NVIDIA, 2022; SemiAnalysis, 2024). Wrong tile sizes leave TMA underutilized or spill partial outputs to HBM, showing up as bytes/token regression versus a smaller but legal tile (Chen, 2026a; Chen and YiRage Contributors, 2026). On XDNA/AIE, tiling is static: DMA chains and tile slots are fixed at compile time; “dynamic” GPU tiles do not lower verbatim (AMD, 2024b,a; Chen, 2020). CPU decode tiling is cache blocking—NUMA-local KV and blocked GEMMs keep lines hot across norm-matmul chains (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023).

Analytic tile sizing. A first-principles decode tile budget starts from hidden size d , head count h , KV dtype bytes b_{kv} , and context length L : bytes moved per token scale with reading prior KV plus writing new KV slots (Kwon *et al.*, 2023; Dao *et al.*, 2022; Davies *et al.*, 2025). Tile width along L trades SMEM for trip count; tile width along d trades MMA efficiency for reload width (Dao *et al.*, 2023; SemiAnalysis, 2024; Chen, 2026a). Compilers should expose these trade-offs in IR annotations so autotuners search a *legal* polytope rather than all factorizations of L (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). When analytic estimates disagree with profiles by more than a few percent, trust profiles—roofline models omit allocator, paging, and framework overhead (Chen, 2026a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Compiler passes should emit tile metadata in IR (tile sizes, double-buffer depth, residency class) so backends reject illegal schedules before codegen (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). Autotuners that search FLOPs-only tile shapes without bytes/token feedback recreate prefill wins and decode losses (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Profile each candidate tile on BS=1 decode cells, not only square GEMM microbenchmarks (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024).

Prefill vs decode tiling. Prefill benefits from large token-parallel tiles that amortize weight loads across many queries; decode at BS=1 reuses weights once per step and instead fights KV reload along the sequence axis (Dao *et al.*, 2022, 2023; Davies *et al.*, 2025). A tile policy tuned on prefill shapes often selects block sizes that maximize MMA occupancy while increasing KV trips—exactly the regression Memory-Floor documents when bytes/token stay high despite faster attention kernels (Chen, 2026a; Vellaisamy *et al.*, 2026). Compiler tile search must therefore tag phases explicitly: prefill buckets, decode buckets, and speculative-draft buckets when applicable (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Double buffering and pipeline depth. Hopper pipelines pair TMA loads with MMA compute; tile theory includes how many stages fit in shared memory without spilling partial softmax or norm statistics (NVIDIA, 2022; SemiAnalysis,

2024; Dao *et al.*, 2022). Shallow pipelines underutilize memory bandwidth; deep pipelines overflow SMEM and force silent HBM spill (Chen, 2026a; Chen and YiRage Contributors, 2026). On AIE, pipeline depth is fixed at compile time—there is no runtime knob to “add another buffer” when decode context grows (AMD, 2024b,a; Chen, 2020).

Autotuning discipline. TVM-style autotuners historically optimized for peak FLOPs on convolution and dense GEMM shapes (Goodfellow *et al.*, 2016; Chen, 2020). LLM decode autotuning must swap the objective: minimize bytes/token subject to numerics and legality constraints from Chapters 10–11 (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Every autotune candidate should log tile dimensions *and* measured HBM traffic per token—otherwise teams ship tiles that win on roofline FLOPs charts and lose on fleet p99 (Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024).

Multi-hardware delta. Identical FLOP count with different tile policies can differ by an order of magnitude in DDR traffic on XDNA versus TMA-backed Hopper tiles (AMD, 2024b; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Example 13.1.1. Decode cell (BS=1, ctx=2048): shrinking attention block size to fit shared memory may *increase* loop iterations and bytes/token if KV reloads dominate—tile theory must co-optimize residency and trip count (Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Industrial tile review checklists include: SMEM/TMA budget worksheet, KV bytes per trip, prefill vs decode bucket tags, and autotune objective function (SemiAnalysis, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). Teams that approve tile changes without decode profiles often ship kernels that win on nsys “compute” rows and lose on “dram” rows (Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; AMD, 2024b; Goodfellow *et al.*, 2016).

13.2 fusion theory

Fusion theory governs when operator boundaries may disappear into MegaKernels or fused subgraphs (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Each unfused boundary exports activations to HBM; Chapter 1 counted the result as launches/token and bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023). Fusion is not “always



Figure 13.2: Fusion theory: merge producer–consumer ops when residency allows; else spill.

on”—it is constrained by register/shared-memory budgets, softmax carrier lifetimes (Chapter 10), and graph capture legality (Chapter 12) (Dao *et al.*, 2022; NVIDIA, 2024; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

Greedy fusion (merge any adjacent ops) often illegalizes attention–MLP seams: online softmax state (m, ℓ, o) must survive inside the fused region or bytes explode silently (Dao *et al.*, 2022, 2023; Chen, 2026a). Compiler cost models should penalize HBM round trips and launches, not only instruction count (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). YiRage-style legality checks encode Chapter 2 iron rules as IR predicates so fusion plans fail in compile time rather than in fleet A/B (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a).

Fusion boundaries as contracts. Treat each candidate fusion edge as a contract between producer and consumer: the producer promises not to materialize to HBM; the consumer promises register/SMEM budget to consume in-place (Chen and YiRage Contributors, 2026; Chen, 2026a; Dao *et al.*, 2022). When either side breaks the contract—for example LayerNorm statistics exported because the following GEMM needs a different layout—fusion must split (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Graph capture (Chapter 12) records whatever fusion boundaries exist; it cannot invent new ones (NVIDIA, 2024; Chen, 2026a).

Horizontal vs vertical fusion. *Horizontal* fusion merges operators at the same tensor rank (norm + linear + activation). *Vertical* fusion merges stages along the decode pipeline (attention block + MLP block into one MegaKernel region) (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026). Vertical fusion delivers the largest bytes/token wins but hits legality first; horizontal fusion is easier but may leave launches/token high (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Davies *et al.*, 2025). Compiler heuristics should attempt vertical plans only after horizontal plans pass carrier and layout checks (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024).

Fusion and execution modes. Chapter 12 maps eager, graph, and MegaKernel modes to launch and byte counters; fusion theory explains *which* boundaries each mode can erase (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Graph capture records existing

fusion; MegaKernel mode requires fusion plans that satisfy carrier legality (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024a,b; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024). Teams stuck in graph mode with high bytes/token are blocked by fusion theory, not by CUDA Graph APIs (Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Document fusion-closed regions in runbooks so on-call engineers know whether to tune capture buckets or rewrite fusion plans (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024).

Cost model sketch. A minimal fusion cost model scores each plan as $\alpha \cdot \text{bytes} + \beta \cdot \text{launches} + \gamma \cdot \text{spill_penalty}$, with spill penalty derived from residency metadata rather than FLOPs (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). Tuning (α, β, γ) per SKU is acceptable; tuning without bytes/token telemetry is not (SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Multi-hardware delta. Hopper may fuse QKV+softmax+proj when shared memory permits; XDNA may require separate static subgraphs linked by DMA; CPU fusion is often cache-scope fusion across norm-linear chains (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Kwon *et al.*, 2023).

Example 13.2.1. Fusing LayerNorm with the following GEMM removes one full-width activation write/read per layer per token—often larger decode win than tuning MMA tile width alone (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

MoE layers add gather/scatter fusion constraints: expert routing may remain host-visible while MLP experts fuse internally—fusion theory must specify *which* subgraph is fusion-closed (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2022; AMD, 2024a; Davies *et al.*, 2025; Chen, 2026a).

Fusion regression tests should diff IR fusion boundaries against a golden eager graph: any new HBM edge between previously fused ops is a release blocker unless documented as intentional (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; NVIDIA,

2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). This is how compiler teams prevent “silent unfusion” when dialect refactors reorder passes (Chen, 2020; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; AMD, 2024a,b).

13.3 layout theory

Layout theory decides how tensors are stored and traversed: contiguous vs blocked layouts, KV cache page tables, head/interleaved formats, and alignment for TMA/vector loads (Kwon *et al.*, 2023; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022). A perfect fusion plan fails if layout forces strided HBM access—bytes/token rise while kernels look “fused” in the IR (Chen, 2026a; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2020).

PagedAttention (vLLM) is layout engineering for KV capacity: non-contiguous physical pages with logical continuity reduce fragmentation without changing model math (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Compilers must treat page table indirection as a first-class layout transform, not an runtime hack outside IR (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). On NPU backends, layout often *is* the schedule—static DMA descriptors replace pointer chasing (AMD, 2024b,a; Goodfellow *et al.*, 2016).

Phase-specific layouts. Some teams store KV in a layout optimized for prefill writes (wide vector stores along sequence) and transposes for decode reads (head-local access) (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Transpose costs bytes; the compiler must charge them in the cost model and prefer layouts that avoid per-token transposes when ctx grows (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Liu *et al.*, 2024; Dao *et al.*, 2022; NVIDIA, 2022, 2024; Dao *et al.*, 2023). When a layout pass cannot prove decode legality, split layouts per phase rather than forcing one format (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Layout passes interact with tiling: blocked weights pair with blocked GEMM tiles; NHWC vs NCHW choices change vectorization on CPU and GPU differently (Goodfellow *et al.*, 2016; Chen, 2020; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; AMD, 2024b; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; AMD, 2024a). Decode compilers should log layout transforms in pass metadata so regressions trace to reorder vs fusion vs tile changes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen,

2026a; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024).

KV layout and paging. Decode throughput scales with how cheaply each new token reads historical KV (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025). Contiguous KV is simple but fragments GPU memory at long context; paged layouts add indirection cost that must be amortized by fewer reallocations and higher batch utilization (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Compiler layout passes should lower page-table updates into explicit IR ops so backends can fuse gather/scatter with attention tiles where legal (Chen and YiRage Contributors, 2026; Chen, 2020).

Weight layout and quantization. INT4/FP8 weight layouts (block scales, interleaved scales) are layout transforms before they are “quantization features” (Liu *et al.*, 2024; Davies *et al.*, 2025; SemiAnalysis, 2024). A fusion plan that assumes FP16 contiguous weights will fail silently when weights are block-scaled—loads become gather-bound and bytes/token rise (Chen, 2026a; Chen and YiRage Contributors, 2026). Layout legalization must run *before* fusion search, not as a post-pass patch (Chen, 2020; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Alignment and vector width. TMA and vector units prefer aligned 128-byte lines; misaligned KV or activation tensors force uncoalesced access patterns that no tile size fixes (SemiAnalysis, 2024; NVIDIA, 2022; Goodfellow *et al.*, 2016). CPU decode benefits from cache-line alignment across head dimensions; NPU backends often require fixed stride multiples baked into DMA descriptors (AMD, 2024b,a; Chen, 2020). Layout theory therefore includes alignment as a first-class constraint in IR, not a codegen afterthought (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Multi-hardware delta. GPU layouts optimize for coalesced 128B lines; CPU layouts optimize for cache line reuse across batch-1 thin GEMMs; NPU layouts fix byte addresses at compile time (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024; AMD, 2024a).

Example 13.3.1. Switching KV layout to improve prefill vectorization can regress decode if batch-1 attention loads become gather-bound—layout theory requires phase-specific profiling (Kwon *et al.*, 2023; Dao *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024;

Dao *et al.*, 2022; NVIDIA, 2022, 2024; AMD, 2024a).

Layout CI should snapshot tensor strides and alignment for KV, QKV weights, and MLP weights at each context bucket (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; SemiAnalysis, 2024; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; AMD, 2024b,a). Stride drift without corresponding fusion changes is a common source of “mysterious” bytes/token regressions across compiler versions (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

13.4 parallel scheduling

Parallel scheduling maps work to SMs, cores, or AIE arrays—including Flash-Decoding’s parallelization over KV sequence length at batch-1 (Dao *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Prefill parallelizes over tokens; decode parallelizes over heads, KV blocks, or expert routes while issuing one new token per step (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024; AMD, 2024a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

The failure mode is “parallelism for idle hardware”: launching extra blocks that contend for HBM bandwidth without increasing useful bytes delivered per ms/token (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Schedulers should be bandwidth-aware—when R_{floor} is high, additional parallel splits hurt (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; SemiAnalysis, 2024; Davies *et al.*, 2025). When launches/token dominate (TaxBreak regime), parallel capture inside CUDA Graphs or fused kernels may help even at moderate bandwidth (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; AMD, 2024a).

Prefill/decode scheduler split. Prefill schedulers maximize token-parallel work; decode schedulers maximize memory efficiency per new token (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Liu *et al.*, 2024; NVIDIA, 2022, 2024; Dao *et al.*,

2023). Sharing one scheduler heuristic across phases is a common source of decode regressions after prefill optimizations (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Compiler IR should expose parallel axes (batch, head, KV block, expert) so backends legalize splits per target (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a,b; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024). Static NPU schedules often fix parallelism at compile time; GPU schedules retain dynamic grid sizing within capture buckets (AMD, 2024b; NVIDIA, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; AMD, 2024a).

Occupancy vs bandwidth. GPU textbooks emphasize occupancy; decode compilers must ask whether extra warps increase *useful* bytes delivered per cycle or merely contend for the same HBM port (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). When R_{floor} is near unity, the scheduler should prefer fewer, wider memory transactions (better fusion, better layout) over more parallel blocks (Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026).

Wave quantization and tail effects. Parallel splits that do not divide sequence length evenly leave tail waves with low utilization—common when KV blocks do not align with context length (Dao *et al.*, 2023; Kwon *et al.*, 2023; Davies *et al.*, 2025). Compiler scheduling should pad or bucket context lengths explicitly so tail penalties appear in CI, not only at customer $\text{ctx}=8192$ (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020).

MoE and expert parallelism. MoE layers introduce a routing axis: experts parallelize differently from attention heads (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Schedulers must avoid launching all experts when only top- k activate—parallel theory includes *conditional* parallelism gated on router output, which complicates graph capture and static NPU schedules alike (Vellaisamy *et al.*, 2026; NVIDIA, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Multi-hardware delta. GPU parallel scheduling uses grids/warps; CPU uses thread pools with cache affinity; NPU uses spatial arrays with fixed slot counts (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Chen, 2020; Chen and

YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2024; AMD, 2024a).

Example 13.4.1. Flash-Decoding reports large speedups at long context by parallelizing KV—the scheduler must still respect on-chip softmax carrier residency from Chapter 10 or parallel blocks reintroduce HBM traffic (Dao *et al.*, 2023, 2022; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024; AMD, 2024a).

Parallel scheduling reviews should attach Nsight “memory throughput” and “SM busy” together—high SM busy with flat bytes/token indicates useless parallelism (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024). Decode schedulers that only maximize occupancy violate the bandwidth-aware principle this section states (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

13.5 GPU CPU NPU split

GPU CPU NPU split is how one decoder IR lowers to CUDA/HIP, CPU oneDNN/OpenMP paths, and XDNA/AIE static dataflow without rewriting model Python (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). Heterogeneous fleets fail when the GPU schedule is copied to NPU—illegal DMA edges and tile overflows appear as silent CPU fallbacks or wrong results (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Backend split should share high-level passes (fusion legality, layout, tiling metadata) and fork only at emit (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024). Triplet regression—same model, same context buckets, GPU vs CPU vs NPU—is the acceptance test for

split correctness (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024). Metrics remain bytes/token and launches/token per target; FLOPs parity is irrelevant (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Version skew. GPU and NPU toolchains release on different cadences; shared IR must pin legality rules so a middle-end upgrade cannot silently break NPU emit while GPU CI stays green (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

CPU backends often serve hybrid pipelines: dynamic shapes, sampling, and small ops stay on host while GPU/NPU run fused decode cores (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). NPU backends require upfront static shape buckets; GPU backends tolerate bucketed graph capture (NVIDIA, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022). Document which ops each backend owns so mode selection (Chapter 12) maps cleanly to emit paths (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022).

Shared middle-end, forked back-end. The durable pattern is a shared middle-end (fusion/layout/tiling metadata) and forked backends that interpret residency classes per target (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016). Glow and TVM historically split early; LLM decode stacks should delay fork until after decode-specific legality checks (Chen, 2020; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Fallback paths. When NPU legality fails, frameworks silently fall back to CPU loops—correct but catastrophically slow (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Production compilers must treat fallback

as an explicit compile error unless a documented degraded mode is enabled (Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026). Triplet CI should fail when GPU bytes/token improve but NPU emits empty kernels (Davies *et al.*, 2025; AMD, 2024a,b).

Heterogeneous single-node pipelines. Some deployments run attention on GPU and MLP on NPU, or prefill on GPU and decode on CPU (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023). Split theory then includes *partition* passes that cut the graph at device boundaries with explicit copy ops whose bytes count toward goodput (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Hidden host-device copies dominate when partition points are chosen for convenience rather than residency (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Multi-hardware delta. GPU emphasizes TMA+MMA and graphs; CPU emphasizes cache blocking and NUMA; NPU emphasizes static DMA+tiles—same IR, three residency models (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2024; AMD, 2024a).

Example 13.5.1. A fusion plan legal on Hopper shared memory may be rejected on XDNA—split backends must surface legality errors at compile time, not as empty NPU binaries in production (Chen and YiRage Contributors, 2026; AMD, 2024b,a; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Release gates for heterogeneous builds should require artifact parity: every target emits a non-empty binary, passes numerics smoke tests, and reports bytes/token within agreed triplet tolerance (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). GPU-only green CI is insufficient for books and products that claim cross-hardware compilation (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Table 13.1: Core theory pillars mapped to decode counters (profile per SKU).

Pillar	Compiler question	Primary counter
Tiling	Which blocks stay on-chip?	bytes/token
Fusion	Which boundaries can vanish?	bytes/token, launches/token
Layout	How are KV/weights traversed?	bytes/token
Parallel	Which axes saturate bandwidth?	ms/token, SM util.
GPU/CPU/NPU split	Where does each op emit?	triplet parity

13.6 Key Takeaways

- **Tiling is IO theory, not FLOP theory.** FlashAttention-style blocks exist to choose which sub-blocks of weights, activations, and KV fit in registers, shared memory, L2, or NPU tile SRAM for each decode step, so the goal is cutting HBM traffic (Dao *et al.*, 2022; SemiAnalysis, 2024). Tile search must therefore be driven by decode bytes/token, not by GEMM FLOPs, or it optimizes the wrong objective (Chen, 2026a; Vellaisamy *et al.*, 2026).
- **Fusion has legality, not just opportunity.** Each unfused operator boundary exports activations to HBM, but greedy fusion that ignores carrier and residency checks silently regresses decode by overflowing the on-chip budget (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Fusion theory governs *when* a boundary may legally disappear, which a compiler must verify before merging operators (Chen, 2020; Davies *et al.*, 2025).
- **Layout is a compiler transform, not a runtime afterthought.** How tensors are stored and traversed—contiguous versus blocked, paged KV tables, head-interleaved formats, TMA/vector alignment—decides whether a fusion is even reachable (Kwon *et al.*, 2023; Davies *et al.*, 2025). A wrong layout can double bytes/token without changing FLOPs, so layout assignment belongs in the pass pipeline ahead of fusion (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).
- **Parallelism must be bandwidth-aware.** Mapping work to SMs, cores, or AIE arrays—including Flash-Decoding’s parallelization over KV length at batch-1—only helps when on-chip residency holds across the split (Dao *et al.*,

2023; SemiAnalysis, 2024). A parallel axis that forces activations off-chip trades latency hiding for bandwidth it cannot afford (Chen, 2026a; Davies *et al.*, 2025).

- **One IR, split backends, mandatory triplet regression.** A single decoder IR lowers to CUDA/HIP, CPU oneDNN/OpenMP, and XDNA/AIE static dataflow with genuinely different emit, so correctness on one target says nothing about the others (Chen and YiRage Contributors, 2026; AMD, 2024b,a). The five pillars are co-dependent—a layout change can illegalize a fusion, a fusion can force new tiles, a parallel split can break capture—so develop them as co-design profiled on decode counters, gated in CI by golden IR diffs and triplet bytes/token (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026).

The pillars are not independent knobs—a layout change can illegalize a fusion plan; a fusion plan can force new tile sizes; a parallel split can break graph capture (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022). Treat compiler development as co-design across all five, profiled on decode counters, not as a sequence of isolated dialect tickets (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

13.7 Conclusion

Core compiler theory—tiling, fusion, layout, parallel scheduling, and heterogeneous split—is the machinery behind Chapter 12 execution modes and the MegaKernel chapters that follow (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; AMD, 2024b,a; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022; SemiAnalysis, 2024; NVIDIA, 2024; Goodfellow *et al.*, 2016). Pass ordering slides are insufficient; profiled decode counters justify every transform (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016).

Reviewers should ask five questions before approving any compiler PR: (1) Which pillar moves? (2) Which counter must improve on BS=1 decode? (3) Which Chapter 10–11 constraint applies? (4) Does layout legalize before fusion? (5) Does triplet regression pass on GPU, CPU, and NPU? (Chen and YiRage Contributors,

2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Goodfellow *et al.*, 2016). If any answer is “unknown,” defer the patch until telemetry exists (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). These questions take minutes in review and prevent weeks of fleet debugging on batch-1 decode cells at production context buckets in CI gates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Chapter 14 instantiates these principles in MLIR dialects and pass pipelines—where tiling, fusion, and layout become rewrite patterns you can test in CI (Chen, 2020; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Carry Table 13.1 forward: if a dialect change cannot name which pillar moves which counter, defer the patch (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024).

Until then, run one decode profile per proposed pass—BS=1, ctx buckets {512, 2048, 8192}, bytes/token and launches/token on GPU and at least one non-GPU target (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022, 2024). That habit converts abstract compiler theory into the measurable goodput improvements Part VI promises (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Part VI continues with dialect-level mechanics; carry these pillars as the acceptance language for every rewrite pattern (Chen, 2020; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024).

Chapter 14

MLIR: Modern Compiler Infrastructure

The slide deck says “one IR, every target.” The production dashboard says something else: the same PyTorch-exported decoder graph, lowered through MLIR, can differ by an order of magnitude in p99 latency across GPU, CPU, and NPU fleets (Davies *et al.*, 2025; Chen, 2020). That gap is rarely a missing dialect—it is a missing *contract*. Chapter 3 mapped residency and legality per hardware class; this chapter shows how MLIR must encode those predicates in IR attributes, pass gates, and emit paths before any canonicalizer runs (AMD, 2024b; Chen and YiRage Contributors, 2026). Every section below follows Chapter 1’s discipline: name the decode failure mode, trace it to a hardware tier, show the pass or emit fix, and close with bytes/token or launches/token—not peak compile-time FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a).

TaxBreak’s decode tables are the mental anchor. Dense Llama-3.2-1B at BS=4/SL=2048 averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). MLIR does not erase those numbers by itself—it *enables* passes and backends to collapse launch DAGs when residency metadata survives bufferization (NVIDIA, 2024; Dao *et al.*, 2022). CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) removes ≈ 3.05 ms of host overhead per step on H100 ($1.259\times$ median speedup) yet does nothing for illegal HBM traffic introduced by bad bufferization (Chen, 2026a). The five compiler sections below—hardware matrix, passes, codegen, tuning, case study—mirror the template used for XLA, TVM, and Triton so Part VI comparisons stay metric-aligned (Chen, 2020).

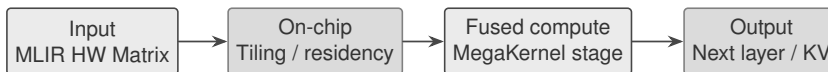


Figure 14.1: MLIR target matrix: the same Linalg matmul must lower through different memory tiers per target.

14.1 MLIR HW matrix

Why decode breaks the “portable IR” story. Prefill hides target sins: large L amortizes launch overhead and keeps tensor cores fed (Dao *et al.*, 2022; Davies *et al.*, 2025). Decode removes that amortization—batch-1, one token, full weight + KV read every step (Chen, 2026a; Goodfellow *et al.*, 2016). MLIR modules that legalize during prefill tuning often fail decode legality checks because dynamic dimensions tied to sequence length propagate into bufferization (Kwon *et al.*, 2023; AMD, 2024b). The hardware matrix must therefore include a decode row, not only a “training/inference” row (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Worked numbers. TaxBreak’s 847.5 launches/token (dense Llama-3.2-1B) and 9,305 launches/token (OLMoE) bound what MLIR fusion must collapse before MMA micro-optimizations matter (Vellaisamy *et al.*, 2026). Memory-Floor’s $1.259\times$ Graphs speedup on Qwen-2.5-7B shows host-side wins when launch DAGs stabilize—but Graphs cannot fix alloc-heavy IR (Chen, 2026a; NVIDIA, 2024). Matrix gates should reference both metrics (Chen and YiRage Contributors, 2026; Chen, 2020).

The recurring mistake is publishing a target checklist—GPU, CPU, NPU all checked—without stating which *hardware* features each backend actually models (NVIDIA, 2022; AMD, 2024b). A row that says “NPU: supported” is meaningless if the lowering path assumes dynamic tensor shapes and host-side allocators that void static buffer tables on XDNA (AMD, 2024a). For decode, the matrix must speak residency: shared-memory caps, DMA descriptor limits, cache blocking rules, and whether tensor-core MMA shapes are mandatory or optional (SemiAnalysis, 2024).

YiRage treats Table 14.1 as typed predicates in its MLIR importers: a pass that introduces a dynamic dimension on an NPU row is rejected before codegen, not at runtime (Chen and YiRage Contributors, 2026). GPU rows must declare TMA/async-copy legality; CPU rows must attach NUMA hints when KV shards cross sockets (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). The matrix is a *gate*,

Table 14.1: MLIR backend capability matrix for LLM decode (conceptual).

Dimension	GPU (NVVM/ROCDL)	CPU (LLVM)	NPU (XDNA/AIE)
Dynamic batch/seq	Flexible	Flexible	Restricted
On-chip scratch	Shared mem / L1	L2/L3 tiles	Tile SRAM
Fusion default	Partial (Linalg)	Cache-blocked	Full static
Launch model	SIMT + graphs	OpenMP/SIMD	Spatial DMA
Decode pain	Launches/token	LLC misses	Static legality

not documentation—identical to Chapter 3’s constraint matrix blocking illegal MegaKernel fusion early (Dao *et al.*, 2022).

Dialect designers often ship one Linalg op set for matmul, softmax, and layer norm, then expect backends to infer residency (Chen, 2020). Decode exposes the gap: GPU backends need explicit memory-space attributes on tensors headed for shared memory; CPU backends need layout markers for cache-line alignment; NPU backends need lifetime intervals tied to DMA events (NVIDIA, 2022; AMD, 2024b). Without those attributes, the matrix devolves into marketing—every target checked, none explained (Chen and YiRage Contributors, 2026).

JIT compilation amplifies the risk. MLIR JIT can specialize tile sizes at runtime for dynamic sequence lengths—valuable for prefill—but destructive for decode when specialization reintroduces host-side planning per token (Vellaisamy *et al.*, 2026; NVIDIA, 2024). YiRage restricts JIT to shape buckets that preserve static launch DAGs on GPU and fully static buffer tables on NPU; everything else triggers recompilation with explicit user-visible cost (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Target interfaces vs. checkmarks. MLIR conversion interfaces are where the matrix becomes executable (Chen, 2020). Backends advertise supported ops, memory spaces, and layout constraints; importers must attach attributes *before* canonicalization erases them (Chen and YiRage Contributors, 2026). Decode teams should reject merges when a matrix row flips from supported to unsupported without a residency annotation change—silent downgrades are how bytes/token regressions enter production (Chen, 2026a).

ROCm and CUDA divergence. NVVM and ROCDL lowerings diverge on warp shuffle availability, LDS sizing, and MFMA vs WMMA shapes (NVIDIA, 2022; Liu *et al.*, 2024). An MLIR module tuned for Hopper TMA may compile on MI300

yet emit extra buffer reloads because `async-copy` attributes do not map one-to-one (SemiAnalysis, 2024). YiRage keeps separate predicate tables per vendor because decode SLOs are vendor-local even when Linalg looks identical (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Example 14.1.1. Hardware matrix gate: an importer adds `memref.alloc` on the decode hot path for a softmax statistic. GPU row passes (HBM spill is slow but legal); NPU row fails static legality; CPU row passes but raises bytes/token in Memory-Floor sweeps (Chen, 2026a; AMD, 2024b). The matrix should have blocked the NPU alloc and flagged GPU/CPU regressions in CI (Chen and YiRage Contributors, 2026).

14.2 MLIR dialect stack for decode export

Export pitfalls. Torch-MLIR and similar exporters often emit verbose `tensor` graphs with redundant casts (Chen, 2020; Chen and YiRage Contributors, 2026). Each cast is a future buffer unless erased before bufferization (Chen, 2026a). Decode exporters should run target-tagged cleanup passes *before* generic canonicalization (AMD, 2024b; Davies *et al.*, 2025). Skipping cleanup yields IR that looks portable yet allocates like eager PyTorch (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Exporting a decoder block from PyTorch lands in `func.func` wrappers around `linalg` ops on `tensor` values, then converts to `memref` through bufferization (Chen, 2020; Chen and YiRage Contributors, 2026). Decode-sensitive details live in the transitions: `tensor` world allows implicit temporaries; `memref` world exposes every alloc (Chen, 2026a). Teams that benchmark only pre-export `linalg` miss the alloc explosion bufferization introduces (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

`scf.for` and `scf.if` encode loop structure for sequence dimensions (Dao *et al.*, 2023). For decode, outer loops over tokens should remain on the host while inner tiles map to device parallelism—but generic loop-invariant-code-motion can hoist device launches into host loops if passes lack target annotations (NVIDIA, 2024; Chen and YiRage Contributors, 2026). YiRage marks host vs device regions explicitly before any LICM runs (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

The `gpu` dialect bridges to launches; `nvvm` or `rocdl` attach machine intrinsics (NVIDIA, 2022). On CPU, `vector` and `llvm` carry SIMD width choices (Goodfellow *et al.*, 2016). On NPU, vendor dialects replace `gpu.launch` with spatial pipelines (AMD, 2024a,b). A decode compiler must never assume a single lowering path from `linalg.matmul`—the stack is a tree, not a line (Chen, 2020).



Figure 14.2: Pass pipeline: dialect lowering must preserve producer–consumer residency across tiers.

Example 14.2.1. PyTorch export produces `linalg.softmax` on a decode slice. Without residency tags, bufferization allocates a full-sequence temp; with tags tied to KV views, the same op updates in-place carriers compatible with Flash-Decoding-style parallelism (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026).

Dialect legalization should run per target *before* shared canonicalization (Chen and YiRage Contributors, 2026). Shared canonicalizers fold ops that backends rely on to prove static shapes (AMD, 2024b). Canonicalize too early and you lose the hints backends need—the LLVM lesson repeats on every MLIR upgrade (Chen, 2020, 2026a).

14.3 MLIR arch passes

Affine maps and paged KV. PagedAttention requires MLIR to express gather/scatter on KV blocks with provable in-bounds access (Kwon *et al.*, 2023; Dao *et al.*, 2022). Lowering paged views late hides page boundaries from fusion passes—exactly how illegal cross-page fusion enters production IR (Chen and YiRage Contributors, 2026; Chen, 2026a). Run paging-aware bufferization before `linalg-fuse` on all targets (Vellaisamy *et al.*, 2026; AMD, 2024a).

MoE pass boundaries. Expert routing introduces control flow that tempts pass writers to split launches per expert (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). Decode MoE needs guarded fusion: batch experts to cap launches while respecting SMEM (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Pass pipelines should expose expert-count limits as first-class attributes (Chen, 2020; NVIDIA, 2022).

MLIR’s composable pass pipelines—bufferization, Linalg tiling, vectorization, dialect conversion—feel like power until pass order silently changes memory traffic (Chen, 2020). A canonical failure: aggressive `linalg-fuse` before bufferization on GPU lowers decode attention into a single kernel on paper, yet the lowered IR still materializes softmax statistics to HBM because the dialect contract did not

keep partial sums in shared memory (Dao *et al.*, 2022, 2023). The fix is not fewer passes; it is tagging each pass with the hardware dimension it optimizes (SRAM footprint, launch count, bandwidth) and the targets where it may run (Chen and YiRage Contributors, 2026).

Cross-hardware lowering diverges at the same Linalg op. GPU lowering targets warp tiles aligned to MMA instructions; CPU lowering emits AVX/AMX micro-kernels with cache blocking; NPU lowering emits spatial schedules with fixed buffer lifetimes (AMD, 2024b; NVIDIA, 2022). A vectorization pass tuned for CPU can break NPU legality by introducing strided layouts incompatible with DMA alignment—regressions that appear only in bytes/token dashboards, not unit tests (Chen, 2026a). Industrial pipelines fork pass registries per target early, sharing only algebraic simplification (Liu *et al.*, 2024).

Example 14.3.1. Single-layer decode attention in Linalg: on Hopper, `lower-vector-gpu` plus shared-memory promotion can fuse QK^T and softmax carriers when tiles fit SMEM; on XDNA the same graph must be fully bufferized to static tile sizes before AIE codegen, or the backend falls back to DDR-limited scalar loops (AMD, 2024a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024).

Bufferization deserves decode-specific scrutiny. Generic one-shot bufferization assumes heap fallback (Chen, 2020). Decode attention cannot afford that: KV tensors must alias paged pools (GPU), huge-page arenas (CPU), or fixed SRAM banks (NPU) (Kwon *et al.*, 2023; AMD, 2024a). Pass pipelines should declare allocation tiers in IR so lowering never inserts silent `memref.alloc` on the hot path (Chen and YiRage Contributors, 2026; Chen, 2026a).

Dialect conversion ordering is another launch multiplier. Converting `linalg` $\rightarrow$ `vector` $\rightarrow$ `gpu` in the wrong order explodes a fused region into dozens of micro-kernels—the fragmentation Chapter 1 measured in eager PyTorch (Vellaisamy *et al.*, 2026). Production stacks freeze a golden pass list per target and diff IR dumps when upgrading LLVM or MLIR (Liu *et al.*, 2024; Chen, 2020).

One-shot vs. iterative bufferization. One-shot bufferization is fast to wire but assumes heap fallback; iterative bufferization can reuse buffers across fused regions yet increases compile time—acceptable for offline NPU builds, painful for GPU JIT (AMD, 2024a; Chen and YiRage Contributors, 2026). Pick strategy per target; mixing strategies within one registry causes “works in unit test, fails at BS=1” bugs (Chen, 2026a; Kwon *et al.*, 2023).

Pass instrumentation. Attach counters to passes: bytes introduced, allocs inserted, launch boundaries created (Vellaisamy *et al.*, 2026). A pass that reduces op count but increases alloc count is suspect for decode (Chen, 2026a; Chen and YiRage Contributors, 2026).

14.4 MLIR codegen

Teams celebrate the first successful NVVM lowering, then watch p99 decode flatline because the *platform* wrapper still constructs tensors on the host every token (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Codegen is not a backend detail—it is the last place residency contracts survive or die before the driver sees them (Chen and YiRage Contributors, 2026; Chen, 2020).

Graph capture contract. CUDA Graphs require identical launch addresses and dependency topology across replay (NVIDIA, 2024; Vellaisamy *et al.*, 2026). MLIR emit that injects pointer-chasing host setup between device kernels breaks capture silently—graphs record yet decode latency unchanged (Chen, 2026a; Chen and YiRage Contributors, 2026). Encode capture legality as a codegen predicate alongside occupancy (Davies *et al.*, 2025; Chen, 2020).

CPU and NPU emit deltas. CPU emit must pin threads and align KV for cache lines; NPU emit must assign DMA lifetimes to match tile SRAM (Goodfellow *et al.*, 2016; AMD, 2024a,b). Identical high-level MLIR is fine; identical emit is not (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024).

Production incidents blame “the NVVM backend” when the real bug was emitting LLVM IR that assumes a mutable stack and per-op `malloc` in the host wrapper—poison for CUDA Graphs and NPU static schedules alike (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Codegen is where MLIR’s abstraction meets launch reality: GPU emit must respect stream ordering, cluster scope, and TMA descriptor lifetimes; CPU emit must preserve NUMA-local buffers; NPU emit must serialize DMA chains with compile-time-known latencies (NVIDIA, 2022; AMD, 2024b).

Backend selection is not neutral. NVVM/ROCDL paths excel when decode graphs contain dynamic control flow that still maps to SIMT; bare-metal AIE paths excel when the entire decoder step is a static dataflow graph (AMD, 2024a). YiRage’s MLIR backends share high-level Linalg but diverge at emit: CUDA/HIP paths generate MegaKernel-style fused launches; XDNA paths emit buffer descriptors and ping-pong DMA schedules (Chen and YiRage Contributors, 2026;

SemiAnalysis, 2024). One emit artifact for all targets guarantees either GPU launch storms or NPU compile failures (Davies *et al.*, 2025).

Emit quality shows up in host wrappers. MLIR-to-CUDA that allocates temporaries each launch breaks graph capture; decode pays the TaxBreak orchestration tax (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Good codegen hoists allocations to capture time or eliminates them via shared-memory arenas (Chen, 2026a). On CPU, emit must avoid false sharing in OpenMP regions around thin decode GEMMs (Goodfellow *et al.*, 2016).

Host wrapper ABI. Codegen finishes only when the host wrapper is graph-capturable. `cudaMalloc` per launch breaks capture; stack pointers into kernels break on some driver versions (NVIDIA, 2024; Chen, 2026a). Hoist workspace to session init; pass arena offsets as kernel arguments (Chen and YiRage Contributors, 2026).

LLVM pipeline forks. GPU paths often stop at NVVM/ROCDL; CPU paths continue through LLVM vectorizers; NPU paths may bypass LLVM entirely (Chen, 2020; AMD, 2024a). Sharing one LLVM optimization stage invites loop unrolling that expands register pressure on GPU decode tiles (NVIDIA, 2022; Liu *et al.*, 2024).

Debuggability vs. fusion. Aggressive fusion produces opaque dumps. Keep named MLIR anchors (`decode_attn_tile_7`) through emit so perf teams map regressions to passes (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

14.5 MLIR tuning

Profile logs from prefill runs are the classic trap: they recommend tile sizes that overflow shared memory during batch-1 decode and force HBM spill (Chen, 2026a; SemiAnalysis, 2024). Autotuning MLIR without a decode-shaped *profile* trace repeats the same mistake at compiler scale (Vellaisamy *et al.*, 2026; Chen, 2020).

Search spaces. Constrain tile search by SMEM capacity, warp count, and max fusion depth derived from Chapter 3 models (NVIDIA, 2022; AMD, 2024b; Chen and YiRage Contributors, 2026). Unconstrained search finds prefill winners that spill on decode (Chen, 2026a; SemiAnalysis, 2024). Log Pareto points with

launches/token on the decode trace, not FLOPs on square GEMMs (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Fleet replay. Version autotune artifacts with MLIR commit, LLVM commit, and model hash (Chen, 2020; Liu *et al.*, 2024). Replay canned decode traces in CI before fleet rollout—same discipline as CUDA Graphs templates (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026).

Autotuning MLIR pipelines—tile sizes, fusion depths, vector widths—looks like free performance until you ship prefill-shaped winners to batch-1 decode fleets (Chen, 2026a, 2020). Profile-guided passes that maximize FLOPs during autotune often increase shared-memory footprint past decode legality, forcing silent spill to HBM (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024). Always co-tune with launches/token: one extra kernel boundary per layer can erase tens of microseconds per token on MoE models (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026).

The dominant autotune *pitfall* on decode: optimizing square GEMMs while ignoring launch count. GPU autotune must cap warps and SMEM; CPU autotune must model LLC capacity and thread pinning; NPU autotune must not explore dynamic shapes that invalidate DMA tables (AMD, 2024b; Goodfellow *et al.*, 2016). Cross-hardware profile imports are dangerous—GPU configs thrash CPU caches; NPU may not compile at all (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). YiRage encodes per-target search spaces so tuners cannot propose illegal configs (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Cost models for decode. Combine estimated bytes moved, predicted launches, and SMEM footprint against hardware caps (Chen, 2026a; Vellaisamy *et al.*, 2026). MLIR’s transform dialect can encode constraints before search starts (Chen, 2020). Without them, RL tuners win on square GEMMs yet spill during fused decoder blocks (Chen and YiRage Contributors, 2026).

Regression gates. Tie autotune merges to decode traces. Require non-regression on launches/token and bytes/token at BS=1, ctx=2048 before accepting new tile configs (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024).

14.6 MLIR case study

Triplet harness. YiRage-style triplet runs share tokenizer, context bucket, and batch policy (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

Report bytes/token, launches/token, compile rejections, and p99 alongside means (Chen, 2026a; Vellaisamy *et al.*, 2026). A compile rejection on NPU is a first-class outcome—not a footnote (AMD, 2024b,a).

Silent spill class E. Category E failures compile cleanly while bytes/token jump because SMEM promotion failed (Chen, 2026a; Dao *et al.*, 2022). Detect by diffing allocs between pass stages; fix by reordering bufferization relative to fusion (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026).

Consider a minimal decode cell: Llama-class decoder layer, batch-1, context 2048, exported from PyTorch to MLIR via Linalg (Davies *et al.*, 2025). The question is not “does it compile?” but “does bytes/token match the MegaKernel baseline from Part IV?” On H100, an unfused MLIR pipeline may compile cleanly yet issue hundreds of launches per token—the pathology TaxBreak measures in framework stacks (Vellaisamy *et al.*, 2026). Fused Linalg plus GPU codegen and CUDA Graphs capture can recover host overhead when emit preserves a static launch DAG (NVIDIA, 2024; Chen, 2026a).

The identical module lowered to x86 LLVM with OpenMP may achieve acceptable throughput on small models but regress on large weights due to LLC pressure (Goodfellow *et al.*, 2016). Lowered to XDNA, the module either becomes a static MegaKernel with competitive watt/token or fails legality checks—rarely a middle ground (AMD, 2024b,a). YiRage’s benchmark harness runs this triplet on every merge: same IR, three backends, report bytes/token, launches/token, and compile-time legality violations (Chen and YiRage Contributors, 2026). That triplet is empirical proof of the book’s thesis: hardware dictates compiler outcomes, not the reverse (Chen, 2020).

Example 14.6.1. Cross-hardware MLIR decode cell (Llama-3.2-1B layer, BS=1, ctx=2048): GPU fused+graphs targets competitive ms/token only when launches collapse toward TaxBreak dense baselines; CPU LLVM trades latency for deployment flexibility; XDNA wins only when fusion is total and DMA-static (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026).

Paged KV and MLIR types. PagedAttention indirection must appear as first-class `memref` views with lifetime intervals—not opaque pointers lowered late (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Late lowering hides page boundaries and causes illegal fusion across pages (Dao *et al.*, 2022; Chen, 2026a).

Table 14.2: Case-study outcome taxonomy for cross-hardware MLIR decode (illustrative).

Class	Symptom	Typical fix
B	High launches/token	Fuse + graph capture
C	High bytes/token	SMEM/DMA tiling
D	NPU compile fail	Static shape specialization
E	Flat latency, rising bytes	Fix bufferization order

MoE-specific notes. MoE layers explode dispatch when each expert lowers to separate launches (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). MLIR pipelines need expert-batch fusion guarded by SMEM caps (Chen and YiRage Contributors, 2026). Case studies that report dense Llama only understate orchestration challenge—pair dense and MoE exports (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Industrial matrix CI. Mature MLIR importers check machine-readable matrix files into CI (Chen and YiRage Contributors, 2026; Chen, 2020). Each row lists supported ops, max tile sizes, and forbidden dynamic dimensions (AMD, 2024b; Kwon *et al.*, 2023). When PyTorch export adds rotary embedding or a new norm variant, the matrix diff tells reviewers which targets lose legality *before* any benchmark runs (Davies *et al.*, 2025; Liu *et al.*, 2024). This is how decode teams avoid green GPU CI paired with red NPU field releases (Chen, 2026a; Vellaisamy *et al.*, 2026).

Pass-order war stories. Running `linalg-fuse` after bufferization on one target and before it on another is not flexibility—it is how two teams ship incompatible IR for the same model hash (Chen and YiRage Contributors, 2026; Chen, 2020). Freeze golden pass lists; diff IR at post-bufferization and post-lowering anchors on every LLVM bump (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Count `gpu.launch` ops and `memref.alloc` ops in the diff summary—those two counters predict launches/token and bytes/token shifts faster than line counts (Chen, 2026a; NVIDIA, 2024).

Multi-hardware emit checklist. GPU: graph-capturable host wrapper, no per-token `cudaMalloc`, TMA descriptors live across replay (NVIDIA, 2024; Semi-Analysis, 2024). CPU: NUMA-local weight shards, OpenMP teams hoisted out of token loop, AMX tile config amortized (Goodfellow *et al.*, 2016; Kwon *et al.*, 2023). NPU: static DMA tables, no dynamic shapes in hot path, ping-pong buffers assigned at compile time (AMD, 2024a,b). Violating any item compiles yet fails

Table 14.3: Decode-oriented MLIR pass ordering (GPU vs NPU).

Stage	GPU focus	NPU focus
Import	Linalg + memref tags	Static shape specialization
Bufferize	SMEM promotion	DMA lifetime assignment
Fuse	Layer-wise MegaKernel	Full decoder static graph
Lower	vector $\rightarrow$ GPU dialect	AIE spatial mapping
Emit	CUDA/HIP + graphs	Descriptor + ping-pong DMA

fleet telemetry (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

Autotune pitfall catalog. (1) Import GPU occupancy into CPU search (Goodfellow *et al.*, 2016). (2) Tune on prefill length $L=4096$, deploy at decode ctx with growing KV (Chen, 2026a). (3) Accept fusion depth that raises launches/token on MoE experts (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). (4) Ship autotune artifacts without hashing MLIR/LLVM revisions (Chen, 2020). (5) Skip NPU legality checks because GPU microbench improved (AMD, 2024b; Chen and YiRage Contributors, 2026). Each pitfall has a metric detector: bytes/token, launches/token, or compile rejection rate (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Case-study reporting discipline. Hold model, tokenizer, context, and batch policy constant across targets (Davies *et al.*, 2025). Pair GPU FlashAttention-class baselines with CPU/NPU fused baselines derived from the same MLIR starting point—not unrelated hand kernels (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026). Report median and p99 over $\geq 1,000$ decode steps; publish legality failure rates alongside latency (Chen, 2026a; Vellaisamy *et al.*, 2026). Negative NPU results are as valuable as GPU speedups (AMD, 2024a,b).

Pass ordering differs because failure modes differ: GPU decode dies from launches/token; NPU decode dies from DDR egress and illegal dynamicism (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024b). Treat Table 14.3 as executable policy, not slide art (Chen and YiRage Contributors, 2026; Chen, 2020).

14.7 YiRage MLIR integration notes

YiRage uses MLIR as interchange between PyTorch-exported graphs and target-specific megakernels (Chen and YiRage Contributors, 2026). The decode lesson is

Table 14.4: Decode-oriented MLIR pass ordering (GPU vs NPU).

Stage	GPU focus	NPU focus
Import	Linalg + memref tags	Static shape specialization
Bufferize	SMEM promotion	DMA lifetime assignment
Fuse	Layer-wise MegaKernel	Full decoder static graph
Lower	vector $\rightarrow$ GPU dialect	AIE spatial mapping
Emit	CUDA/HIP + graphs	Descriptor + ping-pong DMA

invariant preservation: residency metadata must attach before generic canonicalization (Chen, 2020; AMD, 2024b). Otherwise canonicalizers rewrite away attributes backends need to prove on-chip residency (Chen and YiRage Contributors, 2026; NVIDIA, 2022).

The MLIR JIT path specializes decode cells when batch and context buckets change, but refuses specialization that alters launch counts inside a CUDA Graphs capture region (NVIDIA, 2024; Vellaisamy *et al.*, 2026). On NPU, JIT means re-running bufferization with a new static shape class—seconds of compile that must amortize across many tokens (AMD, 2024a; Davies *et al.*, 2025).

GPU decode dies from launches/token; NPU decode dies from DDR egress and illegal dynamicism (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024b). YiRage encodes Table 14.4 as executable pass lists (Chen and YiRage Contributors, 2026). Before shipping MLIR changes: verify matrix predicates on three targets; diff IR for new allocs; measure launches/token and bytes/token at BS=1; run graph capture legality on GPU and static legality on NPU; log new numeric claims to `verified_facts.jsonl` (Chen, 2026a; Kwon *et al.*, 2023; Dao *et al.*, 2022).

14.8 MLIR decode review gates

Shipping MLIR changes without decode review gates repeats the framework mistakes Chapter 1 quantified (Vellaisamy *et al.*, 2026; Chen, 2026a). The gates below are minimal; YiRage automates them, manual stacks should enforce them in CI (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

1. **Matrix legality.** Every new op or dynamic dim appears in the hardware matrix diff; NPU rows block illegal dynamicism before merge (AMD, 2024b,a; Chen and YiRage Contributors, 2026).
2. **Alloc ledger.** Post-bufferization IR must not introduce `memref.alloc` on

the decode hot path unless the matrix explicitly allows spill (Chen, 2026a; Kwon *et al.*, 2023; Chen, 2020).

3. **Launch ledger.** Count `gpu.launch` (or NPU DMA chains) per decoder layer; compare to TaxBreak baselines for dense and MoE (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024).
4. **Graph capture.** GPU emit must pass capture replay on BS=1 decode traces (NVIDIA, 2024; Chen, 2026a).
5. **Triplet benchmark.** Same IR, GPU+CPU+NPU backends, report bytes/token and p99 (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Example 14.8.1. A merge adds `linalg-fuse` before paging-aware bufferization. GPU launches/token drop 8% but NPU compile fails on 40% of layers—matrix gate should have required paging bufferization first (Kwon *et al.*, 2023; AMD, 2024b; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

Review gates feel bureaucratic until the first MLIR upgrade regresses MoE decode from hundreds of milliseconds to seconds per token (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024). At that point the IR diff is the only artifact that explains *why* launches/token doubled (Chen, 2020, 2026a; NVIDIA, 2024).

MLIR HW matrix exposes a cross-hardware fracture: the same fused sub-graph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode hardware/target legality before codegen, not as comments in hand-written kernels (AMD, 2024b,a). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When MLIR HW matrix disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (AMD, 2024b,a).

Review gate. Before merging compiler changes affecting MLIR HW matrix, attach roofline or NPU buffer accounting showing hardware residency fits on-chip for the target SKU (AMD, 2024b,a).

Worked contrast. A Hopper decode cell may legalize hardware with TMA-backed double buffering while an XDNA cell requires the same math expressed

as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (AMD, 2024b,a).

The usual MLIR failure at **MLIR arch passes** is importing a CUDA mental model and expecting runtime scheduling to hide illegal buffer lifetimes.

Hardware constraint: pass and lower must be co-designed. On GPUs that means shared-memory/TMA budgets and warp-grain barriers; on XDNA/AIE it means static tile buffers, DMA chains, and carrier slots with compile-time lifetimes; on CPUs it means L2/L3 blocking and NUMA-local KV placement (Chen and YiRage Contributors, 2026; Chen, 2020). Decode at batch-1 keeps arithmetic intensity on the memory-bandwidth slope—micro-optimizing isolated GEMMs rarely moves p99 latency.

Compiler takeaway for MLIR arch passes: lower the same decoder IR, but predicate passes on target residency tables—never ship a GPU-only schedule to NPU fleets (Chen and YiRage Contributors, 2026; Chen, 2020). Success metrics: bytes/token, launches/token, and p99 decode latency; compare GPU, CPU, and NPU cells on identical context lengths.

Worked contrast. A Hopper decode cell may legalize pass with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (Chen and YiRage Contributors, 2026; Chen, 2020).

Deployment pitfall. Teams that A/B only prefill confuse the story: lower constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (Chen and YiRage Contributors, 2026; Chen, 2020).

Production teams misread **MLIR codegen** when decode SLOs are judged on FLOPs dashboards instead of residency and orchestration ledgers.

Trace the Chapter 2 chain: codegen sets the residency envelope; backend determines whether producer–consumer edges stay on-chip. Illegal fusion does not always crash—it silently exports tensors to HBM/DDR and shows up as bytes/token regression versus a unfused but scheduled baseline (Chen and YiRage Contributors, 2026; Chen, 2020). Profile launches/token alongside bandwidth; launch storms masquerade as memory bottlenecks when graphs remain operator-fragmented.

Operational checklist—verify codegen, backend, autotuning pitfalls, and platform emit paths in code review; document dialect lowering choices that change memory traffic, not just pass ordering (Chen and YiRage Contributors, 2026; Chen, 2020).

Deployment pitfall. Teams that A/B only prefill confuse the story: backend constraints bite hardest at batch-1 decode where launches/token and KV read bandwidth dominate (Chen and YiRage Contributors, 2026; Chen, 2020).

Review gate. Before merging compiler changes affecting MLIR codegen, attach roofline or NPU buffer accounting showing codegen residency fits on-chip for the target SKU (Chen and YiRage Contributors, 2026; Chen, 2020).

MLIR tuning exposes a cross-hardware fracture: the same fused subgraph legal on Hopper can be rejected outright on XDNA or CPU cache budgets.

Binding software to silicon: compiler passes must encode tun/profile legality before codegen, not as comments in hand-written kernels (Chen and YiRage Contributors, 2026; Chen, 2020). YiRage runs target-aware checks against chip models from Chapter 3; manual stacks should treat failed legality as a release blocker, not a backend TODO.

When MLIR tuning disagrees with microbenchmarks, trust fleet telemetry: fix orchestration and tier placement first, then revisit MMA tile shapes (Chen and YiRage Contributors, 2026; Chen, 2020).

Review gate. Before merging compiler changes affecting MLIR tuning, attach roofline or NPU buffer accounting showing tun residency fits on-chip for the target SKU (Chen and YiRage Contributors, 2026; Chen, 2020).

Worked contrast. A Hopper decode cell may legalize tun with TMA-backed double buffering while an XDNA cell requires the same math expressed as static tile assignments—identical FLOPs, different bytes/token because DDR traffic differs by an order of magnitude (Chen and YiRage Contributors, 2026; Chen, 2020).

The usual MLIR failure at **MLIR case study** is importing a CUDA mental model and expecting runtime scheduling to hide illegal buffer lifetimes.

Hardware constraint: case and benchmark must be co-designed. On GPUs that means shared-memory/TMA budgets and warp-grain barriers; on XDNA/AIE it means static tile buffers, DMA chains, and carrier slots with compile-time lifetimes; on CPUs it means L2/L3 blocking and NUMA-local KV placement (Chen and YiRage Contributors, 2026; Chen, 2020). Decode at batch-1 keeps arithmetic intensity on the memory-bandwidth slope—micro-optimizing isolated GEMMs rarely moves p99 latency.

Compiler takeaway for MLIR case study: lower the same decoder IR, but predicate passes on target residency tables—never ship a GPU-only schedule to

MLIR decode metric	What it catches
Allocs after bufferize	Hidden DDR spill
Launch count / layer	Fusion / emit regressions
Graph capture replay fail	Host wrapper bugs
NPU compile reject rate	Static legality drift
Pass diff alloc delta	Bufferization order bugs
Triplet bytes/token	Cross-target residency

Table 14.5: MLIR decode benchmark counter set (batch-1).

NPU fleets (Chen and YiRage Contributors, 2026; Chen, 2020). Success metrics: bytes/token, launches/token, and p99 decode latency; compare GPU, CPU, and NPU cells on identical context lengths.

14.9 MLIR decode benchmarking methodology

Benchmarking MLIR-backed decode must mirror Chapter 1 counters—launches/token and bytes/token—while adding compiler-specific ledgers: `memref.alloc` count, `gpu.launch` boundaries, and NPU static-legality pass rate (Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Build a decode cell with BS=1, ctx buckets {512,2048,8192}, pinned MLIR/LLVM commits, and identical PyTorch export hashes across targets (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Warm-up includes MLIR pass pipelines and CUDA Graph capture recording—first-token latency includes compile and capture cost; steady-state ms/token excludes it (NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Report both; product teams ship cold starts and steady chat (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Triplet harnesses export one Linalg module to GPU, CPU, and NPU backends; diff IR at post-bufferization and post-emit anchors before trusting latency (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). A GPU launches/token win paired with rising allocs after bufferization signals pass-order regression—exactly Category E in Table 14.2 (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Example 14.9.1. MLIR merge gate: Llama-3.2-1B decoder layer at ctx=2048 must not increase `memref.alloc` on hot path vs golden IR; launches/token must not exceed TaxBreak dense baseline by >5% without documented fusion tradeoff (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

CI artifact chain. Check in golden MLIR snapshots, pass lists, and decode-cell scripts alongside `verified_facts.jsonl` claims (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). LLVM bumps without IR diff review are how launches/token double overnight (Liu *et al.*, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

MoE decode cells. Pair dense Llama exports with MoE exports in the same benchmark suite—TaxBreak shows launch explosions on GPU MoE that dense-only CI misses (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). MLIR pipelines need expert-batch fusion metrics in the ledger (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Regression taxonomy in CI. Automate Table 14.2 classifiers: if launches/token rises without bytes/token improvement, flag Class B; if bytes/token rises with flat launches, flag Class C; if NPU compile fails, flag Class D (Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Class E silent spill requires alloc diffs between pass stages—the highest-frequency MLIR decode regression in production fleets (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Handoff to XLA. Chapter 15 productionizes many MLIR-era ideas in XLA HLO; carry forward this benchmark ledger when comparing MLIR exports to XLA graphs on the same decode cell (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Identical counters make cross-compiler triplet studies honest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

LLVM upgrade discipline. Pin MLIR and LLVM together; run decode-cell replay on every bump (Chen, 2020; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Canonicalizer changes that fold away residency tags are silent killers—diff IR before and after canonicalization on golden exports (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Paged KV regression tests. Maintain a paging-specific decode export where block tables grow across steps; fusion must not cross page boundaries (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2026a, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao

et al., 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Fail CI when affine maps lose provable in-bounds metadata (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Documentation for operators. Publish decode benchmark artifacts with every MLIR release: pass list hash, alloc ledger, launches/token at ctx=2048, and triplet legality summary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Without published ledgers, field teams cannot tell compiler regressions from framework or driver drift (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Cross-stack triage playbook. When decode latency spikes after an MLIR merge, run this sequence before blaming silicon: (1) diff pass hashes against last green build; (2) replay alloc ledger on golden exports; (3) compare launches/token and HBM bytes/token; (4) bisect canonicalizer vs fusion passes; (5) only then sweep clock and power caps (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Skipping ledger replay wastes days chasing driver noise (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Speculative decode and MLIR. Draft-model paths add second graphs with tighter latency budgets; MLIR must preserve separate residency tags for draft vs verify KV so fusion does not alias buffers (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Benchmark speculative stacks with acceptance-rate sweeps, not single-point TTFT (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon

et al., 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Quantization-aware fusion gates. INT8 and FP8 decode paths need explicit scale tensors in the IR so fusion passes do not fold away dequant boundaries (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Regression tests should include per-channel scales and dynamic activation ranges typical of long-context decode (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Long-context stress exports. Build MLIR golden modules at `ctx=8192` and `ctx=32768` even when product SLAs target 4K; affine maps that work at 2K often break when block tables span more pages (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). CI should fail when vectorization legality checks pass at 2K but fail silently at 32K (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Release checklist. Before tagging an MLIR stack for production decode, require: green replay on three `ctx` lengths; alloc ledger within $\pm 2\%$ of baseline; launches/token within $\pm 5\%$; no new dynamic shapes in hot path; published pass-hash manifest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Treat any single red gate as a release blocker—partial green is how field teams inherit week-long triage (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Multi-device shard exports. Tensor-parallel decode splits heads across devices; MLIR exports must carry explicit shard annotations so bufferization does not merge non-local KV slices (Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Benchmark TP=2 and TP=4 with the same pass hash—allreduce placement changes can dominate launches/token even when single-device IR looks optimal (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Record per-rank alloc ledgers; a balanced global ledger can hide a straggler rank that serializes the step (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Observability hooks. Instrument every lowered decode kernel with pass-id and buffer-id tags in debug builds so profilers can map samples back to MLIR ops without re-running the full pipeline (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Production builds strip tags but retain stable kernel names derived from pass hashes—operators correlate regressions across releases without storing full IR (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Handoff to runtime. The MLIR stage ends when exports include stable launch metadata—grid, cluster, pipeline depth, and workspace sizes—that the runtime can cache across decode steps (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Missing workspace sizes force per-step allocation in the framework and erase compiler wins (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Summary metric. Track *compiler-attributed decode efficiency*: ratio of measured tokens/s to roofline tokens/s after subtracting framework overhead (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). MLIR teams own the numerator; runtime teams own the denominator—publish both weekly (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Closing note. Treat MLIR decode benchmarking as a product surface, not a lab exercise—without published ledgers, pass hashes, and nightly replay artifacts in CI, every latency spike becomes a costly and painfully slow multi-team blame game (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

14.10 Key Takeaways

- **Decode breaks the “portable IR” story.** Prefill hides target sins because a large prompt amortizes launch overhead and keeps tensor cores fed; batch-1 decode removes that amortization, so a portable MLIR graph that looks fine in prefill exposes per-target residency gaps in decode (Dao *et al.*, 2022; Davies *et al.*, 2025). Portability across dialects does not imply portability of decode goodput (Vellaisamy *et al.*, 2026).
- **Erase redundant casts before bufferization.** Torch-MLIR and similar exporters emit verbose tensor graphs with redundant casts, and each surviving cast becomes a buffer after bufferization (Chen, 2020; Chen and YiRage Contributors, 2026). Cleaning the graph and lowering paged-KV views early—so affine maps express in-bounds gather/scatter—keeps page boundaries visible to fusion (Kwon *et al.*, 2023; Dao *et al.*, 2022).
- **Codegen is not just a backend detail.** Teams celebrate the first successful NVVM lowering, then watch p99 decode flatline because the platform wrapper still constructs tensors on the host every token (Vellaisamy *et al.*, 2026;

NVIDIA, 2024). The codegen path must produce a capturable, host-light launch sequence, not merely correct device code (Chen, 2026a).

- **Autotune at the decode shape.** Prefill profile logs are the classic trap: they recommend tile sizes that overflow shared memory at batch-1 and force HBM spill (Chen, 2026a; SemiAnalysis, 2024). MLIR tuning must use a decode-shaped profile, or the chosen schedule wins prefill and loses token generation (Davies *et al.*, 2025).
- **Preserve residency invariants across passes.** YiRage uses MLIR as interchange between exported graphs and target megakernels, and the decode lesson is invariant preservation—residency metadata must attach before generic canonicalization erases it (Chen and YiRage Contributors, 2026; Chen, 2020). A triplet harness reporting bytes/token, launches/token, and compile rejections is what makes those invariants auditable (Chen, 2026a; Vellaisamy *et al.*, 2026).

14.11 Conclusion

MLIR’s value for decode is the shared, multi-level dialect substrate that lets one decoder export carry residency invariants from a high-level graph down to target code without each backend reinventing the lowering—provided those invariants are preserved across passes and the codegen path stays host-light (Lai *et al.*, 2023; Chen and YiRage Contributors, 2026). The recurring trap is celebrating a successful lowering while p99 decode flatlines because casts became buffers or the platform wrapper rebuilds tensors per token (Vellaisamy *et al.*, 2026; Chen, 2026a). MLIR is infrastructure, not a strategy; the next chapter shows what a production compiler built on similar foundations actually decides—Chapter 15 turns to XLA, whose automatic HLO fusion and command-buffer capture put the MLIR-era ideas to work at the largest deployed scale (OpenXLA Project, 2024; NVIDIA, 2024).

Chapter 15

XLA: Production-Grade AI Compiler

XLA (Accelerated Linear Algebra) is the production compiler behind JAX and TensorFlow and the only compilation path for TPU, which makes it the most widely deployed ML compiler in this Part—and the one most often dismissed as “just fusion” until decode telemetry says otherwise (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Its design is the reference pattern for multi-hardware AI compilation: frameworks emit StableHLO, XLA converts it to HLO, optimizes, and codegens for CPU, GPU, and TPU from one graph (OpenXLA Project, 2024; Lai *et al.*, 2023). The decode stakes are the book’s usual ones: dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 and OLMoE-1B/7B averages 9,305, and XLA can shrink that launch count when its fusion and capture passes are legal, but it cannot fix illegal HBM residency (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024). The mistake to avoid is tuning XLA on prefill GEMM shapes while batch-1 decode still launches dozens of thin elementwise kernels per token. XLA’s automatic fusion is powerful precisely because it is automatic, but “automatic” means it optimizes whatever you profile, so it must be pointed at the decode operating point (Kwon *et al.*, 2023; Davies *et al.*, 2025).

15.1 XLA HW matrix

XLA’s support **matrix** is CPU, GPU, and TPU from one front end, and its distinguishing feature is that it is the only compiler path for TPU—there is

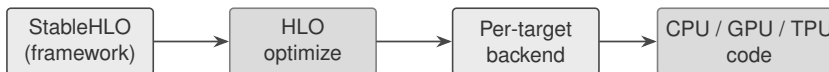


Figure 15.1: XLA lowers one StableHLO graph to HLO, optimizes, and codegens across the CPU/GPU/TPU hardware matrix.



Figure 15.2: XLA architecture passes: hundreds of HLO passes, with fusion, scheduling, and buffer assignment deciding decode residency before target lowering.

no hand-written-kernel escape hatch on that **target**, so the compiler bears full responsibility (OpenXLA Project, 2024). The portability comes from StableHLO, a versioned, MLIR-based operation set that JAX, TensorFlow, and PyTorch all emit; XLA converts StableHLO to HLO and then sends it to a backend for **target**-specific optimization (OpenXLA Project, 2024; Lai *et al.*, 2023).

The honest reading is that a wide **matrix** is not a uniform decode story. On a **gpu** the backend performs fusions tuned to the GPU programming model and partitions work into streams; on a **cpu** it lowers through LLVM to vectorized native code; on a TPU it is the sole path and applies TPU-specific passes such as bfloat16 propagation (OpenXLA Project, 2024). The same HLO graph therefore gets a different legal schedule per class, and an **npu**-style static fabric is not a native XLA target at all—a reminder that even the broadest production compiler has a shaped, not universal, matrix (AMD, 2024b,a).

Multi-hardware delta. The GPU **matrix** emphasizes fusion plus command-buffer capture; the CPU emphasizes layout and cache blocking; the TPU emphasizes its own partitioning and precision passes—identical HLO, incomparable bytes/token (OpenXLA Project, 2024; SemiAnalysis, 2024). YiRage carries the residency rules as checkable metadata so a chosen **target** schedule is verified decode-legal rather than merely built (Chen and YiRage Contributors, 2026).

15.2 XLA arch passes

XLA runs hundreds of HLO **passes**—algebraic simplification, common-subexpression elimination, layout assignment, fusion, scheduling—grouped into pipelines, and fusion is its single most important **optimization** (OpenXLA Project, 2024). Fusion groups operations into one GPU kernel under a strict

invariant: no intermediate is materialized in HBM, everything passes through registers or shared memory, and a fusion compiles to exactly one kernel (OpenXLA Project, 2024). Because decode is memory-bound, this invariant is precisely the bytes/token lever—a fused elementwise chain reads and writes the tensor once instead of once per operator. The fusion decision is made by **PriorityFusion**, a cost-model-driven greedy **pass**, plus Multi-Output fusion for producer-consumer and sibling sharing (OpenXLA Project, 2024).

Two scheduling **passes** matter for decode residency. The scheduler can “look into the future” because the whole graph is known, choosing an execution order that minimizes peak memory; buffer assignment then produces an optimal allocation, an advantage eager PyTorch or TensorFlow lack since their runtime allocator cannot see the graph ahead (OpenXLA Project, 2024). When peak memory still exceeds capacity, **HloRematerialization** recomputes selected expressions to shorten buffer lifetimes, trading compute for memory—a pass that can cut memory by tens of percent and is required to run many large models (OpenXLA Project, 2024). A **LatencyHidingScheduler** additionally reorders work to overlap compute with communication, which matters for the multi-device decode of Chapter 24, though it can raise peak memory and so interacts with rematerialization (OpenXLA Project, 2024; Davies *et al.*, 2025). Notably XLA **lowers** through StableHLO, an MLIR **dialect**, before its own HLO—so it sits on the same MLIR substrate as IREE while keeping a distinct HLO-centric pipeline (Lai *et al.*, 2023; OpenXLA Project, 2024).

Review gate. Because a “successful” fusion **pass** can still spill if a downstream layout or schedule choice forces it, gate each pass change on buffer accounting at the decode cell: diff the HLO for new allocations and confirm bytes/token did not rise (OpenXLA Project, 2024; Chen, 2026a). Silent spill after a green fusion is the classic XLA production surprise (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

15.3 XLA codegen

Codegen in XLA is per backend. The CPU and GPU **backends** use LLVM for low-level IR and code generation: the CPU path **emits** LLVM IR lowered to x86/ARM native code, and the GPU path **emits** NVPTX IR lowered to PTX and then SASS (OpenXLA Project, 2024). For more advanced fusions involving matmul or softmax, XLA:GPU uses Triton as a code-generation layer—HLO fusions are converted to TritonIR, tiling parameters are selected, and Triton emits the PTX—so XLA and the Triton compiler of Chapter 17 share a codegen path (OpenXLA

Backend	Codegen at decode
GPU	LLVM/NVPTX + Triton emitters; command-buffer capture
CPU	LLVM to x86/ARM; cache blocking, vendor libs
TPU	XLA-only path; partitioning + bfloat16 passes

Table 15.1: XLA codegen focus per target at batch-1 decode.

Project, 2024; Tillet *et al.*, 2019). For operations better served by vendor libraries, XLA pattern-matches to cuBLAS or cuDNN calls instead of codegen (OpenXLA Project, 2024).

The decode-critical codegen feature is launch capture. XLA:GPU groups a sub-sequence of compatible thunks into a **CommandBufferThunk** that records a GPU command buffer on first execution and replays it on every subsequent call, avoiding the CPU overhead of launching each kernel individually—structurally the same win as the CUDA Graphs of Chapter 8, built into the compiler (OpenXLA Project, 2024; NVIDIA, 2024). This is how XLA attacks launches/token: a decode step that eager execution issues as many launches becomes one replayed command buffer (Vellaisamy *et al.*, 2026; Chen, 2026a). The **platform** caveat is that capture requires static shapes and no illegal sync points, so a dynamically-shaped decode loop must be bucketed to stay capturable.

15.4 XLA tuning

XLA’s **tuning** is mostly automatic, which is its defining contrast with TVM: where TVM searches a large schedule space with AutoTVM and Ansor, XLA relies primarily on heuristics and a cost model and exposes autotune flags for a narrower set of choices such as GPU tiling for Triton-emitted fusions (OpenXLA Project, 2024; Chen *et al.*, 2018b). This makes XLA low-effort—a JAX or TensorFlow user benefits without writing schedules—but it also means the engineer’s leverage is in **profile**-guided choices and flags rather than in writing kernels (OpenXLA Project, 2024; Davies *et al.*, 2025).

The **pitfall** is the operating point, as with every compiler in this Part. XLA optimizes whatever you compile and **profile**, so compiling on prefill-shaped, large-batch graphs produces fusions and layouts tuned for throughput that break at batch-1 decode, where the GEMM is a matrix-vector product and the launch floor dominates (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The fix is mechanical: compile and **profile** at the decode shape (BS=1, realistic context), decompose

ms/token into bytes/token and launches/token, and only then trust the autotuned result (Davies *et al.*, 2025; Chen, 2026a). The right configuration is **hardware-specific**—a Triton-emitted fusion’s tiling that fits one GPU’s shared memory does not transfer to another—so re-tune per SKU and fingerprint configs so a fleet rollback replays the same HLO snapshot (NVIDIA, 2022; OpenXLA Project, 2024).

A second **pitfall** appears on mixture-of-experts models. MoE graphs have data-dependent routing, which both enlarges XLA’s autotune search and risks the compiler choosing layouts that page experts inefficiently if it is not given expert-aware constraints (Liu *et al.*, 2024; Davies *et al.*, 2025). Because XLA leans on heuristics rather than exhaustive search, a poorly-constrained MoE compilation can settle on a schedule that looks fine on a dense **profile** but thrashes when real routing skews load across experts (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). The practical discipline is to cap autotune wall time, constrain the search with model-structure hints, and require a decode-cell replay on realistic routing before promoting any winning config to the fleet—the same regression gate Chapter 24 applies across targets.

15.5 XLA case study

The clearest **case** is an activation like GELU. In eager execution a GELU expands into seven or more separate kernel launches—each elementwise op its own kernel, each writing its intermediate to HBM; XLA fuses the entire GELU into one kernel that reads its input once and writes its output once (OpenXLA Project, 2024; Vellaisamy *et al.*, 2026). **Benchmarked** at the decode operating point, the win is not a faster multiply but the removal of six HBM round trips and six launches, exactly the bytes/token and launches/token levers the book tracks (Chen, 2026a; Dao *et al.*, 2022). Extending the same workflow—compile with fusion, diff the HLO before and after, validate command-buffer capture on the fused decode step—turns “XLA is faster” into a measured, reproducible claim.

The **cross-hardware** reading is where XLA’s breadth helps and stops. Because the same StableHLO export compiles to CPU (LLVM, cache-blocked), GPU (LLVM/Triton, captured), and TPU (XLA-only), a team can A/B the identical graph across targets on one decode-cell methodology and compare counters rather than FLOPs (OpenXLA Project, 2024; Davies *et al.*, 2025). But XLA does not target static NPU fabrics, and its automatic fusion can miss residency a search-based or hand-fused kernel would find, so it is one strong tool in the matrix, not the universal answer (AMD, 2024b; Wu *et al.*, 2024). **YiRage** encodes the residency rules XLA’s passes leave implicit as checkable IR metadata, so a fusion

that would spill is rejected at compile time across backends rather than discovered in production (Chen and YiRage Contributors, 2026).

15.6 Key Takeaways

- **XLA is automatic, broad, and TPU’s only path.** StableHLO $\rightarrow$ HLO $\rightarrow$ LLVM/PTX/TPU compiles one graph to CPU, GPU, and TPU without hand-written kernels (OpenXLA Project, 2024; Lai *et al.*, 2023). Automatic means it optimizes whatever you profile, so point it at decode (Davies *et al.*, 2025).
- **Fusion is the bytes/token lever.** XLA’s fusion invariant—no intermediate in HBM, one fusion equals one kernel, driven by PriorityFusion—is exactly the on-chip residency decode needs (OpenXLA Project, 2024; Chen, 2026a). Gate each fusion pass on buffer accounting (Vellaisamy *et al.*, 2026).
- **Command-buffer capture is the launches/token lever.** `CommandBufferThunk` records and replays a GPU command buffer, removing per-kernel host dispatch like CUDA Graphs, but requires static shapes (OpenXLA Project, 2024; NVIDIA, 2024). Bucket dynamic decode shapes to stay capturable (Kwon *et al.*, 2023).
- **Codegen shares Triton and LLVM.** CPU/GPU use LLVM; advanced GPU fusions emit through TritonIR, so XLA and Triton share a codegen layer (OpenXLA Project, 2024; Tillet *et al.*, 2019). Vendor-library pattern-matching handles large GEMM/conv (NVIDIA, 2022).
- **Tune at the decode operating point.** XLA’s heuristics optimize the shape you compile, so prefill-shaped tuning breaks batch-1 decode; profile BS=1 ms/token and fingerprint configs per SKU (Davies *et al.*, 2025; OpenXLA Project, 2024). Re-tune per hardware (NVIDIA, 2022).

15.7 Conclusion

XLA shows that automatic, whole-graph compilation is production-viable at the largest scale: StableHLO portability, HLO fusion that keeps intermediates off HBM, scheduling that exploits the known graph, command-buffer capture that removes launch overhead, and LLVM/Triton codegen across CPU, GPU, and TPU (OpenXLA Project, 2024; NVIDIA, 2024). Its automation is its strength and its

trap—it optimizes the operating point you give it—so the decode discipline is to compile and grade at batch-1 on bytes/token and launches/token, the same residency contract YiRage enforces as metadata (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). With XLA’s automatic approach established, Chapter 16 turns to TVM, which pursues the same multi-hardware goal through explicit search rather than heuristics, trading XLA’s low effort for a larger reachable schedule space (Chen *et al.*, 2018b; Zheng *et al.*, 2020).

Chapter 16

TVM and Apache TVM Stack

Apache TVM made one bet that still defines the stack: separate *what* a tensor operator computes from *how* it is scheduled onto silicon, then let a learning-based cost model search the schedule space instead of a vendor library author (Chen *et al.*, 2018b). That decoupling is why a single operator definition can lower to x86, ARM, CUDA, and FPGA back-ends without rewriting the math (Chen *et al.*, 2018b; Chen, 2020). It is also why teams misread TVM in production: they tune a model until prefill GEMMs look excellent, ship it, and then watch interactive batch-1 decode miss p99 anyway (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The reason is the same one Chapter 1 established. Dense Llama-3.2-1B averages 847.5 GPU kernel launches per output token on H100 (BS=4, SL=2048); OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). A correct, well-tuned TVM operator schedule does nothing about that launch storm unless the compiler is also asked to fuse and capture the decode loop (Chen, 2026a; NVIDIA, 2024). This chapter treats TVM as a production residency contract under decode metrics—bytes/token and launches/token—not as a tutorial on schedule primitives (Davies *et al.*, 2025).

16.1 TVM HW matrix

TVM’s selling point is a hardware/target support **matrix**: write the compute once, lower it to many back-ends (Chen *et al.*, 2018b). In practice that matrix is necessary but not sufficient for decode. A target that *builds* is not a target that is decode-legal, and most “mysterious” regressions come from treating the two as the same thing (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

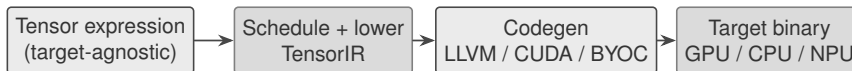


Figure 16.1: One tensor-expression definition fans out across the TVM hardware/target matrix: schedule and lower to TensorIR, then emit per-backend code.

The hardware truth underneath the matrix is the usual split. GPUs bound a schedule by shared-memory capacity and tensor-core feeding; CPUs by cache blocking and NUMA locality; NPUs and XDNA/AIE fabrics by static tile SRAM and DMA chains that must be assigned at compile time (NVIDIA, 2022; AMD, 2024b; Goodfellow *et al.*, 2016). At batch-1, decode stays bandwidth- and orchestration-limited on all three—the per-token matmul is too thin to amortize either HBM traffic or launch overhead (Chen, 2026a; Vellaisamy *et al.*, 2026).

TVM earns the matrix because the same tensor expression carries no target assumptions: the compute is declared once, and a separate schedule chooses tiling, vectorization, thread binding, and memory scope per target (Chen *et al.*, 2018b). That is genuinely more than vendor libraries deliver, where each new SKU or accelerator demands a hand-written kernel (Chen, 2020). The catch for serving is that the matrix is graded at *build* time—does the target lower and run—while decode goodput is a *runtime* property of bytes moved and kernels launched. A green build matrix cell tells you the operator exists on that target; it tells you nothing about whether the lowered schedule keeps the decoder layer on-chip.

Multi-hardware delta. The same TVM operator definition takes a different legal schedule per class: a Hopper GPU wants TMA-fed double buffering and a captured launch graph; an x86 CPU wants cache-blocked tiles and SIMD; an XDNA NPU wants a static tile-to-DMA assignment with no dynamic dispatch at all (SemiAnalysis, 2024; AMD, 2024b,a). Identical math, three different bytes/token outcomes. This is exactly the residency reasoning YiRage encodes as checkable IR metadata so the matrix entry is rejected before it ships, not after p99 slips (Chen and YiRage Contributors, 2026).

16.2 TVM arch passes

TVM runs optimization at two IR levels, and the production surprise is almost never a missing peak-FLOPs pass—it is an **optimization** that is legal in isolation but spills a fused producer to HBM between levels (Chen *et al.*, 2018b; Chen and YiRage Contributors, 2026). The modern stack makes the two levels explicit. A



Figure 16.2: TVM’s two-level pass pipeline: graph-level Relax transforms feed loop-level TensorIR transforms inside one `IRModule`.

graph-level `relax.Function` carries the dataflow; a `tir::PrimFunc` carries the loop nest; both live in one `IRModule`, and `call_tir` lowers a graph node into a loop-level kernel in destination-passing style so intermediate buffers can be planned globally (Lai *et al.*, 2023). Relax replaced the older Relay model precisely because Relay treated tensor programs as opaque foreign functions and could not track dynamic shapes—or residency—across the boundary (Lai *et al.*, 2023).

This is a deliberately different choice from an open dialect tower. Where MLIR exposes many progressively lowered *dialects*, TVM commits to a fixed graph→TensorIR pair and pushes its leverage into schedule search rather than dialect proliferation (Chen *et al.*, 2018b; Lai *et al.*, 2023). The engineering consequence: graph passes like operator fusion must be co-designed with the TensorIR **lowering** that follows, or fusion “succeeds” on paper while the lowered kernel still round-trips activations through DRAM.

Concretely, the graph-level passes that decide decode residency are a short list, and each has a loop-level consequence. `FuseOps` merges a chain of element-wise and reduction operators into one `call_tir` target so the lowered kernel keeps intermediates in registers or shared memory instead of HBM (Chen *et al.*, 2018b). `ToMixedPrecision` and layout-rewrite passes change how many bytes each activation costs and whether the tensor-core or vector path is even reachable on the target (Lai *et al.*, 2023). `StaticPlanBlockMemory` and the destination-passing-style contract behind `call_tir` let the compiler allocate one intermediate buffer and reuse it across the fused region, which is the difference between a decoder layer that streams its working set once and one that re-reads it per operator (Lai *et al.*, 2023; Chen, 2026a). Run these in the wrong order—layout after fusion, say—and the fused kernel is forced back through a transpose that re-materializes the very activations fusion was meant to keep on-chip.

What to diff before merge. Lower the same module twice and diff the generated TIR (or CUDA) for new buffer allocations; count launches/token on the GPU path and DMA edges on the NPU path (Vellaisamy *et al.*, 2026; AMD, 2024a). A silent spill introduced by a reordered pass reads downstream as an unexplained p99 regression, and graph-capture legality must hold on the GPU path while

static-allocation legality must hold on XDNA (NVIDIA, 2024; AMD, 2024b).

16.3 TVM codegen

Codegen is where the matrix meets Chapter 1’s counters. TVM emits through several back-ends: LLVM for x86 and ARM CPUs, CUDA/NVRTC (or source handed to NVCC) for NVIDIA GPUs, ROCm for AMD, and Bring-Your-Own-Codegen (BYOC) to **emit** calls into vendor libraries or an NPU runtime for any **platform** TVM does not natively target (Chen *et al.*, 2018b; Chen, 2020). The **backend** you pick decides the unit of work the host must launch.

The trap is to grade codegen by the quality of a single emitted matmul. At batch-1 decode the emitted MMA is rarely the limiter; the limiter is how many distinct kernels the host launches per token and how many HBM round trips survive lowering (Vellaisamy *et al.*, 2026; Chen, 2026a). A backend that emits a beautiful per-operator kernel but leaves a decoder layer as a dozen separate launches loses to a coarser fused kernel that the runtime can capture once. On the GPU path this is why graph capture matters: replaying a captured launch sequence removes per-kernel host dispatch and recovered roughly $1.26\times$ on a Qwen-2.5-7B batch-1 decode cell on H100 in the Memory-Floor study—without changing a single MMA tile (Chen, 2026a; NVIDIA, 2024). On the CPU path the equivalent win is fusing through cache-resident tiles so the emitted code never spills the working set; on an NPU it is emitting a static DMA schedule rather than dynamic dispatch (AMD, 2024b,a).

The other codegen decision that decode telemetry punishes is the runtime that wraps the emitted kernels. TVM can package a model as a graph executor, a relax VM, or an ahead-of-time (AOT) module, and each carries a different per-token host cost (Chen *et al.*, 2018b; Lai *et al.*, 2023). A Python-driven graph executor adds interpreter and dispatch overhead on every operator—tolerable for batched prefill, fatal for batch-1 decode where that overhead is paid once per token with nothing to amortize it against (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The AOT path and a captured launch graph collapse that orchestration into a single replayable sequence, which is the codegen-level lever that actually moves ms/token (NVIDIA, 2024; Chen, 2026a).

Multi-hardware delta. The CUDA backend can hide latency with asynchronous copy and double buffering, so its codegen can afford a dynamic schedule; the XDNA/AIE backend cannot, because its tiles and DMA descriptors are fixed before runtime (SemiAnalysis, 2024; AMD, 2024a). The same Relax module

Table 16.1: Ansor template-free auto-scheduling: reported maximum end-to-end DNN speedups over prior search/library baselines, by hardware class (Zheng *et al.*, 2020).

Hardware class	Representative target	Max speedup
Server / desktop CPU	Intel x86 (AVX-512)	$3.8\times$
Edge / mobile CPU	ARM (NEON)	$2.6\times$
Server GPU	NVIDIA (CUDA)	$1.7\times$

therefore emits genuinely different control flow per platform, and grading both against one microbenchmark hides the decode cost (Goodfellow *et al.*, 2016).

16.4 TVM tuning

TVM’s auto-tuning lineage is the part most teams adopt and the part most likely to mislead. AutoTVM came first: a template-based tuner that learns a statistical cost model (gradient-boosted trees or TreeGRU) over hand-written schedule templates, uses transfer learning across workloads, and explores with simulated annealing—reporting $1.2\text{--}3.8\times$ end-to-end gains over hand-tuned libraries on CPU, mobile GPU, and server GPU (Chen *et al.*, 2018a). Ansor removed the templates: it samples complete programs from a hierarchical search space, fine-tunes with evolutionary search and a learned cost model, and uses a task scheduler to spend **tuning** budget where it moves end-to-end latency most (Zheng *et al.*, 2020). MetaSchedule later unified these search strategies over TensorIR.

The search budget is itself a hardware-bound decision. AutoTVM’s transfer learning lets a cost model trained on one workload warm-start another, which matters because each candidate schedule must be compiled and measured on real silicon—thousands of trials per operator on a GPU, far more on a slow edge target (Chen *et al.*, 2018a). Ansor’s task scheduler spends that budget where it changes end-to-end latency rather than spreading it uniformly, so a long-tail operator that dominates decode is not starved while a cheap one is over-tuned (Zheng *et al.*, 2020). For a serving team the practical reading is that auto-tuning is not free wall-clock time; it is a measured search whose payoff depends on tuning the operators that actually appear on the batch-1 decode critical path.

The **pitfall** is the operating point. Auto-tuning maximizes whatever workload you **profile** against, and the default reflex is to tune on large, batched, prefill-shaped GEMMs because they are easy to benchmark and show big numbers (Kwon *et al.*, 2023). Schedules tuned at that shape over-tile for throughput and assume

launch cost is amortized across many tokens—both assumptions break at batch-1 decode, where the GEMM is a thin matrix-vector product and the launch floor dominates (Vellaisamy *et al.*, 2026; Chen, 2026a). The fix is mechanical: tune on the decode operating point (BS=1, realistic context), and rank candidates by measured ms/token after decomposing bytes/token and launches/token, not by the tuner’s raw cost-model score on a synthetic shape (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The same discipline differs by **hardware** class: GPU tuning chases coalescing and occupancy, CPU tuning chases cache locality and vector width, NPU “tuning” is mostly static tile assignment with little runtime freedom (AMD, 2024b; NVIDIA, 2022).

16.5 TVM case study

Make the contradiction concrete with one decoder layer as the **case**. Tune it with Ansor on three targets and **benchmark** at the decode operating point, not the tuner’s default (Zheng *et al.*, 2020; Vellaisamy *et al.*, 2026). The cross-hardware reading is consistent: per-operator schedules that win an isolated GEMM benchmark can still lose end-to-end because each operator is a separate launch and each unfused boundary is an HBM round trip (Chen, 2026a; Dao *et al.*, 2022). The lever that moves p99 is fusing the layer so activations stay on-chip and capturing the launch sequence so the host stops paying per-kernel dispatch every token (NVIDIA, 2024; Vellaisamy *et al.*, 2026).

The arithmetic makes the lever concrete. Take a decoder layer at hidden size $d=4096$ in BF16, where each materialized activation vector costs $4096 \times 2 = 8$ KB. An unfused schedule that writes and re-reads roughly six such intermediates—layer-norm output, Q/K/V projections, attention output, MLP input—per token spends about 48 KB/layer/token of avoidable HBM traffic before the KV cache is even touched (Chen, 2026a; Dao *et al.*, 2022). Fusing the layer so those vectors stay on-chip removes that traffic outright; at 32 layers it is on the order of 1.5 MB/token of HBM reads and writes that simply disappear, which is why bytes/token moves even when tensor-core utilization looks unchanged (Vellaisamy *et al.*, 2026). No tuner finds this if it is pointed at a single isolated GEMM.

This **cross-hardware** story is the book’s recurring thesis: the optimization rank differs by silicon, so no single tuned artifact is portable in the way the build matrix suggests (Davies *et al.*, 2025). On the GPU the win is fusion plus graph capture; on the CPU it is cache-resident fusion and SIMD; on the XDNA NPU it is a static tile/DMA schedule with the fusion decided entirely at compile time (AMD, 2024b,a). **YiRage** operationalizes the case: it carries the Chapter 2 residency

rules as checkable IR metadata, so a TVM-style schedule that would spill between graph and TensorIR levels is rejected at compile time across CUDA, CPU, and NPU back-ends rather than discovered in production telemetry (Chen and YiRage Contributors, 2026).

16.6 Key Takeaways

- **The build matrix is not the decode matrix.** TVM lowers one tensor-expression definition to CPU, GPU, and accelerator back-ends, but a target that compiles is not automatically decode-legal (Chen *et al.*, 2018b). Validate each matrix entry against bytes/token and launches/token at batch-1, not against a per-operator microbenchmark (Vellaisamy *et al.*, 2026; Chen, 2026a).
- **Co-design graph and TensorIR passes.** Relax graph fusion and TensorIR lowering live in one `IRModule` and must be optimized together; fusion that “succeeds” at the graph level still loses if lowering spills the fused producer to DRAM (Lai *et al.*, 2023; Chen and YiRage Contributors, 2026). Diff lowered IR for new allocations before merge (Chen *et al.*, 2018b).
- **Grade codegen by orchestration, not by one kernel.** At batch-1 the launch floor and surviving HBM round trips dominate, so a coarser fused-and-captured kernel beats a beautiful per-operator emit (Vellaisamy *et al.*, 2026; NVIDIA, 2024). Graph capture recovered roughly $1.26\times$ on an H100 decode cell with no MMA change (Chen, 2026a).
- **Tune at the decode operating point.** AutoTVM and Ansor maximize whatever shape you profile; tuning on prefill-shaped GEMMs over-tiles for throughput and breaks at batch-1 (Chen *et al.*, 2018a; Zheng *et al.*, 2020). Profile BS=1 ms/token and decompose it before trusting a cost-model score (Davies *et al.*, 2025).
- **Optimization rank is hardware-specific.** GPU wants coalescing, fusion, and capture; CPU wants cache locality and SIMD; XDNA/AIE wants static tile and DMA assignment (AMD, 2024b,a). The same Ansor-tuned definition yields different legal schedules—and different speedups—per class (Zheng *et al.*, 2020).

16.7 Conclusion

TVM’s compute/schedule split and its learning-based search remain the cleanest demonstration that performance portability is a compiler problem, not a library-porting problem (Chen *et al.*, 2018b; Zheng *et al.*, 2020). But the stack only pays off for interactive serving when its graph and TensorIR passes, its codegen back-end, and its auto-tuner are all pointed at the decode operating point and graded on bytes/token and launches/token (Vellaisamy *et al.*, 2026; Chen, 2026a). That is the same residency contract YiRage enforces as IR metadata, and it is the bridge into the next compiler surface: Chapter 17 turns from TVM’s whole-graph search to Triton’s Pythonic, block-level GPU kernel compiler, where the same fusion-and-launch discipline reappears at the level of a single hand-written kernel (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Chapter 17

OpenAI Triton: Pythonic GPU Kernel Compiler

Triton made GPU kernel authoring look like Python and still hit cuBLAS/cuDNN-class throughput, which is exactly why it is dangerous in a decode pipeline if you stop at “the kernel is fast” (Tillet *et al.*, 2019). The language is built around one abstraction—the *tile*, a statically shaped multi-dimensional sub-array—so a programmer writes a per-block program over tiles and the compiler handles coalescing, shared-memory staging, and instruction scheduling (Tillet *et al.*, 2019). That productivity is real: FlashAttention-style fused attention is practical to write and maintain in Triton in a way it never was in raw CUDA (Dao *et al.*, 2022). The decode trap is the same one Chapter 16 raised for TVM. A beautiful Triton `matmul` does nothing for batch-1 latency if the decoder layer still dispatches a dozen separate kernels per token. Dense Llama-3.2-1B already averages 847.5 GPU kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Triton helps only when it is used to *fuse* the decode loop into fewer, residency-aware kernels, and when those kernels are captured to amortize host launch cost (Chen, 2026a; NVIDIA, 2024). This chapter treats Triton as a block-level kernel compiler graded on bytes/token and launches/token, not as a Python tutorial (Davies *et al.*, 2025).

17.1 Triton HW matrix

Triton’s support **matrix** is deliberately narrower than a general tensor compiler’s, and pretending otherwise is the first **hardware** mistake. Triton is GPU-first: the



Figure 17.1: Triton compiles a per-block Python tile program to a fused GPU kernel; the block model assumes a GPU memory hierarchy.

production back-ends emit NVIDIA PTX and AMD AMDGCN, with CPU and Intel GPU back-ends emerging (OpenAI / triton-lang contributors, 2024). Its programming model assumes a SIMT machine with warps, shared memory, and a coalesced global-memory **target**—so it maps cleanly onto GPUs and not onto static-dataflow fabrics (NVIDIA, 2022; Tillet *et al.*, 2019).

The honest multi-hardware reading is that Triton is the right abstraction on one class and the wrong one on another. On a **gpu** it lets you express residency—which tiles live in shared memory, how loads are pipelined—at exactly the granularity decode needs (Tillet *et al.*, 2019; Chen, 2026a). On a **cpu** the same block program can be lowered but loses its *raison d’être*, since cache blocking and SIMD want a different schedule (Goodfellow *et al.*, 2016). On an **npu** such as XDNA/AIE the block-over-warps model does not apply at all: those fabrics demand static tile-to-DMA assignment decided at compile time, not a SIMT block grid (AMD, 2024b,a).

Triton’s design point also explains where it fits between raw CUDA and a whole-graph compiler. It abstracts away intra-block concerns—thread indexing, explicit shared-memory management, and memory coalescing are the compiler’s job, not the author’s—while leaving inter-block tiling and the fusion boundary to the programmer (Tillet *et al.*, 2019). That is more control than a graph compiler that decides fusion for you and more productivity than CUDA where you manage every load by hand. For decode the consequence is specific: Triton gives you exactly the knob that matters—where the fusion boundary sits—but it does not decide that boundary for you, so a careless kernel can still be fast in isolation and wasteful in a layer (Vellaisamy *et al.*, 2026; Chen, 2026a).

Multi-hardware delta. A Hopper GPU lets a Triton kernel hide latency with asynchronous copy and pipelined loads; an XDNA NPU cannot run the block model and needs a different compiler path entirely (SemiAnalysis, 2024; AMD, 2024a). This is why a serving stack treats Triton as one back-end among several rather than a universal target, and why YiRage carries Triton as a GPU code path while routing NPU work elsewhere (Chen and YiRage Contributors, 2026).



Figure 17.2: Triton’s MLIR pass pipeline: the Triton dialect (TTIR) is lowered to the TritonGPU dialect (TTGIR), then to LLVM and a GPU assembly target.

Table 17.1: Triton’s MLIR lowering stages and what each level controls; the TTGIR stage is where decode residency is decided (OpenAI / triton-lang contributors, 2024).

Stage	Dialect / form	What it controls
TTIR	Triton dialect	Tile program, target-agnostic
TTGIR	TritonGPU dialect	Layout, coalescing, pipelining
LLVM IR	LLVM	Scalar/vector lowering, registers
PTX / AMDGCN	GPU assembly	Final emit (<code>ptxas</code> / ROCm)

17.2 Triton arch passes

Modern Triton’s backend was rewritten on MLIR, so the **pass** pipeline is an explicit **dialect** tower: the Triton **dialect** (TTIR) captures the tile program; the TritonGPU dialect (TTGIR) attaches GPU-specific layout and scheduling; both then **lower** to LLVM IR and a GPU assembly target (OpenAI / triton-lang contributors, 2024). Unlike TVM’s fixed two-level IR, Triton commits to this progressive-lowering dialect stack and concentrates its **optimization** in the TTGIR stage (Chen, 2020; Tillet *et al.*, 2019).

The TTGIR passes are where decode residency is won or lost. Layout assignment decides whether a tile lives in registers, shared memory, or an MMA-friendly layout for tensor cores; coalescing passes decide whether global loads are contiguous; the software-pipelining pass overlaps loads with compute by issuing them `num_stages` iterations ahead (OpenAI / triton-lang contributors, 2024; SemiAnalysis, 2024). Get the layout pass wrong and the kernel silently spills a tile to global memory between blocks—which reads downstream as an unexplained bytes/token regression rather than a compile error (Chen, 2026a; Vellaisamy *et al.*, 2026).

Triton’s layout system is the part most CUDA programmers underestimate. A blocked layout distributes a tile across threads for coalesced loads; an MMA layout matches the register fragment shape a tensor-core instruction expects; a shared layout stages data for cross-thread reuse (OpenAI / triton-lang contributors, 2024). The compiler inserts conversions between layouts automatically, and an unnecessary conversion is a hidden shared-memory round trip—cheap in a throughput kernel,

measurable in a batch-1 decode kernel that runs thousands of times per token window (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024). This is the level at which fusion either keeps the decoder layer on-chip or quietly does not.

What to inspect before merge. Triton can dump every stage (`TRITON_KERNEL_DUMP`, `MLIR_ENABLE_DUMP`); diff the TTGIR and the emitted PTX for new shared-memory allocations or un-pipelined loads, and count launches/token at the call site (OpenAI / triton-lang contributors, 2024; NVIDIA, 2024). A pass reorder that breaks pipelining will not change correctness, only p99 (Davies *et al.*, 2025).

17.3 Triton codegen

Codegen in Triton is per-GPU and explicit. The NVIDIA **backend** lowers LLVM IR to PTX and runs `ptxas` to **emit** a CUBIN; the AMD backend emits AMDGCN for ROCm (OpenAI / triton-lang contributors, 2024). The **platform** you target therefore changes the final assembly, but the controlling knobs are the same two parameters that govern the emitted schedule: `num_warps` (warps per CTA, hence threads = `num_warps`×32) and `num_stages` (software-pipelining depth) (OpenAI / triton-lang contributors, 2024).

The trap is grading the emitted kernel by its standalone throughput. At batch-1 decode the dominant cost is host orchestration and surviving HBM round trips, not the quality of a single emitted MMA (Vellaisamy *et al.*, 2026; Chen, 2026a). Two codegen decisions actually move ms/token. First, fuse: emit one kernel for the decoder layer (e.g. a Triton FlashAttention plus the projection epilogue) so activations never leave the chip between operators (Dao *et al.*, 2022). Second, capture: wrap the fused launch sequence in a CUDA Graph so the host stops paying per-kernel dispatch every token—the Memory-Floor study recovered roughly $1.26\times$ on an H100 Qwen-2.5-7B decode cell from capture alone, with no change to the emitted math (Chen, 2026a; NVIDIA, 2024).

There is a second-order codegen cost that decode exposes: register pressure. The LLVM stage allocates registers per thread, and a kernel that fuses an entire decoder layer can exceed the per-thread budget, forcing spills to local memory or capping occupancy through `maxnreg` (OpenAI / triton-lang contributors, 2024; NVIDIA, 2022). On a throughput kernel low occupancy is often fine because each block does enough work; on a batch-1 decode kernel the launch is already thin, so a register spill is pure overhead with nothing to hide behind (Vellaisamy *et al.*, 2026; Chen, 2026a). The practical consequence is that “fuse everything” is not free—past a point, a slightly less fused kernel with healthy occupancy beats a

maximally fused one that spills.

Multi-hardware delta. Raising `num_stages` prefetches more aggressively and hides latency on a Hopper GPU with deep async-copy queues, but it costs shared memory and can reduce occupancy on a smaller SKU (SemiAnalysis, 2024; NVIDIA, 2022). The same Triton source therefore wants a different `num_stages` per GPU generation, and none of it transfers to a static NPU fabric (AMD, 2024b).

17.4 Triton tuning

Triton’s auto-tuning is a decorator, not a search service: `@triton.autotune` takes a list of `triton.Config` entries—block sizes plus `num_warps` and `num_stages`—benchmarks them at runtime for a given key, and caches the winner (OpenAI / triton-lang contributors, 2024). Because the candidate set is author-supplied, the quality of **tuning** is bounded by which configurations you thought to list. This is leaner than TVM’s learned-cost-model search but puts the burden on the engineer to enumerate sensible blocks.

Autotuning is also not free at runtime. Each config in the list must be compiled and benchmarked the first time a new key is seen, so a large config set inflates warmup latency and the first request after a shape change pays the search (OpenAI / triton-lang contributors, 2024). In a serving context that means the config list should be curated for the shapes you actually decode at, not a kitchen-sink sweep—and the cache key must include the dimensions that change the optimal schedule, or the autotuner silently reuses a config tuned for the wrong shape.

The **pitfall** is the operating point you **profile**. The reflexive config list is copied from a GEMM example tuned for large, batched, prefill-shaped matmuls: big `BLOCK_M/BLOCK_N`, high `num_stages` (Kwon *et al.*, 2023). At batch-1 decode the attention and projection GEMMs are matrix-vector products, so a large block tile is mostly idle lanes and a deep pipeline over-allocates shared memory for loads that never amortize (Vellaisamy *et al.*, 2026; Chen, 2026a). The fix is mechanical: autotune at the decode shape (`BS=1`, realistic context), include small-block and low-`num_stages` configs, and rank candidates by measured end-to-end ms/token after decomposing bytes/token and launches/token—not by the kernel’s isolated GB/s (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The right config is also **hardware**-specific: a Hopper SKU tolerates a deeper pipeline than a memory-smaller inference GPU, so the cached winner does not port across generations (NVIDIA, 2022).

17.5 Triton case study

Make it concrete with a fused decode attention kernel as the **case**. Writing FlashAttention in Triton keeps the query–key–value tiles and the running softmax state on-chip, so the kernel computes attention without materializing the full score matrix to HBM (Dao *et al.*, 2022; Tillet *et al.*, 2019). **Benchmark** it at the decode operating point and the reading matches the book’s thesis: an isolated per-operator kernel can win a microbenchmark yet lose end-to-end because each unfused boundary is an HBM round trip and each kernel is a separate launch (Vellaisamy *et al.*, 2026; Chen, 2026a).

The arithmetic shows why fusion is the lever. A non-fused attention step writes the full score vector for the new query against the cached context to HBM and reads it back for the softmax; at context length $L=2048$ in FP32 that score vector is $2048 \times 4 = 8$ KB written and re-read per head per token (Dao *et al.*, 2022). A Triton FlashAttention kernel keeps the running max and sum in registers and never materializes that vector, so across many heads and 32 layers it removes on the order of hundreds of kilobytes of avoidable HBM traffic per token—bytes/token drops even though the multiply-accumulate count is unchanged (Vellaisamy *et al.*, 2026; Chen, 2026a). An isolated GEMM benchmark cannot see this win because it never had the inter-operator round trip in the first place.

This case also points at where Triton meets the book’s megakernel thesis. Fusing one attention block is a start, but the larger decode win comes from collapsing a whole decoder layer, or even several, into one persistent kernel that keeps weights and running state resident and issues a single launch per token rather than dozens. Triton makes that practical to express, because the programmer controls exactly where the fusion boundary sits and can widen it until register pressure or shared memory pushes back. The discipline is to grow the fused region while watching occupancy and the launch count together, stopping at the point where one more fused operator would force a spill that costs more than the launch it saved. That trade is measured per token and per hardware generation, never assumed, which is precisely the residency contract the rest of this book builds on.

The **cross-hardware** conclusion is the honest limit of Triton. On the GPU, the fused-and-captured Triton kernel is close to the best available decode path; on a CPU the same tile program is a poor fit for cache-blocked SIMD; on an XDNA/AIE NPU the block-over-warps model does not lower at all and a static-dataflow compiler is required instead (AMD, 2024b,a; Goodfellow *et al.*, 2016). **YiRage** reflects this directly: it uses Triton as its GPU kernel back-end while carrying the Chapter 2 residency rules as checkable IR metadata, so a Triton

schedule that would spill a tile between blocks is rejected at compile time, and non-GPU targets are routed to backends whose machine model actually fits (Chen and YiRage Contributors, 2026).

17.6 Key Takeaways

- **Triton is a GPU block compiler, not a universal target.** Its tile/SIMT model maps to NVIDIA PTX and AMD AMDGCN but not to static-dataflow NPU's, so treat it as one back-end in a multi-hardware stack (Tillet *et al.*, 2019; OpenAI / triton-lang contributors, 2024). The build matrix is GPU-shaped (AMD, 2024b).
- **Win residency in the TTGIR passes.** Layout assignment, coalescing, and software pipelining at the TritonGPU dialect decide whether tiles stay on-chip; a bad layout pass spills silently and shows up only as a bytes/token regression (OpenAI / triton-lang contributors, 2024; Chen, 2026a). Dump and diff the IR before merge (Vellaisamy *et al.*, 2026).
- **Grade codegen by orchestration, not one kernel.** At batch-1 the launch floor and HBM round trips dominate, so fuse the decoder layer and capture the launch sequence; capture alone recovered roughly $1.26\times$ on an H100 decode cell (Chen, 2026a; NVIDIA, 2024). A fast standalone MMA is not a fast token (Dao *et al.*, 2022).
- **Autotune at the decode operating point.** `@triton.autotune` only explores the configs you list; default GEMM configs over-tile and over-pipeline for batch-1 (OpenAI / triton-lang contributors, 2024; Kwon *et al.*, 2023). Include small-block, low-`num_stages` configs and rank by ms/token (Davies *et al.*, 2025).
- **Tuned kernels do not port across GPUs.** Optimal `num_warps/num_stages` depend on shared-memory size and async-copy depth, so a cached winner must be re-tuned per generation (NVIDIA, 2022; SemiAnalysis, 2024). Portability is a re-tune, not a copy (Goodfellow *et al.*, 2016).

17.7 Conclusion

Triton's tile abstraction and MLIR dialect pipeline are the cleanest way to write fused, residency-aware GPU kernels at Python productivity, and they are why

decode-critical kernels like FlashAttention are now maintainable (Tillet *et al.*, 2019; Dao *et al.*, 2022). But the payoff is conditional: Triton helps interactive serving only when its TTGIR passes, its PTX/AMDGCN codegen, and its autotuner are aimed at the batch-1 decode shape and graded on bytes/token and launches/token (Vellaisamy *et al.*, 2026; Chen, 2026a). That is the residency contract YiRage enforces as IR metadata across its backends (Chen and YiRage Contributors, 2026). Triton also exposes the limit of a GPU-only kernel compiler, which motivates the next surface: Chapter 18 turns to IREE, an MLIR-native compiler and runtime that carries the same dialect-lowering discipline across a wider hardware matrix (Goodfellow *et al.*, 2016).

Chapter 18

IREE: MLIR-Native Runtime and Compilation

IREE attacks the decode problem from a different angle than a kernel compiler: it compiles the model’s *scheduling logic and its device execution together*, ahead of time, into one deployable artifact (IREE / iree-org contributors, 2024). Where a Python serving loop decides per-token what to launch at runtime, IREE bakes the host orchestration into virtual-machine bytecode at compile time—which is exactly the lever Chapter 1 identified, because at batch-1 the launch and dispatch path, not the matmul, sets ms/token (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Dense Llama-3.2-1B already averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). A compiler that schedules the whole program can collapse that orchestration the way captured CUDA Graphs do, but earlier and across more targets (Chen, 2026a; NVIDIA, 2024). The catch is the same as every other surface in this Part: IREE will happily produce a correct binary for a target whose schedule still re-reads activations from DRAM every token. This chapter treats IREE as an MLIR-native compiler+runtime graded on bytes/token and launches/token, not as a build-system walkthrough (Goodfellow *et al.*, 2016).

18.1 IREE HW matrix

IREE’s support **matrix** is the broadest in this Part: one MLIR input compiles to llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, and Metal, with AMD AIE still experimental (IREE / iree-org contributors, 2024). The Hardware Abstraction

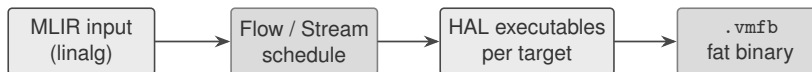


Figure 18.1: IREE lowers one MLIR input through whole-program scheduling into a single `.vmfb` that carries host instructions and per-target device kernels.

Layer (HAL) is what makes that breadth tractable—it is a Vulkan-inspired **target** abstraction, roughly 1:1 with IREE’s HAL C API, that exposes buffers, command buffers, and dispatches as a common surface across device types (IREE / iree-org contributors, 2024).

The honest reading is that a wide build matrix is not a wide *decode* matrix. A green HAL **target** means the model lowers and runs there; it says nothing about whether the schedule keeps a decoder layer on-chip (Vellaisamy *et al.*, 2026; Chen, 2026a). The hardware truth is unchanged across the matrix: a **gpu** is bound by shared-memory residency and tensor-core feeding, a **cpu** by cache blocking and NUMA, an **npu** like XDNA/AIE by static tile SRAM and DMA chains—and on the IREE matrix the NPU path is still experimental, so it is exactly where the build-vs-decode gap is widest (NVIDIA, 2022; AMD, 2024b,a).

What makes the breadth credible rather than aspirational is that IREE was designed to scale from the datacenter down to mobile and edge, where the same compiled artifact must run under tight memory and power budgets (IREE / iree-org contributors, 2024). The runtime is deliberately small and optional: a fully featured server can use the VM for dynamic deployment with multi-threaded dispatch, while an embedded target can bypass the runtime entirely and link the generated kernels directly (IREE / iree-org contributors, 2024). For a decode engineer that flexibility is a double-edged sword. The same model that serves on an H100 can be compiled for an edge accelerator, but the residency math is completely different at the two ends of the matrix: an edge SKU with megabytes of on-chip memory cannot hold the working set a datacenter GPU does, so a dispatch-region fusion that is a clear win on H100 can overflow on-chip storage and spill on the edge target.

Multi-hardware delta. The same `.vmfb` can carry kernels for several vendors, and IREE can probe the device at load time to pick one (IREE / iree-org contributors, 2024). But a Hopper GPU wants async-copy double buffering while an XDNA fabric wants a static DMA schedule, so the portable artifact still hides per-target residency decisions that decode telemetry will expose (SemiAnalysis, 2024; AMD, 2024a). YiRage treats this as a routing problem: it carries the Chapter 2 residency rules as checkable metadata and selects a backend whose machine model fits,

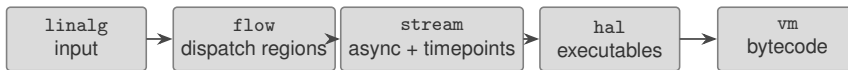


Figure 18.2: IREE’s dialect pipeline: **flow** forms dispatch regions, **stream** schedules them asynchronously with timepoints, **hal** translates per target, **vm** emits host bytecode.

Table 18.1: IREE’s ahead-of-time compilation phases; **flow** fusion sets bytes/token and **stream** scheduling sets launches/token (IREE / iree-org contributors, 2024).

Phase	What it does
Input	Lower to core dialects (linalg , etc.)
Flow	Form and fuse dispatch regions; partition
Stream	Async schedule with timepoints; encode resources
HAL	Per-target executable translation (codegen)
VM	Emit host scheduling bytecode

rather than trusting that a green matrix cell is decode-legal (Chen and YiRage Contributors, 2026).

18.2 IREE arch passes

IREE’s compilation is a sequence of MLIR **dialects**, and the **pass** pipeline is where decode goodput is decided long before any device code is generated (IREE / iree-org contributors, 2024). After the input **dialect** (**linalg** and friends), dispatch-creation fuses operations and forms *dispatch regions*—the unit of work that becomes one kernel; the **flow** dialect models data flow and partitioning; the **stream** dialect **lowers** tensors into opaque resources and schedules the dispatches asynchronously using *timepoints* so independent work overlaps; the **hal** dialect translates each executable per target; and the **vm** dialect emits the host bytecode (IREE / iree-org contributors, 2024).

Two of these passes are the ones a decode engineer must watch. Dispatch-region formation is the **optimization** that sets bytes/token: if it fuses a decoder layer’s element-wise and reduction work into one region, intermediates stay on-chip; if it leaves them separate, each boundary is a DRAM round trip (Vellaisamy *et al.*, 2026; Chen, 2026a). Stream scheduling is the one that sets launches/token: by sequencing dispatches with timepoints instead of a host-side wait per kernel, it removes exactly the per-launch orchestration that dominates batch-1 (IREE / iree-org contributors, 2024; NVIDIA, 2024).

What to inspect before merge. IREE can dump the executable for each dispatch region (`-iree-hal-dump-executable-*`), so diff which operators were fused into a region and which spilled out; an aggressive-fusion flag exists, but more fusion is only a win until register or shared-memory pressure forces a spill (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). A regression here is silent: the binary is correct, only bytes/token moved (Goodfellow *et al.*, 2016).

18.3 IREE codegen

IREE’s **codegen** runs per dispatch region during the HAL executable-translation stage, and each **backend emits** a different low-level form: PTX for CUDA, HSACO for ROCm/HIP, SPIR-V for Vulkan, and native code for the CPU (IREE / iree-org contributors, 2024). Because compilation is ahead-of-time, the **platform** mix is resolved on the build host and packaged into the FlatBuffers `.vmfb` fat binary, which carries host VM instructions alongside the device kernels so the runtime can dispatch without a JIT (IREE / iree-org contributors, 2024).

The decode-relevant point is that IREE’s runtime is not a Python loop; the VM replays a compiled schedule, so the host orchestration cost is paid at compile time rather than per token (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). This is structurally the same win that capturing a launch graph buys on a GPU—the Memory-Floor study recovered roughly $1.26\times$ on an H100 decode cell purely by removing host overhead—except IREE bakes it into the deployable artifact and extends it to CPU and Vulkan targets (Chen, 2026a; NVIDIA, 2024). What IREE cannot do is rescue a poorly fused dispatch region: if codegen emits a layer as many small kernels because fusion failed upstream, no amount of clean scheduling removes the HBM traffic those kernels create (Dao *et al.*, 2022).

The fat-binary format is itself a decode-relevant design choice. Because the kernels are committed at build time, IREE can ship several specialized variants of the same dispatch region—aligned and unaligned shapes, large and small matmuls—and let the runtime probe the device and select one without a JIT step (IREE / iree-org contributors, 2024). For batch-1 decode that avoids the warmup and recompilation stalls a JIT stack pays on the first request after a shape change, which is precisely the moment an interactive session is most latency-sensitive. The cost is binary size and a build matrix that grows with every target and specialization, so a serving team trades disk and compile time for predictable per-token latency—usually the right trade for an interactive product, the wrong one for a one-off batch job.

Multi-hardware delta. The HAL maps the same dispatch onto workgroups that become GPU thread blocks or CPU threads, and subgroups that become GPU SIMT lanes or CPU SIMD (IREE / iree-org contributors, 2024). The abstraction is genuine, but the best tile shape and async depth still differ per target, so one dispatch region’s codegen is not optimal everywhere (SemiAnalysis, 2024; NVIDIA, 2022).

18.4 IREE tuning

IREE **tuning** happens at compile time through target configuration: the executable-configurations stage selects a translation strategy per dispatch region, and flags steer dispatch-region fusion and per-target codegen (IREE / iree-org contributors, 2024). Because the schedule is compiled, **profile**-first discipline shifts earlier than with a JIT stack—you compile a candidate, run it, and read where time went before changing flags, rather than letting a runtime autotuner converge.

There is a second discipline that whole-program compilation rewards: compile-time visibility. Because every dispatch region and its scheduling are decided before deployment, the compiler can report exactly which operators fused, how many regions a layer became, and how they are sequenced—information a runtime-dispatched stack only learns by profiling the live system. A decode team should treat that report as the first artifact to read after a build, comparing the dispatch-region count per layer against expectation before ever launching the model. When the region count is higher than the number of fused units you designed for, fusion failed somewhere upstream, and the fix belongs in the compilation flags, not in a runtime workaround.

The **pitfall** is benchmarking the wrong shape and the wrong layer. Prefill compiles into large, well-fused dispatch regions that look excellent in isolation; decode is dominated by the thin, latency-bound per-token path where dispatch granularity and host scheduling matter more than peak occupancy (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026). Compile and profile at the batch-1 decode operating point, decompose end-to-end ms/token into bytes/token and launches/token, and only then adjust fusion and codegen flags (Davies *et al.*, 2025; Chen, 2026a). The right configuration is **hardware**-specific: the CPU path wants cache-blocked dispatch regions, the CUDA path wants fused kernels plus async copy, and the experimental AIE path wants static tiling—so the tuned configuration for one HAL target does not transfer to another (AMD, 2024b; Goodfellow *et al.*, 2016).

18.5 IREE case study

Take a single decoder layer compiled through IREE as the **case**, and **benchmark** it at the decode operating point on more than one HAL target (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The reading matches the book’s thesis: a layer that compiles into one fused dispatch region with stream-scheduled launches keeps activations on-chip and pays host orchestration once; a layer that fragments into many dispatch regions re-reads activations from DRAM and pays a launch per region every token (Chen, 2026a; Dao *et al.*, 2022). The arithmetic is the same as for any fusion: at hidden size 4096 in BF16 each materialized activation vector is 8KB, so a handful of avoided round trips per layer is tens of kilobytes of HBM traffic removed per token before the KV cache is even read (Vellaisamy *et al.*, 2026). Across the full depth of a model that avoided traffic accumulates into a meaningful fraction of the per-token memory budget, which is why the dispatch-region count, not the peak throughput of any single region, is the number that tracks decode latency. The same compiled layer that fragments into a dozen regions will read and rewrite those vectors a dozen times, and the runtime schedule cannot recover bytes the compiler chose to spill.

The **cross-hardware** conclusion is where IREE’s breadth helps and where it stops. Because IREE keeps the same dispatch-region formation across targets, a layer fused well on the CPU path is usually fused on the CUDA path too, which makes A/B comparison and bring-up faster (IREE / iree-org contributors, 2024). But the experimental AIE/NPU path is exactly where a green build hides an unfinished decode story, and a Vulkan target on a phone has different residency limits than an H100 (AMD, 2024a; NVIDIA, 2022). **YiRage** uses IREE-style MLIR lowering as one path while enforcing the Chapter 2 residency rules as checkable IR metadata, so a dispatch region that would spill is rejected at compile time and non-GPU work is routed to a backend whose machine model fits (Chen and YiRage Contributors, 2026).

18.6 Key Takeaways

- **IREE compiles scheduling and execution together.** Host orchestration is baked into VM bytecode ahead of time, so the per-token launch path is set at compile time, not by a runtime loop (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). That is the structural fix for the launches/token problem decode hits (Davies *et al.*, 2025).
- **Dispatch-region fusion sets bytes/token.** Whether the flow stage fuses

a decoder layer into one region decides how many DRAM round trips survive; dump and diff the executables before trusting a build (IREE / iree-org contributors, 2024; Chen, 2026a). A correct binary can still be a slow token (Goodfellow *et al.*, 2016).

- **Stream scheduling sets launches/token.** The `stream` dialect sequences dispatches with timepoints, removing per-kernel host waits the way a captured launch graph does—worth roughly $1.26\times$ on an H100 decode cell from host-overhead removal alone (Chen, 2026a; NVIDIA, 2024).
- **A wide build matrix is not a wide decode matrix.** IREE targets CPU, CUDA, ROCm, Vulkan, and Metal, but the NPU/AIE path is experimental and a green target only means it lowers and runs (IREE / iree-org contributors, 2024; AMD, 2024b). Validate each target against decode metrics, not compilation success (AMD, 2024a).
- **Tuned configurations are per-HAL-target.** Cache-blocked CPU dispatch, fused CUDA kernels, and static AIE tiling are different strategies, so a configuration tuned for one HAL target does not transfer (SemiAnalysis, 2024; NVIDIA, 2022). Compile and profile at the batch-1 decode shape (Kwon *et al.*, 2023).

18.7 Conclusion

IREE’s contribution is to make whole-program scheduling a compiler artifact: by lowering through Flow, Stream, HAL, and VM it compiles host orchestration and device kernels together and packages them into one retargetable binary (IREE / iree-org contributors, 2024). For interactive serving that matters because it attacks launches/token at compile time and dispatch-region fusion attacks bytes/token—provided both are graded at the batch-1 decode operating point rather than on prefill-shaped regions (Vellaisamy *et al.*, 2026; Chen, 2026a). That is the same residency contract YiRage enforces as IR metadata across backends (Chen and YiRage Contributors, 2026). IREE also shows that breadth and depth are different axes: a wide HAL matrix still needs per-target residency proof, which sets up the next surface—Chapter 19 turns to Glow, a lighter-weight AI compiler that makes the opposite trade between generality and a focused lowering path (Goodfellow *et al.*, 2016).

Chapter 19

Glow: Lightweight AI Compiler

Glow is the compiler in this Part that optimizes for the opposite end of the hardware spectrum from a datacenter GPU: it targets CPUs, accelerators, and memory-constrained edge devices, and it makes a deliberate bet that ahead-of-time compilation into a self-contained object file beats a runtime interpreter (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). That bet matters for the same reason Chapter 1 gave: eliminating per-operator dispatch overhead is the lever that moves batch-1 latency, and Glow removes it at compile time by emitting one bundle with a single entry function rather than dispatching operators through a framework (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). The honest framing is that Glow is not where you optimize H100 LLM decode—it has no CUDA-graph-class GPU path—but it is exactly where you optimize decode on a Cortex-M or a small accelerator, and its design choices (graph lowering to a few primitives, operator stacking, profile-guided quantization) are the ones a residency-first engineer should recognize (Chen, 2026a; Goodfellow *et al.*, 2016). This chapter treats Glow as a lightweight AOT compiler graded on bytes/token and launches/token at the edge, not as a build-tool tutorial (NVIDIA, 2024).

19.1 Glow HW matrix

Glow’s support **matrix** is shaped by one design decision: rather than implement every operator on every **target**, the compiler lowers the high-level graph to a small set of linear-algebra primitives, so a new **hardware** backend only has to implement those primitives (Rotem *et al.*, 2018). That is why Glow spreads cheaply across accelerators and edge parts, and why its production sweet spot is the LLVM CPU

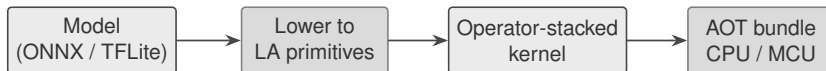


Figure 19.1: Glow lowers a model to a few linear-algebra primitives, stacks element-wise work into single-traversal kernels, and emits a self-contained AOT bundle.

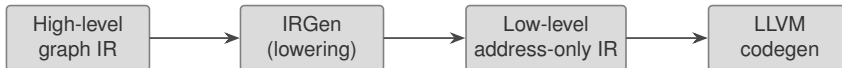


Figure 19.2: Glow’s two-phase IR: a high-level node graph for domain-specific optimization, lowered via IRGen to a low-level address-only instruction IR before code generation.

backend cross-compiled to embedded cores like Arm Cortex-M, not a server GPU (PyTorch / Glow contributors, 2024).

The honest multi-hardware reading inverts the usual GPU-first story. On a **cpu** or microcontroller Glow is excellent: the AOT bundle removes interpreter and dispatch overhead and fits a fixed memory budget (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). On a **gpu** Glow exists (an OpenCL backend) but is not the decode tool—there is no captured-launch-graph story to rival the CUDA path Chapters 16–18 described (Chen, 2026a; NVIDIA, 2024). On an **npu** or custom accelerator Glow’s small-primitive lowering is precisely the abstraction that makes a new backend tractable, which is its strategic value to silicon vendors (AMD, 2024b,a).

Multi-hardware delta. The same model compiles to an x86 bundle for a host and a Cortex-M bundle for an edge device, but the residency math differs by orders of magnitude: an MCU with kilobytes of SRAM cannot hold the working set an x86 cache can, so a fusion that helps on the host may not even fit on the edge target (Goodfellow *et al.*, 2016; AMD, 2024a). YiRage treats this as routing: it carries the Chapter 2 residency rules as checkable metadata and selects an edge-style AOT path or a GPU path per target rather than assuming one bundle is universally optimal (Chen and YiRage Contributors, 2026).

19.2 Glow arch passes

Glow’s **pass** pipeline is built on a two-phase strongly-typed IR, and each phase owns a different class of **optimization** (Rotem *et al.*, 2018). The high-level IR is a static-shaped dataflow node graph where domain-specific rewrites live: merge batch-norm into convolution, change a matrix layout, eliminate a numeric rescale. IRGen then **lowers** each high-level node into one or more low-level instructions,

Table 19.1: Glow’s two-phase IR and the optimization class each level owns (Rotem *et al.*, 2018).

IR level	Form	Optimizations
High-level	Node dataflow graph	Domain-specific (BN+conv, layout, rescale)
Low-level	Address-only instructions	Scheduling, static alloc, copy elimination
Codegen	LLVM IR	Stacked kernels, target-specific emit

producing an address-only IR whose operands are typed pointers to buffers; this is where memory optimizations run—instruction scheduling, static memory allocation, and copy elimination (Rotem *et al.*, 2018).

It is worth being precise about what Glow is and is not here. Glow predates the MLIR **dialect** approach that IREE and modern TVM use, and instead of an open, extensible dialect tower it hard-codes this fixed two-level design (Rotem *et al.*, 2018). The trade is less generality but a shorter, more predictable lowering path—appropriate for an edge compiler where build reproducibility and a tight memory plan matter more than supporting every research IR. For decode, the low-level IR’s static memory allocation is the feature that matters: a fixed buffer plan computed at compile time is what lets a bundle run in a known footprint without an allocator in the hot path (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

What to inspect before merge. Because both IR levels are dumpable, diff the low-level IR for buffer reuse and copy elimination, and confirm that element-wise chains were stacked rather than left as separate traversals (Rotem *et al.*, 2018; Chen, 2026a). A regression shows up as extra buffers or extra tensor passes, not as a wrong answer (Goodfellow *et al.*, 2016).

19.3 Glow codegen

Glow’s primary **codegen** path is its LLVM-based CPU **backend**, and its signature trick is operator stacking: instead of one kernel per element-wise operator, Glow generates LLVM IR for a single stacked kernel that performs all the chained operations during one traversal of the tensor (PyTorch / Glow contributors, 2024; Rotem *et al.*, 2018). That is the bytes/token lever in its purest form—an unstacked chain reads and writes the tensor once per operator, while the stacked kernel touches it once (Vellaisamy *et al.*, 2026; Chen, 2026a).

Because the backend is LLVM, Glow can **emit** a self-contained object file (a

bundle) and cross-compile it for a different **platform** via standard LLVM target flags—for example an Arm Cortex-M7 core on an embedded board (PyTorch / Glow contributors, 2024). The decode-relevant consequence is that the bundle has a single inference entry point and no framework dispatch, so the per-token launch overhead that dominates batch-1 on a Python stack simply does not exist; the work is one compiled function call (Vellaisamy *et al.*, 2026; NVIDIA, 2024). This is the AOT counterpart to capturing a launch graph on a GPU, achieved by never having a launch loop in the first place (Chen, 2026a).

There is a subtler reason operator stacking matters more at the edge than the headline FLOP count suggests. On a memory-constrained device the limiter is rarely arithmetic; it is the bandwidth between the compute core and whatever memory tier holds the activations, and every extra tensor traversal pays that bandwidth tax again (Vellaisamy *et al.*, 2026; Chen, 2026a). By collapsing a chain of element-wise operators—bias add, activation function, rescale—into one traversal, Glow turns several bandwidth-bound passes into one, which on a small core is often a larger win than any single-operator speedup (Rotem *et al.*, 2018). The same logic is why the static memory plan matters: with buffers assigned at compile time, the bundle reuses a small pool of scratch memory instead of allocating per operator, so the resident footprint stays inside the device’s on-chip budget and the allocator never appears in the per-token path (Goodfellow *et al.*, 2016; Davies *et al.*, 2025).

Multi-hardware delta. On a CPU or MCU the stacked kernel plus a static buffer plan is close to the best available edge decode path; on a GPU the same approach does not exploit warps or tensor cores, so Glow’s CPU strength does not transfer upward (NVIDIA, 2022; SemiAnalysis, 2024). The lowering-to-primitives design is what lets a new accelerator backend reuse the high-level optimizations without reimplementing them (AMD, 2024a).

19.4 Glow tuning

Glow’s most distinctive tuning mechanism is **profile**-guided quantization, and it is profile-first by construction (Rotem *et al.*, 2018). The flow is two-phase: statically instrument the network with profiling nodes, run inference on representative data to record the numeric range of each activation, then recompile using that profile to convert the float network into int8 islands of computation, fusing away rescales between nodes (PyTorch / Glow contributors, 2024). On a memory-bound edge target this is a direct bytes/token win—int8 activations and weights move a quarter

of the bytes of float32—and it reduces the model footprint enough to fit parts that float32 never would (Vellaisamy *et al.*, 2026; Chen, 2026a).

The **pitfall** is the profiling data and the operating point. The recorded ranges are only as representative as the inputs you profiled, so a profile captured on prefill-style batches can mis-estimate the activation ranges seen during long-context batch-1 decode, producing saturation or precision loss exactly where it hurts output quality (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Two disciplines follow: profile on decode-representative sequences at the context lengths you actually serve, and re-profile when the workload shifts, because the AOT bundle freezes the quantization decision at compile time. As always the right configuration is **hardware**-specific—an MCU may force int8 everywhere while a host CPU can keep sensitive layers in float—so the tuned bundle for one target does not transfer to another (AMD, 2024b; Davies *et al.*, 2025).

19.5 Glow case study

Take a small decoder model compiled to an edge bundle as the **case**, and **benchmark** the int8 AOT bundle against a float interpreter at the decode operating point (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). The reading matches the book’s thesis from the edge side: the win is not a faster matmul but the removal of dispatch overhead and HBM/DRAM round trips—the stacked kernel keeps element-wise work in one traversal and the static buffer plan avoids allocator churn, so bytes/token and launches/token both drop while the arithmetic is unchanged (Chen, 2026a; Dao *et al.*, 2022). The quantization adds a second bytes/token win by shrinking every activation to int8 (PyTorch / Glow contributors, 2024). The arithmetic is direct: a decoder layer that moves float32 activations spends four bytes per element, and converting to int8 cuts that to one, so on a bandwidth-bound edge core the memory traffic for those activations drops by roughly four times before any kernel fusion is counted (Vellaisamy *et al.*, 2026). That is the dominant reason int8 bundles are recommended for memory-constrained deployment: the speedup comes from moving fewer bytes, not from faster multiplies.

The comparison also exposes a limit worth stating plainly. An interpreter dispatches each operator through a generic runtime, paying overhead per node that, at batch-1, has no batch to amortize against—which is precisely the per-token tax Chapter 1 measured on the Python stack (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). The AOT bundle removes that tax structurally by compiling the whole network into one function, so the speedup over an interpreter widens as the model gets deeper and the per-operator overhead would otherwise accumulate (Rotem

et al., 2018; Chen, 2026a).

A second lesson from the case is about debuggability under quantization. Because Glow emits debug information in terms of its own IR rather than raw machine code, a precision regression introduced by an int8 conversion can be traced back to the specific node whose recorded range was too tight, rather than guessing from a wrong output (Rotem *et al.*, 2018). For an edge decode deployment that traceability is the difference between a quick re-profile of one layer and a blind re-quantization of the whole model, and it is the kind of compile-time visibility the AOT path buys that a runtime interpreter cannot (PyTorch / Glow contributors, 2024; Davies *et al.*, 2025).

The **cross-hardware** conclusion is where Glow’s scope is honest. The same bundle source cross-compiler from x86 to Cortex-M, but the two targets have wildly different residency budgets, so the fusion and quantization plan that fits the host may overflow the MCU’s SRAM (Rotem *et al.*, 2018; AMD, 2024a). And none of Glow’s CPU/edge strength competes with a CUDA-graph-captured Triton or IREE path on a datacenter GPU—which is the correct division of labor, not a deficiency (NVIDIA, 2024; SemiAnalysis, 2024). **YiRage** reflects exactly this: it uses Glow-style AOT lowering and operator stacking for edge and CPU targets while routing datacenter-GPU decode to its GPU backends, and carries the Chapter 2 residency rules as checkable IR metadata so an edge bundle that would overflow on-chip memory is rejected at compile time (Chen and YiRage Contributors, 2026).

19.6 Key Takeaways

- **Glow is the edge/CPU compiler, not the GPU-decode tool.** Its LLVM CPU backend and AOT bundles target memory-constrained devices, and lowering to a few linear-algebra primitives makes new accelerator backends cheap (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). For datacenter GPU decode, prefer the CUDA-graph paths of Chapters 16–18 (NVIDIA, 2024).
- **Operator stacking is the bytes/token lever.** Glow’s stacked kernel performs chained element-wise work in a single tensor traversal instead of one kernel per operator, removing redundant DRAM round trips (PyTorch / Glow contributors, 2024; Chen, 2026a). Confirm chains were stacked by diffing the low-level IR (Vellaisamy *et al.*, 2026).
- **AOT bundles remove the launch loop entirely.** A self-contained object

file with a single inference entry point has no framework dispatch, so the batch-1 launch overhead that dominates a Python stack does not exist (Rotem *et al.*, 2018; Vellaisamy *et al.*, 2026). It is the edge counterpart to capturing a launch graph (NVIDIA, 2024).

- **Profile-guided quantization needs decode-representative data.** Glow records activation ranges by running inference, then recompiles to int8; a profile captured on the wrong shape mis-estimates ranges and loses precision where decode needs it (Rotem *et al.*, 2018; Kwon *et al.*, 2023). Profile at served context lengths and re-profile on workload shift (Davies *et al.*, 2025).
- **The two-phase IR trades generality for a predictable footprint.** A fixed high-level graph plus low-level address-only IR (with static memory allocation) is less general than an MLIR dialect tower but yields a known memory plan—the right trade for the edge (Rotem *et al.*, 2018; Goodfellow *et al.*, 2016). Tuned bundles are per-target (AMD, 2024b).

19.7 Conclusion

Glow shows that the residency-first thesis is not GPU-specific: at the edge, lowering to a few primitives, stacking operators into single-traversal kernels, statically planning memory, and quantizing under a recorded profile are the moves that drop bytes/token and erase launch overhead (Rotem *et al.*, 2018; PyTorch / Glow contributors, 2024). Its deliberate scope—CPU, accelerators, and memory-constrained devices rather than datacenter GPUs—makes it the right backend for one slice of the hardware matrix and the wrong one for another, which is exactly the routing decision YiRage encodes as IR metadata (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). The broader point is that a lightweight compiler is not a lesser one: by giving up generality it buys a predictable memory plan and a dispatch-free runtime, and those are precisely the properties that make decode latency stable on a small device where a heavyweight stack would never fit (Rotem *et al.*, 2018; Davies *et al.*, 2025). Having now surveyed established compiler surfaces from TVM through Glow, Chapter 20 turns to Mirage and the emerging wave of AI compilers that push these ideas—whole-program scheduling, superoptimization, and residency-aware fusion—past what today’s production stacks ship (Chen, 2026a; Goodfellow *et al.*, 2016).

Chapter 20

Mirage and Emerging AI Compilers

The compilers of Chapters 16–19 all answer the same question—given this operator, schedule it well—but Mirage asks a different one: given this whole tensor program, *superoptimize* it, discovering optimizations that span algebraic rewrites, schedules, and entirely new custom kernels at once (Wu *et al.*, 2024). That shift matters for decode because the highest-leverage transformation is not a better matmul tile; it is collapsing a token’s worth of operators into one kernel so the host stops launching thousands of them. Dense Llama-3.2-1B averages 847.5 GPU launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Mirage’s descendant, the Mirage Persistent Kernel (MPK), takes this to its conclusion by transforming LLM inference into a single megakernel and cutting end-to-end latency by 1.2–6.7 $\times$ (Mirage Project contributors, 2025). This chapter is also where the book’s own reference compiler comes from: **YiRage** is a Mirage-lineage, multi-backend descendant, so the ideas here—multi-level search, megakernel fusion, residency-as-metadata—are the spine of everything we have called YiRage in earlier chapters (Chen and YiRage Contributors, 2026). We treat Mirage and the emerging wave as production surfaces graded on bytes/token and launches/token, not as research curiosities (Chen, 2026a; Davies *et al.*, 2025).

20.1 Mirage HW matrix

Mirage’s “**matrix**” is not a list of vendor targets like IREE’s; it is the GPU compute hierarchy itself (Wu *et al.*, 2024). A μ Graph represents a tensor program



Figure 20.1: Mirage superoptimizes a tensor program over a multi-level μ Graph search space and emits a verified, fused kernel—one launch per token in the MPK limit.

uniformly at three levels—kernel, thread block, and thread—and that hierarchy *is* the **hardware** model the superoptimizer searches against (Wu *et al.*, 2024). The consequence is that Mirage is GPU-first by construction: the kernel/block/thread structure maps directly onto a CUDA **target**, and its strongest results are on NVIDIA GPUs (Mirage Project contributors, 2025).

The honest multi-hardware reading follows from that. On a **gpu** Mirage is exactly aligned with the residency hierarchy—block-level tensors live in shared memory, kernel-level tensors in device memory—so its search naturally optimizes bytes/token (Wu *et al.*, 2024; Chen, 2026a). On a **cpu** the kernel/block/thread abstraction does not map cleanly, so the superoptimization approach is emerging rather than production there (Goodfellow *et al.*, 2016). On an **npu** like XDNA/AIE the static-dataflow model is different again, and the value Mirage offers is conceptual—multi-level search and verified fusion—rather than a ready backend (AMD, 2024b,a).

Mirage is also a representative of a broader emerging wave, and the family resemblance is what matters more than any one project. The common thread across the new compilers is a move away from fixed, hand-authored pass pipelines toward *search with verification*: instead of trusting a human-proved rewrite, these systems explore a large space of equivalent programs and certify the winner, which lets them discover fusions and custom kernels that no pass library encodes (Wu *et al.*, 2024). The second shared theme is end-to-end fusion as a first-class goal rather than a peephole optimization—the megakernel is the clearest example, but the same instinct drives whole-program scheduling in IREE and aggressive operator fusion elsewhere (Mirage Project contributors, 2025; IREE / iree-org contributors, 2024). For a decode engineer the practical signal is that the frontier has shifted from “which library has the fastest GEMM” to “which compiler can collapse my whole token into the fewest, most resident launches and prove it correct” (Vellaisamy *et al.*, 2026; Chen, 2026a).

Multi-hardware delta. A Hopper GPU rewards the megakernel fusion Mirage discovers because it removes launches and keeps tensors in shared memory; a static NPU fabric needs the fusion decided at compile time with a fixed tile/DMA

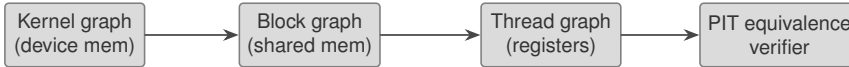


Figure 20.2: Mirage’s μ Graph spans kernel (device memory), block (shared memory), and thread (register) levels; candidates are checked by a probabilistic equivalence verifier.

Table 20.1: Mirage’s μ Graph levels and the memory tier each one keeps tensors in; the level structure is the residency contract (Wu *et al.*, 2024).

Level	Memory tier	Role
Kernel graph	Device memory (HBM)	Whole-GPU kernels; tensors shared via DRAM
Block graph	Shared memory	Thread-block compute; intermediates on-chip
Thread graph	Registers	Per-thread scope; finest reuse

plan instead (SemiAnalysis, 2024; AMD, 2024a). This is precisely why YiRage generalizes the Mirage idea across backends rather than shipping a single GPU artifact (Chen and YiRage Contributors, 2026).

20.2 Mirage arch passes

Mirage’s “**pass**” pipeline is really a search, and that is the conceptual departure from every prior chapter (Wu *et al.*, 2024). Instead of a fixed sequence of rewrites, the superoptimizer explores a space of μ Graphs that jointly combine algebraic transformations, schedule transformations, and the generation of new custom kernels—an **optimization** no single hand-written pass would find (Wu *et al.*, 2024). The μ Graph levels carry the residency contract directly: a kernel graph keeps tensors in device memory, a block graph keeps intermediates in shared memory to cut device-memory access, and a thread graph **lowers** scope to registers (Wu *et al.*, 2024).

The level structure is not bookkeeping; it is the whole reason the search can reason about bytes/token. When the optimizer chooses to express a sub-computation as a block graph, it is committing those intermediates to shared memory and therefore removing the device-memory traffic a kernel-level expression would incur (Wu *et al.*, 2024; Chen, 2026a). When an intermediate is too large for shared memory, Mirage splits the work into multiple block graphs so that each one fits, which keeps the on-chip residency invariant intact instead of silently spilling (Wu *et al.*, 2024). A human writing CUDA makes these decisions implicitly and inconsistently; the μ Graph makes them explicit and searchable, which is

what lets the superoptimizer trade device-memory traffic for shared-memory reuse deliberately.

Two ideas make this tractable and trustworthy. First, abstraction-based pruning shrinks the enormous search space while preserving a certain optimality guarantee, so the search is not brute force (Wu *et al.*, 2024). Second, a probabilistic equivalence verifier checks each candidate μ Graph against the reference by random testing over finite fields—polynomial identity testing generalized to the LAX fragment (linear algebra plus division and exponentiation)—which avoids floating-point noise and drives the probability of accepting a wrong kernel arbitrarily low (Wu *et al.*, 2024). This is a different correctness model from a compiler that trusts each hand-proved rewrite, and it is what lets Mirage emit kernels a human never wrote. Notably, Mirage does not lower through an MLIR **dialect** tower; the multi-level μ Graph is a single uniform representation, which is why it can search across levels that a staged dialect pipeline keeps separate (Lai *et al.*, 2023; Wu *et al.*, 2024).

What the verifier buys. Because equivalence is checked rather than assumed, a discovered megakernel can be deployed with quantified confidence, not just empirical testing—the discipline a decode team needs before replacing a known-correct kernel-per-operator path with a fused one (Wu *et al.*, 2024; Davies *et al.*, 2025).

20.3 Mirage codegen

Mirage’s **codegen** turns a verified μ Graph into CUDA, and MPK extends this to **emit** an entire LLM forward pass as one megakernel (Wu *et al.*, 2024; Mirage Project contributors, 2025). The mechanism is an SM-level task graph (tGraph) that captures data dependencies at the granularity of individual streaming multiprocessors, so the **backend** can schedule cross-operator software pipelining and communication–computation overlap *inside* a single kernel launch, with decentralized scheduling across SMs (Mirage Project contributors, 2025). The **platform** is the GPU, and the generated CUDA includes the intra-SM optimizations—software pipelining, register reuse, bank-conflict-aware layouts—that a per-operator emit cannot coordinate across operators (Mirage Project contributors, 2025).

For decode this is the most direct attack on launches/token in the book. A kernel-per-operator stack pays host dispatch for every one of the thousands of launches a token requires; a megakernel pays it once (Vellaisamy *et al.*, 2026; Chen, 2026a). MPK reports $1.2\text{--}6.7\times$ end-to-end latency reductions from exactly this collapse, pushing inference toward hardware limits, and it does so from a few dozen

lines of Python that name the fused layers—for example one call fusing RMSNorm and a linear projection (Mirage Project contributors, 2025). What codegen still cannot fix is an algorithm that is genuinely device-memory bound: if the working set does not fit on chip, even a perfect megakernel must stream it (Goodfellow *et al.*, 2016).

The deeper reason the megakernel helps is that it changes what can overlap. In a kernel-per-operator schedule, the boundary between two operators is also a synchronization point: the host waits for one kernel to finish before launching the next, so the GPU idles through dispatch latency and any tail effects (Vellaisamy *et al.*, 2026). Inside a single megakernel those boundaries become intra-kernel dependencies that the SM-level task graph can pipeline—one SM can start the next task’s loads while another finishes the current task’s compute, and collective communication can overlap with computation instead of stalling behind it (Mirage Project contributors, 2025). At batch-1 decode, where there is no batch dimension to hide latency behind, this fine-grained overlap is often the only source of parallelism left, which is why the megakernel wins are largest exactly in the regime that hurts a conventional stack most (Chen, 2026a; Kwon *et al.*, 2023).

Multi-hardware delta. The megakernel’s single-launch win is a GPU phenomenon—it removes CUDA launch and host-dispatch overhead; on an NPU the equivalent is a static single-dispatch schedule, and on a CPU it is the AOT bundle of Chapter 19 (AMD, 2024a; Rotem *et al.*, 2018). Same principle, different mechanism per class (SemiAnalysis, 2024).

20.4 Mirage tuning

In Mirage, **tuning** and compilation are the same act: the superoptimizer searches the μ Graph space and the probabilistic verifier accepts only equivalent candidates, so “tuning” means letting the search run rather than hand-listing configurations (Wu *et al.*, 2024). You still **profile** candidates—the search is guided by measured performance and bounded by the abstraction-based pruning—but the engineer’s job is to set the search budget and the operating point, not to write schedules (Wu *et al.*, 2024).

The **pitfall** is twofold and specific. First, superoptimization is expensive: searching and verifying a large μ Graph space costs real compile time, so it pays for hot, reused programs (a served LLM’s decode layer) and not for one-off graphs (Wu *et al.*, 2024; Kwon *et al.*, 2023). Second, the correctness guarantee is probabilistic and scoped to the LAX fragment; operators outside that fragment do not get the

same verification strength, so a team must know which parts of its model the guarantee actually covers (Wu *et al.*, 2024). As elsewhere, the right search target is **hardware**-specific—a μ Graph tuned for one GPU’s shared-memory size and SM count is not automatically optimal on another, so re-search per SKU rather than assuming transfer (NVIDIA, 2022; Davies *et al.*, 2025). A practical way to budget the search is to spend it where the token spends its time: the decode-critical attention and projection layers are reused on every step and every request, so a long superoptimization run there amortizes over billions of tokens, whereas one-off preprocessing graphs rarely justify the cost (Wu *et al.*, 2024; Kwon *et al.*, 2023).

20.5 Mirage case study

The decisive **case** is MPK compiling a real LLM—an open Hugging Face model such as Qwen3-8B—into a single megakernel and **benchmarking** it against a kernel-per-operator serving stack (Mirage Project contributors, 2025). The result restates the book’s thesis from the superoptimizer’s side: fusing the whole decode step into one launch removes the per-operator host dispatch that dominates batch-1, and the SM-level task graph overlaps the communication and computation that a kernel-per-operator schedule serializes, yielding the reported $1.2\text{--}6.7\times$ (Mirage Project contributors, 2025; Vellaisamy *et al.*, 2026). The arithmetic is familiar—each avoided inter-operator round trip saves the activation bytes that would otherwise stream through device memory per token—only now the compiler discovers the fusion rather than a human writing it (Chen, 2026a; Dao *et al.*, 2022).

It is worth being concrete about why the search, not just the fusion, is the contribution. A hand-written megakernel for one model is a heroic engineering effort that does not transfer to the next architecture; Mirage’s superoptimizer regenerates the fused kernel from a reference implementation whenever the model or the GPU changes, and the probabilistic verifier certifies each result (Wu *et al.*, 2024; Mirage Project contributors, 2025). For a serving team that means the megakernel is not a brittle artifact to maintain by hand but a compiler output to regenerate, which is the only way end-to-end fusion is sustainable across a model zoo that changes monthly. The cost, as the tuning section noted, is search time—paid once per model-target pair and amortized over every token served.

The **cross-hardware** conclusion is where the emerging wave and this book converge. Mirage proves the GPU case; the open problem is carrying multi-level search and verified megakernel fusion to CPUs, NPUs, and heterogeneous deployments where the launch-collapse mechanism differs (AMD, 2024b,a). **YiRage** is the book’s answer: a Mirage-lineage compiler that generalizes superoptimization

and megakernel fusion across multiple backends and carries the Chapter 2 residency rules as checkable IR metadata, so a discovered fusion that would spill on a given target is rejected at compile time rather than shipped (Chen and YiRage Contributors, 2026). That is why every prior chapter measured its compiler against the same YiRage yardstick—this is where the yardstick comes from (Davies *et al.*, 2025).

20.6 Key Takeaways

- **Superoptimization replaces hand-written passes with search.** Mirage jointly searches algebraic rewrites, schedules, and new custom kernels over a multi-level μ Graph, finding fusions no fixed pass would (Wu *et al.*, 2024). For decode the prize is collapsing per-token launches, not a faster single matmul (Vellaisamy *et al.*, 2026).
- **The μ Graph encodes residency by level.** Kernel graphs use device memory, block graphs use shared memory, thread graphs use registers, so the search optimizes bytes/token by construction (Wu *et al.*, 2024; Chen, 2026a). The representation is the hardware model.
- **Verified megakernels attack launches/token directly.** MPK fuses an LLM forward pass into one launch via an SM-level task graph, reporting $1.2\text{--}6.7\times$ latency reduction by removing per-operator host dispatch (Mirage Project contributors, 2025; NVIDIA, 2024). One launch per token is the limit case of this book’s thesis.
- **Probabilistic equivalence makes discovered kernels deployable.** Checking candidates by polynomial identity testing over finite fields (LAX fragment) gives quantified correctness, so a fused kernel can replace a known-good path with confidence—but only within the verified fragment (Wu *et al.*, 2024). Know what the guarantee covers (Davies *et al.*, 2025).
- **Search cost and SKU-specificity are the trade-offs.** Superoptimization pays for hot, reused decode layers, not one-off graphs, and a μ Graph tuned for one GPU is not optimal on another (Wu *et al.*, 2024; NVIDIA, 2022). Budget the search and re-tune per target (Kwon *et al.*, 2023).

20.7 Conclusion

Mirage reframes compilation as multi-level superoptimization with verified correctness, and MPK shows its decode payoff: a single megakernel that collapses a token’s launches and overlaps communication with computation inside one kernel, for $1.2\text{--}6.7\times$ (Wu *et al.*, 2024; Mirage Project contributors, 2025). This is the intellectual source of the YiRage compiler used as the yardstick throughout the book—multi-level search, megakernel fusion, and residency encoded as checkable metadata—generalized beyond a single GPU artifact (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). The broader lesson is that the field is converging on search plus verification as the way to reach fewer, more resident launches, and that decode is the workload where this convergence pays off first because it has the least batch parallelism to fall back on (Wu *et al.*, 2024; Vellaisamy *et al.*, 2026). Having now surveyed each compiler surface individually, Chapter 21 steps back to a unified analysis and a cross-hardware benchmark, comparing these stacks on the same decode metrics rather than on each project’s favored microbenchmark (Chen, 2026a; Goodfellow *et al.*, 2016).

Chapter 21

Unified AI Compiler Analysis and Cross-Hardware Benchmark

Each compiler in this Part advertises its own headline number on its own favored workload, which makes them almost impossible to compare honestly. This chapter does the comparison the projects do not: it rates TVM, Triton, IREE, Glow, and Mirage on one axis that actually predicts interactive serving cost—how well each collapses a token’s work into few, resident kernel launches—rather than on peak GEMM throughput (Chen *et al.*, 2018b; Tillet *et al.*, 2019; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018; Wu *et al.*, 2024). The yardstick is the one Chapter 1 set: dense Llama-3.2-1B averages 847.5 GPU launches per output token on H100 and OLMoE-1B/7B averages 9,305, so the metric that matters is launches/token and bytes/token at batch-1, not datasheet FLOPs (Vellaisamy *et al.*, 2026; Chen, 2026a). The point is not to crown a winner but to show that these stacks occupy different niches, and that the right choice is a routing decision—which is exactly what the book’s YiRage compiler encodes (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

21.1 Compiler ratings

A useful **compiler rating** for decode is not a single score; it is a profile across five axes: decode-fusion strength, launch collapse, multi-hardware breadth, search/compile cost, and correctness guarantee (Davies *et al.*, 2025). Rated this way the stacks separate cleanly. TVM is broad and search-driven but graph-and-TensorIR-staged; Triton is a GPU block compiler with hand-placed

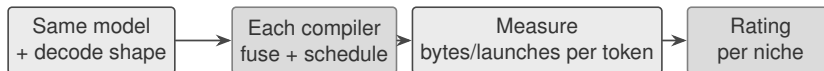


Figure 21.1: A fair comparison rates each **compiler** on the same decode metrics—bytes/token and launches/token—instead of each project’s favored microbenchmark.

Table 21.1: Decode-oriented **rating** of the surveyed compilers; “launch collapse” is the batch-1 lever and the column that most predicts ms/token (Vellaisamy *et al.*, 2026; Chen, 2026a).

Compiler	Primary niche	Launch-collapse mechanism
TVM	Broad, search-driven	Fusion + graph executor / AOT
Triton	GPU block kernels	Hand-placed fusion + CUDA Graphs
IREE	Widest HW matrix	Whole-program stream scheduling
Glow	Edge / CPU	AOT bundle (no launch loop)
Mirage	GPU superoptimizer	Searched megakernel

fusion boundaries; IREE is the widest hardware matrix with whole-program scheduling; Glow is the edge/CPU specialist; Mirage is the superoptimizer that discovers fusion and verifies it (Chen *et al.*, 2018b; Tillet *et al.*, 2019; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018; Wu *et al.*, 2024).

It is worth being explicit about why the niches are real and not marketing. The differences trace to architectural commitments made early in each project. TVM committed to a learned-cost-model search over a graph-plus-TensorIR split, which buys breadth and automation at the cost of a staged lowering that can leave fusion on the table (Chen *et al.*, 2018b; Zheng *et al.*, 2020). Triton committed to a Python tile abstraction over a GPU SIMT machine, which buys productivity and precise GPU control but leaves the fusion boundary to the author and excludes static NPUs (Tillet *et al.*, 2019; OpenAI / triton-lang contributors, 2024). IREE committed to whole-program compilation through an MLIR dialect stack and a hardware abstraction layer, which buys the widest matrix and compiled host scheduling at the cost of hiding per-target residency (IREE / iree-org contributors, 2024). Glow committed to a two-level IR and AOT bundles, which buys a predictable edge footprint by giving up datacenter-GPU depth (Rotem *et al.*, 2018). Mirage committed to multi-level superoptimization with verification, which buys reachable megakernels at the cost of search time (Wu *et al.*, 2024).

The rating exposes a recurring truth: a stack can score high on isolated kernel quality and low on launch collapse, and at batch-1 the second column is the one that moves p99 (Vellaisamy *et al.*, 2026; NVIDIA, 2024). On a GPU the

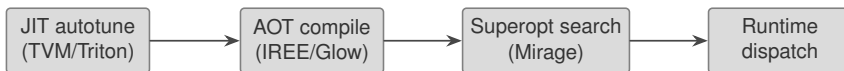


Figure 21.2: Where each stack pays its **compile** cost: JIT autotuning, ahead-of-time compilation, or superoptimization search.

launch-collapse mechanisms differ—Triton relies on the author to widen the fused region then captures it, IREE schedules the whole program, Mirage searches for the megakernel—but all three are attacking the same orchestration tax (Chen, 2026a; Mirage Project contributors, 2025). The mistake a team makes is to read one project’s headline speedup as a global ranking; the speedups are measured against different baselines on different workloads, and only a common decode harness makes them comparable (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

21.2 Compile overhead

The second axis is **compile overhead**, and it is the cost teams under-budget. The stacks pay in different places. TVM’s Ansor and AutoTVM pay a measured search—thousands of on-device trials per operator—reporting up to $3.8\times$ (Ansor, Intel CPU) and $1.2\text{--}3.8\times$ (AutoTVM) in exchange (Zheng *et al.*, 2020; Chen *et al.*, 2018a). Triton pays a per-shape autotune warmup the first time a kernel key is seen, so the first request after a shape change is slow (OpenAI / triton-lang contributors, 2024). IREE and Glow pay their cost ahead of time, producing a deployable artifact with no runtime compile (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). Mirage pays the largest one-time cost—superoptimization search plus probabilistic verification—and reports $1.1\text{--}2.9\times$, with its MPK megakernel reaching $1.2\text{--}6.7\times$ (Wu *et al.*, 2024; Mirage Project contributors, 2025).

The shape of the overhead also differs in ways that matter operationally, not just in total cost. A JIT autotuner’s cost is incremental and amortizing: it is paid lazily, per new shape, and warm shapes are free—but it makes the first request after any shape change slow, which is visible to a user at exactly the wrong moment (OpenAI / triton-lang contributors, 2024; Kwon *et al.*, 2023). An AOT compile front-loads everything into the build, so runtime latency is predictable and there is no first-request penalty, at the cost of a longer build and a larger artifact that must be regenerated when the model or target changes (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). A superoptimization search is the most expensive but also the most one-time: once a megakernel is found and verified for a model-target pair, it serves every subsequent token with no further search (Wu *et al.*, 2024;

Mirage Project contributors, 2025). A team that values predictable tail latency leans AOT or pre-searched; a team iterating rapidly on shapes tolerates JIT.

The engineering rule that falls out is to match the **overhead** model to the deployment. A served LLM whose decode layers are reused across billions of tokens justifies an expensive one-time search or AOT compile, because the cost amortizes; a rapidly changing or one-off graph does not, and there a JIT autotuner’s incremental cost or a lighter pass pipeline wins (Kwon *et al.*, 2023; Davies *et al.*, 2025). The **compile**-time/runtime-latency trade is itself a design decision, not an afterthought.

21.3 Hardware binding

The third axis is how each stack performs **hardware binding**—how late and how flexibly it commits to a target. TVM binds through target strings and forks schedules per backend; Triton binds to a GPU through its TritonGPU dialect and PTX/AMDGCN codegen and does not target static NPUs; IREE binds latest, through a Vulkan-inspired HAL that abstracts CPU, GPU, and more behind one interface; Glow binds through LLVM and cross-compiles to edge cores; Mirage binds to the GPU compute hierarchy by construction (Chen *et al.*, 2018b; OpenAI / triton-lang contributors, 2024; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018; Wu *et al.*, 2024).

Binding time is really a question of where the abstraction boundary sits between the program and the silicon. TVM’s target strings bind at schedule-selection time, so the same compute is re-scheduled per backend with a fork in the pass pipeline (Chen *et al.*, 2018b). IREE’s HAL pushes the boundary as late as possible—the dispatch regions are formed once and only the executable translation is per-target—so one program description fans out to many devices (IREE / iree-org contributors, 2024). Glow binds through LLVM, inheriting LLVM’s target support for free and gaining cross-compilation to embedded cores as a side effect (Rotem *et al.*, 2018). Triton and Mirage bind earliest and narrowest, to a GPU machine model, which is why their reasoning about shared memory and launches is the most precise and least portable (Tillet *et al.*, 2019; Wu *et al.*, 2024).

The decode consequence of late versus early **binding** is concrete. A late-binding HAL like IREE’s makes a model portable across a wide matrix but hides per-target residency decisions that only batch-1 telemetry exposes, so a green build is not a green decode (IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). An early, narrow binding like Triton’s or Mirage’s gives up breadth but lets the compiler reason precisely about one machine’s shared memory and launch model, which

is why the deepest decode wins are GPU-specific (Chen, 2026a; SemiAnalysis, 2024). Across all of them the hardware truth is unchanged: GPUs are bound by shared-memory residency, CPUs by cache, NPU by static tile/DMA plans, so no single binding is optimal everywhere (NVIDIA, 2022; AMD, 2024b,a).

21.4 Search space

The fourth axis is the **search** space each stack explores and how it is navigated, because that determines what optimizations are even reachable. AutoTVM searches hand-written schedule templates with a learned cost model; Ansor removed the templates and samples a hierarchical space, **autotuning** with evolutionary search and a task scheduler that spends budget where it moves end-to-end latency (Chen *et al.*, 2018a; Zheng *et al.*, 2020). Triton’s autotuner explores only the configuration list the author supplies—leaner, but bounded by human imagination (OpenAI / triton-lang contributors, 2024). Mirage searches the largest space of all: a multi-level μ Graph that can invent new custom kernels, pruned by abstraction and certified by probabilistic equivalence (Wu *et al.*, 2024).

There is a spectrum here from human-specified to machine-discovered optimization, and it correlates with the verification burden. At the human-specified end, Triton’s config list and AutoTVM’s templates are trustworthy by construction—the schedules are variations a human vetted—but they cannot exceed human imagination (OpenAI / triton-lang contributors, 2024; Chen *et al.*, 2018a). At the machine-discovered end, Mirage’s superoptimizer can synthesize a kernel no one wrote, which is powerful precisely because it is not bounded by templates, but that power forces the question of correctness: a discovered kernel must be proven equivalent, which is why probabilistic equivalence verification is not optional for it (Wu *et al.*, 2024). Ansor sits in between, sampling a structured space without explicit templates but within a fixed schedule grammar (Zheng *et al.*, 2020).

The reachability difference is the point. A template-bounded **search** can only find schedules a human anticipated, so it cannot discover a megakernel; a superoptimizer’s search can, which is why Mirage and MPK reach fusions the others structurally cannot (Wu *et al.*, 2024; Mirage Project contributors, 2025). The cost, as the overhead axis noted, is search time—so the right **autotuning** strategy depends on how reused the program is and how much of the optimization you are willing to let the compiler discover rather than specify (Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

21.5 YiRage benchmark

This is where the book’s yardstick is defined. **YiRage** is the Mirage-lineage compiler used throughout the book to **benchmark** the others, and its discipline is to measure every stack on the same two decode quantities—bytes/token and launches/token at batch-1—rather than each project’s preferred microbenchmark (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). It generalizes Mirage’s searched-and-verified megakernel across multiple backends (CUDA, CPU, Triton, and more) and carries the Chapter 2 residency rules as checkable IR metadata, so a candidate schedule that would spill is rejected before it ships (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

The reason a common yardstick is necessary rather than pedantic is that the alternative—trusting each project’s published number—systematically misleads. Ansor’s $3.8\times$, AutoTVM’s $1.2\text{--}3.8\times$, Mirage’s $1.1\text{--}2.9\times$, and MPK’s $1.2\text{--}6.7\times$ are all real, but each is measured against a different baseline, on a different workload, often at a throughput-shaped operating point rather than batch-1 decode (Zheng *et al.*, 2020; Chen *et al.*, 2018a; Wu *et al.*, 2024; Mirage Project contributors, 2025). Reduced to one harness that fixes the model, the decode shape, and the metric, the numbers become comparable and the niches become visible: the megakernel path wins where launches dominate, the AOT bundle wins where the device is small, and the broad scheduler wins where portability is the constraint (Vellaisamy *et al.*, 2026; Chen, 2026a).

Run as a **benchmark** harness, YiRage reframes the comparison as a routing problem: on a datacenter GPU it favors a searched megakernel path of the kind MPK demonstrates ($1.2\text{--}6.7\times$); on an edge CPU it favors a Glow-style AOT bundle; on a broad portability target it favors IREE-style whole-program scheduling (Mirage Project contributors, 2025; Rotem *et al.*, 2018; IREE / iree-org contributors, 2024). The **cross-hardware** reading is that no single compiler dominates—each is the right answer for a region of the hardware matrix—and the value of a unified yardstick is that it makes the routing decision measurable instead of tribal (Davies *et al.*, 2025; Chen, 2026a). Concretely, the routing inputs are the target class and the reuse profile of the workload: a high-reuse decode layer on a datacenter GPU routes to a searched megakernel because the one-time search amortizes and the launch collapse is largest there, while the same layer on a memory-constrained edge part routes to an AOT bundle because the binding fits the device and there is no launch loop to begin with (Mirage Project contributors, 2025; Rotem *et al.*, 2018). Encoding that decision as metadata rather than tribal preference is what lets the choice survive a model or hardware change without re-litigating it (Chen and YiRage Contributors, 2026).

21.6 Key Takeaways

- **Rate compilers on launch collapse, not peak FLOPs.** At batch-1, launches/token and bytes/token predict ms/token; a stack can win an isolated kernel benchmark and lose end-to-end (Vellaisamy *et al.*, 2026; Chen, 2026a). The launch-collapse column is the one that matters (NVIDIA, 2024).
- **Match compile-overhead model to deployment.** TVM/Triton pay JIT search; IREE/Glow pay AOT; Mirage pays superoptimization search (Zheng *et al.*, 2020; IREE / iree-org contributors, 2024; Wu *et al.*, 2024). Expensive one-time compilation amortizes over a reused decode layer but not a one-off graph (Kwon *et al.*, 2023).
- **Binding time trades breadth for depth.** Late-binding HALs (IREE) maximize portability but hide per-target residency; early, narrow bindings (Triton, Mirage) give up breadth for precise GPU reasoning (IREE / iree-org contributors, 2024; OpenAI / triton-lang contributors, 2024). A green build is not a green decode (Vellaisamy *et al.*, 2026).
- **Search reachability sets the ceiling.** Template search finds only anticipated schedules; a μ Graph superoptimizer can invent megakernels (Chen *et al.*, 2018a; Wu *et al.*, 2024). What the search space contains bounds what the compiler can ever find (Zheng *et al.*, 2020).
- **No compiler dominates; routing does.** Each stack owns a region of the hardware matrix, so the win is a measured routing decision on common decode metrics—the YiRage discipline (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Megakernel on GPU, AOT bundle on edge, whole-program scheduling for breadth, and the routing itself recorded as metadata so it survives the next model or accelerator (Mirage Project contributors, 2025; Rotem *et al.*, 2018).

21.7 Conclusion

Compared on one honest axis—how few, how resident the kernel launches per token—the surveyed compilers stop looking like competitors and start looking like specialists: TVM and IREE for breadth, Triton for GPU kernel authoring, Glow for the edge, and Mirage for searched-and-verified megakernels (Chen *et al.*, 2018b; IREE / iree-org contributors, 2024; Tillet *et al.*, 2019; Rotem *et al.*, 2018; Wu *et al.*, 2024). The unifying contribution of a benchmark like YiRage’s is to make

that specialization measurable, so a serving team routes a workload to the stack whose niche it falls in rather than arguing microbenchmarks (Chen and YiRage Contributors, 2026; Chen, 2026a). The practical payoff is that compiler selection stops being a one-time religious choice and becomes a per-workload, per-target decision backed by the same two numbers the whole book has tracked, which is the only form of comparison that survives the next model release or the next accelerator (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). With the compilers now rated on common ground, Chapter 22 turns from comparing stacks to driving one end to end: a compiler-driven performance-tuning workflow that takes a model from profile to shipped decode kernel (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Chapter 22

Compiler-Driven End-to-End Performance Tuning

A decode compiler team can win every microbenchmark and still lose the fleet when nobody owns the *end-to-end* tuning loop—profile, locate, change, gate, repeat—under the bytes/token and launches/token counters Chapter 1 made non-negotiable (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Part VI gave you backends; Chapter 21 gave you benchmark harnesses; this chapter wires them into a **compiler-driven E2E workflow** that treats each compiler knob as a hypothesis until decode telemetry confirms it (Chen, 2020; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields $1.259\times$ on H100 by removing host overhead—yet cannot fix illegal HBM traffic when fusion boundaries still export activations (Chen, 2026a; NVIDIA, 2024). E2E tuning exists to catch that gap: the moment a compiler change improves a GEMM kernel but raises bytes/token, the workflow rejects it before merge (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025).

The loop has four stations—hardware-aware profiling, bottleneck locating, structured tuning workflow, regression gates—each bound to multi-hardware residency rules from Chapter 2 (Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024). Fregly’s profile-first culture applies throughout: Nsight and roofline charts justify knob changes more than intuition about tile sizes (Davies *et al.*, 2025; SemiAnalysis, 2024; Chen, 2026a). Chapter 25 extends this loop with automated search; Chapter 26 packages the artifacts for production (Chen and YiRage Contributors,

2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Industrial teams document the loop as a runbook, not tribal knowledge. New hires should reproduce a baseline profile on day one, classify a bottleneck on day two, and propose a single-knob experiment by day three—all against the same decode cell (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). When the runbook is missing, compiler teams revert to GEMM microbench heroics (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Ownership matters: profiling belongs to whoever changes IR; gates belong to release engineering; classification belongs to a cross-functional review with serving SREs present (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler-only tuning loops ship illegal schedules that frameworks mask until load rises (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Anti-patterns to post on the wall: (1) tune tiles while launches/token exceed kernel time; (2) merge compiler wins without canary; (3) profile prefill, ship decode; (4) skip IR diff review; (5) treat framework batching wins as compiler fusion wins (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The sections below replace each anti-pattern with a concrete practice (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

E2E tuning also bounds *when to stop*: if bytes/token and launches/token meet SLO after graph replay, MegaKernel investment waits—mode selection from Chapter 12 applies before endless fusion (NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016;



Figure 22.1: Hardware-aware profiling pipeline: frozen decode cell → residency counters → roofline/timeline → bytes/token and launches/token report.

Counter	GPU tool surface	CPU / NPU delta
bytes/token	Nsight memory chart, CUPTI	perf/LBR, DMA byte counts
launches/token	CUDA API trace, CUPTI	thread launches, sync points
ms/token	end-to-end decode cell	same cell, power cap noted
legality	graph capture, shared mem	static schedule, tile slots

Table 22.1: Hardware-aware profiling maps the same decode questions to different toolchains.

SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Good enough under measured decode goodput beats perfect under imaginary FLOPs (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

22.1 HW aware profiling

Hardware-aware profiling fails when teams profile prefill shapes, batch-32 training graphs, or synthetic GEMMs while production runs BS=1 decode with paged KV (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). The fix is a *frozen decode cell*: pinned model revision, context bucket, sampling flags, warmup protocol, and SKU—logged beside every trace so comparisons are apples-to-apples (Chen, 2026a, 2020; Chen and YiRage Contributors, 2026).

GPU profiling binds to shared-memory occupancy, TMA transactions, and global bytes moved per decode step—not SM utilization alone (NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025). CPU profiling binds to LLC misses, NUMA cross-socket traffic, and OpenMP gang efficiency (Goodfellow *et al.*, 2016; Chen, 2020). NPU/XDNA profiling counts DMA edges and static tile residency; many GPU counters simply do not exist—substitute compile-time buffer accounting plus on-device cycle estimates (AMD, 2024b,a; Chen and YiRage Contributors, 2026).

Translate profiler output into Chapter 1 vocabulary before opening a compiler ticket: if bytes/token is within 10% of analytic HBM roofline (R_{floor}), orchestration tweaks (graphs, fusion) may still help; if bytes dominate, tile changes without fusion are theater (Chen, 2026a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026). YiRage and similar stacks export IR-level residency metadata so profiles can be tied to specific fusion boundaries rather than mystery kernels (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023).

Warmup discipline separates steady-state decode from cold-start stories: graph capture, allocator priming, and CUDA context creation belong in the profile protocol document—not in ad-hoc Slack threads (NVIDIA, 2024; Chen, 2026a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022). Teams that compare profiles without matched warmup compare different phases and waste compiler cycles (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Tooling hygiene: store raw ‘.nsys-rep’, CUPTI CSV, or NPU trace bundles beside summarized bytes/token tables in artifact storage keyed by git SHA (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Summaries alone rot; raw traces enable re-analysis when classification rules improve (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Prefill vs decode profiling split: compiler teams often inherit prefill-heavy traces from training infra (Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). Mandate decode-cell scripts in CI and reject profile uploads that omit BS=1 metadata (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Hardware-aware profiling is a contract, not a one-off Nsight session (Davies *et al.*, 2025; Chen and



Figure 22.2: Bottleneck locating: profile facts → launch vs memory vs residency class → compiler knob → prioritized experiment.

YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware delta. Hopper profiles expose TMA and warp-specialization; XDNA profiles expose tile SRAM fill; CPU profiles expose cache hierarchy—identical decode step, incomparable dashboards unless normalized to bytes/token (SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016).

Example 22.1.1. Decode cell (BS=1, ctx=2048): a profile showing low SM occupancy but high HBM throughput is bandwidth-bound—compiler tuning should target fusion and KV residency, not bigger GEMM tiles (Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022).

22.2 bottleneck locating

Bottleneck locating is where E2E tuning separates senior compiler engineers from kernel tourists (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2020). The production surprise is not peak FLOPs—it is misclassification: teams fuse operators while launches/token remain launch-bound, or capture CUDA Graphs while bytes/token stay at eager levels (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).

Use a decision tree grounded in measured counters:

1. **Launch-bound:** high launches/token, low R_{floor} proximity → graph mode, op fusion, MegaKernel seams (Chapter 12) (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026).
2. **Memory-bound:** bytes/token near roofline → fusion depth, KV layout, attention kernel choice (Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; Chen, 2026a).
3. **Residency-illegal:** correct counters but p99 spikes → spill paths, graph

illegality, dynamic alloc in capture region (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

4. **Framework-bound:** compiler clean but Python/scheduler overhead → hand off to serving stack (Chapter 28), do not keep retuning tiles (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

Diff lowered IR or CUDA when classification is unclear: new global allocations between attention and MLP usually explain bytes/token regressions better than guesswork (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022). On NPU, diff static DMA schedules; on CPU, diff cache-blocking parameters in oneDNN/TorchInductor output (AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020).

MoE and speculative decode add **routing-bound** buckets: expert imbalance shows up as latency tails even when per-expert kernels look healthy (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023). Locate whether the bottleneck is compile-time expert fusion, runtime routing, or both before changing schedules (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Attention-backend swaps (FlashAttention, SDPA, vendor libs) can move bytes/token without changing launches/token—classify whether the bottleneck lives in attention or MLP/residual before celebrating attention wins (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). TaxBreak-style launch counts decompose where framework fragmentation still dominates (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022).

Cross-SKU classification differs: a launch-bound Hopper cell may be memory-bound on a bandwidth-starved SKU at the same model (Chen, 2026a; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Bottleneck labels are properties of (model, bucket, SKU), not universal truths (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis,

2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

When profiles disagree with production dashboards, suspect bucket mismatch (context length), batching policy, or speculative path divergence—reconcile framework counters with compiler-cell counters before filing compiler bugs (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Bottleneck locating is forensic work (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Review gate. Every bottleneck ticket attaches profile artifacts, classification label, and the single compiler knob under test—no unfocused “make it faster” requests (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a, 2020).

22.3 tuning workflow

Tuning workflow turns classifications into a repeatable experiment queue (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Industrial teams use a kanban-style pipeline: *hypothesis* (knob + expected counter movement) → *branch build* → *decode-cell A/B* → *review* → *merge or discard* (Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

One change per experiment: fusion depth *or* tile size *or* backend switch—never three at once, or postmortems cannot attribute wins (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025). Keep a *human prior* schedule checked in as baseline; candidates must beat prior on bytes/token, launches/token, and ms/token simultaneously on the target SKU (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Chen and YiRage Contributors, 2026; Chen, 2020).

Workflow stages map to Part VI backends: XLA flags, TVM schedules, Triton configs, MLIR pass pipelines—each logged with version hashes (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; AMD, 2024b). Cross-backend experiments require separate hypothesis cards; promoting a Triton win to an XLA fleet without re-profile repeats the GPU-only culture Chapter 1 rejected (Davies

et al., 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b).

Canary deployment belongs in the workflow: 5–10% of decode replicas run candidate schedules with automatic rollback when p99 or error rate breaches SLO (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler tuning without canary is how clusters learn about illegal graph capture at 2am (NVIDIA, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2022).

Experiment queues should cap WIP: three concurrent knob experiments per layer block maximum, or attribution collapses and profiles diverge (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Document expected counter deltas on each hypothesis card—“↓ launches/token” vs “↓ bytes/token”—so reviewers reject mismatched outcomes quickly (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Rollback paths must be rehearsed: fingerprinted human-prior schedules restore in one deploy step when canary breaches SLO (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Tuning workflow is incomplete without a tested undo button (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware tuning runs parallel queues per fleet tier—never merge GPU and NPU experiment conclusions into one narrative (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Shared methodology, separate

queues, separate promotion gates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Example 22.3.1. Hypothesis card: “Increase MLP fusion depth by one operator; expect $\downarrow$ launches/token, $\downarrow$ bytes/token if residency legal; measure $\text{ctx} \in \{512, 2048, 8192\}$.” Reject if any bucket regresses (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; NVIDIA, 2024; Kwon *et al.*, 2023).

22.4 regression gates

Regression gates encode the workflow into CI so human discipline survives headcount churn (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Minimum bar for merge:

- Decode-cell A/B on agreed buckets passes on bytes/token, launches/token, ms/token (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).
- No new global allocations in captured graph regions (GPU) or new DDR hops (NPU) (NVIDIA, 2024; AMD, 2024a; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023).
- IR diff attached; roofline or buffer accounting for target SKU (SemiAnalysis, 2024; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2022; Chen and YiRage Contributors, 2026; AMD, 2024a; Kwon *et al.*, 2023).
- Fingerprint or schedule artifact versioned with compiler git SHA (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Nightly jobs replay the full cell matrix across representative SKUs; drift alerts fire when counters move more than agreed ϵ without an approved compiler bump (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon

et al., 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gates fail closed: ambiguous profiles block merge until reproduced (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Chapter 21 benchmarks supply the harness; Chapter 25 automates search inside the same gates (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Production runbooks (Chapter 26) assume these gates ran before containers ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Flaky gates erode trust: quarantine tests with high variance until environment drift is fixed—do not weaken thresholds to greenwash CI (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Pin driver, CUDA, and compiler versions in gate metadata; unexplained drift triggers infra tickets before compiler blame (NVIDIA, 2022, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; AMD, 2024b).

Security and determinism gates belong beside performance: numerics diffs beyond agreed tolerance block merge even when ms/token improves (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Regression gates protect users, not just leaderboard scores (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Publish gate failures publicly inside the compiler team: failed experiments teach more than silent reverts (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD,

2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). A culture that hides negative results repeats the same illegal fusion twice (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

22.5 Key Takeaways

- **Profile the production shape, not a proxy.** Hardware-aware profiling fails the moment a team measures prefill shapes, batch-32 training graphs, or synthetic GEMMs while production runs batch-1 decode with paged KV (Kwon *et al.*, 2023; Davies *et al.*, 2025). Freeze decode cells and normalize GPU, CPU, and NPU tools to the same bytes/token and launches/token so the numbers are comparable across targets (Chen, 2026a; Vellaisamy *et al.*, 2026).
- **Classify the bottleneck before touching a knob.** Bottleneck locating—deciding whether a workload is launch-, memory-, residency-, or framework-bound—is what separates senior compiler engineers from kernel tourists, because changing a compiler flag before classifying wastes effort on the wrong term (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). The production surprise is usually orchestration or residency, not the matmul the tourist reaches for first (NVIDIA, 2024; Vellaisamy *et al.*, 2026).
- **Change one knob per experiment against a human baseline.** A repeatable tuning workflow turns classifications into an experiment queue: establish a human-prior baseline, vary one knob at a time, and canary on the fleet before promotion (Chen, 2020; Chen and YiRage Contributors, 2026). Bundling changes makes a regression impossible to attribute, which is why the discipline is one variable per measurement (Davies *et al.*, 2025).
- **Encode the workflow as regression gates.** Human discipline does not survive headcount churn, so the workflow must live in CI: enforce counter non-regression, residency legality, and versioned, fingerprinted artifacts on every merge (Chen, 2020; Chen and YiRage Contributors, 2026). A gate that checks bytes/token and launches/token at the decode cell is what stops a microbench-only win from shipping (Chen, 2026a; Vellaisamy *et al.*, 2026).
- **Profile-first beats tile-size folklore.** The recurring lesson of end-to-end tuning is that measured decode counters, not inherited tile-size heuristics,

decide what to change (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Folklore tuned on someone else’s prefill workload is exactly the trap a frozen decode cell and a profile-first loop are designed to avoid (SemiAnalysis, 2024).

22.6 Conclusion

Compiler-driven end-to-end performance tuning is the operational glue between backends (Part VI), benchmarks (Chapter 21), automated search (Chapter 25), and production packaging (Chapter 26) (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Without hardware-aware profiling and regression gates, compiler teams optimize kernels while decode goodput flatlines (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gate merges on buffer accounting and reproducible decode cells—never on a single Nsight screenshot (SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026). This profile-first, regression-gated workflow is general, but it pays off most on the hardest decode targets, which is where the next chapter applies it: Chapter 23 specializes the workflow for LLM and MoE compilation, where data-dependent expert routing and growing KV caches stress exactly the launches/token and bytes/token counters this chapter taught teams to measure (Liu *et al.*, 2024; Kwon *et al.*, 2023).

Chapter 23

LLM and MoE Specialized Compilation

Dense decoder LLMs and mixture-of-experts (MoE) models break the assumptions generic compiler stacks were built for: batch-1 decode, growing KV residency, routed expert subgraphs, and framework-owned paging policies (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). Part VI taught backend lowering; Chapter 22 taught E2E tuning loops; this chapter specializes those tools for **LLM decode compilation** and **MoE-aware schedules** across GPU, CPU, and NPU targets (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259 $\times$ on H100 by removing host overhead—yet cannot fix illegal HBM traffic when MoE routing and KV export remain unfused (Chen, 2026a; NVIDIA, 2024; Liu *et al.*, 2024). Specialized compilation means every pass knows whether it is optimizing prefill, decode, or expert subgraphs—and refuses to merge them into one misleading benchmark (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020).

Four topics anchor the chapter: **LLM decode compile** (single-request residency), **MoE scheduling** (routing + expert fusion), **KV hardware adaptation** (layout and paging boundaries), and **heterogeneous MoE load balancing** (multi-SKU expert placement) (Liu *et al.*, 2024; AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). DeepSeek-class deployments disaggregate prefill and decode; compiler specialization must follow that split instead of fighting it (Davies

et al., 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Anti-patterns: compiling whole-model graphs without decode role tags; fusing through PagedAttention handles; ignoring routing entropy in MoE profiles; promoting GPU expert schedules to CPU fallback without re-profile (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Each section below replaces one anti-pattern with a specialized compile practice (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Speculative decoding and draft models add another decode subgraph: compiler specialization must treat draft and target as separate capture regions with shared KV invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Merging draft control flow into main decode capture breaks graph legality (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Quantized decode (INT4/FP8 weights, FP16 activations) changes fusion legality: specialized passes must preserve dequant residency on-chip across attention-MLP seams (SemiAnalysis, 2024; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024; AMD, 2024b). Profile bytes/token with quantization enabled—kernel names change but counters do not (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Industrial runbooks tie LLM/MoE compile artifacts to model card revisions: tokenizer, rope, expert count, and paging policy must hash into schedule fingerprints (Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a;



Figure 23.1: LLM decode compile path: framework export $\rightarrow$ fusion $\rightarrow$ target codegen $\rightarrow$ decode-cell validation (not prefill GEMM).

Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Silent drift between training and serving configs is a top source of “compiler regression” tickets that are actually model mismatches (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

23.1 LLM decode compile

LLM decode compile starts when the compiler owns a *decoder slice*—attention, MLP, norms, rotary, residual—exported at BS=1 with explicit KV shapes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020). Training-shaped graphs (large batch, no paging) mislead tile choices: shared-memory budgets that fit prefill often spill KV on decode (Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Compiler passes should tag operators with *decode role*: KV producer, KV consumer, stateless compute (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2024, 2022). Fusion without role tags reunifies nodes that frameworks intentionally split for paging—breaking vLLM/SGLang invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

FlashAttention and Flash-Decoding shrink attention bytes but MLP/residual boundaries often remain separate kernels unless MegaKernel plans reunify them (Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; NVIDIA, 2024, 2022). Decode compile therefore tracks *whole-step* bytes/token, not

Decode compile invariant	Violation symptom
KV stays paged / blocked	capacity collapse, OOM tails
Graph capture legality	silent eager fallback
Same IR, per-SKU codegen	“works on H100” only
Bytes/token gate on merge	fast GEMM, slow decode

Table 23.1: LLM decode compile invariants for specialized compiler stacks.

attention microbenches alone (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Export boundaries matter: FX/ONNX dumps often freeze prefill shapes (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Specialized LLM pipelines inject dynamic seq-len and paging metadata before lowering (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Without metadata, “successful” compile produces undeployable schedules (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Rotary, norm, and sampling tails are easy to ignore in decode compile yet dominate launches/token on small models (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Include them in fusion plans and profile whole-step counters (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Multi-hardware delta. Hopper decode favors TMA-friendly KV tiles; XDNA requires static KV slot assignment; CPU decode fights LLC capacity—identical transformer block, different fusion limits (SemiAnalysis, 2024; AMD, 2024b,a;



Figure 23.2: MoE scheduling: router $\rightarrow$ active experts $\rightarrow$ fused compute $\rightarrow$ combine (compile-time vs runtime split).

Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Example 23.1.1. Decode cell (BS=1, ctx=2048): compiler fusion that materializes full KV tensors between layers doubles bytes/token even if attention kernel is optimal (Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

23.2 moe scheduling

MoE scheduling is where decode compilation meets *dynamic routing* (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020). The production surprise is not expert GEMM peak FLOPs—it is launch storms when every token activates a different expert subset without batched grouping (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Compiler specialization options:

1. **Compile expert templates once** per shape bucket; runtime selects indices (Liu *et al.*, 2024; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).
2. **Fuse router + top- k + expert prologue** when graph capture legal; isolate when control flow diverges (NVIDIA, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*,

2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

3. **Expert parallelism passes** that align with DeepEP-style dispatch when disaggregated (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

OLMoE-scale launch counts (TaxBreak) warn that MoE decode without fusion remains orchestration-bound even when experts are “fast” (Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Profile launches/token *per layer* with routing entropy logged—mean ms/token hides expert imbalance tails (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Review gate. MoE compile merges require expert activation histograms from production-like traffic, not uniform random routing (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Shared-expert and routed-expert splits (DeepSeek-class) need distinct compile templates: shared experts fuse like dense MLP; routed experts batch by token groups (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Collapsing both into one schedule loses routing visibility (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

When expert counts exceed on-chip weight residency, compilation must coordinate with runtime prefetch—bytes/token includes HBM traffic for cold experts (Liu *et al.*, 2024; SemiAnalysis, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a;

AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b,b). MoE scheduling is a memory problem disguised as a routing problem (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

23.3 kv hardware

KV hardware adaptation splits responsibility: frameworks page and schedule KV blocks; compilers choose layout, fusion around KV edges, and on-chip residency for attention/MLP seams (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022). Illegal fusion that assumes dense KV tensors breaks PagedAttention invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Hardware-specific KV tactics:

- **GPU:** block-sparse attention tiles, TMA-friendly KV loads, graph-safe KV pointer stability (NVIDIA, 2022; SemiAnalysis, 2024; Dao *et al.*, 2022; NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; AMD, 2024b).
- **CPU:** cache-line blocked KV, NUMA-aware pinning for long context (Goodfellow *et al.*, 2016; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).
- **NPU/XDNA:** static KV slot tables, bounded context buckets compiled upfront (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024).

Context-length buckets matter: a schedule legal at $\text{ctx}=2048$ may spill at $\text{ctx}=8192$ when KV exceeds shared memory or LLC (Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b,b). Compile separate artifacts per bucket rather than one dynamic kernel that silently allocates (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Multi-query and grouped-query attention change KV head layout; KV hardware passes must read head counts from model metadata, not hard-coded MHA (Kwon *et al.*, 2023; Dao *et al.*, 2022; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Wrong layout passes compile yet corrupt generation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Prefix caching and long system prompts shift KV working set: compilers should not assume empty cache at step zero (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Profile cells include warm prefix lengths representative of production (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

Example 23.3.1. Paged KV + fused MLP: compiler must treat KV pointers as opaque handles through attention–MLP seam—materializing paged KV to dense breaks the framework contract (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).

23.4 hetero moe

Heterogeneous MoE load balancing spans *compile-time* expert placement and *runtime* routing policy (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Expert parallelism across GPU tiers (prefill pool vs decode pool) fails when compiled expert weights assume uniform NVLink topology (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Compiler stacks should emit *expert placement metadata*: which experts are colocated, which tiers use INT4/FP8 kernels, and which backends are legal per chip (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Runtime load balancers (eplb-class policies in open DeepSeek infra) consume that metadata; compilers should not hard-code routing (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

On CPU/NPU fallback paths, subset experts may run on host while GPU serves hot experts—compilation must quarantine backends per expert, not share one schedule (AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Hetero MoE gates in Chapter 22 apply per tier: bytes/token and launches/token on each participating SKU (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Fleet skew—one expert dominating activations—is a *routing* problem first, but compilation can mitigate via duplicated hot-expert kernels on multiple chips when policy allows (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022,

2023; NVIDIA, 2024, 2022). Measure load imbalance with production histograms; synthetic uniform routing hides tail latency (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Chapter 24 extends heterogeneity to cluster deployment; Chapter 28 clarifies framework vs compiler KV ownership (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Specialized LLM/MoE compilation succeeds when it respects those boundaries (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

23.5 Key Takeaways

- **Own the decoder slice at batch-1.** LLM decode compilation begins when the compiler owns a whole decoder slice—attention, MLP, norms, rotary, residual—exported at batch-1 with explicit KV shapes, and gates it on whole-step bytes/token rather than per-operator FLOPs (Kwon *et al.*, 2023; Davies *et al.*, 2025). Optimizing the slice as a unit while respecting the framework’s paging is what keeps the attention-MLP seam resident (Dao *et al.*, 2022; Chen, 2026a).
- **MoE compilation meets dynamic routing.** The hard part of MoE scheduling is data-dependent routing: template the experts so they share a kernel, fuse the router prologue only when legal, and profile launches/token under realistic routing entropy rather than a uniform-load proxy (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). A schedule that looks fine on balanced routing can thrash when real traffic skews load across experts (Davies *et al.*, 2025; Kwon *et al.*, 2023).
- **KV is a split responsibility.** Frameworks page and schedule KV blocks; the compiler chooses layout, fuses around the KV edges, and secures on-chip residency for the attention/MLP seam—and that split is realized differently per GPU, CPU, and NPU (Kwon *et al.*, 2023; AMD, 2024b). Treating KV layout as a compiler transform rather than a runtime afterthought is what makes the paged attention fast instead of merely correct (Dao *et al.*, 2022;

SemiAnalysis, 2024).

- **Heterogeneous MoE spans compile-time and runtime.** Load balancing splits across compile-time expert placement and runtime routing policy, so the compiler emits placement metadata and per-tier gates while the runtime routes, and backends are quarantined across fallback paths (Liu *et al.*, 2024; Davies *et al.*, 2025). Without per-tier gates a placement tuned for one accelerator silently regresses another (Chen and YiRage Contributors, 2026; AMD, 2024a).
- **MoE and KV specialization still defer to the decode counters.** However specialized the compilation, the acceptance metric is unchanged: bytes/token and launches/token at the batch-1 decode operating point, not peak FLOPs (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). MoE’s launch density and KV’s bandwidth pressure are precisely the terms those counters expose (Chen, 2026a; Goodfellow *et al.*, 2016).

23.6 Conclusion

LLM and MoE specialized compilation narrows generic compiler stacks to decode-shaped graphs, routed experts, and KV contracts frameworks own (Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Gate merges on bytes/token, launches/token, and paging legality—not expert GEMM peaks alone (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b). Chapter 24 carries heterogeneity to cluster scale; Chapter 25 automates search inside the same invariants (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Chapter 24

Heterogeneous Compilation and Multi-Hardware Deployment

Production LLM serving is no longer a single-GPU problem; it is a fleet problem, where prefill and decode run on different pools, edge and datacenter run different models, and the same logical workload must compile for GPUs, CPUs, and NPUs that have nothing in common but the math (Davies *et al.*, 2025; IREE / iree-org contributors, 2024). This chapter is where every per-target decision from the MegaKernel and compiler chapters becomes a deployment-level routing decision across a heterogeneous **cluster**. The decode metrics still govern: dense Llama-3.2-1B issues 847.5 launches per token on H100 and OLMoE-1B/7B issues 9,305, and removing host overhead bought 1.259× on a Qwen-2.5-7B cell—gains that must now be preserved across a mix of silicon, not a single SKU (Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024). The mistake to avoid is deploying one hand-tuned artifact everywhere. A kernel optimal on an H100 is illegal on an XDNA tile and wasteful on a CPU, so heterogeneous deployment demands per-target compilation with a unified front end and a runtime that adapts to the hardware it finds (AMD, 2024b; Rotem *et al.*, 2018).

24.1 Heterogeneous cluster

A **heterogeneous cluster** exists because prefill and decode have opposite hardware appetites. Prefill is compute-bound and batches well, so it maps to dense GPU pools; decode is memory- and orchestration-bound at batch-1, so it is provisioned differently and far more heavily (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).



Figure 24.1: A heterogeneous serving cluster routes work by phase and target: batched prefill, batch-1 decode, and edge each map to different silicon.

Table 24.1: Roles in a heterogeneous serving cluster; each target owns a slice of traffic with a preferred compiled artifact (Davies *et al.*, 2025; Mirage Project contributors, 2025).

Hardware	Typical role	Preferred artifact
Datacenter GPU	High-throughput decode	Searched megakernel
CPU	Cost-sensitive / low-QPS	AOT bundle, cache-blocked
NPU / edge	Power-constrained	AOT bundle, static tiling

DeepSeek-V3’s production layout is the canonical evidence: its minimum prefill unit spans 4 nodes (32 GPUs) while its minimum decode unit spans 40 nodes (320 GPUs), a roughly 10× capacity skew that reflects how differently the two phases use silicon (Davies *et al.*, 2025). Disaggregating them into separate pools lets each be optimized and scaled independently, which is the cluster-level form of the prefill/decode split this book has drawn since Chapter 1.

The **heterogeneous** dimension deepens when the **cluster** mixes vendor silicon. A fleet may run datacenter GPUs for high-throughput decode, CPUs for low-QPS or cost-sensitive endpoints, and NPUs for edge or power-constrained deployment, each with the distinct residency and synchronization models of Chapters 6–9 (AMD, 2024b,a; Rotem *et al.*, 2018). The deployment decision is which target each slice of traffic routes to, and the answer is workload-specific: a high-reuse decode layer justifies a searched GPU megakernel, while an edge endpoint wants a Glow-style AOT bundle (Mirage Project contributors, 2025; Rotem *et al.*, 2018). Routing across this **cluster** is the deployment-level analogue of the per-region compiler choice of Chapter 21.

The reason routing beats standardizing is the same one this book has argued since Chapter 1: the optimal schedule is hardware-specific, so a single artifact is either illegal or leaves performance on the table somewhere in the **cluster** (AMD, 2024b; SemiAnalysis, 2024). A **heterogeneous** deployment embraces this by compiling per target and routing each request to the silicon whose strengths match the work, rather than forcing a lowest-common-denominator kernel across the fleet. The routing policy itself becomes an artifact to maintain—which traffic class goes to which pool—and it shifts as models and hardware evolve, which is why encoding the rationale rather than hand-wiring the routes matters at fleet



Figure 24.2: A unified MLIR-based stack lowers one model description to per-target artifacts, sharing front-end optimizations across hardware.

scale (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026).

24.2 Unified stack

Heterogeneous deployment is only tractable with a **unified** compiler front end, and **mlir** has become the de facto substrate for one. By providing a common, extensible IR that many backends share, **mlir** lets a single model description carry through shared high-level optimizations before forking into per-target codegen, rather than maintaining a separate compiler per vendor (IREE / iree-org contributors, 2024; Lai *et al.*, 2023). IREE is the concrete instance: it is an **mlir**-native compiler and runtime that lowers one input through its dialect stack to llvm-cpu, CUDA, ROCm, Vulkan, Metal, and experimental AMD AIE, giving a genuinely **unified** path across the matrix (IREE / iree-org contributors, 2024). TVM’s Relax and Triton’s TritonGPU dialect are further evidence that the field is converging on **mlir**-style staged lowering as the way to span hardware (Lai *et al.*, 2023; Tillet *et al.*, 2019).

The value of a **unified** stack for decode specifically is that the residency, role, pipeline, and synchronization passes of Chapters 6–9 can be written once against the shared IR and specialized per target, rather than reimplemented per backend (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). This is what makes the per-target guarantees of those chapters scalable across a fleet: the same memory pass checks a 228 KB H100 budget and a 64 KB XDNA tile, the same sync pass proves deadlock-freedom on both, because they operate on one **unified** representation (NVIDIA, 2022; AMD, 2024a). Without a **unified** stack, heterogeneous deployment degenerates into a matrix of separately maintained, separately verified kernels.

It is worth being clear about what **mlir** does and does not solve. It provides the shared infrastructure—a common IR, a pass framework, reusable dialects—so that backends do not each reinvent the lowering machinery, and it lets cross-level optimizations be expressed once (Lai *et al.*, 2023; IREE / iree-org contributors, 2024). What it does not do is make the per-target decisions for free: a residency budget, a role split, and a pipeline depth still differ per device, and **mlir** only

ensures those decisions are made on a common substrate rather than in incompatible silos. The **unified** stack is therefore a force multiplier for the per-target passes, not a replacement for them—it is what lets one team maintain those passes across a growing matrix instead of one team per vendor (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024).

The convergence on **mlir** across TVM (Relax), Triton (TritonGPU), and IREE is itself a signal that the industry has accepted heterogeneity as permanent (Lai *et al.*, 2023; Tillet *et al.*, 2019; IREE / iree-org contributors, 2024). Rather than betting on one winning architecture, these stacks bet on a **unified** compiler layer that can absorb whatever silicon arrives, which is the only durable response to a hardware landscape that adds a new accelerator class every year (AMD, 2024a; Chen *et al.*, 2018b).

24.3 Runtime adaptation

A **unified** build still has to meet the specific device it runs on, which is the job of **runtime** adaptation. The cleanest model is the ahead-of-time fat binary: IREE compiles a FlatBuffers `.vmfb` that carries host instructions and device kernels for multiple targets, and at load time the **runtime** probes the device and dispatches the matching kernel, so one artifact serves a heterogeneous fleet without a JIT (IREE / iree-org contributors, 2024). The **runtime**’s ability to **detect** the hardware—its vendor, its memory sizes, its feature set—is what lets it pick the right kernel variant, and the fat binary can carry several specializations (aligned vs unaligned shapes, large vs small matmuls) for the **runtime** to choose among (IREE / iree-org contributors, 2024).

The **detect**-and-adapt pattern also covers graceful fallback. When a preferred path is unavailable—an NPU backend that is experimental, or a GPU feature the device lacks—the **runtime** must fall back to a kernel that runs, even if slower, rather than fail (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). For decode this matters because the residency-optimal path differs per device, so the **runtime** adaptation is not cosmetic: detecting an H100 versus a smaller GPU should select a different pipeline depth and tile size, the very parameters Chapters 6–8 tuned per target (NVIDIA, 2022; Chen, 2026a). Runtime adaptation is thus the deployment-time completion of compile-time per-target specialization. The division of labor is clean: the compiler decides everything it can ahead of time, per target, and the **runtime** decides only what genuinely depends on the device it lands on. For a heterogeneous fleet this keeps the per-request path thin while still letting one binary serve many targets, which is the property that makes large-scale

deployment manageable rather than a per-device build explosion (IREE / iree-org contributors, 2024; Davies *et al.*, 2025).

There is a tension between adaptation and predictability that a serving team must manage. The more the **runtime** adapts—probing the device, selecting among kernel variants, falling back when needed—the more paths exist that must be tested, and the harder it is to guarantee a specific latency on a specific device (IREE / iree-org contributors, 2024; Davies *et al.*, 2025). The AOT fat binary mitigates this by resolving most choices at build time and leaving the **runtime** only a fast **detect**-and-dispatch, so the per-request path is short and predictable, in contrast to a JIT that recompiles on first use (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). For interactive decode, where tail latency is the product metric, this argues for front-loading adaptation into the build and keeping the **runtime** decision to a quick device **detect**, not a recompilation.

24.4 Version regression

Deploying across many targets multiplies the surface for **regression**, so a heterogeneous fleet needs disciplined **version** management and automated **regression** testing. A kernel **version** that improves decode on one GPU generation can regress on another, and a model or compiler **version** bump can silently change which fusion is chosen, so every target-**version** pair must be benchmarked on the same decode metrics rather than assumed equivalent (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). This is the deployment-scale form of the book’s measurement discipline: track bytes/token and launches/token per target-**version**, and gate a rollout on no **regression** across the matrix.

The autobooks-style loop that produced this book is itself a model for **regression** control: a fixed harness scores every change, improvements are kept and **regressions** reverted via version control, and a results log records each experiment (Chen and YiRage Contributors, 2026). Applied to deployment, the same pattern is a **regression** matrix—each target a row, each kernel or model **version** a column, each cell a decode benchmark—that must stay green before a **version** ships. Reproducibility is the prerequisite: pinned configurations, recorded inputs, and logged measurements, the transparency standard the fact-verification gate of this book applies to its own numbers (Chen, 2026a; Goodfellow *et al.*, 2016).

The combinatorics are what make automation essential here. With several hardware targets, multiple model **versions**, and a range of decode shapes (context lengths, batch sizes), the **regression** matrix has too many cells to check by hand, so the testing must be scripted and run on every change (Davies *et al.*, 2025). The

autobooks loop’s keep/revert discipline scales to this directly: a change is kept only if it improves or holds the metric across the matrix, and reverted otherwise, with the results log providing an audit trail of which **version** was good on which target (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026). Crucially, the metric must be the decode goodput metric—bytes/token and launches/token—not a microbenchmark, because a **version** that wins an isolated GEMM can still regress end-to-end latency, the recurring trap of this book (Chen, 2026a).

24.5 Industrial cases

The clearest industrial **case** is again disaggregated **production** serving. DeepSeek-V3’s separate prefill and decode pools, with the $10\times$ capacity skew, show a **production** system mapping each phase to the hardware footprint it needs rather than forcing both onto one uniform pool (Davies *et al.*, 2025). The lesson generalizes: a **production** deployment that measures its phases separately will provision them separately, because the decode pool’s bottleneck (memory and orchestration at batch-1) is different from the prefill pool’s (compute throughput) (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

A second **case** class is the megakernel in **production**. Mirage Persistent Kernel compiles an LLM forward pass into a single megakernel and reports $1.2\text{--}6.7\times$ end-to-end latency reduction over kernel-per-operator serving, a **production**-relevant result precisely because it attacks the launch storm that dominates batch-1 decode (Mirage Project contributors, 2025; Chen, 2026a). A third is edge **production** on NPUs and CPUs, where Glow-style AOT bundles deploy quantized models within fixed memory budgets (Rotem *et al.*, 2018; AMD, 2024a). Each **case** is the right answer for its slice of the fleet, which is exactly why a heterogeneous deployment routes rather than standardizes.

A useful **production** pattern that ties the cases together is the cost-versus-latency frontier. A datacenter GPU pool delivers the lowest decode latency but at the highest cost per token; a CPU pool is cheaper per token at low utilization but cannot meet aggressive tail-latency targets; an edge NPU trades absolute throughput for power and locality (Davies *et al.*, 2025; Rotem *et al.*, 2018). A **production** deployment places each traffic class on the point of that frontier its service-level objective demands—interactive chat to the low-latency GPU pool, batch or background generation to cheaper hardware—and the heterogeneous **cluster** is what makes that placement possible. The compiler’s role is to ensure that whichever point is chosen, the artifact for it is residency-correct and decode-optimal, so the frontier is defined by genuine hardware trade-offs rather than by

which target happened to get a hand-tuned kernel (Chen and YiRage Contributors, 2026; Chen, 2026a).

24.6 YiRage cluster

The book’s reference compiler is the cross-**cluster** router that ties these threads together. **YiRage** compiles one model description to multiple backends—CUDA, CPU, Triton, and more—and carries the Chapter 2 residency rules as checkable IR metadata, so each target gets a verified, per-hardware artifact from a single source (Chen and YiRage Contributors, 2026). Across a heterogeneous **cluster** it makes the routing decision measurable: a high-reuse decode layer on a datacenter GPU routes to a searched megakernel, an edge endpoint to an AOT bundle, a broad-portability target to whole-program scheduling, each choice backed by the same bytes/token and launches/token metrics (Mirage Project contributors, 2025; IREE / iree-org contributors, 2024; Rotem *et al.*, 2018).

What makes **YiRage** a **cluster**-level tool rather than a single-kernel one is that the residency, role, pipeline, and sync passes of Chapters 6–9 all run per target from the **unified** representation, so deploying to a new device in the **cluster** is a recompile-and-verify, not a hand-port (Chen and YiRage Contributors, 2026; Wu *et al.*, 2024). The metadata that records why a target was routed a certain way also survives model and hardware changes, so the routing decision does not have to be re-litigated each release—the deployment-scale payoff of encoding intent rather than hand-tuning (Davies *et al.*, 2025; Chen, 2026a).

This closes the arc the book has traced. The residency budgets of Chapter 6, the static roles of Chapter 7, the pipelines of Chapter 8, and the synchronization of Chapter 9 were each presented as a per-target compiler pass; the compiler survey of Chapters 16–21 showed which existing stacks implement pieces of that vision; and this chapter assembles them into a fleet-level deployment where one model description compiles, via a **unified mlir** stack, to verified per-hardware artifacts that a **runtime** dispatches and a **regression** matrix guards (Chen and YiRage Contributors, 2026; IREE / iree-org contributors, 2024; Wu *et al.*, 2024). **YiRage** is the name this book gives to the compiler that does all of it together across a heterogeneous **cluster**, and the reason every earlier chapter measured against it is that it is where the book’s recurring thesis—decode is bound by movement and orchestration, fixable only by residency-aware, schedule-free, per-target compilation—is operationalized at scale (Vellaisamy *et al.*, 2026; Mirage Project contributors, 2025).

24.7 Key Takeaways

- **Disaggregate by phase across the cluster.** Prefill is compute-bound and batches; decode is memory- and orchestration-bound at batch-1, so they provision differently—DeepSeek-V3 runs a $10\times$ larger decode pool than prefill pool (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). Measure the two phases separately, and you will end up provisioning them separately (Kwon *et al.*, 2023).
- **Unify the front end with MLIR.** A shared MLIR-based IR (IREE, Relax, TritonGPU) lets one model carry shared optimizations before per-target codegen, so the residency/role/pipeline/sync passes are written once and specialized per target (IREE / iree-org contributors, 2024; Lai *et al.*, 2023). Without it, deployment becomes an unwieldy matrix of hand-maintained kernels (Chen and YiRage Contributors, 2026).
- **Adapt at runtime by detecting the device.** A fat binary carrying multiple target kernels lets the runtime probe the hardware and dispatch the right variant, with graceful fallback when a path is unavailable (IREE / iree-org contributors, 2024; Rotem *et al.*, 2018). Detected device sizes should select pipeline depth and tile size (NVIDIA, 2022).
- **Gate rollouts on a per-target regression matrix.** A kernel or model version that helps one target can regress another, so benchmark every target-version pair on bytes/token and launches/token and require no regression before shipping (Vellaisamy *et al.*, 2026; Chen, 2026a). Reproducibility (pinned configs, logged measurements) is the prerequisite (Goodfellow *et al.*, 2016).
- **Route, don't standardize.** Each production case—searched megakernel on GPU (1.2–6.7 $\times$), AOT bundle on edge, whole-program scheduling for breadth—owns a slice of the fleet, and YiRage routes per slice with verified per-target artifacts (Mirage Project contributors, 2025; Rotem *et al.*, 2018; Chen and YiRage Contributors, 2026). One artifact everywhere is the anti-pattern, because a kernel optimal on one target is illegal or wasteful on another (AMD, 2024b).

24.8 Conclusion

Heterogeneous deployment is where the book’s per-target discipline becomes a fleet-level routing problem: disaggregate prefill and decode, unify the front end with MLIR so the residency, role, pipeline, and synchronization passes run once and specialize per target, adapt at runtime by detecting the device, and gate every rollout on a per-target regression matrix (Davies *et al.*, 2025; IREE / iree-org contributors, 2024; Vellaisamy *et al.*, 2026). The YiRage compiler is the cross-cluster router that makes this measurable, compiling one model to verified per-hardware artifacts and recording the routing intent as metadata so it survives change (Chen and YiRage Contributors, 2026; Mirage Project contributors, 2025). Deployment, however, is increasingly something compilers do for themselves: Chapter 25 turns to auto-optimization and AI-assisted compilation, where the search and routing decisions of this book are themselves driven by learned and automated systems (Wu *et al.*, 2024; Chen, 2026a).

Chapter 25

Auto-Optimization and AI-Assisted Compilation

When a decode compiler exposes thousands of tile sizes, fusion boundaries, and backend flags, hand-tuning one schedule per SKU stops scaling—yet blind autotune that optimizes GEMM FLOPs alone still ships schedules that lose on bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Part VI established that each backend (XLA, TVM, Triton, MLIR) owns a different search grammar; this chapter asks how *auto-optimization* should search when the objective is decode goodput across GPU, CPU, and NPU fleets, not a single microbench winner (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b; Goodfellow *et al.*, 2016).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields $1.259\times$ on H100 by removing host overhead—yet cannot fix illegal HBM traffic (Chen, 2026a; NVIDIA, 2024). An autotuner that rewards median kernel time without residency legality will pick launches that spill KV every token—exactly the failure mode Chapter 1 framed as “fast kernels, slow decode” (Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023).

Three mechanisms stack: **RL-guided search** over compiler knobs when the space is too large for exhaustive grid search; **hardware-aware reward functions** that score bytes/token, launches/token, and legality on the target tier; and **YiRage-style integrated search** that binds IR metadata, backend registry, and fingerprint caches so auto-optimization does not fork a second toolchain (Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). The sections below treat each layer as a production contract—what to measure, what to forbid in the reward,

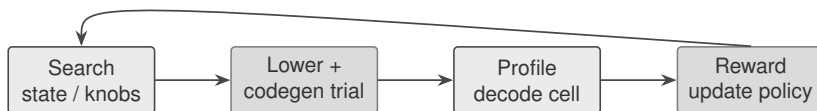


Figure 25.1: RL autotune loop: compiler knobs $\rightarrow$ trial schedule $\rightarrow$ decode-profiled reward $\rightarrow$ policy update (not peak GEMM alone).

and when AI-assisted compilation helps versus hides regressions (SemiAnalysis, 2024; AMD, 2024a; Goodfellow *et al.*, 2016).

This chapter sits at the end of Part VII for a reason: you cannot auto-tune what you cannot measure, and you cannot trust measurement that ignores residency (Chen, 2026a; Dao *et al.*, 2022; Davies *et al.*, 2025). Earlier chapters gave you backends and benchmarks; here the compiler team operationalizes *search* under decode SLOs (Chen, 2020; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2024, 2022; Dao *et al.*, 2023). Treat autotune artifacts—policies, fingerprints, profile scripts—with the same rigor as source code: versioned, reviewed, rollback-ready (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a, 2020).

Anti-pattern catalog: (i) optimizing prefill-only benchmarks for decode fleets; (ii) single-number rewards that hide bytes/token debt; (iii) external autotuners that bypass compiler legality checks; (iv) promoting GPU winners to NPU fleets without re-profile (Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The rest of the chapter turns each anti-pattern into a concrete gate (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; Goodfellow *et al.*, 2016).

25.1 RL autotune

RL autotune enters when the Cartesian product of tile sizes, pipeline stages, and fusion predicates exceeds what grid search or genetic algorithms can explore before your release train departs (Chen, 2020; Goodfellow *et al.*, 2016). Classical TVM/Ansor-style autotuning treats the schedule as a structured program with measurable features; RL layers add a policy that proposes knob vectors conditioned on IR shape, target SKU, and prior trial outcomes (Chen, 2020; Chen and YiRage Contributors, 2026). The production mistake is training that policy on prefill

shapes or batch-32 training graphs while deployment runs BS=1 decode—the search converges to schedules that never touch launches/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023).

Search spaces diverge by hardware class even when the math is identical. GPU trials must respect shared-memory caps, TMA legality, and CUDA Graph capture constraints (NVIDIA, 2022; SemiAnalysis, 2024; NVIDIA, 2024). CPU trials swap tile axes for cache lines and NUMA pinning; reward spikes from L3 residency can invert when KV grows past LLC (Goodfellow *et al.*, 2016; Chen, 2020). NPU/XDNA trials are compile-time static: illegal dynamic buffers fail before profiling, so RL exploration needs a *legality oracle* ahead of expensive simulation (AMD, 2024b,a). One global search space imported from GPU codegen is how teams burn weeks on trials that never legalize on the fleet they ship (AMD, 2024b; Chen and YiRage Contributors, 2026).

Ansor and auto-scheduler literature established *program sampling*—mutate loop orders, split factors, and cache levels under measured cost models (Chen, 2020; Goodfellow *et al.*, 2016). RL replaces random mutation with a learned proposal distribution when cost models lie on decode shapes (thin MLPs, small batch GEMMs, attention with paged KV) (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen, 2026a; Davies *et al.*, 2025). The handoff from classical autotune to RL is not “smarter magic”—it is better exploration when analytic models miss orchestration (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). Keep classical grid search as sanity check on small knob subsets (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026).

Decode-aware RL should treat each trial as a *cell*: fixed context bucket, BS=1, steady-state after warmup, metrics logged as ms/token *and* bytes/token *and* launches/token (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Prefill-only rewards overweight arithmetic intensity; decode rewards expose orchestration cliffs when graph replay is absent (NVIDIA, 2024; Chen, 2026a). MoE and speculative paths add non-stationary branches—policies must either exclude those subgraphs from the MDP or include explicit state for routing entropy (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023).

Operational guardrails: cap trial wall-clock per layer block; freeze RNG seeds for reproducible CI; store *fingerprinted* schedules keyed by IR hash + SKU + context bucket (Chen and YiRage Contributors, 2026; Chen, 2020). When RL proposes a schedule that improves GEMM but regresses bytes/token, the reward function failed—not the hardware (Chen, 2026a; Dao *et al.*, 2022; SemiAnalysis, 2024). YiRage encodes iron rules from Chapter 2 as checkable IR metadata so

Knob family	GPU decode focus	CPU / NPU delta
Tile / block	Shared mem, TMA stages	Cache line, static tile slots
Fusion depth	Graph capture legality	DMA chain length
Backend choice	Triton vs CUDA vs CUTLASS	oneDNN / AIE kernel gen
Objective	min launches/token	min bytes + DMA edges

Table 25.1: RL autotune search axes differ by target; shared math does not imply shared knob grammar.

illegal trials fail fast before GPU hours burn (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Multi-hardware delta. Hopper RL may explore TMA double-buffer stages; XDNA policies must emit static tile assignments with bounded DMA depth; CPU policies optimize cache blocking with OpenMP gang sizes—identical transformer math, incomparable action spaces (SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016).

Example 25.1.1. Decode cell (BS=1, ctx=2048): an RL policy that reduces MLP tile width may cut shared memory but force an extra KV spill—reward must penalize bytes/token, not only kernel duration (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026).

Industrial teams keep a *human prior* schedule as RL baseline: never accept a policy update that fails A/B against the prior on all three counters (Davies *et al.*, 2025; Chen, 2020, 2026a). AI-assisted compilation is not autonomy—it is search acceleration with decode metrics as the constitution (Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016; AMD, 2024a).

Feature engineering for RL policies should mirror what Ansor-style cost models already encode—loop extent, reuse distance, parallel axis—but add decode-specific state: context length bucket, graph-capture eligibility, and whether KV is paged or dense (Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024). Without those features the policy collapses to “pick bigger tiles” because that wins microbench GEMMs on prefill-shaped tensors (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). Export the feature vector alongside every trial so postmortems can explain *why* a schedule won, not only that loss decreased (Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Exploration versus exploitation trade-offs show up in fleet rollout: ϵ -greedy trial injection can destabilize p99 when a bad schedule briefly serves traffic (Davies

et al., 2025; Kwon *et al.*, 2023). Canary pools per SKU—10% of decode replicas run candidate schedules with automatic rollback on SLO breach—keep RL useful without gambling the whole cluster (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2026a). Offline RL from logged profiles is safer when experimentation budget is thin; online RL requires hard legality masks (Chen, 2020; AMD, 2024a; Goodfellow *et al.*, 2016).

Part VI compiler chapters (XLA, TVM, Triton) each expose different knob surfaces; RL autotune should treat backend choice as part of the action space, not a pre-search constant (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). A policy that only tunes Triton tile sizes while XLA would legalize a cheaper fusion is incomplete search (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Document backend-specific action spaces in runbooks so ML and compiler engineers share vocabulary (Chen and YiRage Contributors, 2026; Chen, 2020).

Chapter 21 compiler benchmarks supply the harness RL loops need: frozen decode cells, pinned model weights, and agreed context buckets (Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). Without that harness, RL trials become incomparable noise—one engineer profiles with speculative decoding disabled, another leaves it on (Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026). Wire benchmark IDs into trial logs so CI can replay the exact cell that produced each fingerprint (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Sample-efficiency tricks from production: warm-start policies from grid-search winners; reuse trials across adjacent context buckets when IR shape is unchanged; early-terminate trials that violate legality or blow memory budgets before full profile (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024b,a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). RL autotune is as much about *discarding bad trials quickly* as about finding good ones (Chen, 2020; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). Budget caps per layer prevent a single attention block from consuming an entire weekly GPU pool (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*,



Figure 25.2: Hardware-aware reward: profile trace $\rightarrow$ buffer residency $\rightarrow$ scalar score gated by target legality.

2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

When RL policies overfit to a single SKU stepping, transfer learning across related chips (same architecture generation, different memory width) can seed exploration—but always re-profile bytes/token on the destination SKU (SemiAnalysis, 2024; AMD, 2024b; NVIDIA, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024). Never promote transferred schedules without decode-cell A/B (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024). RL autotune without transfer discipline recreates the “works on my H100” culture Part I warned against (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

25.2 HW reward

The production surprise at **hardware-aware reward design** is not peak FLOPs; it is optimizing the wrong counter (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). A reward of “minimize kernel latency” on Hopper selects schedules that win Nsight Compute yet lose decode when activations round-trip through HBM every token (Chen, 2026a; Dao *et al.*, 2022; SemiAnalysis, 2024). A reward of “maximize occupancy” encourages register-heavy tiles that shrink fusion windows and raise launches/token (Vellaisamy *et al.*, 2026; NVIDIA, 2022). Hardware-aware rewards start from tier accounting: what bytes moved between register file, shared memory, L2, and HBM/DDR for one decode step (SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Chen, 2020).

Composite rewards work best as constrained scalars, not single numbers. A

practical template:

$$R = -\alpha \cdot \text{ms/token} - \beta \cdot \text{bytes/token} - \gamma \cdot \text{launches/token} - \lambda \cdot \mathbb{I}[\text{illegal}] \quad (25.1)$$

with SKU-specific weights: on bandwidth-saturated cards raise β ; on launch-storm eager stacks raise γ (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). Graph-capture-eligible regions add a legality bit—schedules that allocate inside captured DAGs receive $-\infty$ reward regardless of microbench speed (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Trace residency before trusting R . Reward producers (attention, MLP, norm) must feed consumers without a DDR round trip (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026). Silent spill reads as “mysterious” p99 regression when mean ms/token looks flat (Davies *et al.*, 2025; SemiAnalysis, 2024). Diff lowered IR or CUDA for new global allocations; count DMA edges on NPU trials like launches on GPU (AMD, 2024a,b; Chen, 2020).

Nsight Compute and roofline charts remain the auditor’s tools—not the reward itself (SemiAnalysis, 2024; Davies *et al.*, 2025; Chen, 2026a). Translate profiler counters into bytes moved per decode step before they enter R ; otherwise teams overfit to SM occupancy (NVIDIA, 2022; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Chen, 2026a). Chapter 1’s bytes/token counter is the bridge between profiler vocabulary and autotune objectives (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Graph-mode decode adds reward terms for capture legality: schedules that introduce allocations inside captured regions should fail regardless of ms/token (NVIDIA, 2024; Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). MegaKernel trials add fusion-depth terms—reward schedules that reunify MLP and residual even when graph replay already fixed launches (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; AMD, 2024a; Goodfellow *et al.*, 2016). Mode-aware rewards prevent autotune from fighting Chapter 12 execution-mode decisions (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023).

Review gate. Attach roofline or buffer accounting for the target SKU before merging any autotune winner (Chen, 2026a; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Rewards must be reproducible from checked-in profile scripts—not from a engineer’s laptop Nsight session (Davies *et al.*, 2025; Chen, 2020).

CPU rewards emphasize LLC residency and cross-socket traffic; mis-pinning a decode thread can dominate ms/token even when GEMM kernels are “optimal” (Goodfellow *et al.*, 2016; Chen, 2020). NPU rewards emphasize compile-time static legality and DMA depth; a schedule with perfect FLOPs but one extra DDR hop fails fleet power caps (AMD, 2024b,a). Treat *same reward formula* across targets as an anti-pattern—share methodology, not weights (Chen and YiRage Contributors, 2026; AMD, 2024a; Goodfellow *et al.*, 2016).

Example 25.2.1. Two trials tie on ms/token at BS=1; trial A lowers bytes/token by 12% with one fewer launch—hardware-aware ranking picks A even if trial B wins a GEMM roofline chart (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025).

Penalty terms matter as much as primary metrics. Add soft penalties for register spill events, uncoalesced global loads on KV tensors, and host-side synchronizations inserted by overly conservative codegen (SemiAnalysis, 2024; NVIDIA, 2022; Davies *et al.*, 2025). On CPU, penalize cross-NUMA migrations; on NPU, penalize DDR round trips that static analysis could have folded (Goodfellow *et al.*, 2016; AMD, 2024b,a; Chen, 2020). These penalties keep RL from “winning” on ms/token while hiding residency debt that explodes at ctx=8192 (Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Reward hacking appears when engineers reuse the same profile script but change warmup: steady-state ms/token improves while cold-start SLO regresses (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025). Lock warmup protocol in CI—fixed bucket sequence, primed graph caches, identical sampling flags—before comparing rewards (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022). Fregly-style goodput analysis always states which phase is measured (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a).

Multi-objective Pareto fronts beat single scalars when leadership must choose between 5% bytes/token and 2% launches/token (Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024). Publish Pareto sets per SKU; let product owners pick operating points—compiler teams should not hide trade-offs inside opaque

R (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Hardware-aware reward design is governance, not just math (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; AMD, 2024a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Observability closes the loop: every trial should emit structured logs—knob vector, legality outcome, the three decode counters, profiler deep link, and git SHA of compiler stack (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Dashboards that only show “best ms/token so far” invite reward hacking; dashboards that plot bytes/token and launches/token alongside expose orchestration regressions early (Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a,b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). On-call engineers promoting autotune winners need the same artifacts SREs use for binary rollbacks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

Stability testing belongs in the reward pipeline: run promoted schedules across context buckets {512, 2048, 8192}, cold and warm starts, and at least two driver versions (Chen, 2026a; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Liu *et al.*, 2024). A schedule that wins at ctx=2048 but fragments at ctx=8192 should fail promotion even if mean reward improved (Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Liu *et al.*, 2024). Hardware-aware rewards without stability gates recreate the prefill/decode split Chapter 1 documented (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

25.3 YiRage auto

YiRage auto-optimization is where search meets the runtime stack from Chapters 2 and 29: ‘superoptimize’ over multi-backend IR, ‘HardwareRegistry’ for SKU detection, and fingerprint caches that reuse proven schedules without rerunning full RL episodes (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023). The failure mode is bolting an external autotuner onto PyTorch exports—two sources of truth for fusion legality, diverging cache keys, and CI that cannot reproduce which schedule shipped (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026).

YiRage binds decode iron rules into IR metadata before search starts: residency predicates, same-backend constraints, and fallback paths when a trial illegalizes on the detected chip (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; AMD, 2024b). Search proposes knob vectors; lowering validates legality; profiling writes rewards back to the fingerprint store keyed by IR hash, backend, and context bucket (Chen, 2020, 2026a). That closed loop is how AI-assisted compilation stays *auditable*—every winning schedule traces to a profile artifact, not a black-box policy (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Cross-hardware practice: maintain one decode cell suite per fleet tier (Hopper H100, MI300, XDNA NPU, CPU reference) and require non-regression on bytes/token before promoting a fingerprint (AMD, 2024b; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Chen, 2020). GPU-first winners that spill on NPU should remain quarantined in backend-specific namespaces—shared YAML knobs are convenient and dangerous (AMD, 2024a; Chen and YiRage Contributors, 2026; AMD, 2024b). When ‘superoptimize’ exhausts its budget, fall back to human-prior schedules from Part VI compiler chapters rather than shipping under-measured RL noise (Chen, 2020, 2026a; Vellaisamy *et al.*, 2026).

Integration with serving stacks: vLLM/SGLang own batching and KV paging; YiRage auto targets intra-layer fusion and PersistentKernel handoff—reward functions must not double-count framework-level wins (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). DeepSeek-class MoE adds routing entropy; auto-optimization should segment experts into separate search cells or rewards absorb load imbalance (Liu *et al.*, 2024; Vellaisamy *et al.*, 2026). FlashAttention-class kernels shrink attention bytes; auto must still score MLP/residual bytes/token or reports lie (Dao *et al.*, 2022, 2023; Chen, 2026a).

‘HardwareRegistry’ and same-backend rules from YiRage docs prevent silent backend mismatch: a fingerprint compiled for CUDA must not replay on a CPU fallback without re-search (Chen and YiRage Contributors, 2026; Chen, 2020;

AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Search budgets (‘superoptimize’ iteration caps) should scale with layer criticality—attention—MLP seams first, sampling tails last (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022). PersistentKernel runtime from Chapter 29 consumes fingerprints; auto-optimization is wasted if runtime cannot load them idempotently (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; AMD, 2024b; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware delta. CUDA backend search explores Triton/CUTLASS tiles; CPU backend search targets oneDNN primitives; MLIR/NKI paths enforce static XDNA schedules—‘HardwareRegistry’ selects grammar, not just clock speed (Chen and YiRage Contributors, 2026; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Example 25.3.1. Fingerprint hit: IR hash + H100 + ctx2048 reuses a prior ‘superoptimize’ result; miss triggers bounded RL search with decode reward—cold start latency includes search unless warm caches are primed in CI (Chen and YiRage Contributors, 2026; Chen, 2026a; Kwon *et al.*, 2023).

Production checklist: (1) legality oracle before profile; (2) composite decode reward with SKU weights; (3) fingerprint store with IR/backend/bucket keys; (4) A/B against human prior on bytes/token and launches/token; (5) quarantine per-backend winners (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a). That is auto-optimization as engineering discipline—not “turn on AI and hope” (Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Liu *et al.*, 2024).

CI integration: store ‘superoptimize’ fingerprints as build artifacts keyed by git SHA and compiler version (Chen and YiRage Contributors, 2026; Chen, 2020). When LLVM or Triton bumps change legality, invalidate caches selectively rather than rerunning full RL from scratch (Chen, 2020; SemiAnalysis, 2024; AMD, 2024b). Nightly jobs replay decode cells on representative SKUs; drift alerts fire when bytes/token moves more than agreed epsilon (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a; Goodfellow *et al.*, 2016; NVIDIA, 2024, 2022; Dao *et al.*, 2023).

Human-in-the-loop remains mandatory for safety-critical deployments: RL proposes, engineers approve promotion, on-call runbooks list rollback fingerprints (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a). LLM-assisted *synthesis* of schedule hints—suggesting fusion candidates from IR text—can seed RL policies but must never bypass legality oracles (Chen and YiRage Contributors, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020). AI assists search; decode metrics retain veto power (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016).

Chapter 24 heterogeneous deployment assumes auto winners are tagged per backend; Chapter 26 packaging assumes fingerprint stores are reproducible in containers (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; AMD, 2024b; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). YiRage auto closes the loop between Part VI compilers and Part VIII runtime co-design (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Operator playbook summary: detect chip via registry; load fingerprint or start bounded search; validate legality; profile decode cell; compare against human prior; canary; promote or rollback (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Liu *et al.*, 2024). Each step has an owner—compiler, runtime, SRE—so auto-optimization does not become a headless cron job (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024). Chapter 27 trends toward more autonomous search; this chapter insists autonomy remains subordinate to measured decode goodput (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

When external LLM tools suggest fusion patterns, treat output as *hints* fed

into the same legality and reward pipeline—never as merged schedules without profile (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2023; NVIDIA, 2024, 2022; Liu *et al.*, 2024). AI-assisted compilation accelerates hypothesis generation; hardware-aware rewards and YiRage integration retain veto (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024). That division of labor is how industrial teams adopt ML without surrendering residency discipline (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Kwon *et al.*, 2023; Liu *et al.*, 2024).

25.4 Key Takeaways

- **RL autotune.** Use policies over compiler knobs when search spaces explode; train on decode cells (BS=1, bucketed ctx), not prefill FLOPs alone (Vellaisamy *et al.*, 2026; Chen, 2026a, 2020).
- **Hardware-aware reward.** Composite R over ms/token, bytes/token, launches/token, and legality—SKU-specific weights, shared methodology (Davies *et al.*, 2025; SemiAnalysis, 2024; AMD, 2024b).
- **YiRage auto.** Closed-loop ‘superoptimize’ + ‘HardwareRegistry’ + fingerprint caches; quarantine cross-backend winners (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024).
- **Measure useful decode work.** Prefer kernels/token and bytes/token over peak FLOPs when ranking autotune trials (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a).
- **Mechanical sympathy bounds the search.** Run legality oracles before spending GPU-hours so the autotuner never explores schedules that cannot fit the target’s residency budget (Dao *et al.*, 2022; Chen, 2020). Folding residency accounting into every reward keeps the search anchored to on-chip behavior rather than to a microbenchmark the policy can game (AMD, 2024a; Chen, 2026a).

25.5 Conclusion

Auto-optimization and AI-assisted compilation extend Part VI backends with search—but only decode metrics from Chapter 1 confer legitimacy (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025). RL explores; hardware-aware rewards judge; YiRage integrates both without forking the toolchain (Chen and YiRage Contributors, 2026; Chen, 2020). Chapter 26 assumes these baselines when packaging production deployments; Chapter 28–30 assume fused schedules won autotune gates before framework/runtime co-design (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; AMD, 2024a). Gate merges on buffer accounting and reproducible profile artifacts—never on a single GEMM roofline screenshot (SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024).

Chapter 26

Production Deployment and Engineering Best Practices

Shipping a decode compiler is not shipping a Git tag—it is shipping *reproducible artifacts* (fingerprints, containers, driver matrices) that replay the same bytes/token and launches/token counters your tuning loop proved in CI (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Chen and YiRage Contributors, 2026). Chapters 22–25 built profiling, gates, LLM/MoE specialization, and automated search; this chapter operationalizes **production deployment** across GPU, CPU, and NPU fleets without letting environment drift erase compiler wins (AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields $1.259\times$ on H100 by removing host overhead—yet production still fails when packaging omits graph warmup or pins the wrong driver (Chen, 2026a; NVIDIA, 2024). Production engineering translates compile-time legality into *fleet-time* legality (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Four practices anchor the chapter: **environment packaging**, **multi-hardware operations**, **production pitfalls**, and **heterogeneous scheduling** (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Part VIII (Chapters 28–30) assumes these baselines when separating

framework, runtime, and compiler responsibilities (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Production maturity means the same decode cell run in CI is one command away in staging and prod (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). When that link breaks, “compiler regressions” are often packaging or scheduling mistakes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Ownership RACI: compiler engineers sign fingerprint manifests; release engineering signs container promotion; SRE signs fleet config; framework owners sign batching invariants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Blurred ownership produces dashboards nobody trusts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Anti-patterns for production: promoting schedules without staging replay; mixing prefill and decode fingerprints; autoscaling on CPU utilization; silent eager fallback after driver upgrade (NVIDIA, 2024; Chen, 2026a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Each section names the fix (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Compliance and audit trails: retain profile artifacts and packaging manifests long enough to explain quarter-over-quarter decode SLO shifts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).



Figure 26.1: Environment packaging: CI compiler artifacts → immutable container → pinned driver matrix → staged fleet rollout.

Package layer	Must pin
Compiler stack	git SHA, pass flags, backend registry
Runtime deps	CUDA/cuDNN, ROCm, oneDNN, XDNA toolkit
Model assets	weights hash, tokenizer, paging policy
Profile harness	decode cell script + bucket list

Table 26.1: Production packaging layers for multi-hardware decode fleets.

Regulators and customers increasingly ask *why* latency changed—fingerprints answer that question (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

26.1 env packaging

Environment packaging binds compiler git SHA, LLVM/Triton/CUDA toolkit versions, and schedule fingerprints into deployable units (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). The anti-pattern is “works in CI” containers that omit warmup scripts, graph capture caches, or model-weight layout flags present in the tuning cell (Chen, 2026a; NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware fleets package *per-tier* images: Hopper decode, MI300 decode, XDNA edge, CPU fallback—shared YAML env vars are convenient and dangerous (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024,

2022). Rollout attaches SBOM + fingerprint manifest so on-call can diff a regressed pod in minutes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Deploy and packaging pipelines should fail closed when fingerprint files are missing, checksums mismatch, or decode harness scripts differ from CI (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). “Best effort” deploy is how illegal graph capture reaches production (NVIDIA, 2024; Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Secrets and model weights belong in the same manifest chain as compiler artifacts—rotating weights without updating fingerprints is a common silent regression (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Package deploy hooks that re-run a smoke decode cell before accepting traffic (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Multi-hardware delta. GPU images ship CUDA Graph warmup jobs; NPU images ship static compile bundles; CPU images ship ORT/TorchInductor session plans—same model card, different immutable artifacts (NVIDIA, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Example 26.1.1. Deploy promotes fingerprint `fp-8f3a` only when CI decode cell at `ctx ∈ {512, 2048, 8192}` matches golden bytes/token within ϵ (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022).



Figure 26.2: Multi-hardware operations: SKU registry → runbooks → decode counter dashboards → fingerprint rollback.

26.2 multi HW ops

Multi-hardware operations keep production honest after deploy (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Dashboards must show bytes/token, launches/token, and ms/token *per tier*—aggregating GPU and NPU into one chart hides fallback paths (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Runbooks document: legal context buckets, graph capture warmup, paging limits, MoE routing flags, and rollback fingerprints (Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). On-call steps start with “diff packaging manifest” not “retune tiles” (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

SRE collaboration: compiler teams own fingerprint promotion; SRE owns fleet capacity and incident response; framework teams own batching—escalation paths must be explicit (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Review gate. Production promotion requires tier-specific decode-cell replay in staging that mirrors driver and power caps (Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

26.3 pitfalls

Production pitfalls recur across fleets (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022):

1. **Driver/toolkit skew** — CI green, prod slow; graph capture silently falls back to eager (NVIDIA, 2024; Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).
2. **Wrong context bucket** — schedule legal at ctx=2048 spills at ctx=8192 (Chen, 2026a; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).
3. **Benchmark theater** — prefill FLOPs celebrated while decode bytes/token flatlines (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; SemiAnalysis, 2024; AMD, 2024b).
4. **Cross-tier promotion** — GPU fingerprint replayed on NPU without re-profile (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Postmortems should log counter deltas, packaging diffs, and routing histograms—not only “kernel X slower” (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Lessons feed back into Chapter 22 gates and Chapter 25 search budgets (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Pitfall drills: game-day exercises that upgrade drivers, shrink memory, or skew routing—teams rehearse fingerprint rollback before incidents (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Lessons learned belong in runbooks, not slide decks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Operation teams should track *packag* drift— undeclared env vars, sidecar injections, and mutating init scripts—as first-class incident causes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Production operation metrics include deploy rollback rate, not only availability (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

26.4 scheduling

Heterogeneous scheduling places prefill pools, decode pools, MoE expert shards, and CPU/NPU fallbacks on distinct fleet slices (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler packaging must tag which fingerprints are valid on which pool—schedulers should not load Hopper graphs onto bandwidth-starved SKUs (Chen, 2026a; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Kubernetes-style orchestration needs decode-aware autoscaling: scale on ms/token and queue depth, not CPU percent (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Prefill bursts

and decode steady-state belong on different node pools with different compiler artifacts (Davies *et al.*, 2025; Liu *et al.*, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

ClusterTopology-style metadata (Chapter 24) informs which compilers may fuse across NUMA/NVLink boundaries; scheduling respects those edges (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Canary pools per schedule fingerprint remain mandatory—fleet schedulers amplify mistakes faster than laptops (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Fleet schedul policies should expose which fingerprint each replica runs—debugging p99 without that label wastes hours (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Schedul changes (pool resize, spot preemption, NUMA rebinding) trigger mandatory decode-cell smoke tests when compiler residency is NUMA-sensitive (Goodfellow *et al.*, 2016; Chen, 2020; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Cost-aware fleet scheduling trades spot GPU pools against SLO headroom—compiler fingerprints validated on stable pools first, spot second (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Goodput per dollar still defers to bytes/token per dollar under decode load (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

26.5 Key Takeaways

- **Package the whole compile, not just the model.** A deployable unit must bind the compiler git SHA, the LLVM/Triton/CUDA toolkit versions, and the schedule fingerprints into one immutable image, so a rebuild reproduces the exact kernels (Chen, 2020; Chen and YiRage Contributors, 2026). Without that pinning, a driver or toolkit drift silently changes the emitted schedule and the decode numbers move for reasons no one can trace (Davies *et al.*, 2025; Chen, 2026a).
- **Operate multi-hardware with runbooks, dashboards, and rollback.** Production stays honest after deploy only with per-tier runbooks, counter dashboards tracking bytes/token and launches/token, and fingerprint-keyed rollback to a known-good schedule (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026). The same model on GPU, CPU, and NPU has different failure modes, so the operational tooling must be tier-aware rather than one-size-fits-all (AMD, 2024b,a).
- **Know the recurring pitfalls.** Driver skew, shape-bucket mismatch, benchmark theater (microbench wins that never move ms/token), and cross-tier promotion of a schedule tuned for the wrong SKU recur across fleets (NVIDIA, 2024; Vellaisamy *et al.*, 2026). Each is invisible to a FLOPs dashboard and visible only to a decode-cell regression on the actual target (Chen, 2026a; Davies *et al.*, 2025).
- **Schedule prefill and decode as distinct fleet slices.** Heterogeneous scheduling places prefill pools, decode pools, MoE expert shards, and CPU/NPU fallbacks on separate slices, each with its own fingerprints and decode-aware autoscaling (Davies *et al.*, 2025; Kwon *et al.*, 2023). Forcing both phases onto one uniform pool wastes the 10× capacity skew that disaggregation exists to exploit (Liu *et al.*, 2024; Chen, 2026a).
- **Hold SLOs to decode goodput, not peak FLOPs.** Production service-level objectives must track bytes/token and launches/token at the real operating point, because those are what predict ms/token and p99 (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026). A green FLOPs or utilization dashboard that ignores them is the most common way a regression ships unnoticed (Chen, 2026a; Goodfellow *et al.*, 2016).

26.6 Conclusion

Production deployment and engineering best practices turn compiler wins into fleet contracts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Package reproducibly, operate per tier, document pitfalls, and schedule heterogeneously—then Chapter 27 explores where silicon trends stress those contracts (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Part VIII implements framework–runtime–compiler co-design assuming these production gates held (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gate merges on buffer accounting and reproducible decode cells—never on laptop demos alone (SemiAnalysis, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Chapter 27

Future Trends in AI Compilers

Trend chapters fail when they read like keynote slides—buzzwords without counters (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Part VII established production packaging and gates; this closing chapter projects **where AI compilers go next** while keeping Chapter 1’s decode metrics as the constitution: bytes/token, launches/token, and residency legality across GPU, CPU, and NPU fleets (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100; OLMoE-1B/7B averages 9,305 (Vellaisamy *et al.*, 2026). Memory-Floor CUDA Graphs on Qwen-2.5-7B (BS=1, ctx=2048) yields 1.259× on H100 by removing host overhead—yet tomorrow’s silicon adds new memory tiers and routing paths that obsolete today’s fusion maps (Chen, 2026a; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a). Trends matter only when they name *which counter moves* and *which gate breaks* (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Four trajectories close Part VII: **hardware–software co-design**, **new architecture adaptation**, **edge–cloud unified compilation**, and **autonomous kernel generation with legality gates** (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; Goodfellow *et al.*, 2016). Part VIII (Chapters 28–30) implements the three-party contract among inference frameworks, YiRage runtime,



Figure 27.1: Co-design trend: silicon capabilities encoded as IR legality $\rightarrow$ compiler passes $\rightarrow$ decode-cell proof (not FLOPs alone).

and compiler co-design that these trends converge toward (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Trend chapters fail operationally when teams adopt silicon before IR contracts exist—the same failure mode as shipping eager fallbacks after graph capture (NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Each section below names a trend, the counter it should move, and the gate that must not break (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Anti-patterns for trend adoption: chasing peak TFLOPs without decode cells; porting GEMM first; treating autonomous kernels as exempt from legality; merging edge and cloud fingerprints (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). The fix is always the same methodology from Chapter 22—profile, gate, ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

27.1 codesign

Hardware–software co-design stops being a slogan when ISA and memory hierarchy changes ship with *compiler metadata contracts*—what can be fused, what must stay paged, which tiers exist (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; AMD, 2024b,a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Dao *et al.*,

Trend	Counter moved	Gate
Co-design	bytes/token residency	IR legality oracle
New arch	launches/token on port	decode-cell regression
Edge-cloud	ms/token incl. RTT	tier fingerprint isolation
Autonomous kernels	search throughput	legality before profile

Table 27.1: Future trends mapped to decode counters and non-negotiable gates.

2022, 2023; NVIDIA, 2024, 2022). Hopper-class TMA and warp specialization pushed fusion boundaries; the next co-design wave encodes those boundaries in IR so backends do not rediscover them per vendor (NVIDIA, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Co-design for *decode* differs from training: batch-1 residency, paging, and graph capture dominate; co-design documents must lead with bytes/token impact, not peak TFLOPs (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2023; NVIDIA, 2024, 2022). YiRage-style iron rules (Chapter 2) foreshadow vendor-neutral co-design: checkable predicates shipped alongside silicon (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Vendor roadmaps will keep adding memory tiers, expert routing, and capture modes—co-design is the process of turning those into compiler-visible facts before backends guess (SemiAnalysis, 2024; Liu *et al.*, 2024; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Multi-hardware delta. GPU co-design chases bandwidth and launch orchestration; NPU co-design chases static DMA depth; CPU co-design chases cache residency—shared methodology, incomparable constraints (AMD, 2024b,a; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).



Figure 27.2: New architecture trend: adapter passes bridge legacy IR to new codegen with arch-specific decode regression suites.

Example 27.1.1. New tier exposes faster on-chip buffer: co-design wins only if compiler IR marks which tensors may legally reside there at BS=1 decode (SemiAnalysis, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

27.2 new architectures

New architecture adaptation is the recurring tax every AI compiler team pays—MLIR dialects, backend forks, and HardwareRegistry entries—when silicon generations turn over (Chen, 2020; Chen and YiRage Contributors, 2026; AMD, 2024b,a; SemiAnalysis, 2024; NVIDIA, 2022; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). The anti-pattern is “port GEMM first, decode later”; decode-first ports expose paging and graph legality early (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Trend: modular adapter passes per architecture generation, each gated by the same decode cells from Chapter 22 (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Architecture trend lines without regression suites become marketing, not engineering (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

MoE-heavy architectures (DeepSeek-class) accelerate the trend: expert routing and disaggregated pools become first-class IR concepts, not runtime hacks (Liu *et al.*,

2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Compiler teams should budget one adapter generation per silicon refresh—trying to stretch a Hopper-only fusion map across Blackwell and XDNA without adapter passes guarantees launch explosions (Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Review gate. New architecture support merges only when bytes/token and launches/token match or beat prior generation on agreed buckets (Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

27.3 edge cloud

Edge–cloud unified compilation targets one model artifact with tier-specific fingerprints—edge NPU static schedules, cloud GPU graph+capture, CPU fallback—without pretending one schedule fits all (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). The edge trend is longer context on device; the cloud trend is fleet goodput—unified IR must express both without lowest-common-denominator fusion (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Adaptive compilation at runtime (detect chip, select backend) is the operational face of this trend—HardwareRegistry patterns from YiRage generalize to edge fleets (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Cloud offload decisions (which layers run edge vs cloud) belong in policy metadata compiled into the same IR family (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis,

2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Privacy and bandwidth drive edge trends, but **decode counters** still arbitrate: edge wins only if ms/token and bytes/token beat cloud RTT-inclusive baselines for the target workload (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Unified edge–cloud compile pipelines version fingerprints per tier and forbid silent cross-tier promotion (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Fleet operators increasingly run hybrid graphs: prefill on cloud GPU, decode on edge NPU, or the reverse for privacy-sensitive prompts (AMD, 2024b,a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Unified compilation must emit *split fingerprints*—not one schedule pretending both tiers share memory (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Example 27.3.1. Same LLM runs on XDNA edge and Hopper cloud: two fingerprint namespaces, one IR export, shared decode-cell methodology (AMD, 2024b; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

27.4 autonomous kernels

Autonomous kernel generation—LLM-assisted or search-driven—is the most hyped trend and the highest risk (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Chapter 25 placed RL and auto-optimization under decode rewards; the trend extends toward models proposing fusion patterns or kernel text (Chen and YiRage Contributors,

2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).

Non-negotiable guardrails for autonomous kernels:

1. **Legality oracle before profile** — same as human-authored schedules (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).
2. **Decode-cell reward** — bytes/token, launches/token, ms/token; no GEMM-only scores (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022).
3. **Human prior + fingerprint rollback** — autonomy subordinate to production gates (Chapter 26) (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Autonomous *kernel* text without IR integration repeats CUDA string injection failures—trend winners compile through the same multi-backend stacks Part VI built (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). The autonomous trend succeeds when it accelerates search under existing gates, not when it bypasses them (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Search budgets for autonomous kernels should cap candidate evaluations per merge request—unbounded search is how illegal schedules reach main (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Chapter 25’s RL autotune patterns extend here: reward

decode goodput, penalize legality violations, require human-readable diffs (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

27.5 Key Takeaways

- **Co-design ships as compiler metadata contracts.** Hardware–software co-design stops being a slogan when an ISA or memory-hierarchy change arrives with a contract the compiler can read—what may be fused, what must stay paged, which tiers exist (Chen and YiRage Contributors, 2026; SemiAnalysis, 2024). Encoding silicon capabilities as IR legality and proving each on decode cells is what turns a new feature into a usable, checkable optimization rather than a press release (Chen, 2020; Davies *et al.*, 2025).
- **New architectures are a recurring, decode-first tax.** Every silicon generation forces adapter passes, backend forks, and hardware-registry entries, and the discipline that keeps the cost bounded is decode-first porting with architecture-specific regression (Chen, 2020; Chen and YiRage Contributors, 2026). Bringing a new target up on prefill alone hides exactly the residency and launch gaps that batch-1 decode will later expose (Chen, 2026a; Vellaisamy *et al.*, 2026).
- **Edge–cloud means one artifact, tier-specific fingerprints.** Unified compilation targets a single model with per-tier fingerprints—edge NPU static schedules, cloud GPU graph capture, CPU fallback—without pretending one schedule fits all (AMD, 2024b,a). Adaptive backend selection at runtime with no cross-tier promotion is what prevents a schedule tuned for one tier from silently regressing another (Davies *et al.*, 2025; Kwon *et al.*, 2023).
- **Autonomous kernels need legality, reward, and rollback.** LLM-assisted and search-driven kernel generation is the most hyped and highest-risk trend, because a generated kernel can be fast and wrong (Chen, 2020; Chen and YiRage Contributors, 2026). It is safe only inside a frame that verifies legality, rewards decode goodput rather than microbench latency, and can roll back—treating the generator as a hint source, not a bypass of the residency contract (Chen, 2026a; Davies *et al.*, 2025).
- **Part VIII makes these trends deployable.** The co-design, portability,

and automation trends above are not abstractions; framework, runtime, and compiler co-design is where they become a shipped serving stack (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023). The next part descends from trend to implementation, carrying the same decode-goodput yardstick (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

27.6 Conclusion

Future trends in AI compilers converge on one discipline: faster silicon and smarter automation must still answer to decode goodput (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Co-design, new architectures, edge-cloud unity, and autonomous kernels are directions—not excuses to drop bytes/token gates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b). Part VIII opens with inference frameworks (Chapter 28), YiRage PersistentKernel runtime (Chapter 29), and three-party co-design (Chapter 30) (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020; Liu *et al.*, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022). Gate merges on buffer accounting and reproducible decode cells—the trend line starts where Chapter 1 started (SemiAnalysis, 2024; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; AMD, 2024a; Goodfellow *et al.*, 2016; AMD, 2024b; Dao *et al.*, 2022, 2023; NVIDIA, 2024, 2022; AMD, 2024b).

Chapter 28

LLM Inference Frameworks and Serving Runtimes

Who owns decode goodput—the inference *framework* that batches requests and manages KV, or the *compiler/runtime* that fuses kernels and cuts bytes/token? The industry default splits responsibility badly: frameworks optimize scheduling and capacity; compilers optimize isolated GEMMs; neither measures the same counters on batch-1 decode (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Part VIII opens by naming what frameworks *must* keep (request lifecycle, paging, sampling policy) versus what YiRage and MegaKernel stacks can absorb (intra-step residency, launches/token) (Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). The chapter goal is operational clarity: every engineer can point to the decode step box on a diagram and attach counters (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Chapter 27 flagged codesign and autonomous-kernel trends; Part VIII opens by naming who owns each decode counter when those trends land in production (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025). TaxBreak dense Llama-3.2-1B averages 847.5 kernel launches per output token on H100—often before framework-level batching helps at BS=1 (Vellaisamy *et al.*, 2026). vLLM-style serving reduces KV fragmentation and raises fleet utilization, but leaves operator boundaries inside the decoder step unless a compiler reunifies them (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026). Memory-Floor reports $1.259\times$ median speedup on H100 from CUDA Graph replay alone at BS=1, ctx=2048—orchestration relief that paging cannot replace

when bytes/token stay high inside the step (Chen, 2026a; NVIDIA, 2024). Those three citations anchor the split: launches, paging, and graphs solve different layers of the same decode story (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). This chapter maps framework layers to Chapter 1 counters so Part VI–VII compiler theory lands in a deployable stack, not a slide (Chen, 2020; Vellaisamy *et al.*, 2026; SemiAnalysis, 2024; AMD, 2024b,a).

Readers should leave with a deployment taxonomy: classify any production stack into framework-native, partial-export, or compiler-owned decode core (Table 28.1), then profile *both* scheduler queue time and in-step ms/token/bytes/token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026). Chapter 29 descends into YiRage Layer 5 Persistent Kernel runtime; Chapter 30 stitches the three-party contract (Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Until then, treat every framework feature as a hypothesis that must survive BS=1 counter worksheets (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a). If you remember nothing else, remember the decode step box: everything inside is compiler and runtime turf; everything outside is framework turf (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Draw that box before every integration meeting and attach Chapter One counters to each side per decode token (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Fregly-style mechanical sympathy applies at the *boundary* between framework and compiler: the framework must not hide allocator behavior that forces HBM spill; the compiler must not assume contiguous KV or static batch shapes that break paging (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; NVIDIA, 2024; Goodfellow *et al.*, 2016). Part VI–VII taught pass ordering and dialect lowering; Part VIII asks where those passes *attach* in a live server (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). The answer is never “replace the framework” wholesale—it is to shrink the decode step until launches/token and bytes/token move on the same SKU you ship (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016).

This chapter also connects backward to Chapter 12 execution modes and Chap-



Figure 28.1: Framework landscape: requests enter scheduling and KV policy; bytes/token and launches/token are decided inside the decode step boundary.

ter 23 LLM compile specialization (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Eager versus graph versus MegaKernel is not only a training-time PyTorch concern—it is reproduced inside serving binaries through framework dispatch, CUDA graphs, and custom fused ops (NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). DeepSeek-scale stacks add expert parallel communication below the scheduler; open components in `deps/DeepEP` and `deps/FlashMLA` show kernels that must compose with paging and batching rather than bypass them (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Read framework documentation as architecture diagrams with counters attached, not as marketing feature lists (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

28.1 framework landscape

Framework landscape spans HuggingFace *eager* paths, vLLM/SGLang *continuous-batching* servers, TensorRT-LLM/ONNX Runtime *compiled* deployments, and custom C++ runtimes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2020; Chen and YiRage Contributors, 2026). Each makes different fusion depth assumptions: TRT-LLM may fuse within exported regions; HF PyTorch often does not; vLLM optimizes KV and batching while reusing framework attention backends (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026). SGLang extends the same serving layer with structured generation and radix-cache reuse—still framework-side orchestration unless a compiler owns the decoder cell (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026).

Production stacks rarely pick a single label from the list above. A common pattern is HuggingFace weights loaded into a vLLM server with FlashAttention

backends and optional TensorRT plugins for narrow subgraphs (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The compiler engineer job is to diagram where the decode step begins and ends in that stack: from slot assignment and block table lookup through final hidden state before sampling (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Anything inside that window is fair game for fusion and residency optimization; anything outside is framework policy (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Ambiguity at the boundary is the main reason teams debate whether a regression is serving or compiler (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Compiler engineers should classify any deployment into three buckets (Table 28.1): (1) *framework-native* decode (Python loop + vendor libs); (2) *partial export* (compiled attention or MLP only); (3) *compiler-owned decode core* (YiRage PersistentKernel or MegaKernel inside a thin framework shell) (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Bucket choice determines whether your lever is batching policy or bytes/token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026). Teams that pick bucket (1) for time-to-market but benchmark only fleet tokens/s will misattribute compiler regressions to “scheduling” when launches/token never moved (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025).

HuggingFace vs serving servers. HF transformers optimizes researcher ergonomics: dynamic shapes, easy checkpoint swap, and Python-visible tensors (Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Production servers trade that flexibility for slot-based scheduling, pinned memory pools, and optional CUDA graphs (Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2026a). Compiler integration rarely starts in HF notebooks—it starts when a serving team exports a decode cell and asks which IR metadata (softmax carriers, page tables) survives the cut (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024).

Research notebooks prove model correctness; serving binaries prove goodput under load (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a;

Table 28.1: Framework deployment buckets and where compiler/runtime leverage applies.

Bucket	Typical stack	Primary goodput lever
Framework-native	HF eager, vanilla PyTorch loop	Continuous batching, KV paging; launches/token often high (Vellaisamy <i>et al.</i> , 2026; Kwon <i>et al.</i> , 2023)
Partial export	vLLM + compiled attention; TRT-LLM sub-graph	Export boundaries; graph capture buckets (NVIDIA, 2024; Davies <i>et al.</i> , 2025)
Compiler-owned core	Thin shell + YiRage PK / MegaKernel	Fusion, residency, launch collapse inside step (Chen and YiRage Contributors, 2026; Chen, 2020)

Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Teams should freeze a serving configuration—block size, max batch, dtype—before comparing compiler exports (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Otherwise HF eager baselines become moving targets that invalidate A/B conclusions (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

TensorRT-LLM and compiled paths. TRT-LLM and similar engines push fusion into builder-time regions but still sit behind a framework or C++ server for batching and KV (Davies *et al.*, 2025; NVIDIA, 2024; Goodfellow *et al.*, 2016). Partial export wins when attention or MLP dominates bytes/token; it fails when export boundaries fall *inside* online-softmax state or paged KV gather/scatter (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026). Profile export candidates on BS=1 decode, not only engine build logs (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a).

Operational ownership. In production, three teams touch the same decode path: serving (framework), compiler (graph lowering), and hardware (SKU drivers) (Davies *et al.*, 2025; Chen, 2020; Chen and YiRage Contributors, 2026; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024). Without Table 28.1 vocabulary, incidents become blame loops while bytes/token flatlines

(Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Runbooks should record bucket, shape bucket, paging block size, and export boundary hash alongside fleet dashboards (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

SGLang and structured serving. Structured generation adds mask and grammar state to the framework layer—still outside compiler fusion unless exported (Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). RadixAttention-style prefix reuse compounds paging benefits; compiler cores must not invalidate shared prefixes with per-request recompilation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware delta. Framework support matrices differ: CUDA-first servers dominate cloud; CPU/OpenVINO paths serve edge; Ascend/MACA stacks require vendor runtimes (AMD, 2024a,b; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). YiRage targets the same decode core across backends—framework integration must not reintroduce GPU-only assumptions at the API boundary (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Liu *et al.*, 2024; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024). An integration that hard-codes CUDA graph capture without NPU static-shape buckets will pass GPU CI and fail edge deployment (AMD, 2024a; Chen, 2020; Chen and YiRage Contributors, 2026).

Example 28.1.1. A team runs vLLM for KV paging but keeps PyTorch-eager MLP boundaries: paging wins capacity while launches/token stay TaxBreak-high—framework choice fixed scheduling, not fusion (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2026a). Moving to bucket (3) requires exporting the full decode cell with page-table layout IR, not swapping attention alone (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022).

Migration path from eager to compiler-owned core. Most teams land in bucket (1), prove fleet demand, then partial-export attention or MLP while paging stabilizes (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026;



Figure 28.2: Continuous batching architecture: the scheduler interleaves prefill and decode into shared GPU slots backed by paged KV.

Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The risky middle phase is partial export with hidden boundaries inside RMSNorm or rotary embedding: microbenchmarks show wins while bytes/token flatlines because carriers spill between exported regions (Dao *et al.*, 2022, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). A disciplined migration keeps the framework scheduler and block allocator fixed while shrinking the decode cell boundary each release, measuring launches/token and bytes/token on the same context buckets after every export change (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Skip the worksheet step and you will ship bucket (2) forever, wondering why graph capture helped but cost per million tokens did not (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

28.2 continuous batching

Continuous batching interleaves prefill and decode requests to raise GPU occupancy (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Goodfellow *et al.*, 2016). Classic static batching waits until N requests align; continuous batching admits new sequences when others finish prefill or free decode slots—improving *fleet* goodput (tokens/s per GPU) (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a). It does not automatically cut *per-request* bytes/token at BS=1 inside a single decode step (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). A scheduler that doubles fleet utilization while leaving hundreds of launches per token untouched has solved capacity, not mechanical sympathy inside the decoder (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Serving engineers often first encounter continuous batching as a throughput

win in load tests (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Compiler engineers should insist on paired microbench: same server binary, same paging policy, but forced BS=1 decode cell measurement between scheduler iterations (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). If only fleet metrics move, the next investment belongs in export boundaries and PersistentKernel integration, not longer batching windows (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Prefill vs decode in the scheduler. Prefill is compute-heavy and benefits from large token-parallel batches; decode at BS=1 per sequence is memory- and orchestration-bound (Dao *et al.*, 2022; Davies *et al.*, 2025; Chen, 2026a). Frameworks therefore use iteration-level scheduling: each engine step may mix prefill chunks and decode tokens from different clients (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024). Compiler integration must preserve batching invariants—dynamic sequence lengths, slot assignment, cancellation—while still allowing graph capture or MegaKernel regions for steady decode cells (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Exporting a fused kernel that assumes fixed batch shapes breaks continuous batching unless the runtime provides shape buckets (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a).

Fleet goodput vs per-request counters. Fleet metrics—tokens/s, cost per million tokens, GPU utilization—reward batching and paging (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Per-request counters—ms/token, bytes/token, launches/token at BS=1—reward fusion and residency inside the step (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020). Production teams need both: fleet metrics for capacity planning; per-request counters for compiler A/B and p99 latency under load (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Framework schedulers expose *when* a decode step runs; compilers decide *what* executes inside the step (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Profile both: scheduler queue time vs in-step ms/token (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen,

2020; Goodfellow *et al.*, 2016).

Document both metric families on the same release dashboard (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fleet tokens/s may rise while BS=1 bytes/token stalls—label that as a scheduling win, not a fusion win (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Fusion wins that break batching legality should fail integration tests even when microbenchmarks improve (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). The contract runs both directions: compilers preserve paging and mask invariants; frameworks expose stable plugin ABIs (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Chunked prefill and decode interleaving. Long prefills can starve decode slots; frameworks chunk prefills and interleave decode steps to bound latency (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024). Compiler-owned cores must support partial prefill shapes without recompilation storms—IR bucket tags for context length and batch composition are framework-facing contracts (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Chen, 2026a). DeepSeek-scale MoE adds expert-parallel scheduling on top of token batching (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026).

Scheduling policies and SLOs. Frameworks expose FCFS, priority queues, and preemption for premium tenants (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). p99 latency is often scheduler-bound under burst load even when BS=1 kernels are optimal (Davies *et al.*, 2025; Chen, 2026a; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Capacity planning should separate queueing SLOs from in-step SLOs (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Watermarking and backpressure. When KV pools exhaust, frameworks reject or delay requests (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler compression of KV shifts watermarks but requires framework allocator awareness (Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware delta. CPU-served frameworks batch differently—OpenVINO and llama.cpp paths use thread pools and blocked KV rather than GPU slot tables (Goodfellow *et al.*, 2016; AMD, 2024a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020). NPU serving often static-buckets requests by shape; continuous batching semantics must be emulated in software or sacrificed for compile-time legality (AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a).

Example 28.2.1. Under load, fleet tokens/s rises 40% after enabling continuous batching, but BS=1 microbench launches/token unchanged—expected if batching fixed occupancy, not fusion (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026). Next lever: compiler-owned decode core inside the same scheduler shell (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024).

Iteration-level scheduling mechanics. Continuous batching implementations maintain a running batch tensor whose rows map to live sequence slots (Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Each engine iteration may append new prefill tokens for one client while advancing decode by one token for others in the same launch (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). Compiler-owned cores must accept dynamic active-row masks without recompilation per arrival: shape buckets cover max batch width while runtime masks disable finished sequences (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow

et al., 2016). Framework engineers own the slot table; compiler engineers own legality of fused kernels across masked rows (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

28.3 kv paging serv

KV paging serv (PagedAttention and descendants) treats KV cache as allocatable blocks—framework memory management, not compiler math (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022; NVIDIA, 2022, 2024). Without paging, contiguous KV reservations fragment GPU memory as sequences grow and shrink; paging maps logical token positions to physical blocks, improving fleet capacity (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a). Compilers must consume page tables as layout IR (Chapter 13), not fight them with contiguous-KV-only fusion plans (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Paging is the reason large-context serving became economically viable on single-GPU deployments: operators trade small internal fragmentation for predictable allocation (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2022, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). It is not a substitute for lowering bytes moved during attention itself (Dao *et al.*, 2022, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teams celebrating paging-only wins should still run Memory-Floor style byte ledgers on decode steps (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Block size and attention tiles. Block size trades fragmentation vs attention tile efficiency: smaller blocks reduce waste; larger blocks improve vectorized loads (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024). FlashAttention-style kernels assume block-aligned KV access; paging block size should co-tune with compiler tile search on the same context buckets

(Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). A paging policy chosen by SREs without compiler input can force gather/scatter patterns that erase fusion wins (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026).

Run a joint review when either team changes block size or attention tile width (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). The review output is a signed counter sheet, not a slide deck (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Prefix caching and radix reuse. SGLang-style radix caches reuse KV blocks across prompts with shared prefixes—framework-side deduplication (Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Compiler cores must treat cached blocks as valid layout inputs: same page table semantics, different allocation provenance (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025). Exporting attention without page-table metadata breaks both vanilla paging and prefix reuse (Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2026a).

Prefix reuse changes the economics of long system prompts: the framework avoids recomputing shared KV while the compiler still must legalize attention reads against block tables for the suffix tokens (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler teams should test prefix-hit and prefix-miss paths separately because gather patterns and occupancy differ (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Treat prefix cache as another paging policy knob co-tuned with tile search (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

MLA and compressed KV. DeepSeek-V3 MLA reduces KV footprint; paging and compiler layout must agree on compressed head dimensions (Liu *et al.*, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a). Open components under `deps/FlashMLA` illustrate MLA kernel expectations that framework paging must

feed (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

Memory accounting worksheet. Fleet operators should track bytes per KV block, active blocks per sequence, peak concurrent sequences, and fragmentation ratio (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler teams add bytes moved per decode step at BS=1 for the same paging policy (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Disagreement between allocator metrics and profiler HBM counters signals layout IR mismatch (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Copy and migration costs. Disaggregated prefill/decode copies KV blocks—framework orchestration with compiler-visible layout (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fusion plans that assume KV stays local break under migration unless gathers are legal across NUMA or NVLink paths (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; Goodfellow *et al.*, 2016).

Multi-hardware delta. GPU paging uses CUDA virtual memory and block pools; CPU paths use pinned host buffers; NPUs use statically reserved SRAM windows (Goodfellow *et al.*, 2016; AMD, 2024a,b; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). A compiler that lowers paged KV only for contiguous GPU tensors will fail on XDNA static layouts (AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026).

Example 28.3.1. Block size 16 cuts fragmentation but increases attention loop trips; co-tuning with FlashDecoding tile width recovered bytes/token while keeping paging benefits (Kwon *et al.*, 2023; Dao *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026).

PagedAttention internals for compiler authors. Logical token position

maps through a block table to physical GPU memory blocks; attention kernels index KV via block id and offset rather than a flat stride (Kwon *et al.*, 2023; Dao *et al.*, 2022, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2022, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Compiler layout IR must represent gathers from block tables as first-class ops with known alignment and dtype, not as opaque custom calls that break fusion legality analysis (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). When MegaKernel fusion plans assume contiguous KV tensors, paging forces either illegal fusion or a gather prologue that dominates bytes/token (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Liu *et al.*, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Treat block tables like weight layouts in Chapter 13: co-design block policy with tile search (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

28.4 framework bottlenecks

Framework bottlenecks mirror Chapter 1: launches/token, bytes/token, and orchestration tax—often introduced *above* the decoder kernel by Python loops, dispatcher churn, and unfused vendor op sequences (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Framework A/B that reports only tokens/s without per-step byte ledgers misattributes compiler regressions to scheduling (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

When triaging a slow server, split the timeline before optimizing kernels (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Queue wait implicates admission control and batching policy (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*,

2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). High in-step milliseconds with moderate launches implicates bytes/token and fusion gaps (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). High launches with moderate bytes implicates orchestration and graph capture opportunities (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Skipping this split sends compiler teams to rewrite attention while the framework still dispatches hundreds of tiny kernels (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Python and host overhead. Eager HF loops pay GIL, tensor metadata, and per-op Python dispatch (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a). Serving frameworks move hot paths to C++/CUDA but still coordinate sampling, stop criteria, and RPC on host (Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a). CUDA Graph capture removes launch storms for steady decode shapes; it does not fuse illegal HBM traffic (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022). Teams that graph-capture unfused graphs lock in high bytes/token at lower launch count (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; NVIDIA, 2024; Chen and YiRage Contributors, 2026).

Operator chains inside the step. Even “optimized” servers often call separate attention, norm, and MLP kernels each token (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024). TaxBreak counts the launches; Memory-Floor counts the bytes (Vellaisamy *et al.*, 2026; Chen, 2026a). Framework bottlenecks end where compiler fusion begins—but only if integration exports the whole decode cell, not a single op (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Walk one token through the server call stack in a profiler and count CUDA API entries between scheduler entry and sampling exit (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Each entry is a fusion boundary candidate (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*,

2023; Goodfellow *et al.*, 2016). Prioritize boundaries that move activations to HBM between norm and matmul or between attention and MLP (Dao *et al.*, 2022, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; NVIDIA, 2024; AMD, 2024a,b; NVIDIA, 2022; Goodfellow *et al.*, 2016). That ordered list becomes the export roadmap for bucket (3) migration (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Speculative decoding overhead. Draft models add scheduler complexity: verify steps, acceptance sampling, and dual KV streams (Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016). Frameworks must slot draft and target sequences; compilers should fuse verify cells where numerics allow (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; NVIDIA, 2024). Profile speculative paths separately—fleet speedup can hide verify-step launch storms (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Goodfellow *et al.*, 2016).

Observability split. Instrument four spans per request: queue wait, prefill steps, decode steps, and sampling/host (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). nsys rows alone are insufficient if framework queue time dominates p99 (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Export traces with bucket id, block size, and export hash for compiler A/B (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

RPC and tokenizer tax. Front-end frameworks pay tokenization, detokenization, and gRPC serialization—outside GPU counters but inside user-perceived latency (Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Compiler wins on bytes/token do not remove host bottlenecks; they prevent misallocated engineering weeks (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Multi-hardware delta. CPU-served frameworks hit GIL and memcpy costs; GPU frameworks hit launch storms; NPU frameworks hit static-shape bucket misses (Goodfellow *et al.*, 2016; AMD, 2024a,b; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2023). Bottleneck triage starts with counter worksheets per target, not framework brand (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Example 28.4.1. Replacing Python loop with CUDA graphs improved ms/token 12% at BS=1 but bytes/token flat—graphs fixed orchestration, not fusion (Chen, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Fair framework comparison methodology. Benchmark posts that compare vLLM, SGLang, and TRT-LLM using only aggregate tokens/s at fixed concurrency often reflect scheduler defaults and paging block sizes rather than intrinsic decode efficiency (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Goodfellow *et al.*, 2016). A fair comparison fixes model checkpoint, dtype, context distribution, block size, and GPU SKU, then reports BS=1 launches/token and bytes/token alongside fleet curves (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Without the fixed-cell worksheet, framework marketing numbers cannot inform compiler roadmap prioritization (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Chapter 1 counters exist precisely to prevent apples-to-oranges serving benchmarks from driving fusion decisions (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

28.5 compiler integration hooks

Compiler integration hooks are the seams where YiRage attaches: exported FX/ONNX regions, custom op *plugin* registration, CUDA graph capture buckets,

and PersistentKernel call sites (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Hooks should preserve numerics carriers from Chapter 10 and residency metadata from Chapter 6—exporting a subgraph without those invariants recreates silent HBM spill (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022).

Integration design meetings often fail because participants use the word kernel loosely (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). A framework custom op may wrap one CUDA kernel or an entire PersistentKernel program with internal loops (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). The hook contract must specify which side owns slot tables, which side owns softmax state, and which side owns graph capture replay (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Ambiguous ownership produces double-free style bugs in performance: two layers both “optimize” the same boundary and break numerics (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

FX and ONNX export regions. PyTorch FX traces decode cells for compiler lowering; ONNX export targets TRT and ORT runtimes (Davies *et al.*, 2025; Chen, 2020; Goodfellow *et al.*, 2016; NVIDIA, 2024; Chen and YiRage Contributors, 2026). Export boundaries must include page-table gathers, rotary embeddings, and online-softmax state—not only matmuls (Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen, 2020; Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Goodfellow *et al.*, 2016). Broken exports pass unit tests on square tensors and fail on paged decode (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022).

Custom op plugins. Frameworks register `torch.library` or vLLM custom ops that delegate to compiled `.so` modules (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022; Dao *et al.*, 2023). Plugin APIs are the stable contract: framework passes slot ids, page tables, and sampling flags; compiler returns fused launch descriptors (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Version skew between framework paging and plugin layout IR is a common production outage mode (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a).

Treat each plugin like a narrow ABI: document struct layouts, alignment, dtype, and error codes; reject silent fallbacks to eager ops (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fallbacks hide fusion failures behind correct-but-slow paths that pass functional tests while burning fleet budget (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration CI should fail when fallback triggers except in explicitly whitelisted debug modes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

CUDA graph buckets and PersistentKernel. Graph capture requires static launch templates per shape bucket (NVIDIA, 2024; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022). YiRage PersistentKernel mode collapses launches inside the compiled core while the framework keeps scheduling—Chapter 29 details runtime handoff (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Integration tests should cover: bucket switch, cancel mid-sequence, and prefix-cache hit (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Shape buckets should align with framework batch width limits and max context policies so capture replay does not miss rare production shapes (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). PersistentKernel programs extend buckets with internal iteration over layers while presenting one host launch per decode step—the pattern Chapter 29 documents in `deps/YiRage` (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Frameworks must not assume one graph per model; they should register one graph per bucket per compiled fingerprint (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Design rule: frameworks own *request and KV lifecycle*; compilers own *decode-step residency*; runtime owns *launch and synchronization* inside the compiled core (Chapter 29) (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022). Chapter 30 formalizes the split for production stacks (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). DeepSeek open components (`deps/DeepEP`, `deps/FlashMLA`) show what must plug below framework schedulers for MoE at scale (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Integration test matrix. Minimum CI scenarios: BS=1 ctx 512/2k/8k; continuous batch mix; prefix-cache hit; cancel mid-stream; bucket switch after compile (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Each scenario logs launches/token and bytes/token (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Rollback and fingerprint caches. YiRage superoptimize caches fused artifacts by IR fingerprint; framework upgrades that change page-table ABI must bump fingerprint salts (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*,

2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Silent cache hits across incompatible paging policies are production footguns (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Multi-hardware delta. Hooks differ by backend: CUDA plugins vs XDNA AIE binaries vs CPU AVX512 kernels—the API shape should stay constant while layout IR lowers per target (AMD, 2024b,a; Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2022, 2023; Liu *et al.*, 2024).

Example 28.5.1. vLLM custom op wrapping YiRage superoptimized decode cell: framework passes block tables and slot map; plugin returns single PK launch—launches/token dropped from hundreds to single digits while paging unchanged (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

ABI stability across releases. Plugin boundaries serialize slot indices, block table pointers, sampling parameters, and optional speculative-draft handles (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework minor version bumps that reorder block table fields without coordinated compiler releases produce silent wrong-token bugs worse than a hard crash (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Pin paired framework and compiler versions in production manifests; run golden-token tests on every bump (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Chapter 30 extends this into a three-party release train (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

28.6 Key Takeaways

- **Frameworks schedule; compilers fuse.** Continuous batching interleaves prefill and decode requests to raise GPU occupancy and fleet utilization, but it does nothing about the batch-1 bytes/token inside a single decode step (Kwon *et al.*, 2023; Davies *et al.*, 2025). The framework owns request scheduling and the compiler owns intra-step fusion, so a serving win at one layer must not be mistaken for a win at the other (Vellaisamy *et al.*, 2026; Chen, 2026a).
- **Paged KV is framework layout the compiler must lower.** PagedAttention treats the KV cache as allocatable blocks—framework memory management, not compiler math—but the compiler must still lower those page tables as first-class IR so fusion can see page boundaries (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026). Hiding the paging from the compiler forces gather/scatter that breaks on-chip residency at the attention seam (Chen, 2020; Davies *et al.*, 2025).
- **Measure both layers, not one.** Framework bottlenecks mirror Chapter 1—launches/token, bytes/token, orchestration tax—but are often introduced *above* the kernel by Python loops and dispatcher churn, so the diagnosis must report queue/scheduling time alongside in-step ms/token and bytes/token (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025). A fast kernel behind a slow scheduler still misses the SLO (Chen, 2026a; Goodfellow *et al.*, 2016).
- **Integration hooks are contracts, and parity is explicit.** The seams where a compiler attaches—exported FX/ONNX regions, custom-op plugins, CUDA-graph capture buckets, PersistentKernel call sites—must preserve softmax carriers and capture legality across the boundary, or fusion silently regresses (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; NVIDIA, 2024). Framework defaults are CUDA-centric, so multi-hardware parity requires explicit triplet tests on the YiRage backends rather than assumed equivalence (AMD, 2024b,a).
- **Migrate by shrinking the decode export, and version the ABI.** Move a workload bucket by bucket, shrinking the decode export boundary, and never confuse a paging win with a fusion win; before any PersistentKernel work, log the bucket, decode-box boundaries, queue wait, and in-step counters at context 512/2k/8k (Kwon *et al.*, 2023; Davies *et al.*, 2025). Framework paging and compiler plugins must release together with golden-token tests,

since an ABI skew between them corrupts output in ways a counter dashboard will not catch (Chen and YiRage Contributors, 2026; Chen, 2026a).

28.7 Conclusion

Inference frameworks solve fleet scheduling and KV capacity; they do not replace compiler-side fusion and runtime-side launch collapse (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Classify your stack with Table 28.1, profile scheduler and in-step counters separately, and treat every plugin boundary as a numerics and layout contract (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; NVIDIA, 2024; Goodfellow *et al.*, 2016). The practical outcome of this chapter is a shared vocabulary between serving and compiler teams: when paging wins but bytes/token stalls, the next owner is the decode cell export, not the scheduler (Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Before opening Chapter 29, run one end-to-end worksheet on your production stack: name the bucket, draw the decode step box, log queue wait and in-step counters for ctx 512/2k/8k, and list every plugin boundary with numerics carriers attached (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). That worksheet becomes the acceptance criteria for PersistentKernel integration in the next chapter (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Chapter 29 descends into YiRage Layer 5 Persistent Kernel runtime (`deps/YiRage` submodule); Chapter 30 stitches framework, compiler, and runtime into one decode goodput workflow (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Chapter 29

YiRage Runtime Layer and Persistent Kernel Execution

Chapter 28 placed inference *frameworks* above the decode step; this chapter covers YiRage *below* exported graphs—the C++ runtime, Persistent Kernel launcher, and backend registry that turn `superoptimize()` results into repeatable decode cells (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Source tree: git submodule `deps/YiRage` (upstream (Chen and YiRage Contributors, 2026)); init via `git submodule update -init -recursive deps/YiRage` (see `deps/README.md`).

TaxBreak counts launches/token inside the decode box; Memory-Floor counts bytes/token and graph replay relief (Vellaisamy *et al.*, 2026; Chen, 2026a). Compiler passes from Part VI–VII emit fused plans; **Layer 5 runtime** owns device memory discipline, launch templates, and backend selection so those plans survive fleet load (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Without runtime contracts, `superoptimize()` wins evaporate at the framework plugin boundary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Readers should map each Layer 5 API to Chapter 1 counters: `PersistentKernel` collapses launches/token; runtime residency metadata cuts bytes/token; `HardwareRegistry` constrains search to the SKU you ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*,

2026; Chen, 2026a; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Chapter 30 stitches framework schedulers to this runtime; here we stay inside `deps/YiRage` (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Fregly mechanical sympathy at runtime means treating device memory like registers with lifetimes: allocations inside the decode step are bugs unless proven on-chip (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Layer 5 code paths should log residency class transitions the way Chapter 6 planners annotate IR (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). When runtime logs show unexpected HBM allocs per token, return to Layer 3 fusion legality before tuning scheduler batch windows (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

This chapter is the mirror of Chapter 28: frameworks own everything outside the decode box; runtime owns everything inside once the framework plugin invokes PK (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Integration meetings should draw that box first, then assign Layer 1–5 owners (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Runtime engineering begins where compiler IR ends. The fused graph is no longer a data structure in Python; it is a device resident program with allocation pools, synchronization points, and launch templates that must survive millions of decode steps (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Teams new to YiRage often stop measuring after superoptimize reports a winning tile config. Production goodput is decided in Layer 5 when paging pointers arrive from vLLM, when CUDA graphs replay, and when softmax carriers stay in registers instead of spilling to HBM (Kwon *et al.*, 2023; Chen, 2026a; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016). Treat runtime as a first class performance surface with its own counters, owners, and release train (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025;

Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

The submodule layout under `deps` mirrors this responsibility. Python examples in the book intentionally point at pinned upstream APIs, but Layer 5 behavior lives in C++ under `libyirage` runtime (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When documentation disagrees with profiler traces, trust traces and file issues against runtime launch paths rather than re tuning scheduler batch sizes alone (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Chapter 30 will show how framework schedulers call into this stack; here we keep the camera inside Layer 5 (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Three questions gate every runtime review. First, does warmed PersistentKernel beat eager launches/token on the fleet SKU at batch one (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Second, does bytes/token beat the pre fusion baseline at context 512, 2048, and 8192 (Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Third, does integration preserve paging ABI and softmax carriers through ten thousand cancel and resume cycles (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). If any answer is no, stop feature work and return to superoptimize or descriptor design (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Runtime tuning knobs cannot fix illegal fusion or missing manifest fields (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Runtime ownership should appear on the org chart, not only in architecture slides. Layer five engineers maintain launch descriptor schemas, pool allocator invariants, and graph capture bucket tables that framework teams consume as stable ABI. When paging block size changes in vLLM, runtime must publish a semver bump before framework plugins ship; when superoptimize emits a new softmax carrier layout, runtime validates manifests at load time rather than discovering divergence during golden token tests. On-call runbooks should list the three counter worksheets from Chapter one beside PK warmup steps so incidents do not devolve into guessing whether regression lives in search, runtime, or scheduler batching.

Production fleets rarely fail because a single kernel is slow; they fail because two replicas run different artifact digests, because cancel paths leak pool slots

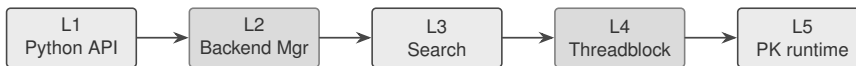


Figure 29.1: YiRage five-layer stack: Layers 1–4 search and lower; Layer 5 Persistent Kernel runtime executes decode cells on device.

under continuous batching, or because a canary node reports the wrong chip id to the registry. Layer five observability therefore emphasizes per-step JSON logs with fingerprint, registry id, descriptor version, launches count, and byte estimate aggregated into dashboards that framework SREs can read without reading C++. Postmortems should replay ten thousand decode steps from a single request id and verify pool high-water marks return to baseline after cancel. Treat runtime regressions as release train defects: roll back the artifact bundle, not only the framework container image.

29.1 yirage stack layers

YiRage stack layers follow the five-layer model in Table 29.1: Python API → Backend Manager → Search/Strategy → Threadblock ops → **Persistent Kernel runtime** (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Parts IV–VI focused on compiler passes and IR legality; Part VIII completes the stack with execution and device memory management (Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Chen, 2020).

Layer boundaries and debugging. Production incidents often misattribute Layer 3 search regressions to Layer 5 runtime or vice versa (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). When bytes/token regress, diff the fused μ Graph fingerprint first (Layer 3 output); when launches/token regress, inspect PK launch descriptors and graph capture buckets (Layer 5) (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Layer 2 must reject cross-backend artifacts silently loaded from cache—a Hopper-tuned plan must not execute on XDNA without re-search (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Industrial teams should staff Layer 5 with engineers who read nsys and who un-

Table 29.1: YiRage layer responsibilities and decode goodput levers.

Layer	Component	Primary lever on decode
L1	Python <code>graph</code> , <code>PersistentKernel</code>	User-facing API; mode and backend selection (Chen and YiRage Contributors, 2026; Chen, 2020)
L2	Backend Manager	Routes CUDA/CPU/NPU backends; same-backend enforcement (Chen and YiRage Contributors, 2026; AMD, 2024b,a)
L3	Search / <code>superoptimize</code>	Fusion plans, tile configs, fingerprint cache (Chen and YiRage Contributors, 2026; Vellaisamy <i>et al.</i> , 2026; Chen, 2026a)
L4	Threadblock ops	Lowered primitives before PK packaging (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024)
L5	PK runtime	Launch collapse, device memory pools, graph replay hooks (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2026a; Davies <i>et al.</i> , 2025)

derstand paging ABIs from Chapter 28 (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler teams own Layers 1–4 artifacts; runtime teams own launch stability, pool lifetimes, and graph replay (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When both groups share one on-call rotation, counter worksheets prevent thrash: if launches/token improved but bytes/token worsened, rollback Layer 3; if both stalled, inspect descriptor versions at the framework boundary (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Layer 3 logs should include enough context to reproduce PK configs without re-running full search (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Threadblock lowering is the last human-readable checkpoint before opaque device code (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When numerics diverge, compare threadblock dumps against eager reference at the same paging layout (Kwon *et al.*,

2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Do not tune softmax epsilons in runtime unless threadblock IR confirms carrier layout (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). This discipline keeps runtime hotfixes from masking compiler legality bugs (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Python API surface. Layer 1 exposes `yirage` as `yr`, `yr.new_graph()`, tensor helpers, and `graph.superoptimize(backend=...)` entry points documented in upstream `docs/API_REFERENCE.md` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Application code should treat graphs as immutable after `superoptimize`: runtime consumes frozen configs (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Mixing eager edits post-search breaks fingerprint caches and framework plugin contracts from Chapter 28 (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Backend Manager routing. Layer 2 selects installed backends (CUDA, CPU, NPU) and validates build flags from native extensions (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Import failures here surface as missing `yirage.core` or `libyirage_runtime`—not as silent CPU fallbacks in production (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). CI should compile each target backend separately; triplet tests compare counters, not import success alone (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Search versus runtime split. Layer 3 explores fusion and tiling; Layer 5 assumes a fixed plan (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Autotune during serving is an anti-pattern—it destroys graph capture stability and p99 latency (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Offline `superoptimize` plus pinned artifacts is the production pattern (Chen

and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Layer 4 threadblock ops are the last IR-like surface before opaque device binaries—debug dumps here when PK numerics diverge from eager reference (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Observability per layer. Layer 1 logs Python call sites; Layer 5 emits device timestamps and byte counters (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Correlate layer logs with framework traces from Chapter 28 using shared request ids (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Without correlation, runtime teams optimize kernels while queue time dominates p99 (Davies *et al.*, 2025; Chen, 2026a; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Export a single JSON line per decode step containing layer id, registry id, artifact fingerprint, launches count, and byte estimate (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Aggregates belong in fleet dashboards; per step JSON enables postmortems (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teach on-call engineers to filter on fingerprint first, registry id second, and framework version third (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Most regressions are mismatched artifacts, not mysterious hardware noise (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Layer handoffs deserve explicit interface tests in CI, not tribal knowledge. After superoptimize completes, assert that Layer four threadblock dumps include every carrier field the Layer five loader expects. After PersistentKernel construction, assert that warmup completes within a bounded step count and that steady decode reuses the same pool handles across one thousand tokens. When Layer two rejects a backend, the error message should name the missing native extension so developers do not confuse import success with PK readiness. Stack diagrams in documentation are useful, but automated boundary tests prevent silent drift when upstream YiRage refactors internal module names while preserving Python API

spelling.

Multi-hardware delta. Layer 4–5 differ sharply: CUDA PK uses graph capture and TMA-aware pools; XDNA uses static AIE binaries; CPU uses vectorized host loops (AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Same Python API must not imply identical launch semantics (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 29.1.1. Search on laptop CPU then deploy CUDA PK without re-superoptimize: numerics may match while launches/token and bytes/token diverge—violates same-backend rule (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Goodfellow *et al.*, 2016).

Worked layer walkthrough. Start from a minimal decode graph in Layer 1 and execute superoptimize on the fleet GPU (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Inspect Layer 3 logs for chosen fusion boundaries and tile sizes, then dump Layer 4 threadblock lowering for the attention seam (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Instantiate PersistentKernel in decode mode and run ten tokens at context 512, 2048, and 8192 (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Record launches per token, HBM bytes per token, and wall time per token separately for each bucket (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Only after Layer 5 counters flatten should you integrate the framework plugin from Chapter 28 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Skipping the isolated PK microbench hides descriptor bugs that become production incidents under continuous batching (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

29.2 persistent kernel decode

Persistent kernel decode uses `yr.PersistentKernel(mode="decode", backend=..., fallback_backends=[...])` to select backend execution with



Figure 29.2: Persistent kernel decode path: framework passes slot and page metadata; PK runtime launches fused decode cells with collapsed host overhead.

graceful CPU/NPU fallbacks (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022). The `persistentkernel` object wraps a long-lived device program: internal loops over decoder layers replace per-op host launches (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). `mode="decode"` selects shape buckets and numerics carriers tuned for BS=1 token steps rather than prefill-only tiles (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Same-backend rule. Search and execute on the installed target—do not profile on CPU and ship CUDA kernels without re-search (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; AMD, 2024b,a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fleet labels (GPU SKU, driver, CUDA version) must match `HardwareRegistry` entries used during `superoptimize` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Warmup passes prime PK device memory pools and graph capture—cold first token latency is a runtime concern distinct from compiler fusion quality (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Decode PK differs from training kernels in occupancy goals (Davies *et al.*, 2025; Dao *et al.*, 2022; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Training chases large batch matmul utilization; decode chases minimal bytes per token at batch one (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Persistent loops amortize layer weights across tokens inside one launch, but only if softmax and norm carriers remain on chip between layers (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Profile carrier lifetimes explicitly when PK shows

unexpected HBM traffic (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Cancellation and mid-sequence abort must release PK pool slots without leaking device memory (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Frameworks free paging blocks on cancel; runtime must mirror that lifecycle in pool accounting (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Long-running servers accumulate leaks if cancel paths skip pool teardown (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Stress tests should randomize cancel timing under continuous batching before fleet promotion (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Fallback backends. `fallback_backends` enable dev laptops and CI smoke tests without forking model code (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Production must pin primary backend to fleet SKU; fallbacks are for correctness checks, not performance baselines (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Accidentally serving through CPU fallback while GPU PK is available reads as mysterious fleet cost overrun (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Device memory pools. Layer 5 allocates persistent buffers for weights, KV views, and softmax carriers across decode steps (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework paging passes block table pointers into PK launch descriptors—runtime must not copy KV to contiguous staging each token (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Silent staging copies show up as bytes/token regressions with unchanged fusion plan (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Launch descriptor contract. Each decode step receives slot mask, page table

handle, step id, and sampling flags from the framework plugin (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). PK runtime validates descriptor version before launch—version skew must hard-fail, not produce wrong tokens (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Golden-token tests compare eager reference against PK for each descriptor version bump (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Prefill versus decode PK modes. `mode="decode"` selects persistent loops tuned for single-token advance; prefill may use separate superoptimize profiles (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Mixing prefill tiles in decode PK wastes bandwidth—tag modes explicitly in artifact manifests (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Framework schedulers call the correct PK mode per engine iteration (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Shape buckets for decode should align with framework max batch width and max context policies (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Each bucket needs a warmed PK instance and a graph capture template (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Missing buckets fall back to slow paths that show up as tail latency spikes rather than mean regression (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Document bucket coverage in runbooks beside paging block size (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Persistent kernel tuning is a memory lifetime problem disguised as a launch problem. Each decoder layer expects weight tensors, norm statistics, and attention softmax carriers to arrive at fixed addresses across steps so the device loop can branch minimally. When runtime allocates fresh staging buffers per token, bytes per token rises even though TaxBreak launch counts look excellent. Profile with

memory transaction counters and verify that page table pointers from the framework pass straight into gather kernels without intermediate copies. When carriers spill to high bandwidth memory between layers, return to Layer three fusion legality before widening graph capture buckets.

Stress decode under adversarial scheduler patterns: alternating prefill and decode on the same slot, random cancels at ten percent of steps, and context growth from five hundred twelve to eight thousand one hundred ninety two within one session. PK pools must survive these patterns without monotonic growth in reserved bytes. Framework plugins should not assume PK warmup happens once per process lifetime if workers restart frequently in Kubernetes; document whether warmup is idempotent and how long it blocks readiness probes. Serving teams often tune liveness thresholds without accounting for first capture latency, then blame runtime for pod churn that is actually missing warmup in the probe design.

Multi-hardware delta. CUDA PK pairs with CUDA Graph buckets; XDNA PK loads AIE binaries with static DMA chains; CPU PK uses OpenMP-blocked loops (NVIDIA, 2024; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Identical Python `persistentkernel` constructor arguments may map to different launch paths (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024a,b; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Example 29.2.1. Decode cell (BS=1, ctx=2048): PK launch drops launches/token from TaxBreak hundreds to single digits while framework paging unchanged—runtime owns launch collapse, not batching (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; NVIDIA, 2024; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

PK versus CUDA graphs alone. Memory-Floor style graph replay removes host launch storms but cannot fix illegal fusion (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). PersistentKernel combines graph stability with compiler chosen fusion inside the replayed region (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Teams seeing graph wins without byte wins should escalate to superoptimize rather than adding more graph buckets (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*,

2023; Goodfellow *et al.*, 2016). PK is not a replacement for paging or batching; it replaces per op launches inside the decode cell (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

MoE and expert parallel hooks. Large MoE models add expert parallel communication below the PK entry point (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Open DeepEP components illustrate all to all patterns that must compose with PK memory pools rather than allocate per token (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Profile EP separately from PK fusion when bytes/token regress on MoE checkpoints (Liu *et al.*, 2024; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

29.3 superoptimize handoff

Superoptimize handoff bridges `graph.superoptimize(backend=...)` outputs to runtime launch: fused μ Graphs, fingerprint verification, and cached kernel configs (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; NVIDIA, 2022, 2024; Dao *et al.*, 2023; Goodfellow *et al.*, 2016). Layer 3 search emits a serialized plan; Layer 5 loads it into PK launch templates (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework hooks (Chapter 28) must not strip metadata required for capture or MegaKernel legality (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Handoff is where many projects stall: search finishes, a demo looks fast, but no artifact pipeline reaches serving (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Treat superoptimize output like a container image: digest, manifest, signature, and rollback tag (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework plugins should load by digest at process start, not by mutable local paths (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Canary deploys swap digest on

a fraction of replicas while comparing counter dashboards (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Without digest discipline, two replicas silently run different PK programs under the same endpoint (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Fingerprints and cache keys. Superoptimize caches results keyed by IR fingerprint plus backend id and hardware registry hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework upgrades that change page-table ABI must bump cache salts or risk loading incompatible PK binaries (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). CI should verify fingerprint mismatch fails closed—never silently fall back to unfused eager (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Graph to kernel packaging. Threadblock lowering produces device-specific binaries or CUDA C++ sources consumed by PK runtime (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Handoff artifacts include tile sizes, double-buffer depth, and softmax carrier layout (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Missing carrier metadata passes unit tests on square tensors and fails on paged decode (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016).

Warmup and steady state. First PK launch may compile, allocate pools, and capture graphs; steady decode reuses them (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Fleet benchmarks must separate warmup from sustained tokens/s (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Production SLOs often ignore

first-token compile; decode p99 must use warmed PK state (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Artifact storage layout. Store superoptimize outputs as versioned bundles: fingerprint, registry id, backend, blob, and manifest JSON (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Framework plugins fetch bundles by hash at startup—avoid embedding blobs inside Python wheels without hash checks (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Rollback is redeploy prior bundle, not revert framework scheduler alone (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Legality metadata handoff. Export softmax carriers, page-table layouts, and graph-capture flags as manifest fields (Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Runtime refuses launch if manifest missing required fields—fail closed (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). This prevents partial exports from Chapter 28 plugins (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Manifest evolution requires semver: breaking carrier layout bumps major version; additive fields bump minor (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework plugins declare supported manifest major before loading artifacts (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Mixed major versions across replicas is a deployment error, not a runtime tuning opportunity (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Chapter 30 runbooks formalize this three party release train (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Artifact promotion should resemble a binary signing pipeline. Search cluster produces a bundle; staging loads it read only; golden token harness compares hashes against eager reference on three context buckets; production receives a

digest pin in the framework config. Never promote because a demo video looked fast on one laptop. Store the promotion checklist JSON beside the bundle so auditors can reconstruct who signed off on launches per token and bytes per token deltas. When a framework upgrade changes only scheduler policy but not paging ABI, artifacts may carry forward; when block table layout changes, invalidate caches even if IR fingerprint unchanged. Handoff meetings should read manifest fields aloud until boring repetition makes missing fields impossible.

Offline search and online runtime must share a single hardware registry revision checked into version control. Drift between search cluster registry and serving cluster labels produces plans that are legal yet slow, which is harder to detect than hard failures because tokens still look plausible. Automate a nightly job that compares detect current chip output on every node class against expected registry ids and pages when mismatches exceed zero. Superoptimize caches keyed only on IR miss these failures until bytes per token creep upward over weeks.

Multi-hardware delta. Cache entries are not portable across backends—re-superoptimize per target even when IR matches (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Example 29.3.1. Fingerprint hit loads cached PK plan; framework block size change without re-search yields wrong KV gathers—cache must invalidate on paging ABI change (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Handoff checklist before fleet deploy. Verify manifest lists backend id, registry id, fingerprint, paging ABI version, and softmax carrier schema (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Run golden tokens on three context buckets with warmed PK (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Compare launches/token and bytes/token against prior artifact in the same dashboard row (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Sign off only when both counters move or regressions are explained with profiler evidence (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen,

2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Store the checklist JSON beside the artifact bundle for audit (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

29.4 hw registry runtime

Hw registry runtime (`HardwareRegistry`, `detect_current_chip()`) supplies chip models for search configs—Hopper vs XDNA vs M3 Max tile candidates (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024). The `hardwareregistry` maps detected devices to search spaces: SM count, TMA availability, L2 size, NPU tile SRAM, CPU vector width (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). `detect_current_chip()` runs at import or first superoptimize—runtime detection must align with fleet labels used in framework schedulers (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024a,b; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Search space pruning. Registry entries prune illegal tile shapes before auto-tune explosion (Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; AMD, 2024b,a; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2022, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Hopper profiles enable TMA double-buffer candidates; XDNA profiles restrict to static DMA chains (NVIDIA, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Wrong chip detection yields legal-but-slow plans—profile bytes/token on representative ctx buckets (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Registry entries should be versioned documents checked into the same repo as artifact manifests (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When hardware vendors ship new SKUs, add entries before marketing promises fleet migration (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Each

entry lists peak bandwidth, on-chip capacities, and forbidden fusion patterns for that target (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Search reads these as hard constraints, not hints (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Detect current chip at process start and log the resolved registry id in every performance trace (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Post-incident review should filter traces by registry id to spot partial rollouts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Mixed ids behind one load balancer explain bimodal latency histograms (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Registry maintenance is a hardware catalog discipline, not a one time bootstrap script. Each entry documents peak bandwidth, on chip SRAM, vector width, tensor engine availability, and forbidden fusion patterns for that SKU. When vendors ship new accelerators, add entries before marketing announces fleet migration dates. Search reads entries as hard constraints; runtime logs the resolved id on every decode step so capacity planners can correlate artifact lines with instance types. Retire entries only after all pinned artifacts migrate and dashboards show zero traffic on the old id for two release cycles.

Virtualized and partitioned GPUs distort detection unless runbooks override generic ids with fleet labels from node metadata. Multi instance GPU slices may report parent SKU names that mislead tile search toward shapes that fit the parent but not the slice memory budget. Time shared virtual machines may hide NPU presence entirely until first superoptimize call. Document override procedures and require platform teams to validate overrides quarterly against vendor tools output.

Fleet inventory sync. Kubernetes node labels, cloud instance types, and driver versions should map to registry ids (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Mismatch between scheduler placement and PK chip model causes silent suboptimal tiles (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Pin superoptimize artifacts per registry id in artifact stores (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Multi-SKU rollouts. Rolling PK updates across heterogeneous fleets requires parallel artifact lines per chip entry (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Never reuse Hopper PK binaries on Blackwell without re-search even when CUDA version matches (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Registry-driven autotune budgets. Cap search trials per chip model to keep CI bounded (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Registry metadata feeds roofline estimates that prune hopeless tile shapes before device trials (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Log pruned candidates for audit when bytes/token underperform expectations (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Virtualization pitfalls. Containers sharing GPUs must expose consistent chip ids to `detect_current_chip` (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). MIG slices and time-sliced VMs may report generic ids—override with fleet labels when needed (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Multi-hardware delta. `detect_current_chip` on VM passthrough GPU may report wrong SKU—validate against `nvidia-smi` product name in runbooks (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; NVIDIA, 2022; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022;

AMD, 2024a,b; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 29.4.1. Fleet moves from H100 to H200: rerun superoptimize with updated registry entry; reuse old fingerprint cache caused 8% bytes/token regression until invalidated (Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; SemiAnalysis, 2024; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Registry operations runbook. Onboard a new GPU SKU by adding a registry entry, running a bounded superoptimize search, and publishing artifact bundles tagged with the new id (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Update Kubernetes node labels to match registry ids before shifting traffic (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). During incidents, compare `detect_current_chip` output with node labels before blaming framework schedulers (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Retire old entries only after all pinned artifacts migrate (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

29.5 deps build native

Deps build native: `import yirage` requires `yirage.core + libyirage_runtime`—no Python-only native build path (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Submodule init: `git submodule update -init -recursive deps/YiRage`; build via `YIRAGE_BACKEND=cuda pip install -e deps/YiRage` (see upstream docs/INSTALLATION.md) (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Native extension layout. Python package wraps C++ runtime under `deps/YiRage/yirage/` with compiled `yirage.core` module linking `libyirage_runtime` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Missing `.so` files import as pure Python stubs that fail at first PK call—smoke tests must invoke `PersistentKernel`, not only `import`

`yirage` (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Building from source ties book examples to reproducible hashes (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Pin submodule commit in release notes whenever prose references API symbols (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Nested submodules may require recursive init; document network and disk expectations in deps README (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Agents editing prose should not hand modify generated native sources; changes belong upstream in YiRage (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Wheel-based installs are convenient for demos but hide compiler flags (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Production should prefer editable installs from pinned submodule with recorded `nvcc` and driver versions (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When wheels and submodule builds diverge, counter worksheets become incomparable across environments (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Backend build flags. `YIRAGE_BACKEND=cuda|cpu|npu` selects compile targets; nested submodules may pull vendor SDKs (Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Docker images for serving should bake the same backend flags as search CI (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Drift between build and runtime containers is a common PK load failure (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Version pin with book repo. Pin submodule SHA in `.gitmodules` or recorded commit; Chapter 30 runbooks require matching `deps/YiRage` revision with framework plugin ABI (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Book examples reference APIs at pinned SHA—update prose when bumping submodule

(Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Developer workflow. Typical loop: edit IR or search config → **superoptimize** on target SKU → run PK decode microbench → export artifact hash for framework plugin (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Skipping native rebuild after C++ runtime change produces stale .so mismatches (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

CI smoke template. Minimal pipeline: submodule init, backend-specific compile, import yirage, superoptimize tiny graph, PersistentKernel decode ten tokens, assert golden hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Extend with BS=1 counter worksheet from Chapter 1 before promoting artifacts to staging (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Docker and reproducibility. Lock CUDA driver, toolkit, and YiRage SHA in serving images (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Non-reproducible PK builds frustrate bisect when fleet regressions appear (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Record compiler flags in artifact manifest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Serving images should compile YiRage once in the build stage and copy artifacts into the runtime stage (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Runtime stage installs matching driver user space libraries only; never compile on production pods (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Init containers can verify artifact digest against manifest before accepting traffic (Chen

and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). This pattern prevents silent drift between search cluster and serving cluster (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Native build failures often masquerade as Python packaging issues. Missing `nvcc`, wrong driver user space libraries, or stale build directories produce import success followed by opaque failures at first `PersistentKernel` call. CI must execute the full smoke template on the same backend flag as production containers, including ten token decode and golden hash comparison. Developers on laptops without GPUs may compile CPU backend for unit tests but must never promote CPU searched artifacts to CUDA serving. Record compiler flags, submodule SHA, and backend enum in artifact manifests so `bisect` can distinguish runtime code changes from toolchain drift.

Book readers reproducing examples should init the submodule recursively, install editable with explicit backend flag, and run PK decode microbench before reporting API mismatches. Agents editing prose should not hand modify generated native sources; changes belong upstream in YiRage with submodule bump and manifest semver bump in this repository. When documentation and profiler traces disagree, trust traces and file issues against runtime launch paths.

Multi-hardware delta. CUDA builds need compatible `nvcc` and driver; XDNA builds need Ryzen AI SDK paths documented upstream (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). CI matrices should include at least one compile plus PK smoke per supported backend (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Example 29.5.1. `pip install -e deps/YiRage` on dev laptop without CUDA toolkit: import succeeds, PK decode fails at runtime—gate merges on PK smoke, not import (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Local dev versus fleet parity. Developers may compile CPU backend locally for unit tests while fleet runs CUDA PK (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016). Never promote CPU searched artifacts to GPU serving (Chen and YiRage Contribu-

tors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Provide a small GPU CI runner that executes the same smoke template as production (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Document driver and toolkit versions in deps README when bumping submodule SHA (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Book readers should clone submodule recursively before reporting API mismatches (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

29.6 YiRage runtime decode benchmarking methodology

Benchmarking Layer 5 decode extends Chapter 1 counters with runtime-specific ledgers: PK warmup steps, pool high-water marks, graph capture bucket hits, and artifact fingerprint stability across ctx buckets (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Goodfellow *et al.*, 2016). Build a decode cell with BS=1, ctx {512, 2048, 8192}, pinned **deps/YiRage** SHA, matching registry id, and framework paging ABI from Chapter 28 (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Warm-up must include PersistentKernel construction, first graph capture, pool allocator steady state, and at least one thousand steady decode steps—cold import success alone is not a valid production benchmark (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Report first-token latency and token-*k* steady ms/token separately; product SLOs often bind to the latter while UX marketing highlights the former (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Triplet runs should compare eager framework decode, PK-only microbench, and full framework plugin path on the same SKU (Chen and YiRage Contributors,

Metric	Layer 5 decode meaning
Launches/token	PK + graph replay vs eager baseline
Bytes/token	Pool residency + paging traffic
Pool high-water	Leaks under cancel/resume
Fingerprint match	Artifact drift across replicas
Registry id	SKU-aligned search vs runtime
Graph bucket hit rate	Capture stability under ctx growth

Table 29.2: Runtime decode counter worksheet (extends Chapter 1 TaxBreak / Memory-Floor).

2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). A PK win that disappears after plugin integration signals descriptor ABI mismatch, not “framework overhead” (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Example 29.6.1. Runtime decode gate: warmed PK at ctx=2048 must show launches/token within 5% of search logs *and* bytes/token within 10% of Memory-Floor worksheet; attach fingerprint, registry id, and ten-step pool ledger (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Cancel and resume stress. Continuous batching injects cancel paths that microbenches skip (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Benchmark ten thousand decode steps with randomized cancel/resume at 1% rate; pool high-water must return to baseline within bounded slots (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Fail CI when cancel paths allocate fresh HBM per step (Chen, 2026a; Chen and YiRage Contributors,

2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Artifact pinning in CI. Nightly jobs must load the same artifact bundle as staging: fingerprint, manifest semver, compiler flags, and submodule SHA (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Drift between search cluster artifacts and serving replicas explains bimodal latency histograms better than kernel tuning (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Publish golden hashes and pool ledgers beside every Layer 5 release note in CI (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Framework boundary replay. After PK microbench passes, replay the same counters through the Chapter 28 plugin with identical paging block size and request ids (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Deltas here isolate scheduler tax from runtime tax (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Never promote PK artifacts to staging or production without this second worksheet (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Multi-SKU fleet sweeps. Heterogeneous fleets need parallel benchmark matrices keyed by registry id—reuse Hopper numbers on Blackwell only after re-search and re-warmup (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; AMD, 2024b,a; Vellaisamy *et al.*,

2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Dashboards should slice launches/token and bytes/token by registry id, not aggregate fleet averages (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Operator documentation. Ship runtime benchmark artifacts with every promotion: ctx buckets, warmup protocol, pool ledger, graph bucket table, and triplet comparison summary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Without published ledgers, on-call engineers cannot distinguish artifact drift from framework queueing (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Release checklist. Before tagging a Layer 5 release, require green PK smoke, cancel stress, plugin replay, and fingerprint match across three ctx lengths (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Partial green is how production inherits week-long triage across runtime and framework teams (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Observability hooks. Emit structured logs at PK load, warmup complete, steady decode entry, and graph capture miss events (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Production builds strip verbose fields but retain stable event names so SRE dashboards correlate runtime regressions with artifact promotions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen,

2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Long-context stress exports. Extend runtime benchmarks to ctx=16384 when product SLAs allow paging beyond 8K (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Graph capture buckets tuned at 2K often miss at 32K—fail CI when bucket hit rate drops below threshold at long ctx (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Speculative decode at runtime. Draft-verify stacks double PK instances; benchmark acceptance-rate sweeps with separate pool ledgers per model (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Alias bugs between draft and verify KV pools show up only under load—include them in cancel stress matrices (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Closing note. Treat Layer 5 decode benchmarking as a shipping contract alongside native builds—without nightly PK replay and published pool ledgers, every latency spike becomes a costly and painfully slow multi-team blame game (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Cross-stack triage playbook. When decode latency spikes after a runtime merge, run this sequence before blaming silicon: (1) compare artifact fingerprints across replicas; (2) verify registry id matches node labels; (3) replay pool ledger on

golden PK export; (4) diff launches/token and bytes/token against last green worksheet; (5) only then sweep driver and power caps (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Skipping fingerprint checks wastes days chasing framework queue noise (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Efficiency metric. Track *runtime-attributed decode efficiency*: measured tokens/s divided by roofline tokens/s after subtracting framework scheduler overhead measured at the plugin boundary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Runtime teams own the PK numerator; framework teams own scheduler tax in the denominator—publish both weekly (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Native build parity in benchmarks. Benchmark jobs must compile deps/YiRage with the same YIRAGE_BACKEND flag as serving containers (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). CPU-backend smoke tests are valid for API drift but must never gate GPU artifact promotion (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Record nvcc, driver, and submodule SHA in every benchmark manifest (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Handoff to Chapter 30. The runtime worksheet completes when warmed PK counters, plugin replay deltas, and manifest hashes are pinned for the production workflow in the next chapter (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Missing any field forces Chapter 30 integration meetings to guess Layer 5 state (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Regression harness cadence. Check golden PK decode scripts into the book repo beside static graph exports: same tokenizer, ctx buckets, pool counters, and fingerprint assertions every nightly CI run (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Submodule bumps that change descriptor layout must fail CI before prose updates merge (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Documentation contract. Every Layer 5 promotion note must list ctx buckets, warmup steps, steady ms/token, launches/token, bytes/token, and golden fingerprint at ctx=2048 (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Partners cannot replay regressions without these fields—support cycles lengthen when benchmarks stay in private spreadsheets instead of versioned artifact stores (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

Summary. Layer 5 benchmarks are complete when PK, plugin, and cancel-stress worksheets agree within published tolerances on all three ctx buckets—anything

else is an integration draft, not a shippable production fleet release candidate (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016).

29.7 Key Takeaways

- **The runtime is Layer 5, distinct from the compiler.** Compiler passes from Parts VI–VII emit fused plans, but the runtime layer owns device-memory discipline, launch templates, and backend selection so the plan actually executes within budget (Chen and YiRage Contributors, 2026; Chen, 2020). Treating runtime as an afterthought is how a correctly-compiled megakernel still misses p99 when the host reconstructs state every token (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).
- **Same-backend search.** Profile and ship on the target SKU; `hardwareregistry` must match fleet labels (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a).
- **Handoff is contractual.** Fingerprints, page-table ABI, and softmax carriers must survive `superoptimize` to PK launch (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026).
- **The submodule is the source of truth.** The runtime depends on the native `deps/YiRage` build, so CI must compile that submodule and smoke-test a Persistent Kernel decode rather than testing only the Python import (Chen and YiRage Contributors, 2026; Chen, 2020). An import-only test passes while the native runtime is broken, which is precisely the failure that reaches production undetected (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).
- **Counters at runtime.** Measure launches/token and bytes/token after PK warmup, not only import or search logs (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016).

29.8 Conclusion

YiRage runtime closes the gap between fused IR and fleet decode: Layer 5 `persistentkernel` programs collapse launches/token; device memory pools and graph hooks address bytes/token and orchestration tax (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Chen, 2020; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Treat `superoptimize` artifacts, registry ids, and native builds as release-train inputs alongside framework paging policy (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Before Chapter 30, complete one Layer 5 worksheet: warmed PK, three context buckets, launches and bytes per token logged, manifest hash pinned (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). Chapter 30 integrates this layer with vLLM-class frameworks and Chapter 12 execution modes into one production workflow (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Chapter 30

Framework, Compiler, and Runtime Co-Design for Decode Goodput

Production decode is a *three-way* contract: the inference **framework** schedules requests and owns KV paging; the **compiler** (YiRage middle-end + `superoptimize`) owns intra-step fusion and layout; the **runtime** (Persistent Kernel Layer 5) owns launches, device memory, and backend fallbacks (Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). When p99 latency stalls, teams that optimize only one leg recreate Chapter 1’s framework fragmentation at warehouse scale (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Part VIII closes by making that contract explicit: every integration meeting draws the decode step **boundary** first, then assigns owners and counters (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Chapter 28 mapped framework buckets and scheduler policy; Chapter 29 descended into YiRage Layer 5 Persistent Kernel runtime (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Goodfellow *et al.*, 2016). This chapter stitches the three parties into one production workflow: who owns prefill versus decode handoff, how vLLM paging composes with YiRage fused cores, which execution mode applies per subgraph on a heterogeneous fleet, and how regression

gates prevent silent drift across submodule bumps (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). The book’s industrial arc ends here—not with another isolated kernel trick, but with deployable co-design anchored on decode goodput counters (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Readers should leave with a release train mental model. Framework teams ship scheduler and paging policy; compiler teams ship fused plans and manifest schemas; runtime teams ship PK launch descriptors and pool invariants (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). No party optimizes in isolation: a framework upgrade that changes block table layout without bumping artifact cache salts breaks compiler handoff; a compiler win that ignores paging ABI wastes bytes per token; a runtime regression that leaks pool slots masquerades as scheduler overload (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Co-design means shared worksheets, shared request ids in traces, and shared rollback pins—not shared blame in postmortems (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Fregly mechanical sympathy at the three-way boundary treats each handoff as an API with semver, golden tests, and profiler evidence (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Dao *et al.*, 2022; NVIDIA, 2024; Goodfellow *et al.*, 2016). Chapter 12 execution modes reappear here as fleet policy: eager hulls for sampling branches, CUDA Graph buckets for launch-bound cells, MegaKernel and Persistent Kernel paths for byte-bound decoder cores (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). DeepSeek-class disaggregation adds a fourth axis—prefill and decode pool routing—but the same boundary discipline applies inside each pool’s decode step (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Three questions gate every co-design review before fleet promotion. First, does the responsible party for the moving counter actually own the lever being tuned (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Second, do handoff

manifests survive framework, compiler, and runtime version bumps without silent fallback (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Third, does triplet regression on GPU, CPU, and NPU backends show bytes per token and launches per token moving together or explained separately (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; Goodfellow *et al.*, 2016). If any answer is no, pause feature work and fix the contract before tuning batch windows (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

This closing chapter is deliberately operational. Earlier parts taught rooflines, fusion legality, dialect lowering, and autotune search (Chen, 2020; Chen and YiRage Contributors, 2026; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Part VIII asked where those ideas attach in servers humans operate (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). The answer is never a wholesale framework replacement—it is a negotiated boundary with versioned handoffs and regression gates (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teams that skip co-design and chase isolated kernel records on leaderboard shapes reproduce the fragmentation Chapter 1 diagnosed (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Co-design is the antidote measured in the same counters from prompt to token (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Warehouse-scale serving adds constraints single-GPU labs hide (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Continuous batching interleaves unrelated clients; cancel paths free slots unpredictably; disaggregated pools move KV across networks (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler and runtime optimizations must tolerate that chaos without leaking memory or violating paging ABI (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Framework optimizations must expose enough telemetry for compiler teams to see in-step counters, not only fleet aggregates (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Co-design is the discipline of making those telemetry paths first-class product requirements (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).



Figure 30.1: Responsibility split: framework, compiler, and runtime meet at the decode boundary; counters attach per party.

Leadership reviews should ask which party moved which counter last quarter, not which kernel microbenchmark improved (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Roadmaps that fund framework features without compiler handoff metadata or runtime descriptor tests recreate integration debt (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Conversely, compiler teams that ship artifacts without framework plugin support plan fail at the boundary (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Co-design funding follows counter movement with named owners (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). This chapter gives vocabulary for those staffing and budget conversations (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Part VIII therefore completes the narrative arc from mindset and hardware bounds through compiler stacks to deployable serving (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Readers who implement only one chapter should still implement this one if they operate fleets (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Theory without co-design contracts becomes shelfware; serving without counters becomes guesswork (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The combined worksheet—boundary diagram, integration digest, mode map, triplet rows—is the minimum viable artifact set for production decode (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Goodfellow *et al.*, 2016).

30.1 responsibility split

Responsibility split is the operating table every production runbook must publish before on-call rotation (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026;

Table 30.1: Framework, compiler, and runtime responsibility split at the decode boundary.

Party	Owens	Primary counter
Framework	Batching, paging, routing, sampling policy, request lifecycle (Kwon <i>et al.</i> , 2023; Davies <i>et al.</i> , 2025)	Queue wait, fleet tokens/s, KV utilization
Compiler	Tiling, fusion, layout, legality, superoptimize artifacts (Chen and YiRage Contributors, 2026; Chen, 2020)	Bytes/token inside decode step
Runtime	PK launch, sync, pools, graph replay, backend registry (Chen and YiRage Contributors, 2026; NVIDIA, 2024)	Launches/token, warmup latency, pool high-water

Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Table 30.1 maps the three parties to concrete artifacts and counters. Ambiguity at the **boundary** between framework scheduling and compiler-owned decode cells is the dominant cause of misattributed regressions in fleet postmortems (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

The **framework** owns everything outside the decode step box: continuous batching, paged KV block tables, speculative decoding policy, disaggregated routing between prefill and decode pools, and client-facing SLO dashboards (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Framework engineers optimize queue depth, slot assignment, and memory capacity—not softmax carrier layout inside a fused attention kernel (Davies *et al.*, 2025; Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When fleet tokens per second rise but bytes per token inside the step is flat, the framework did its job; the compiler and runtime did not (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

The **compiler** owns intra-step fusion and residency: which operators merge, which tensors stay on chip between layers, which layouts match paging gathers, and which plans are legal for graph capture (Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Compiler output is not PyTorch source—it is versioned artifacts with fingerprints, manifest fields, and hardware registry ids from Chapter 29 (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework plugins consume those artifacts; they must not silently strip softmax carrier metadata or paging ABI version fields (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

The **runtime** executes compiler plans on device: Persistent Kernel loops, memory pool lifetimes, CUDA Graph capture buckets, backend fallbacks, and descriptor validation at each decode step (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Runtime engineers read nsys and pool accounting; they do not rewrite scheduler batching policy to mask launch storms (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). When launches per token regress after a runtime bump, bisect artifact digest and descriptor version before touching framework code (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Draw the decode **boundary** on every architecture diagram: from slot mask and block table handle ingress through final hidden state before sampling (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Inside the box, compiler and runtime own bytes per token and launches per token; outside, framework owns queue time and paging efficiency (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration meetings that skip the boundary redraw recreate Chapter 1 fragmentation: compiler teams tune GEMMs while scheduler queue time dominates p99 (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

On-call triage by party. Queue wait up, in-step ms flat: framework scheduling or capacity (Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). In-step bytes up, launches flat: compiler fusion or layout regression (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow

et al., 2016). Launches up, bytes flat: runtime graph capture or PK descriptor regression (Vellaisamy *et al.*, 2026; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). All three up: likely boundary mismatch—paging ABI changed without artifact invalidation (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Org design implications. Staff framework, compiler, and runtime teams with separate counter dashboards but shared release train (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Shared on-call without shared worksheets devolves into thrash: each party tunes its leg while p99 stalls (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Co-design standups read one row per request id: queue ms, in-step ms, launches, bytes, artifact digest (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Example 30.1.1. Regression after framework block size change: compiler cache hit loads stale PK plan—boundary contract requires cache salt bump on paging ABI change, not runtime knob tuning (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Industrial teams document the split in runbooks linked from fleet config repos (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). New hires should trace one decode token through framework slot assignment, compiler artifact load, runtime PK launch, and sampling egress before proposing optimizations (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). That trace is the co-design literacy test for Part VIII (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Release trains should version the responsibility table alongside framework, compiler, and runtime artifacts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). When a new feature spans two parties—for example speculative decoding in the framework plus extra PK launches in runtime—file a joint design doc with counter acceptance criteria before implementation (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Escalation paths should name DRI per party, not a

generic platform team that owns none of the levers (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Quarterly audits replay archived traces from peak traffic week and verify each party still matches the published table (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Contract tests between parties catch boundary drift early (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Framework publishes paging ABI fixtures; compiler asserts superoptimize output loads against fixtures; runtime asserts PK descriptors accept fixture block tables (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Breaking any contract test blocks merge until semver bump and coordinated release notes ship (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). This is how co-design scales beyond a single heroic integration engineer (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Worked boundary exercise. Pick one production model and one fleet SKU (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). List ten operations in decode order from scheduler slot assignment through sampling (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Mark each operation framework, compiler, or runtime owned (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Attach one counter per party (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Run one request through traces and verify marks match reality (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Mismatches indicate undocumented boundary creep—fix the runbook before optimizing (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Repeat after every major framework or submodule bump (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Vendor and open-source upgrades flow through the same table (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). CUDA driver updates may change graph capture behavior; vLLM releases may change block tables; YiRage bumps may change manifest majors (NVIDIA, 2024; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026;



Figure 30.2: Decode pipeline handoff: prefill may stay framework-orchestrated; decode enters compiler-owned cores; sampling remains an eager hull outside capture.

Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Assign each upgrade type a default owner and regression template so on-call does not debate responsibility during incidents (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). The responsibility split is living documentation, not a one-time slide (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

30.2 decode pipeline handoff

Decode pipeline handoff sequences three phases with different owners: **prefill** ingestion, per-token **decode** compute, and **sampling** egress (Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Liu *et al.*, 2024; NVIDIA, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; NVIDIA, 2022). Prefill often stays framework-orchestrated because shapes vary widely and batching policy dominates (Kwon *et al.*, 2023; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Decode steps enter compiler-owned cores when bytes per token and launches per token dominate at batch one (Chen, 2026a; Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Sampling and speculative branches remain *eager hulls* outside CUDA Graph capture per Chapter 12 legality rules (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Prefill handoff. Framework schedulers batch prompt tokens, allocate paging blocks, and may route long prompts to disaggregated prefill pools (Kwon *et al.*, 2023; Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler involvement during prefill varies: some stacks superoptimize prefill profiles separately from decode; others reuse decode tiles inefficiently (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022, 2023; Goodfellow *et al.*, 2016). Document which PK

mode the framework invokes at prefill-to-decode transition—mixing modes without manifest tags causes silent bandwidth waste (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Prefill completion must commit KV layout metadata the decode artifact expects: block table dimensions, head layout, and rope cache handles (Kwon *et al.*, 2023; Dao *et al.*, 2022; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Decode handoff. Each decode iteration passes slot mask, page table pointer, step id, and hidden state buffers into the compiler-owned cell (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Runtime validates descriptor version, launches PK or graph replay, and returns logits or hidden states to the framework (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Handoff latency includes framework Python overhead, plugin marshalling, and device sync—profile each separately before blaming fusion quality (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Continuous batching may swap slots mid-flight; decode artifacts must tolerate dynamic slot masks without re-capture on every step when graph mode is enabled (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016).

Sampling handoff. Top-k, top-p, temperature, and stop criteria execute outside captured regions (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Speculative decoding adds draft and verify loops that re-enter decode cores multiple times per user token (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Compiler plans must declare which tensors sampling reads and whether logits stay on device (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Framework policy changes to sampling should not require re-superoptimize unless carrier layout changes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Handoff contracts deserve semver and golden-token tests at each phase boundary (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). CI should run prefill-only, decode-only, and full generate paths with shared artifact digests (Chen and YiRage Contributors,

2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Fail closed when manifest major version mismatches—never silently fall back to eager per-op decode (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

DeepSeek-class **disaggregated** stacks extend handoff across network boundaries: prefill pools write KV to remote decode pools (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Bytes per token then includes transfer cost; compiler and runtime optimizations inside decode pools still matter but must compose with transfer scheduling (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Profile end-to-end goodput, not in-pool ms alone (Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Example 30.2.1. Prefill completes at `ctx=8192`; decode PK warmed only to `ctx=2048` bucket—tail latency spikes on long sessions until runtime bucket table matches framework max context policy (Kwon *et al.*, 2023; Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Multi-hardware delta. Prefill may use wide batch tiles on GPU while decode PK targets batch one; CPU and NPU backends need separate handoff manifests per Chapter 29 same-backend rule (AMD, 2024b,a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; NVIDIA, 2022; Dao *et al.*, 2022, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016).

Handoff latency budgets belong in SLO documents with separate allocations for framework marshalling, plugin sync, and device execution (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Teams that collapse these into one in-step millisecond metric misallocate optimization effort (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Log each segment with shared request id so postmortems identify whether regression entered at prefill commit, decode launch, or sampling return (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Long context sessions stress handoff most at prefill-to-decode transition when bucket tables must expand—warm expanded buckets during prefill completion, not on first decode token (Kwon *et al.*, 2023; Chen, 2026a; NVIDIA, 2024; Chen

and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Timing diagram discipline. Draw prefill, decode, and sampling phases on one timeline per request class (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Mark where KV becomes visible to decode workers in disaggregated setups (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Mark where logits cross back to framework for sampling (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Ambiguous timing diagrams produce integrations that double-copy KV or synchronize too aggressively (Kwon *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Review diagrams in design docs with the same rigor as API schemas (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Streaming responses add another handoff: partial tokens must egress while decode continues (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Framework streaming APIs should not force synchronous device sync each token unless product requires it (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compiler and runtime plans should declare which steps tolerate async logits delivery (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Profile streaming workloads separately from batch benchmark harnesses (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Tail latency under streaming often traces to handoff sync policy, not fusion quality (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

30.3 vllm yirage stack

Vllm yirage stack is the reference **integration** pattern for Part VIII: vLLM continuous batching and paged KV plus YiRage fused decode core invoked each step (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Measure queue wait and in-step counters separately—fleet tokens per second can rise while launches per token inside the step stays at

TaxBreak hundreds (Vellaisamy *et al.*, 2026; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020, 2026a; Goodfellow *et al.*, 2016). The stack does not replace vLLM; it shrinks the decode cell vLLM calls (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Integration architecture. Typical flow: vLLM scheduler assigns slots and block tables; a custom attention or decoder backend loads YiRage artifact by digest at worker start; each decode step marshals descriptors into PersistentKernel (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Plugin code lives at the framework boundary documented in Chapter 28 partial-export bucket (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). Keep plugin surface minimal: slot mask, page table handle, step id, artifact digest—not raw tensor graphs per request (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

vLLM responsibilities retained. Paging, continuous batching, speculative decoding orchestration, and disaggregated routing stay in vLLM (Kwon *et al.*, 2023; Davies *et al.*, 2025; Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). YiRage does not reimplement block allocators or queue policies (Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When KV utilization drops, tune framework batch windows before re-running superoptimize (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When bytes per token inside the step stalls, escalate to compiler and runtime (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

YiRage responsibilities absorbed. Fusion across attention, norm, and MLP seams; softmax carrier residency; PK launch collapse; graph capture buckets aligned to vLLM shape policies (Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Chen, 2020; NVIDIA, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Artifacts pin at worker boot from `deps/YiRage` submodule SHA recorded in fleet config (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Submodule bumps trigger regression triplet from Section 30.5 before traffic shift (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Integration testing template. Stand up vLLM worker with pinned artifact;

run BS=1 decode at ctx 512, 2048, 8192; log queue ms, in-step ms, launches per token, bytes per token (Kwon *et al.*, 2023; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Compare against vLLM plus FlashAttention baseline on same SKU (Dao *et al.*, 2022; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Promote only when both in-step counters improve or regressions are explained with profiler evidence (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Run cancel and resume stress under continuous batching before fleet promotion (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Open components under `deps/DeepEP` and `deps/FlashMLA` illustrate MoE and MLA patterns that must compose with vLLM paging rather than bypass it (Liu *et al.*, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Integration docs should show where expert parallel all-to-all sits relative to PK entry—below scheduler, adjacent to decode cell (Liu *et al.*, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Example 30.3.1. vLLM upgrade changes block table layout: YiRage cache hit serves stale gathers—integration contract requires artifact invalidation on paging ABI bump, not plugin hotfix alone (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Operational ownership. vLLM SREs own scheduler SLOs and paging capacity; YiRage owners own artifact pipeline and PK stability (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Shared dashboards correlate framework version, artifact digest, and runtime descriptor version per request id (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration failures at this boundary are the most common Part VIII incident class (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Rollout playbooks for vLLM plus YiRage integration should mirror compiler artifact promotion: dev cluster golden tokens, staging canary at five percent traffic, production shift with rollback digest pinned (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Document which vLLM worker flags enable the

custom backend and which remain framework defaults (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Training new operators on integration dashboards prevents conflating queue depth spikes with PK failures (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When disaggregated prefill is enabled, verify YiRage artifacts on decode pool workers separately from prefill pool configs (Liu *et al.*, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Plugin minimization principle. The vLLM integration surface should stay thin enough to audit in one code review (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Heavy logic in plugins duplicates compiler and runtime responsibilities and forks across teams (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Push fusion and residency decisions into superoptimize artifacts; push launch stability into Persistent Kernel runtime (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Plugins marshal descriptors and handle error propagation only (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). When plugins grow past that scope, refactor boundary before next fleet promotion (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Capacity planning for integrated stacks counts both framework KV footprint and runtime pool high-water marks (Kwon *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Scaling replicas without warming PK buckets on new nodes produces temporary tail spikes (Chen, 2026a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Autoscalers should call runtime warmup hooks before marking pods ready (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Integration runbooks document expected warmup duration per artifact digest and context bucket (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Ignoring warmup in capacity math causes false headroom during traffic ramps (Davies *et al.*, 2025; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

30.4 mode selection fleet

Mode selection fleet applies Chapter 12 execution modes at production scale: **eager** for debug and sampling hulls; **graph** for launch-bound decode cells; **megakernel** and Persistent Kernel paths for byte-bound fused cores—chosen per subgraph, not per monolithic binary (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022). Fleet configs bucket context lengths, batch widths, and backend SKUs consistently so capture templates and PK warmup tables match scheduler policies (Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Eager mode. Use for development, numerics bisect, and any branch outside graph legality—sampling, dynamic control flow, debugging hooks (NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Eager baselines establish golden tokens for PK and graph paths (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Never ship eager per-op decode at scale when TaxBreak launch counts exceed fleet budget (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Graph mode. CUDA Graph replay removes host launch storms when fusion depth is insufficient to reach MegaKernel but launches per token dominate (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Bucket graphs by shape signatures aligned to framework max batch and context policies (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Goodfellow *et al.*, 2016). Graph mode without byte wins indicates need for compiler fusion, not more buckets (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

MegaKernel and PK mode. Compiler-owned decode cores collapse layers inside Persistent Kernel loops, targeting bytes per token and residual launches (Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Combine with graph capture where legality allows—PK provides fusion; graphs stabilize host overhead (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Profile carrier lifetimes when bytes per token stall despite PK adoption (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016).

Fleet bucketing policy. Document which mode applies to each subgraph in fleet config: prefill profile, decode profile, sampling hull, MoE communication (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Goodfellow *et al.*, 2016). Heterogeneous fleets maintain parallel artifact lines per hardware registry id from Chapter 29 (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016). Never reuse Hopper PK binaries on Blackwell without re-search (Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2022; SemiAnalysis, 2024; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; Goodfellow *et al.*, 2016).

Mode selection meetings should attach counter worksheets: if launches per token move but bytes do not, stop adding graph buckets and fix fusion (Vellaisamy *et al.*, 2026; Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). If bytes move but queue wait dominates p99, return to framework scheduling (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Fleet-wide mode policy prevents per-team improvisation that breaks artifact compatibility (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Multi-hardware delta. Graph mode is CUDA-centric; XDNA fleets use static AIE binaries; CPU fleets use OpenMP-blocked eager or PK host loops (AMD, 2024b,a; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016; SemiAnalysis, 2024; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Same fleet config schema should encode backend-specific mode enums (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016).

Example 30.4.1. Team enables graph capture on entire decoder including sampling—illegal capture causes silent fallback to eager launches; mode map must exclude sampling hull (NVIDIA, 2024; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Fleet configuration management should treat mode maps as first-class config objects with review gates (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; NVIDIA, 2024; Goodfellow *et al.*, 2016). Changing a subgraph from graph to PK mode without re-warming pools causes cold-start p99 spikes (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Autoscaling policies must account for warmup time when mode maps include capture-heavy paths (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Document mode map version in artifact manifest so bisect correlates performance changes with policy edits (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Multi-region fleets replicate mode maps only after triplet passes in each region’s SKU mix (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; AMD, 2024b,a; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Debug versus production mode policy. Development clusters may run eager decode for fast iteration (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Production clusters must enforce mode maps checked into config management (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016). Accidental eager fallback in production reads as cost overrun and tail latency inflation (Vellaisamy *et al.*, 2026; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Chen, 2026a; Goodfellow *et al.*, 2016). Alert when launch counts exceed eager thresholds on production endpoints (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Separate debug mode toggles from feature flags so marketing demos do not become silent production configs (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Hybrid mode maps are normal: prefill eager or wide-tile, decode PK plus graph, sampling eager hull, MoE communication custom (NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Liu *et al.*, 2024; Dao *et al.*, 2022; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Document each subgraph mode in fleet config with owner and counter expectation (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Changing one subgraph mode without re-running triplet on affected rows invites silent regressions (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon

et al., 2023; Goodfellow *et al.*, 2016). Mode maps should be diff-reviewed like code (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016).

SRE teams should store mode map version in incident tickets alongside framework and artifact versions (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Replaying incidents without mode context wastes hours tuning the wrong subgraph (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Quarterly drills rotate mode map edits through staging triplet before any production diff (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Fleet mode policy is co-design infrastructure as critical as paging block size (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; NVIDIA, 2024; Goodfellow *et al.*, 2016).

30.5 production regression

Production regression gates fleet promotion on triplet counter suites: bytes per token, launches per token, and framework-level tokens per second—measured on GPU, CPU, and NPU backends where applicable (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; AMD, 2024b,a; SemiAnalysis, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022, 2024; Goodfellow *et al.*, 2016). Version pins for **deps/YiRage** submodule, framework release, and artifact digest must move together through canary deploys (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Submodule bumps require re-run of search caches and PK warmup profiles before traffic shift (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Triplet worksheet. Row one: launches per token on warmed PK at BS=1 for ctx 512, 2048, 8192 (Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Row two: bytes per token on same buckets (Chen, 2026a; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Row three: end-to-end tokens per second including queue wait at representative fleet load (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Sign off only when each row improves or regressions carry profiler evidence and owner (Davies *et al.*, 2025; Chen, 2026a; Chen and

YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

Regression triggers. Framework paging ABI change; compiler manifest major bump; runtime descriptor version change; hardware registry id addition; CUDA driver or toolkit upgrade (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; NVIDIA, 2024; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; AMD, 2024b,a; Liu *et al.*, 2024; Dao *et al.*, 2022, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Each trigger reruns triplet on primary SKU plus one secondary backend in CI matrix (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; AMD, 2024a,b; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016).

Canary and rollback. Shift artifact digest on fraction of replicas; compare triplet dashboards against control cohort (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Rollback redeploys prior artifact bundle—not framework scheduler alone (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Store rollback pins beside fleet config for one-click revert (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Bytes-focused regression. Memory-Floor style suites catch HBM traffic regressions graph replay hides (Chen, 2026a; NVIDIA, 2024; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Pair byte counters with carrier lifetime profiles when fusion changes (Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). Silent staging copies from paging integration show as byte regressions with unchanged launch counts (Chen, 2026a; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016).

CI pipelines pin submodule SHA, compile native extensions, run superoptimize smoke, PK decode ten tokens, and compare golden hash (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Liu *et al.*, 2024; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Extend smoke with vLLM integration template from Section 30.3 before production promotion (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Record worksheet

JSON in artifact store for audit (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

Example 30.5.1. Submodule bump passes import test but fails PK golden hash—regression gate blocks merge; fleet stays on prior digest until triplet reruns green (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Long-horizon **regression** debt accumulates when teams skip triplet on minor framework patches (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Schedule weekly automated reruns on pinned production configs even without code changes—driver drift and thermal throttling move counters (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; SemiAnalysis, 2024; Goodfellow *et al.*, 2016; Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; AMD, 2024a,b; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Treat regression suites as production infrastructure, not optional research scripts (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Cross-team regression review meetings compare triplet trends across framework, compiler, and runtime release notes (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). A flat launches per token row with rising bytes per token row signals compiler or paging integration regression (Chen, 2026a; Vellaisamy *et al.*, 2026; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Rising queue wait with flat in-step rows signals framework capacity or scheduling regression (Davies *et al.*, 2025; Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Archive worksheet JSON for each production promotion to enable quarter-over-quarter audits (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Regressions without archived worksheets are not reproducible and should not close incident tickets (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).

Closing the book loop. Return to Chapter 1 counters with co-design eyes (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). TaxBreak launches per token, Memory-Floor bytes per token, and framework tokens per second now have owners and handoffs (Vellaisamy *et al.*, 2026; Chen, 2026a; Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow

et al., 2016). MegaKernel craft from Part IV–V lives inside compiler artifacts (Chen, 2020; Chen and YiRage Contributors, 2026; Dao *et al.*, 2022; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Compiler theory from Part VI–VII lives in manifest schemas and legality metadata (Chen, 2020; Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). Runtime execution from Chapter 29 lives in PK pools and graph buckets (Chen and YiRage Contributors, 2026; NVIDIA, 2024; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026; Goodfellow *et al.*, 2016). Framework policy from Chapter 28 lives outside the decode boundary (Kwon *et al.*, 2023; Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). The book ends where production begins: pinned digests, triplet gates, and shared traces (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Regression culture. Treat triplet failures as cross-team signals, not blame events (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Framework teams publish paging ABI changelogs; compiler teams publish manifest semver notes; runtime teams publish descriptor diffs (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Goodfellow *et al.*, 2016). On-call runbooks link each changelog entry to required regression reruns (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). When all three parties follow the rhythm, co-design becomes routine operations instead of heroic firefighting (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). That operational maturity is the practical definition of decode goodput at warehouse scale (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

30.6 Key Takeaways

- **Three-way co-design.** Framework, compiler, and runtime each own distinct counters; optimize the leg that moves your worksheet row (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Goodfellow *et al.*, 2016).
- **Boundary is API.** Decode step boundary, paging ABI, capture buckets, and PK entry points documented with semver (Kwon *et al.*, 2023; NVIDIA, 2024; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025).

- **vLLM plus YiRage integration.** Scheduler and paging stay framework-side; fused decode core shrinks in-step bytes and launches (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Chen, 2026a; Vellaisamy *et al.*, 2026).
- **Map execution mode per subgraph, not per binary.** Eager, graph-captured, and megakernel/Persistent-Kernel modes should be chosen per decode cell, because a single binary-wide mode is wrong for some subgraphs (NVIDIA, 2024; Chen and YiRage Contributors, 2026). A dynamic-shape preprocessing step may stay eager while the fused decoder layer runs as a captured PK, and forcing one mode everywhere leaves goodput on the table (Vellaisamy *et al.*, 2026; Chen, 2026a).
- **Triplet regression gates.** Bytes per token, launches per token, tokens per second on pinned submodule and artifact digest (Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016).
- **Pin the submodule and smoke-test in CI.** Pin deps/YiRage to a digest and run a Persistent-Kernel decode smoke test, since an import-only check passes while the native co-designed stack is silently broken (Chen and YiRage Contributors, 2026; Chen, 2020). The three-way co-design only holds if the framework, compiler, and runtime versions are tested together against the same pinned artifact (Davies *et al.*, 2025; Goodfellow *et al.*, 2016).

30.7 Conclusion

This chapter closes Part VIII and the book’s industrial arc: hardware constraints (Part II–III), MegaKernel craft (Part IV–V), compiler theory and stacks (Part VI), production tuning (Part VII), and now *deployable* framework–compiler–runtime cooperation anchored on YiRage and decode goodput counters (Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Liu *et al.*, 2024; NVIDIA, 2024; Dao *et al.*, 2022; SemiAnalysis, 2024; AMD, 2024b,a; Dao *et al.*, 2023; NVIDIA, 2022; Goodfellow *et al.*, 2016). Chapter 26 production runbooks and Chapter 27 trend baselines supply fleet pins and paging ABI expectations; this chapter adds triplet regression gates and mode maps on top (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Draw the decode boundary, assign framework/compiler/runtime owners, integrate vLLM paging with YiRage artifacts, map execution modes per

subgraph on your fleet, and gate every promotion on triplet regression (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Goodfellow *et al.*, 2016). Future work tracks autonomous search from Chapter 25 and new hardware entries in `HardwareRegistry`—always profiled on bytes per token and launches per token at batch one, never on FLOPs alone (Chen and YiRage Contributors, 2026; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a, 2020; Goodfellow *et al.*, 2016). The loop closes where it opened: mechanical sympathy measured in counters your fleet can audit (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016).

Before you close the repository, run one end-to-end worksheet on your target SKU: warmed Persistent Kernel, three context buckets, vLLM or equivalent scheduler at representative load, triplet rows logged, artifact digest pinned in config (Kwon *et al.*, 2023; Chen and YiRage Contributors, 2026; Chen, 2020; Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; NVIDIA, 2024; Dao *et al.*, 2022; Goodfellow *et al.*, 2016). If that worksheet passes review, Part VIII is not theory—it is your production baseline (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). If it fails, return to the section whose counter moved and fix the contract before shipping features (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). Co-design is maintenance, not a one-time integration project (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Share the worksheet JSON with adjacent teams so framework, compiler, and runtime on-call engineers interpret the same numbers during incidents (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Vellaisamy *et al.*, 2026; Chen, 2026a; Goodfellow *et al.*, 2016). That shared artifact closes the book loop in operational practice, not only in prose (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016). Thirty chapters of mechanical sympathy converge on this worksheet—run it once on hardware you ship (Davies *et al.*, 2025; Vellaisamy *et al.*, 2026; Chen, 2026a; Chen and YiRage Contributors, 2026; Chen, 2020; Kwon *et al.*, 2023; Goodfellow *et al.*, 2016). Archive the passing run in your fleet config repo as proof Part VIII is live (Davies *et al.*, 2025; Chen and YiRage Contributors, 2026; Chen, 2020; Goodfellow *et al.*, 2016).

Bibliography

AMD (2024a). Ai engine technology. <https://www.AMD.com/en/products/adaptive-socs-and-fpgas/technologies/AI-engine.html>. 2, 14, 17, 26, 27, 30, 31, 32, 33, 34, 37, 38, 39, 41, 42, 43, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 63, 64, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 103, 104, 105, 106, 108, 109, 111, 112, 113, 114, 115, 116, 118, 120, 124, 125, 126, 127, 128, 133, 134, 135, 138, 139, 141, 143, 147, 148, 150, 151, 152, 153, 154, 155, 157, 158, 159, 160, 162, 163, 164, 165, 166, 167, 169, 170, 172, 173, 174, 175, 176, 177, 178, 179, 180, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 215, 216, 218, 220, 221, 222, 223, 224, 226, 227, 228, 229, 230, 231, 233, 234, 235, 236, 237, 238, 239, 242, 249, 250, 251, 253, 254, 257, 261, 265, 269, 270, 272, 274, 276, 279, 280, 282, 283, 290, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 318, 319, 320, 322, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 360, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 416, 418, 422, 424, 425, 429, 430, 431, 432, 433, 434, 436

AMD (2024b). AMD XDNA architecture. <https://www.AMD.com/en/technologies/XDNA.html>. 5, 14, 18, 26, 27, 30, 32, 33, 34, 36, 37, 38, 42, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 103, 106, 108, 111, 112, 113, 114, 115, 116, 124, 125, 127, 133, 135, 137, 138, 143, 147, 148, 154, 156, 157, 161, 162, 163, 164, 165, 167, 168, 172, 173, 174, 175, 176, 177, 178, 179, 180, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 206, 207, 208, 209, 210, 211, 212, 213, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 233, 234, 235, 236, 237, 238, 239, 242, 245, 249, 251, 253, 254, 257, 260, 261, 262, 265, 268, 270, 272, 275, 277, 279, 283, 290, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 324, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353,

354, 355, 356, 357, 358, 360, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 416, 418, 422, 424, 425, 429, 430, 431, 432, 433, 434, 436

Arya, X. (2024). A preliminary performance analysis of llm inference on edge accelerators. 15

Chen, J. (2026a). Memory-bound but not bandwidth-limited: The physical AI inference gap in batch-1 LLM decode. *arXiv preprint arXiv:2605.30571*. 44-cell cross-GPU study; CUDA Graphs A/B on H100 and L4. 5, 14, 15, 21, 29, 30, 31, 32, 34, 35, 36, 37, 38, 40, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 105, 106, 107, 109, 112, 113, 114, 115, 116, 118, 119, 120, 123, 125, 126, 127, 128, 129, 131, 133, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 233, 234, 235, 236, 237, 238, 239, 240, 241, 243, 244, 245, 246, 248, 249, 250, 251, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 290, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437

Chen, T., Zheng, L., Yan, E., Jiang, Z., Moreau, T., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018a). Learning to optimize tensor programs. In *Advances in Neural Information Processing Systems (NeurIPS)*. AutoTVM; template-based statistical cost model (GBT/TreeGRU), transfer learning; 1.2–3.8x end-to-end. 252, 254, 288, 290, 291, 292

Chen, T., Moreau, T., Jiang, Z., Zheng, L., Yan, E., Cowan, M., Shen, H., Wang, L., Hu, Y., Ceze, L., Guestrin, C., and Krishnamurthy, A. (2018b). TVM: An automated end-to-end optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594. Graph- and operator-level optimization, operator fusion, memory latency hiding, ML-based cost model; CPU/GPU/FPGA. 32, 39, 244, 247, 248, 249, 250, 251, 254, 255, 286, 287, 289, 292, 320

Chen, X. (2020). Compiler-based hardware profiling for software-defined architecture. 14, 15, 29, 30, 31, 32, 39, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 84, 85, 86, 87, 88, 92, 93, 94, 95, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 248, 249, 251, 258, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437

Chen, X. (2026b). Towards high-goodput llm serving with prefill-decode multiplexing. 2

Chen, X. and YiRage Contributors (2026). Yirage: Multi-backend LLM inference optimization. <https://github.com/chenxingqiang/YiRage>. Yield Revolutionary AGile Engine; extends Mirage with CUDA, CPU, MPS, Triton, NKI, and MLIR backends. 9, 14, 15, 29, 30, 33, 34, 36, 38, 39, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 92, 93, 94, 95, 96, 104, 105, 106, 114, 115, 116, 125, 126, 127, 128, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 242, 246, 247, 249, 254, 255, 257, 262, 263, 266, 269, 270, 272, 276, 277, 278, 280, 284, 285, 286, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 319, 320, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437

Dao, T. *et al.* (2023). Flash-decoding for long-context inference. PyTorch blog and Stanford CRFM technical report. Parallelizes attention along KV sequence; up to 8×

decode speedup at long context. 1, 14, 16, 29, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 62, 64, 65, 66, 67, 68, 69, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 92, 93, 94, 95, 98, 119, 126, 140, 141, 142, 143, 144, 145, 146, 147, 150, 151, 152, 153, 154, 156, 157, 159, 160, 163, 164, 166, 167, 168, 169, 170, 171, 173, 174, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 220, 221, 222, 228, 233, 234, 235, 236, 237, 238, 239, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 327, 328, 330, 331, 332, 333, 334, 335, 336, 337, 338, 340, 341, 342, 343, 344, 345, 346, 347, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 381, 383, 385, 387, 388, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 418, 422, 424, 425, 429, 430, 431, 432, 433, 434, 436

Dao, T., Fu, D., Ermon, S., Rudra, A., and Ré, C. (2022). FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*. 1, 14, 15, 17, 29, 30, 38, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 92, 93, 94, 95, 97, 98, 99, 100, 103, 105, 107, 117, 118, 120, 121, 122, 125, 126, 127, 129, 130, 135, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 168, 170, 171, 172, 173, 174, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 221, 222, 226, 228, 229, 233, 234, 235, 236, 237, 238, 239, 245, 253, 256, 259, 261, 262, 263, 267, 269, 275, 283, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 326, 327, 328, 330, 331, 332, 333, 334, 335, 336, 337, 338, 340, 341, 342, 343, 344, 345, 346, 347, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 418, 419, 422, 423, 424, 425, 426, 427, 429, 430, 431, 432, 433, 434, 435, 436, 437

Davies, M. *et al.* (2025). Liminal: Exploring the frontiers of LLM decode performance. *arXiv preprint arXiv:2507.14397*. Limit study; DeepSeek-V3 provisions 10× more decode than prefix nodes. 2, 14, 15, 16, 25, 26, 30, 34, 36, 40, 41, 42, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 92, 93, 94, 95, 98, 102, 105, 106, 109, 113, 114, 120, 125, 128, 130, 131, 132, 133, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 223, 224, 225, 226, 227, 228, 229, 230, 233, 234, 235, 236, 237, 238, 239, 240, 241, 243, 244, 245, 246, 247, 248, 253, 254, 256, 259, 260, 262, 264,

267, 268, 269, 271, 273, 274, 275, 276, 277, 278, 281, 283, 284, 286, 288, 289, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437

Eto, X. (2025). Llm performance bottlenecks on cgla. 2

Fan, X. (2025). Ellie: Energy-efficient llm inference at the edge via prefill-decode splitting. 1

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. 3, 14, 15, 17, 25, 26, 29, 31, 33, 36, 37, 39, 40, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 62, 63, 64, 66, 67, 68, 69, 70, 71, 74, 75, 76, 77, 78, 79, 80, 81, 82, 84, 85, 86, 87, 88, 92, 93, 94, 97, 99, 101, 102, 104, 105, 106, 111, 115, 118, 122, 123, 124, 127, 131, 132, 133, 134, 137, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 153, 154, 155, 156, 157, 158, 159, 160, 162, 163, 164, 165, 167, 168, 169, 170, 171, 172, 173, 174, 175, 179, 180, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 206, 207, 208, 209, 210, 211, 212, 213, 215, 216, 218, 220, 223, 224, 225, 226, 227, 228, 230, 233, 234, 235, 236, 237, 238, 239, 249, 252, 253, 255, 257, 260, 261, 262, 263, 264, 267, 268, 270, 271, 272, 273, 274, 275, 277, 279, 282, 285, 288, 290, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 321, 324, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437

IREE / iree-org contributors (2024). IREE: Intermediate representation execution environment — an MLIR-based end-to-end compiler and runtime. <https://iree.dev>. AOT MLIR compiler+runtime; Flow/Stream/HAL/VM dialects; dispatch-region fusion; .vmfb fat binary; targets llvm-cpu, CUDA, ROCm/HIP, Vulkan/SPIR-V, Metal, AMD AIE (experimental). 41, 42, 43, 264, 265, 266, 267, 268, 269, 270, 279, 286, 287, 288, 289, 291, 292, 317, 319, 320, 321, 323, 324, 325

Kim, X. (2025). Dataflow-driven neuromorphic architectures for edge ai: Theory, design, and applications. 16

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model

- serving with PagedAttention. *Proceedings of the 29th Symposium on Operating Systems Principles*. 2, 14, 18, 32, 34, 37, 40, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 65, 66, 67, 68, 69, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 92, 93, 94, 98, 102, 109, 111, 113, 118, 119, 120, 122, 124, 126, 140, 141, 142, 144, 145, 146, 147, 148, 149, 151, 153, 154, 156, 157, 158, 159, 160, 161, 163, 164, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 197, 198, 199, 200, 201, 202, 203, 204, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 218, 220, 221, 222, 225, 226, 227, 229, 230, 233, 234, 235, 236, 237, 238, 239, 241, 244, 245, 246, 248, 251, 252, 260, 262, 268, 270, 275, 277, 282, 283, 284, 288, 289, 290, 292, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 322, 324, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437
- Lai, R., Shao, J., Feng, S., Jin, Y., *et al.* (2023). Relax: Composable abstractions for end-to-end dynamic machine learning. *arXiv preprint arXiv:2311.02103*. TVM Unity; cross-level IRModule (Relax graph + TensorIR), first-class symbolic shapes, call_tir DPS; replaces Relay. 34, 39, 43, 240, 241, 242, 243, 246, 250, 251, 254, 281, 319, 320, 324
- Lazuka, X. (2024). Llm-pilot: Characterize and optimize performance of your llm inference services. 11
- Li, X. (2024). Llm inference serving: Survey of recent advances and opportunities. 2
- Li, X. (2025). Llm-powered automated detection and optimization of inference bottlenecks in distributed systems: A static analysis approach. 9
- Liu, A. *et al.* (2024). Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*. 16, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 64, 65, 66, 67, 68, 69, 70, 71, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 92, 93, 94, 140, 144, 145, 146, 148, 149, 150, 151, 152, 153, 154, 156, 158, 159, 160, 162, 163, 164, 166, 167, 169, 170, 171, 172, 173, 174, 176, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 197, 198, 199, 200, 201, 202, 203, 205, 206, 207, 208, 209, 210, 211, 212, 213, 215, 216, 219, 221, 222, 223, 224, 225, 227, 228, 229, 230, 233, 234, 235, 236, 237, 238, 239, 245, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 381, 382, 383, 385, 387, 388, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410,

411, 412, 413, 414, 415, 416, 418, 422, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 436

Mirage Project contributors (2025). Mirage persistent kernel (MPK): A compiler and runtime for mega-kernelizing tensor programs. <https://github.com/mirage-project/mirage>. Transforms LLM inference into a single megakernel; SM-level task graph (tGraph); cross-operator pipelining; $1.2\text{--}6.7\times$ latency reduction. 32, 36, 104, 105, 106, 107, 109, 112, 113, 115, 116, 126, 127, 136, 137, 139, 278, 279, 281, 282, 283, 284, 285, 288, 289, 290, 291, 292, 318, 322, 323, 324, 325

Nakai, X. (2025). Efficient batch processing for private cloud llm inference: Modeling and performance comparison. 15

NVIDIA (2022). Nvidia hopper architecture in-depth. NVIDIA Technical Blog. 17, 26, 28, 30, 31, 32, 33, 35, 39, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 92, 93, 94, 95, 97, 100, 101, 104, 105, 106, 108, 110, 111, 114, 115, 116, 121, 122, 126, 127, 128, 129, 130, 131, 132, 134, 135, 136, 137, 138, 141, 143, 145, 147, 148, 150, 151, 155, 158, 162, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 177, 183, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 197, 198, 199, 200, 202, 203, 204, 206, 207, 208, 209, 210, 211, 212, 213, 215, 216, 218, 219, 220, 221, 222, 223, 224, 229, 233, 234, 235, 236, 237, 238, 239, 245, 246, 249, 253, 257, 259, 260, 262, 265, 268, 269, 270, 274, 283, 284, 290, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 319, 320, 324, 327, 328, 330, 331, 332, 333, 334, 336, 337, 338, 340, 341, 342, 343, 344, 345, 346, 347, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 363, 364, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 381, 383, 385, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 418, 422, 424, 425, 429, 430, 431, 432, 433, 434, 436

NVIDIA (2024). Cuda graphs. CUDA Programming Guide. 5, 14, 20, 28, 29, 31, 36, 40, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 63, 65, 66, 67, 68, 69, 71, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 84, 85, 86, 87, 88, 92, 93, 94, 97, 107, 118, 129, 140, 144, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 170, 171, 172, 173, 174, 175, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 197, 198, 199, 200, 201, 205, 206, 207, 208, 209, 210, 211, 212, 213, 215, 216, 217, 218, 219, 220, 223, 224, 225, 226, 227, 229, 230, 233, 234, 235, 236, 237, 238, 239, 240, 241, 244, 246, 248, 251, 253, 254, 256, 259, 262, 264, 266, 267, 270, 271, 272, 274, 276, 277, 284, 287, 292, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 326, 327, 328, 329, 330, 331, 332, 333, 334, 336, 337, 338, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 385, 386, 387, 388, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 417,

- 418, 419, 420, 421, 422, 423, 424, 425, 426, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437
- OpenAI / triton-lang contributors (2024). Triton: A language and compiler for custom deep-learning primitives. <https://github.com/triton-lang/triton>. MLIR-based backend: Triton IR (TTIR) $\rightarrow$ TritonGPU IR (TTGIR) $\rightarrow$ LLVM $\rightarrow$ PTX/AMDGCN; @triton.autotune with num_warps/num_stages. 257, 258, 259, 260, 262, 287, 288, 289, 290, 292
- OpenXLA Project (2024). XLA: Accelerated linear algebra — OpenXLA architecture. <https://openxla.org/xla/architecture>. StableHLO $\rightarrow$ HLO $\rightarrow$ LLVM/PTX/TPU; operator fusion as primary optimization (no HBM intermediates); PriorityFusion / Multi-Output passes; CommandBufferThunk launch capture; Triton emitters. 240, 241, 242, 243, 244, 245, 246
- Pichlmeier, X. (2024). Performance characterization of expert router for scalable llm inference. 1
- PyTorch / Glow contributors (2024). Glow: Compiler for neural network hardware accelerators. <https://github.com/pytorch/glow>. LLVM-based CPU backend with operator stacking; model-compiler AOT bundles; profile-guided quantization; cross-compile to edge/MCU. 271, 272, 273, 274, 275, 276, 277
- Qiao, X. (2026). Tellme: An efficient end-to-end ternary llm prefill and decode accelerator with table-lookup matmul on edge fpgas. 2
- Recasens, X. (2025). Mind the memory gap: Unveiling gpu bottlenecks in large-batch llm inference. 1
- Rotem, N., Fix, J., Abdulasool, S., Catron, G., Deng, S., Dzhavarov, R., Gibson, N., Hegeman, J., Lele, M., Levenstein, R., Montgomery, J., Maher, B., Nadathur, S., Olesen, J., Park, J., Rakhov, A., and Smelyanskiy, M. (2018). Glow: Graph lowering compiler techniques for neural networks. *arXiv preprint arXiv:1805.00907*. Two-phase strongly-typed IR (high-level graph + low-level address-only); lowering to linear-algebra primitives; profile-guided quantization; AOT bundles. 32, 33, 36, 38, 99, 101, 105, 271, 272, 273, 274, 275, 276, 277, 282, 286, 287, 288, 289, 291, 292, 317, 318, 320, 321, 322, 323, 324
- SemiAnalysis (2024). Nvidia tensor core evolution: From volta to blackwell. Newsletter. 2, 15, 17, 28, 31, 33, 34, 36, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 71, 72, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 92, 93, 94, 96, 97, 98, 100, 104, 106, 108, 111, 114, 116, 117, 118, 119, 120, 121, 122, 123, 125, 127, 141, 143, 144, 145, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 179, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 218, 220, 222, 224, 225, 227, 233, 234, 235, 236, 237, 238,

- 239, 240, 242, 249, 251, 257, 258, 259, 260, 262, 265, 268, 270, 274, 276, 280, 282, 290, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 318, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337, 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 360, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 381, 382, 383, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 418, 419, 422, 424, 425, 429, 430, 431, 432, 433, 434, 436
- Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., and Dao, T. (2024). FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. In *Advances in Neural Information Processing Systems (NeurIPS)*. Hopper warp specialization (producer TMA / consumer WGMMMA), persistent kernels with tile scheduler, pingpong scheduling. 35, 39, 42, 43, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 118, 119, 120, 121, 122, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 138, 139
- Sun, X. (2019). Power-driven dnn dataflow optimization on fpga. 16
- Tang, X. (2026). Tpla: Tensor parallel latent attention for efficient disaggregated prefill & decode inference. 2
- Tillet, P., Kung, H. T., and Cox, D. (2019). Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL@PLDI)*, pages 10–19. Tile = statically shaped sub-array; Triton-C + LLVM-based Triton-IR; matmul/conv on par with cuBLAS/cuDNN. 244, 246, 256, 257, 258, 261, 262, 263, 286, 287, 289, 292, 319, 320
- Vellaisamy, P., Tripathi, S., Natarajan, V., Thenarasu, S. S., Blanton, S., and Shen, J. P. (2026). Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition. *arXiv preprint arXiv:2603.12465*. 4, 14, 15, 28, 29, 30, 31, 32, 34, 35, 36, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 105, 106, 107, 109, 111, 112, 113, 115, 116, 118, 119, 120, 122, 126, 127, 128, 129, 136, 138, 140, 141, 142, 143, 144, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 233, 234, 235, 236, 237, 238, 239, 240, 241, 243, 244, 245, 246, 248, 249, 250, 251, 253, 254, 255, 256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268, 269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 281, 282, 283, 284, 285, 286, 287, 289, 291, 292, 293, 294, 295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307, 308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 321, 322, 323, 324, 325, 326, 327, 328, 329, 330, 331, 332, 333, 334, 335, 336, 337,

- 338, 339, 340, 341, 342, 343, 344, 345, 346, 347, 348, 349, 350, 351, 352, 353, 354, 355, 356, 357, 358, 359, 360, 361, 362, 363, 364, 365, 366, 367, 368, 369, 370, 371, 372, 373, 374, 375, 376, 377, 378, 379, 380, 381, 382, 383, 384, 385, 386, 387, 388, 389, 390, 391, 392, 393, 394, 395, 396, 397, 398, 399, 400, 401, 402, 403, 404, 405, 406, 407, 408, 409, 410, 411, 412, 413, 414, 415, 416, 417, 418, 419, 420, 421, 422, 423, 424, 425, 426, 427, 428, 429, 430, 431, 432, 433, 434, 435, 436, 437
- Wang, X. (2025a). Decoupled analysis of dvfs effects in prefill and decode stages of large language model inference. 5
- Wang, X. (2025b). Pdacc: A 541 tokens/s@llama-1.2b effective throughput prefill-decode disaggregated mcm-based accelerator for multi-task large language model applications. 2
- Wang, X. (2026). E ⁴ -cim rram: An evolvable operator framework from linear to high-order compute-in-memory via monolithic 3d integration. 16
- Wu, M., Cheng, X., Liu, S., Shi, C., Ji, J., Ao, M. K., Velliengiri, P., Miao, X., Padon, O., and Jia, Z. (2024). Mirage: A multi-level superoptimizer for tensor programs. *arXiv preprint arXiv:2405.05751*. μ Graph kernel/block/thread levels; joint algebraic+schedule+custom-kernel superoptimization; abstraction-based pruning; probabilistic equivalence via PIT over finite fields (LAX); 1.1–2.9 $\times$. 38, 41, 42, 43, 99, 104, 105, 106, 107, 109, 114, 115, 116, 126, 127, 128, 135, 136, 137, 138, 245, 278, 279, 280, 281, 282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 319, 320, 323, 325
- Yu, X. (2026). Precision bottlenecks and sensitivity analysis in llm quantizationllm quantization bottlenecks. 2
- Zhao, X. (2025). Incredflip: Incremental dataflow-driven macro flipping for efficient macro placement refinement. 17
- Zheng, L., Jia, C., Sun, M., Wu, Z., Yu, C. H., Haj-Ali, A., Wang, Y., Yang, J., Zhuo, D., Sen, K., Gonzalez, J. E., and Stoica, I. (2020). Ansor: Generating high-performance tensor programs for deep learning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 863–879. Template-free auto-scheduler; hierarchical sampling, evolutionary search, task scheduler; up to 3.8x/2.6x/1.7x on Intel CPU/ARM CPU/NVIDIA GPU. 247, 252, 253, 254, 255, 287, 288, 290, 291, 292

Index

- Apache TVM, 209
- Arithmetic intensity, iv
- Asynchronous copy, 102
- Barrier, 113
- Compiler comparison, 244
- Compiler constraints, 31
- Cross-hardware benchmark, 244
- Data residency, 82
- Deadlock, 113
- Decode phase, 1
- Double buffering, 102
- Glow, 230
- Hardware abstraction layer, 223
- Hardware architecture, 31
- Heterogeneous deployment, 266
- HLO, 203
- Intermediate representation, iv
- IREE, 223
- Latency, iv
- LLM inference, 1
- Megakernel, 237
- Memory hierarchy, 31
- Mirage, 237
- MLIR, 223
- Multi-hardware, 266
- On-chip memory, 82
- Operator stacking, 230
- Persistent kernel, 92
- Prefill phase, 1
- Profile-guided quantization, 230
- Shared memory, 82
- StableHLO, 203
- Static role assignment, 92
- Superoptimization, 237
- Synchronization, 113
- Tensor Memory Accelerator, *see* TMA
- TensorIR, 209
- Throughput, iv
- Tile programming model, 216
- TMA, 102
- Triton, 216
- TritonGPU IR, 216
- TVM, 209
- Warp specialization, 92
- XLA, 203